

Thinking Inside the Box

2nd Edition
Includes Real-Time and
Linux Examples

Programming Embedded Systems

*with C and GNU
Development Tools*



O'REILLY®

Michael Barr & Anthony Massa
Foreword by Jack Ganssle

Programming Embedded Systems



If you have programming experience and a familiarity with C, *Programming Embedded Systems*, Second Edition, is exactly what you need to get started writing embedded software.

The first edition of *Programming Embedded Systems* taught the subject to tens of thousands of people and is now considered the bible of embedded programming. This second edition has been updated to cover the latest hardware designs and development methodologies.

The techniques and code examples presented here are directly applicable to real-world embedded software projects of all kinds. Examples use the free GNU software programming tools, the eCos and Linux operating systems, and a low-cost hardware platform specially developed for this book. If you obtain these tools along with *Programming Embedded Systems*, Second Edition, you'll have a full environment for exploring embedded systems in depth. But even if you work with different hardware and software, the principles covered in this book apply.

Whether you are new to embedded systems or have done embedded work before, you'll benefit from the topics in this book, which include:

- Basic debugging techniques—a critical skill when working with minimally endowed embedded systems
- Interrupts, and the monitoring and control of on-chip and external peripherals
- Determining whether you have real-time requirements, and whether your operating system and application can meet those requirements
- Task synchronization with real-time operating systems and embedded Linux
- Optimizing embedded software for size, speed, and power consumption

So, whether you're writing your first embedded program, designing the latest generation of hand-held whatchamacallits, or managing the people who do, *Programming Embedded Systems*, Second Edition, will help you develop the knowledge and skills you need to achieve proficiency with embedded software.

Praise for the first edition:

"This lively and readable book is the perfect introduction for those venturing into embedded systems software development for the first time. It provides, in one place, all the important topics necessary to orient programmers to the embedded development process."

—Lindsey Vereen, Editor-in-Chief, Embedded Systems Programming

www.oreilly.com

US \$49.99

CAN \$64.99

ISBN-10: 0-596-00983-6

ISBN-13: 978-0-596-00983-0



9



Includes
**FREE 45-Day
Online Edition**

Sistemas Embarcados de Programação

com ferramentas de desenvolvimento C e GNU

Outros recursos da O'Reilly

Títulos Relacionados

Construindo Linux Embutido
Sistemas

C resumido

Projeto embutido
Hardware

Projetos de hacking doméstico para
Geeks

Linux in a Nutshell

RFID Essentials

Programação Prática C

oreilly.com

oreilly.com é mais do que um catálogo completo de livros da O'Reilly. Você também encontrará links para notícias, eventos, artigos, weblogs, capítulos de amostra e exemplos de código.



oreillynet.com é o portal essencial para desenvolvedores interessados em tecnologias abertas e emergentes, incluindo novas plataformas, linguagens de programação e sistemas operacionais.

Conferências

O'Reilly reúne diversos inovadores para nutrir as ideias que geram indústrias revolucionárias. Somos especializados em documentar as ferramentas e sistemas mais recentes, traduzindo o conhecimento do inovador em habilidades úteis para aqueles que estão nas trincheiras. Visita *conferences.oreilly.com* para nossos próximos eventos.



Safari Bookshelf (*safari.oreilly.com*) é a principal biblioteca de referência online para programadores e profissionais de TI. Faça pesquisas em mais de 1.000 livros. Os assinantes podem se concentrar em respostas a perguntas críticas em questão de segundos. Leia os livros em sua estante de capa a capa ou simplesmente vire para a página que você precisa. Experimente hoje de graça.

SEGUNDA EDIÇÃO

Sistemas Embarcados de Programação

com ferramentas de desenvolvimento C e GNU

Michael Barr e Anthony Massa

O'REILLY®

Pequim • Cambridge • Farnham • Köln • Paris • Sebastopol • Taipei • Tóquio

Programando Sistemas Embarcados com Ferramentas de Desenvolvimento C e GNU, Segunda Edição
por Michael Barr e Anthony Massa

Copyright © 2007, 1999 O'Reilly Media, Inc. Todos os direitos reservados.
Impresso nos Estados Unidos da América.

Publicado por O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

Os livros da O'Reilly podem ser adquiridos para uso educacional, comercial ou promocional de vendas. As edições online também estão disponíveis para a maioria dos títulos (*safari.oreilly.com*). Para mais informações, entre em contato com nosso departamento de vendas corporativo / institucional: (800) 998-9938 ou *corporate@oreilly.com*.

Editor: Andy Oram

Editor de produção: Lydia Onofrei

Editor de cópia: Lydia Onofrei

Revisor: Mary Brady

Indexador: Ellen Troutman Zaig

Designer de capa: Karen Montgomery

Designer de interiores: David Futato

Ilustradores: Robert Romano e Jessamyn Read

Histórico de impressão:

Janeiro de 1999: Primeira edição.

Outubro de 2006: Segunda edição.

Nutshell Handbook, o logotipo Nutshell Handbook e o logotipo O'Reilly são marcas registradas da O'Reilly Media, Inc. *Programando Sistemas Embarcados com Ferramentas de Desenvolvimento C e GNU*, a imagem de um carrapato e a imagem comercial relacionada são marcas comerciais da O'Reilly Media, Inc. Muitas das designações usadas por fabricantes e vendedores para distinguir seus produtos são reivindicadas como marcas comerciais. Onde essas designações aparecem neste livro e a O'Reilly Media, Inc. estava ciente de uma reivindicação de marca registrada, as designações foram impressas em maiúsculas ou iniciais.

Embora todas as precauções tenham sido tomadas na preparação deste livro, a editora e os autores não assume nenhuma responsabilidade por erros ou omissões, ou por danos resultantes do uso das informações aqui contidas.



Este livro usa RepKover™, uma encadernação plana durável e flexível.

ISBN-10: 0-596-00983-6

ISBN-13: 978-0-596-00983-0

[M]

[12/07]

Para meu filho, Vikram.

- Michael Barr

*Este livro é dedicado a
minhas filhas maravilhosas, Katie e Ashley,
e minha linda esposa, Deanna. Você
significa tudo para mim. Eu amo Você.*

- Anthony Massa

Índice

Prefácio XI

Prefácio..... xiii

1. Introdução 1

O que é um sistema integrado?	1
Variações sobre um tema	4
Exemplos de design incorporado	8
A vida como desenvolvedor de software integrado	11
A linguagem C: o menor denominador comum	13
Algumas palavras sobre hardware	15

2. Conhecendo o hardware..... 19

Compreendendo os fundamentos	19
gerais do hardware	21
Examine a paisagem	28
Aprenda a se comunicar	32
Conhecendo o processador	34
Estude os periféricos	38
externos Inicialize o hardware	39

3. Seu primeiro programa integrado..... 43

Olá Mundo!	43
O Programa de LED Piscando	44
O Papel do Ciclo Infinito	51

4	Compilando, vinculando e localizando.	53
	O processo de construção	54
	Construindo o programa de LED piscando	61
	Uma rápida olhada nos Makefiles	66
5	Baixando e depurando.	70
	Baixando os depuradores remotos do	70
	programa LED piscando	77
	Emuladores	84
	Outras ferramentas úteis	86
	Explore o hardware	92
6	Memória	93
	Tipos de Memória	93
	Acesso direto à memória	98
	Edições Endian	98
	Teste de Memória	102
	Validando o conteúdo da memória	114
	usando a memória Flash	118
7	Periféricos.	122
	Registros de controle e status A	122
	filosofia do driver do dispositivo	130
	Projeto do driver do dispositivo	140
8	Interrupções.	142
	Visão geral	142
	Mapa de interrupção	148
	Interromper a rotina de serviço	150
	Resumo do programa de LED piscando	155
	aprimorado de problemas de interrupção	161
9	Juntando tudo.	164
	Visão geral do aplicativo	164
	Trabalhando com processamento de interface de	166
	linha de comando de portas seriais	167
10	Sistemas operacionais	173
	História e Propósito	173
	The Scheduler	174
	Tarefas	180

Sincronização de Tarefas	185
Passagem de mensagens	190
Outra Funcionalidade	191
Manipulação de interrupção	191
Características em tempo real	192
Usar ou não usar recursos adicionais RTOS	194
	197
11. Exemplos eCos.	198
Introdução	198
Mecânica da Tarefa	199
Mutex Task Synchronization	202
Sincronização de tarefas de semáforo	205
passagem de mensagens	210
Tratamento de interrupções eCos	213
12. Exemplos de Linux integrado.	219
Introdução	219
Acessando Hardware no Linux	220
Task Mechanics	220
Mutex Task Synchronization	222
Sincronização de tarefas de semáforo	224
passagem de mensagens	227
13. Estendendo Funcionalidade.	232
Periféricos Comuns	232
Rede para todos os dispositivos, grandes e pequenos	242
14. Técnicas de otimização.	248
Aumentando a Eficiência do Código	249
Diminuindo o Tamanho do Código	252
Problemas com a otimização de compiladores,	254
reduzindo o uso de memória	255
Técnicas de economia de energia	256
que limitam o impacto do C ++	259

A. O kit de desenvolvimento ArcomVIPER-Lite.	263
B. Configurando Seu Ambiente de Desenvolvimento de Software.	266
C. Construindo as Ferramentas de Software GNU.	271
D. Configurando o Ambiente de Desenvolvimento eCos.	274
E. Configurando o Embedded Linux Development Environment.	277
Índice.	285

Prefácio

Se você mencionar a palavra *embutido* para a maioria das pessoas, eles presumirão que você está falando sobre repórteres em uma zona de guerra. Poucos dicionários, incluindo o canônico *Dicionário de Inglês Oxford* -ligação *embutido* para sistemas de computador. Mesmo assim, os sistemas embarcados são a base de quase todos os dispositivos eletrônicos usados hoje, de telefones celulares a abridores de portas de garagem e instrumentos médicos. Agora, é quase impossível construir qualquer coisa eletrônica sem adicionar pelo menos um pequeno microprocessador e software associado.

Os fornecedores produzem cerca de nove bilhões de microprocessadores a cada ano. Talvez 100 ou 150 milhões deles vão para PCs. Isso é apenas cerca de um por cento das unidades enviadas. Os outros 99% vão para sistemas embarcados; claramente, esse negócio furtivo representa a própria estrutura de nossa sociedade altamente tecnológica.

E o uso dessas tecnologias só aumentará. Soluções para problemas ambientais iminentes certamente dependerão do uso mais inteligente de recursos habilitados por sistemas embarcados. Basta olhar para a rede de processadores de 32 bits no híbrido Prius da Toyota para ter um vislumbre do futuro.

Embora os prognósticos sejam difíceis, é absolutamente claro que os consumidores continuarão a demandar produtos cada vez mais inteligentes, exigindo mais microprocessadores e grandes aumentos no software correspondente. As estimativas sugerem que o conteúdo do firmware da maioria dos produtos dobra a cada 10 a 24 meses. Enquanto a demanda por mais código está aumentando, nossas taxas de produtividade aumentam lentamente. Portanto, também está claro que a indústria precisará de mais pessoas em sistemas embarcados para atender à demanda.

De quais habilidades essas pessoas precisarão? No mundo do PC, é preciso ser um programador C / C++ competente. Mas os desenvolvedores embarcados devem ter um conhecimento profundo das linguagens de programação e do próprio hardware; ninguém pode projetar, codificar e testar uma rotina de serviço de interrupção, por exemplo, sem saber de onde vêm as interrupções, como o hardware as prioriza, os truques por trás da manutenção desse hardware e detalhes no nível da máquina sobre como salvar e preservar o contexto do sistema. Um desenvolvedor de firmware deve ter uma visão detalhada da implementação de hardware dos periféricos de seu sistema antes de escrever uma única linha de código de driver.

No mundo do PC, a magia do hardware está escondida por trás de uma API extensiva. Em um sistema embarcado, essa API é sempre escrita pelos engenheiros que estão desenvolvendo o produto.

Neste livro, Michael Barr e Anthony Massa mostram como o software e o hardware formam uma gestalt sinérgica. Eles não se intimidam com as complicações de interrupções e I / O, ou inversão de prioridade e mutexes.

Os autores demonstram apropriadamente a construção de sistemas embarcados usando uma variedade de ferramentas de código aberto, incluindo o pacote de compiladores GNU, que é uma ferramenta padrão amplamente usada nesta indústria. eCos e Linux, ambos produtos de código aberto / gratuito, são usados para demonstrar sistemas operacionais pequenos e grandes.

A versão original deste livro usava uma placa de destino x86, que foi substituída nesta edição por um produto baseado em ARM. Coincidentemente, como este volume estava em produção, a Intel fez um anúncio do fim da vida útil de todos os seus processadores x86 embarcados. Os leitores podem ter certeza de que o ARM ainda existirá por muito tempo, pois é suportado por uma enorme infraestrutura de fornecedores.

O hardware é barato e facilmente disponível; o software é gratuito. Juntos, eles representam a corrente principal do desenvolvimento de sistemas embarcados. Os leitores podem ter certeza de que usarão essas ferramentas no futuro.

Compre o kit de desenvolvimento, leia o livro e execute os exemplos. Você obterá a experiência prática que os empregadores exigem: criando e trabalhando com aplicativos incorporados reais.

- Jack Ganssle

Prefácio

Primeiro, descubra por que você quer que os alunos aprendam o assunto e o que você quer que eles saibam, e o método resultará mais ou menos pelo bom senso.

- Richard Feynman

O software embutido está em quase todos os dispositivos eletrônicos em uso hoje. Há software escondido dentro de nossos relógios, DVD players, telefones celulares, freios antibloqueio e até mesmo algumas torradeiras. Os militares usam software integrado para guiar mísseis, detectar aeronaves inimigas e pilotar UAVs. Satélites de comunicação, sondas espaciais e muitos instrumentos médicos teriam sido quase impossíveis de criar sem ele.

Alguém precisa escrever todo aquele software, e há dezenas de milhares de engenheiros elétricos, cientistas da computação e outros profissionais que realmente o fazem. Somos dois deles e sabemos por experiência própria como pode ser difícil aprender o ofício.

Cada sistema embarcado é único e o hardware é altamente especializado para o domínio do aplicativo. Como resultado, a programação de sistemas embarcados pode ser uma experiência amplamente variada e pode levar anos para ser dominada. No entanto, um denominador comum em quase todo o desenvolvimento de software embarcado é o uso da linguagem de programação C. Este livro irá ensiná-lo a usar C em qualquer sistema embarcado.

Mesmo se você já sabe como escrever software embarcado, ainda pode aprender muito com este livro. Além de aprender como usar C de maneira mais eficaz, você também se beneficiará das explicações detalhadas e do código-fonte associado a problemas comuns de software embarcado. Entre os tópicos avançados abordados no livro estão teste e verificação de memória, design e implementação de driver de dispositivo, componentes internos do sistema operacional em tempo real e técnicas de otimização de código.

Por que escrevemos este livro

A cada ano, globalmente, aproximadamente um novo processador é fabricado por pessoa. Isso é mais de seis bilhões de novos processadores a cada ano, menos de dois por cento do

que são os Pentiums e PowerPCs no coração dos novos computadores pessoais. Você pode se perguntar se realmente existem tantos computadores ao nosso redor. Mas apostamos que em cinco minutos você provavelmente poderá encontrar dezenas de produtos em sua própria casa que contêm processadores: televisores, aparelhos de som, MP3 players, cafeteiras, despertadores, videocassetes, DVD players, microondas, lava-louças, controles remotos, máquinas de fazer pão, relógios digitais e assim por diante. E esses são apenas pertences pessoais - muitos mais desses dispositivos são usados no trabalho. O fato de que cada um desses produtos contém não apenas um processador, mas também um software, é o ímpeto para este livro.

Uma das coisas mais difíceis sobre esse assunto é saber quando parar de escrever. Cada sistema embarcado é único e, portanto, aprendemos que existe uma exceção para todas as regras. No entanto, tentamos reduzir o assunto à sua essência e apresentar as coisas que os programadores definitivamente precisam saber sobre sistemas embarcados.

Audiência pretendida

Este é um livro sobre programação de sistemas embarcados em C. Como tal, presume que o leitor já tenha alguma experiência de programação e esteja pelo menos familiarizado com a sintaxe da linguagem C. Também ajuda se você tiver alguma familiaridade com estruturas de dados básicas, como listas vinculadas. O livro não pressupõe que você tenha muito conhecimento sobre hardware de computador, mas espera que esteja disposto a aprender um pouco sobre hardware ao longo do caminho. Afinal, isso faz parte do trabalho de um programador embarcado.

Ao escrever este livro, tínhamos dois tipos de leitores em mente. O primeiro leitor é um iniciante - da mesma forma que éramos antes. Ele tem formação em ciência da computação ou engenharia e alguns anos de experiência em programação. O iniciante está interessado em escrever software embarcado para viver, mas não tem certeza de como começar. Depois de ler os primeiros capítulos, ele será capaz de colocar suas habilidades de programação para trabalhar desenvolvendo programas embarcados simples. O restante do livro servirá como referência para os tópicos mais avançados encontrados nos próximos meses e anos de sua carreira.

O segundo leitor já é um programador de sistemas embarcados. Ela está familiarizada com hardware embarcado e sabe como escrever software para ele, mas está procurando um livro de referência que explica os principais tópicos. Talvez o programador de sistemas embarcados tenha experiência apenas com programação em linguagem assembly e seja relativamente novo em C. Nesse caso, o livro irá ensiná-la a usar a linguagem C efetivamente em um sistema embarcado, e os capítulos posteriores fornecerão material avançado em real-time sistemas operacionais, periféricos e otimizações de código.

Quer você se enquadre em uma dessas categorias ou não, esperamos que este livro forneça as informações que você está procurando em um formato amigável e de fácil acesso.

Organização

O livro contém 14 capítulos e 5 apêndices. Os capítulos podem ser divididos muito bem em duas partes. A primeira parte consiste nos Capítulos 1 a 5 e é destinada principalmente para iniciantes em sistemas embarcados. Esses capítulos devem ser lidos na íntegra e na ordem em que aparecem. Isso o deixará atualizado rapidamente e apresentará os fundamentos do desenvolvimento de software embarcado. Depois de concluir o Capítulo 5, você estará pronto para desenvolver pequenos pedaços de software embutido por conta própria.

A segunda parte do livro consiste dos Capítulos 6 a 14 e discute tópicos avançados que são do interesse de programadores embarcados inexperientes e experientes. Esses capítulos são em sua maioria independentes e podem ser lidos em qualquer ordem. Além disso, os Capítulos 6 a 12 contêm programas de exemplo que podem ser úteis para você em um futuro projeto de software incorporado.

Capítulo 1, *Introdução*

Explica o campo da programação embarcada e apresenta os parâmetros do livro, incluindo o hardware de referência usado para exemplos

Capítulo 2, *Conhecendo o Hardware*

Mostra como explorar a documentação do seu hardware e representar os componentes com os quais você precisa interagir em C

Capítulo 3, *Seu primeiro programa integrado*

Cria um aplicativo simples de luz intermitente que ilustra os princípios básicos da programação embarcada

Capítulo 4, *Compilando, vinculando e localizando*

Descreve as maneiras como os sistemas embarcados diferem dos sistemas convencionais de computador durante as etapas de construção do programa, cobrindo questões como compiladores cruzados

Capítulo 5, *Baixando e depurando*

Apresenta as ferramentas de que você precisa para resolver problemas de hardware e software

Capítulo 6, *Memória*

Descreve os diferentes tipos de memória que os desenvolvedores escolhem para sistemas embarcados e os problemas envolvidos no uso de cada tipo

Capítulo 7, *Periféricos*

Apresenta a noção de um driver de dispositivo, junto com outras técnicas de codificação para trabalhar com dispositivos

Capítulo 8, *Interrupções*

Cobre esta área central de trabalho com periféricos

Capítulo 9, *Juntando tudo*

Combina os conceitos e o código do capítulo anterior com funções de conveniência e um programa principal, para criar um aplicativo carregável e testável

Capítulo 10, *Sistemas operacionais*

Apresenta conceitos comuns de sistema operacional, incluindo tarefas (ou threads) e mecanismos de sincronização, junto com os motivos para adicionar um sistema operacional em tempo real

Capítulo 11, *Exemplos eCos*

Mostra como usar alguns recursos do sistema operacional eCos em tempo real

Capítulo 12, *Exemplos de Linux embarcado*

Realiza a mesma tarefa do capítulo anterior, mas para o sistema operacional Linux embarcado

Capítulo 13, *Estendendo Funcionalidade*

Descreve opções para adicionar barramentos, rede e outros recursos de comunicação a um sistema

Capítulo 14, *Técnicas de Otimização*

Descreve maneiras de diminuir o tamanho do código, reduzir o uso de memória e economizar energia

Apêndice A, *O kit de desenvolvimento Arcom VIPER-Lite*

Descreve o quadro usado para os exemplos neste livro e como pedir um para você

Apêndice B, *Configurando Seu Ambiente de Desenvolvimento de Software*

Fornecer instruções para carregar o software descrito neste livro em seu computador host Windows ou Linux

Apêndice C, *Construindo as ferramentas de software GNU*

Mostra como compilar as ferramentas de desenvolvimento GNU

Apêndice D, *Configurando o Ambiente de Desenvolvimento eCos*

Mostra como construir uma biblioteca eCos apropriada para seu sistema embarcado para que você possa compilar programas para rodar em seu sistema

Apêndice E, *Configurando o Embedded Linux Development Environment*

Descreve como instalar as ferramentas Linux integradas para o seu sistema Arcom e construir e executar um programa nele

Ao longo do livro, tentamos encontrar um equilíbrio entre exemplos específicos e informações gerais. Sempre que possível, eliminamos pequenos detalhes na esperança de tornar o livro mais legível. Você vai tirar o máximo proveito do livro se visualizar os exemplos, como fazemos, principalmente como ferramentas para a compreensão de conceitos importantes. Tente não se prender aos detalhes de qualquer placa de circuito ou chip. Se você entende os conceitos gerais de programação C, deve ser capaz de aplicá-los a qualquer sistema embarcado que encontrar.

Para focar o código de exemplo do livro em conceitos específicos, intencionalmente o deixamos incompleto - por exemplo, eliminando certos arquivos de inclusão e declarações de variáveis redundantes. Para obter detalhes completos sobre o código, consulte o código-fonte de exemplo completo no site do livro.

Convenções, tipográficas e outros

As seguintes convenções tipográficas são usadas ao longo do livro:

itálico

Indica nomes de arquivos, programas, métodos e opções quando aparecem no corpo de um parágrafo. Itálico também é usado para dar ênfase e para introduzir novos termos.

Largura Constante

Nos exemplos, indica o conteúdo dos arquivos e a saída dos comandos. No texto normal, este estilo indica palavras-chave, funções, nomes de variáveis, classes, objetos, parâmetros e outros fragmentos de código.

Largura Constante Negrito

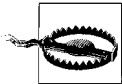
Indica comandos e opções a serem digitados literalmente. Este estilo é usado apenas em exemplos.

Largura Constante Negrito Itálico

Indica o texto a ser substituído por valores do usuário; por exemplo, um nome de arquivo em seu sistema. Este estilo é usado apenas em exemplos.



Este símbolo é usado para indicar uma dica, sugestão ou nota geral.



Este símbolo é usado para indicar um aviso.

Outras convenções estão relacionadas a gênero e funções. Com relação ao gênero, usamos propositalmente “ele” e “ela” ao longo do livro. Com relação às funções, ocasionalmente distinguimos entre as tarefas de engenheiros de hardware, engenheiros de software embarcado e programadores de aplicativos. Mas esses títulos se referem apenas a funções desempenhadas por engenheiros individuais, e deve-se observar que pode acontecer, e muitas vezes acontece, que um único indivíduo preencha mais de uma dessas funções em uma equipe de projeto integrado.

Obtendo os exemplos online

Este livro inclui muitas listagens de código-fonte, e todos, exceto os trechos mais triviais, estão disponíveis online. Esses exemplos são organizados por número de capítulo e incluem instruções de construção (makefiles) para ajudá-lo a recriar cada um dos executáveis. O completo arquivo está disponível em <http://examples.oreilly.com/embsys2>.

Usando exemplos de código

Este livro está aqui para ajudá-lo a realizar seu trabalho. Em geral, você pode usar o código deste livro em seus programas e documentação. Você não precisa entrar em contato conosco para obter permissão, a menos que esteja reproduzindo uma parte significativa do código. Por exemplo, escrever um programa que usa vários pedaços de código deste livro não requer permissão. Vender ou distribuir um CD-ROM de exemplos de livros da O'Reilly *faz* requer permissão. Responder a uma pergunta citando este livro e citando um código de exemplo não requer permissão. Incorporar uma quantidade significativa de código de exemplo deste livro na documentação do seu produto *faz* requer permissão.

Agradecemos, mas não exigimos atribuição. Uma atribuição geralmente inclui o título, autor, editora e ISBN. Por exemplo: " *Programando Sistemas Embarcados com Ferramentas de Desenvolvimento C e GNU*, Segunda edição, de Michael Barr e Anthony Massa. Copyright 2007 O'Reilly Media, Inc., 978-0-596-00983-0. "

Se você achar que o uso de exemplos de código está fora do uso justo ou da permissão fornecida acima, sinta-se à vontade para nos contatar em permission@oreilly.com.

Comentários e perguntas

Envie comentários e perguntas sobre este livro ao editor:

O'Reilly Media, Inc.
Rodovia 1005 Gravenstein Norte
Sebastopol, CA 95472
800-998-9938 (nos Estados Unidos ou Canadá)
707-829-0515 (internacional ou local)
707-829-0104 (fax)

Temos uma página web para este livro, onde listamos erratas, exemplos de código e qualquer informação adicional. Arquivos correspondentes para exemplos de código são mencionados na primeira linha do exemplo. Você pode acessar esta página em:

<http://www.oreilly.com/catalog/progembsys2>

Para comentar ou fazer perguntas técnicas sobre este livro, envie um e-mail para:

bookquestions@oreilly.com

Para obter mais informações sobre nossos livros, conferências, Centros de recursos e a Rede O'Reilly, consulte nosso site em:

<http://www.oreilly.com>

Safari® habilitado



Quando você vir um ícone Safari® Enabled na capa do seu livro de tecnologia favorito, isso significa que o livro está disponível online através da O'Reilly Network Safari Bookshelf.

O Safari oferece uma solução melhor do que e-books. É uma biblioteca virtual que permite que você pesquise facilmente milhares de livros de tecnologia de ponta, recorte e cole amostras de código, baixe capítulos e encontre respostas rápidas quando precisar das informações mais precisas e atuais
ção Experimente gratuitamente em <http://safari.oreilly.com>.

Comentários e agradecimentos pessoais

From Michael Barr

Desde que me lembro, tenho interesse em escrever um ou dois livros. Mas agora que escrevi vários, devo confessar que fui ingênuo quando comecei. Eu não tinha ideia de quanto trabalho seria necessário, ou quantas outras pessoas teriam que se envolver no processo. Outra coisa que me surpreendeu foi como foi fácil encontrar um editor disposto. Eu esperava que essa fosse a parte difícil.

Continuo a ser grato a todas as seguintes pessoas por compartilharem suas ideias e revisar meu trabalho na primeira edição: Toby Bennett, Paul Cabler (e outras pessoas incríveis da Arcom), Mike Corish, Kevin D'Souza, Don Davis, Steve Edwards, Mike Ficco, Barbara Flanagan, Jack Ganssle, Stephen Harpster, Jonathan Harris, Jim Jensen, Mark Kohler, Andy Kollegger, Jeff Mallory, Ian Miller, Henry Neugauss, Chris Schanck, Brian Silverman, John Snyder, Jason Steinhorn, Ian Taylor, Lindsey Vereen, Jeff Whipple e Greg Young.

Também gostaria de agradecer ao nosso editor, Andy Oram. Sem seu entusiasmo por minha proposta inicial, paciência superabundante e incentivo constante, nem a primeira nem a segunda edição deste livro teriam sido concluídas.

E, claro, estou extremamente grato a Anthony Massa. O interesse de Anthony em atualizar este livro com materiais adicionais, novo hardware e exemplos, e uma mudança nas ferramentas GNU veio na hora certa. Tem sido difícil ver alguém atualizar uma primeira edição que me deixou feliz e que vendeu tão surpreendentemente bem. Mas o novo livro é significativamente melhor para os esforços incansáveis de Anthony. Esta segunda edição não existiria se não fosse pelo trabalho árduo e dedicação de Anthony ao projeto.

De Anthony Massa

Esta é minha segunda aventura no reino da escrita de livros. Achei que escrever uma segunda edição daria muito menos trabalho porque a maior parte do material já estava concluída. Rapaz, eu estava errado. A segunda edição foi um pouco difícil e exigiu mais esforço e tempo do que eu esperava, mas acho que o livro saiu melhor como resultado.

Sou muito grato ao nosso editor, Andy Oram. Seu feedback foi fantástico, ele foi um guia para levar o livro até a conclusão, ele sempre forneceu a centelha necessária para juntar as coisas e ele até mesmo interveio para testar o código quando necessário. A segunda edição deste livro é muito melhor por causa dele e não teria sido possível sem seu apoio e determinação.

Gostaria de agradecer a Michael Barr pela oportunidade de trabalhar com ele neste projeto. Sei o quanto um escritor pode se tornar apegado a esse projeto; obrigado por me confiar a nova edição. Michael forneceu informações extremamente úteis e me ajudou a orientar o texto na direção certa. Houve algumas dificuldades para fazer as coisas certas, mas acho que trabalhar com elas melhorou o livro. Michael é realmente uma grande mente na comunidade de desenvolvimento de software embarcado.

Agradeço ao pessoal da Arcom que tão gentilmente forneceram o sistema de desenvolvimento impressionante e de primeira linha para este livro. Um grande obrigado a Glen Middleton, que sempre esteve lá para garantir que eu recebesse tudo o que precisava. E obrigado à equipe de desenvolvimento extremamente útil da Arcom, composta por Ian Campbell, Martyn Blackwell e David Vrabel.

Tive a sorte de que as seguintes pessoas dedicaram seu valioso tempo para ajudar a tornar este livro melhor, compartilhando ideias e revisando a segunda edição. Essa excelente equipe era composta por Michael Boerner, John Catsoulis, Brian Jepson, Nigel Jones, Alfredo Knecht, Jon Masters, Tony Montiel, Andrea Pellegrini, Jack Quinlan, Galen Seitz e David Simon. Um agradecimento especial a Jonathan Larmour por estar lá na embreagem quando eu tinha uma pergunta para você - você veio para mim, de novo.

Um agradecimento especial à minha equipe de revisão A-1 composta por Greg Babbitt, meu irmão Sean Hughes, Brian Kingston, Anthony Taranto e Joseph Terzoli.

Gostaria de agradecer a duas ótimas pessoas por todo o apoio que deram ao longo de minha vida - Nonno e Nonna. Eles sempre estiveram lá para mim com amor e orientação.

Agradeço ao meu irmão, Laurie, e à minha irmã, Catherine, pelo apoio. Sou grato por vocês dois estarem em minha vida.

Gostaria de dar um agradecimento muito especial à minha mãe e ao meu pai por me darem a base para o sucesso. Vocês são pessoas muito especiais, me apóiam em tudo que faço na vida e estão sempre presentes quando preciso de alguma coisa. Eu me sinto abençoado por ter você como meus pais.

Agradeço às minhas filhas, Katie e Ashley, que estão sempre disponíveis para me animar quando estou deprimida ou estressada. Vocês são garotas preciosas e especiais, e amo vocês duas de todo o coração.

Finalmente, gostaria de agradecer a minha esposa maravilhosa, Deanna. Sei que essa jornada foi difícil às vezes, mas você sempre foi paciente e me apoiou. Eu sou grato por você estar na minha vida. Obrigado por ser meu melhor amigo.

Introdução

Acho que existe um mercado mundial para talvez cinco computadores.

- Thomas Watson, presidente da IBM, 1943

Não há razão para alguém querer um computador em sua casa.

- Ken Olson, presidente de equipamentos digitais Corporation, 1977

Um dos desenvolvimentos mais surpreendentes das últimas décadas foi a ascensão dos computadores a uma posição de prevalência nos assuntos humanos. Hoje, há mais computadores em nossas casas e escritórios do que pessoas que vivem e trabalham neles. No entanto, muitos desses computadores não são reconhecidos como tal por seus usuários. Neste capítulo, explicaremos o que são os sistemas embarcados e onde são encontrados. Também apresentaremos o assunto da programação embarcada e discutiremos o que a torna uma forma única de programação de software. Explicaremos por que selecionamos C como linguagem para este livro e descreveremos o hardware usado nos exemplos.

O que é um sistema integrado?

Um *sistema embarcado* é uma combinação de hardware e software de computador - e talvez peças adicionais, mecânicas ou eletrônicas - projetadas para executar uma função dedicada. Um bom exemplo é o forno de microondas. Quase todas as famílias têm um, e dezenas de milhões deles são usados todos os dias, mas muito poucas pessoas percebem que um processador de computador e um software estão envolvidos na preparação de seu almoço ou jantar.

O projeto de um sistema embarcado para executar uma função dedicada está em contraste direto com o de um computador pessoal. Também é composto por hardware e software de computador e componentes mecânicos (unidades de disco, por exemplo). No entanto, um computador pessoal não foi projetado para executar uma função específica. Em vez disso, é capaz de fazer muitas coisas diferentes. Muitas pessoas usam o termo *computador de uso geral* fazer

esta distinção é clara. Na entrega, um computador de uso geral é uma tela em branco; o fabricante não sabe o que o cliente fará com ele. Um cliente pode usá-lo para um servidor de arquivos de rede, outro pode usá-lo exclusivamente para jogar e um terceiro pode usá-lo para escrever o próximo grande romance americano.

Freqüentemente, um sistema embarcado é um componente de algum sistema maior. Por exemplo, carros e caminhões modernos contêm muitos sistemas integrados. Um sistema integrado controla os freios antibloqueio, outro monitora e controla as emissões do veículo e um terceiro exibe informações no painel. Alguns fabricantes de carros de luxo até divulgaram o número de processadores (geralmente mais de 60, incluindo um em cada farol) em anúncios. Na maioria dos casos, os sistemas embarcados automotivos são conectados por uma rede de comunicações.

É importante ressaltar que um computador de uso geral faz interface com vários sistemas embarcados. Por exemplo, um computador típico possui um teclado e mouse, cada um dos quais é um sistema embutido. Cada um desses periféricos contém um processador e software e é projetado para executar uma função específica. Outro exemplo é um modem, projetado para enviar e receber dados digitais por meio de uma linha telefônica analógica; isso é tudo que faz. E a função específica de outros periféricos também pode ser resumida em uma única frase.

A existência do processador e do software em um sistema embutido pode passar despercebida por um usuário do dispositivo. Esse é o caso de um forno de micro-ondas, reproduzidor de MP3 ou despertador. Em alguns casos, seria até possível construir um dispositivo funcionalmente equivalente que não contivesse o processador e o software. Isso pode ser feito substituindo a combinação processador-software por um personalizado *circuito integrado (IC)* que executa as mesmas funções no hardware. No entanto, a combinação de processador e software geralmente oferece mais flexibilidade do que um design com fio. Geralmente é muito mais fácil, mais barato e com menos consumo de energia usar um processador e software em um sistema embarcado.

História e Futuro

Dada a definição de sistemas embarcados apresentada anteriormente neste capítulo, o primeiro desses sistemas não poderia ter surgido antes de 1971. Esse foi o ano em que a Intel lançou o primeiro microprocessador de chip único do mundo. Esse chip, o 4004, foi projetado para uso em uma linha de calculadoras de negócios produzida pela empresa japonesa Busicom. Em 1969, Busicom pediu à Intel para projetar um conjunto de circuitos integrados personalizados, um para cada um de seus novos modelos de calculadora. O 4004 foi a resposta da Intel. Em vez de projetar hardware personalizado para cada calculadora, a Intel propôs um circuito de uso geral que poderia ser usado em toda a linha de calculadoras. Este processador de uso geral foi projetado para ler e executar um conjunto de instruções - software - armazenado em um chip de memória externo.

O microprocessador foi um sucesso da noite para o dia, e seu uso aumentou continuamente na década seguinte. Os primeiros aplicativos incorporados incluíam sondas espaciais não tripuladas, semáforos computadorizados e sistemas de controle de voo de aeronaves. Nas décadas de 1980 e 1990, os sistemas embarcados navegaram silenciosamente nas ondas da era do microcomputador e trouxeram os microprocessadores para todas as partes de nossas vidas pessoais e profissionais. A maioria dos dispositivos eletrônicos em nossas cozinhas (máquinas de pão, processadores de alimentos e fornos de microondas), salas de estar (televisores, aparelhos de som e controles remotos) e locais de trabalho (máquinas de fax, pagers, impressoras a laser, caixas registradoras e leitores de cartão de crédito) são sistemas embarcados; mais de 6 bilhões de novos microprocessadores são usados a cada ano. Menos de 2 por cento (ou cerca de 100 milhões por ano) desses microprocessadores são usados em computadores de uso geral.

Parece inevitável que o número de sistemas embarcados continue a aumentar rapidamente. Já existem novos dispositivos embarcados promissores que têm enorme potencial de mercado: interruptores de luz e termostatos que estão conectados em rede e podem ser controlados sem fio por um computador central, sistemas de airbag inteligentes que não inflam quando crianças ou adultos pequenos estão presentes, médicos dispositivos de monitoramento que podem notificar um médico se as condições fisiológicas de um paciente estão em níveis críticos e sistemas de navegação de painel que informam sobre a melhor rota para o seu destino nas condições de tráfego atuais. Claramente, os indivíduos que possuem as habilidades e o desejo de projetar a próxima geração de sistemas embarcados serão solicitados por um bom tempo.

Sistemas em tempo real

Uma subclasse de sistemas embarcados merece uma introdução neste ponto. *UMA sistema em tempo real* tem restrições de tempo. A função de um sistema em tempo real é, portanto, parcialmente especificada em termos de sua capacidade de fazer certos cálculos ou decisões em tempo hábil. Esses cálculos ou atividades importantes têm prazos para conclusão.

A distinção crucial entre os sistemas de tempo real reside no que acontece se um prazo for perdido. Por exemplo, se o sistema em tempo real fizer parte do sistema de controle de voo de um avião, a vida dos passageiros e da tripulação pode ser colocada em perigo por um único prazo perdido. No entanto, se, em vez disso, o sistema estiver envolvido na comunicação por satélite, o dano pode ser limitado a um único pacote de dados corrompido (que pode ou não ter consequências catastróficas, dependendo do aplicativo e do esquema de recuperação de erro). Quanto mais graves forem as consequências, mais provável será que se diga que o prazo é "difícil" e, portanto, que o sistema é um *sistema rígido em tempo real*. Sistemas de tempo real na outra extremidade deste continuum são considerados como tendo prazos "suaves" - um *sistema soft em tempo real*. A Figura 1-1 mostra alguns exemplos de sistemas de tempo real hard e soft.

O projeto do sistema em tempo real não envolve apenas velocidade. Os prazos para sistemas em tempo real variam; um prazo pode ser em um milissegundo, enquanto outro está a uma hora de distância. A principal preocupação de um sistema em tempo real é que haja a garantia de que os prazos rígidos do sistema sejam sempre cumpridos. Para conseguir isso, o sistema deve ser previsível.

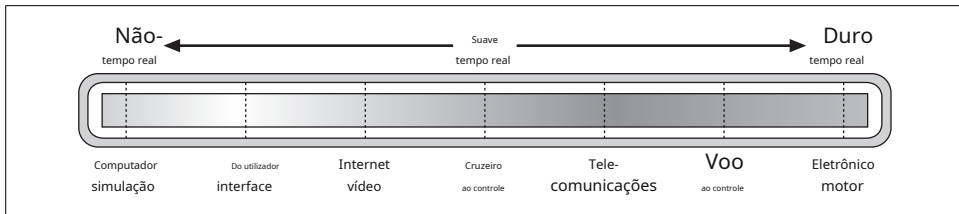


Figura 1-1. Uma variedade de exemplos de sistemas em tempo real

A arquitetura do software embarcado e sua interação com o hardware do sistema desempenham um papel fundamental para garantir que os sistemas em tempo real cumpram seus prazos. Os principais problemas de design de software incluem se a pesquisa é suficiente ou se as interrupções devem ser usadas e quais prioridades devem ser atribuídas às várias tarefas e interrupções. Deve haver premeditação adicional para a compreensão dos requisitos de desempenho do pior caso das atividades específicas do sistema.

Todos os tópicos e exemplos apresentados neste livro são aplicáveis aos projetistas de sistemas de tempo real. O projetista de um sistema de tempo real deve ser mais diligente em seu trabalho. Ele deve garantir a operação confiável do software e hardware em todas as condições possíveis. E, na medida em que vidas humanas dependem da execução adequada do sistema, essa garantia deve ser apoiada por cálculos de engenharia e papelada descritiva.

Variações sobre um tema

Ao contrário do software projetado para computadores de uso geral, o software incorporado geralmente não pode ser executado em outros sistemas incorporados sem uma modificação significativa. Isso se deve principalmente à incrível variedade de hardware em uso em sistemas embarcados. O hardware em cada sistema embarcado é adaptado especificamente para a aplicação, a fim de manter os custos do sistema baixos. Como resultado, os circuitos desnecessários são eliminados e os recursos de hardware são compartilhados sempre que possível.

Nesta seção, você aprenderá quais recursos de hardware são comuns em todos os sistemas embarcados e por que há tantas variações em relação a quase todo o resto. Posteriormente neste livro, veremos algumas técnicas que podem ser usadas para minimizar o impacto das alterações de software, de forma que não sejam necessárias em todas as camadas do software.

Componentes Comuns do Sistema

Por definição, todos os sistemas embarcados contêm um processador e software, mas que outros recursos eles têm em comum? Certamente, para ter software, deve haver um local para armazenar o código executável e armazenamento temporário para manipulação de dados em tempo de execução. Estes assumem a forma de *memória somente leitura (ROM)* e *memória de acesso aleatório (RAM)*, respectivamente; a maioria dos sistemas embarcados tem um pouco de cada. Se apenas um pequeno

quantidade de memória necessária, ela pode estar contida no mesmo chip do processador. Caso contrário, um ou ambos os tipos de memória residem em chips de memória externos.

Todos os sistemas embarcados também contêm algum tipo de entrada e saída. Por exemplo, em um forno de micro-ondas, as entradas são os botões no painel frontal e uma sonda de temperatura, e as saídas são o display legível e a radiação de micro-ondas. As saídas do sistema embarcado são quase sempre uma função de suas entradas e vários outros fatores (tempo decorrido, temperatura atual, etc.). As entradas para o sistema geralmente assumem a forma de sensores e sondas, sinais de comunicação ou botões e botões de controle. As saídas são normalmente exibições, sinais de comunicação ou alterações no mundo físico. Veja a Figura 1-2 para um exemplo geral de um sistema embarcado.

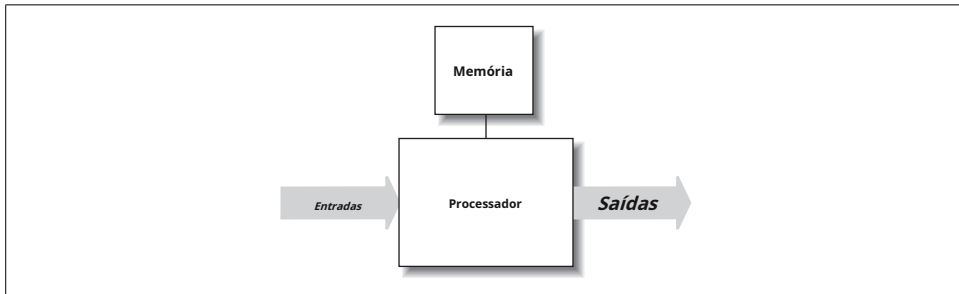


Figura 1-2. Um sistema embarcado genérico

Com exceção desses poucos recursos comuns, o resto do hardware embutido geralmente é único e, portanto, requer um software exclusivo. Essa variação é o resultado de muitos critérios de design concorrentes.

O software para o sistema embarcado genérico mostrado na Figura 1-2 varia dependendo da funcionalidade necessária. O hardware é a tela em branco e o software é a tinta que adicionamos para dar vida à imagem. A Figura 1-3 fornece apenas alguns diagramas de alto nível possíveis que podem ser implementados em um sistema embarcado genérico.

Tanto o diagrama de software embutido básico na Figura 1-3 (a) quanto o diagrama de software embutido mais complexo na Figura 1-3 (b) contêm blocos muito semelhantes. O bloco de hardware é comum em ambos os diagramas.

Os drivers de dispositivo são módulos de software embutidos que contêm a funcionalidade de operar os dispositivos de hardware individuais. O motivo do software do driver de dispositivo é eliminar a necessidade do aplicativo saber como controlar cada peça de hardware. Cada driver de dispositivo individual normalmente precisa saber apenas como controlar seu dispositivo de hardware. Por exemplo, para um forno de micro-ondas, drivers de dispositivo separados controlam o teclado, display, sensor de temperatura e controle de radiação.

Se mais funcionalidade for necessária, às vezes é necessário incluir camadas adicionais no software incorporado para ajudar com essa funcionalidade adicional. Neste exemplo,

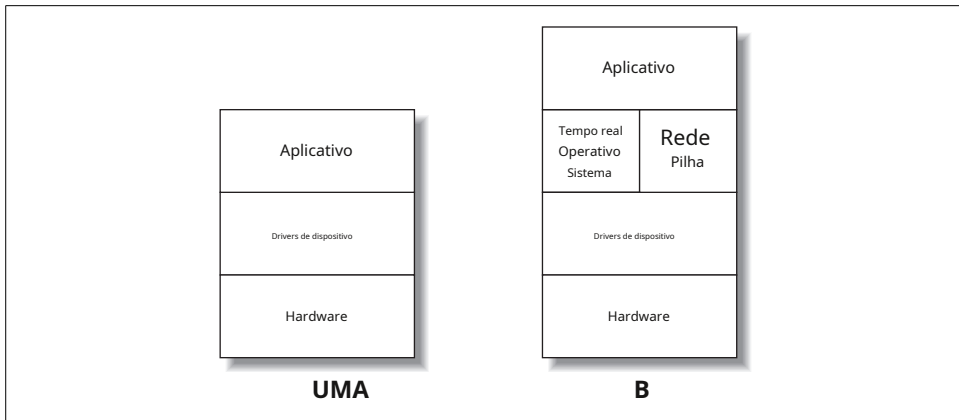


Figura 1-3. (a) Diagrama de software embarcado básico e (b) um diagrama de software embarcado mais complexo

o diagrama complexo inclui um *sistema operacional em tempo real (RTOS)* e uma pilha de rede. O RTOS pode ajudar o programador a separar a funcionalidade do aplicativo em tarefas distintas para uma melhor organização do software do aplicativo e um sistema mais responsivo. Investigaremos o uso de um RTOS posteriormente neste livro. A pilha de rede também aumenta a funcionalidade do sistema embarcado básico; um forno de micro-ondas pode usá-lo para exibir uma mensagem em seu computador desktop quando o almoço estiver pronto.

As responsabilidades da camada de software aplicativo são as mesmas nos diagramas de software embarcado básico e complexo. Em um forno de micro-ondas, o aplicativo processa as diferentes entradas e controla as saídas com base no que o usuário comanda.

Você notará que o software da Figura 1-3 é representado por blocos discretos empilhados uns sobre os outros com bordas fixas. Isso é feito deliberadamente, para indicar a separação das diferentes camadas funcionais do software que compõem o sistema de software embarcado completo. Posteriormente, decomporemos ainda mais esses blocos para mostrar como você pode manter seu software integrado limpo, fácil de ler e portátil. Manter essas camadas de software distintas, com métodos bem definidos que as camadas vizinhas podem usar para se comunicar, ajuda a escrever um bom software embarcado.

Requisitos que afetam as opções de design

Cada sistema embarcado deve atender a um conjunto completamente diferente de requisitos, qualquer um ou todos eles podem afetar os compromissos e compensações feitos durante o desenvolvimento do produto. Por exemplo, se o sistema deve ter um custo de produção inferior a \$ 10, outras características desejáveis - como poder de processamento e confiabilidade do sistema - podem precisar ser sacrificadas para atingir esse objetivo.

Obviamente, o custo de produção é apenas uma das possíveis restrições sob as quais os designers de hardware embarcado trabalham. Outros requisitos de design comuns incluem:

Poder de processamento

A carga de trabalho que o chip principal pode suportar. Uma maneira comum de comparar o poder de processamento é o *milhões de instruções por segundo* (Classificação MIPS). Se dois processadores semelhantes têm classificações de 25 MIPS e 40 MIPS, o último é considerado o mais poderoso. No entanto, outros recursos importantes do processador precisam ser considerados. Uma é a largura do registro, que normalmente varia de 8 a 64 bits. Os computadores de uso geral de hoje usam exclusivamente processadores de 32 e 64 bits, mas os sistemas embarcados ainda são construídos principalmente com processadores de 4, 8 e 16 bits menos caros.

Memória

A quantidade de memória (ROM e RAM) necessária para manter o software executável e os dados que ele manipula. Aqui, o designer de hardware geralmente deve fazer sua melhor estimativa antecipada e estar preparado para aumentar ou diminuir o valor real à medida que o software está sendo desenvolvido. A quantidade de memória necessária também pode afetar a seleção do processador. Em geral, a largura do registro de um processador estabelece o limite superior da quantidade de memória que ele pode acessar (por exemplo, um registro de endereço de 16 bits pode endereçar apenas 64 KB (2^{16}) locais de memória). *

Número de unidades

A execução de produção esperada. O trade-off entre custo de produção e custo de desenvolvimento é afetado principalmente pelo número de unidades que se espera serem produzidas e vendidas. Por exemplo, raramente faz sentido desenvolver componentes de hardware personalizados para um produto de baixo volume.

Consumo de energia

A quantidade de energia usada durante a operação. Isso é extremamente importante, especialmente para dispositivos portáteis alimentados por bateria. Uma métrica comum usada para comparar os requisitos de energia de dispositivos portáteis é mW / MIPS (miliwatts por MIPS); quanto maior esse valor, mais energia é necessária para realizar o trabalho. O menor consumo de energia também pode levar a outras características favoráveis do dispositivo, como menos calor, baterias menores, menos peso, tamanho menor e design mecânico mais simples.

Custo de desenvolvimento

O custo dos processos de design de hardware e software, conhecido como *Engenharia não recorrente* (NRE). Este é um custo fixo único, portanto, em alguns projetos, o dinheiro não é problema (geralmente para produtos de alto volume), enquanto em outros projetos, esta é a única medida precisa do custo do sistema (para a produção de um pequeno número de unidades).

* Quanto mais estreita for a largura do registro, maior será a probabilidade de o processador empregar truques como vários espaços de endereço para oferecer suporte a mais memória. Ainda existem sistemas embarcados que fazem o trabalho com algumas centenas de bytes. No entanto, vários milhares de bytes é o mínimo mais provável, mesmo em um processador de 8 bits.

Quanto tempo o produto deve permanecer em uso. A vida útil necessária ou esperada afeta todos os tipos de decisões de design, desde a seleção de componentes de hardware até quanto o desenvolvimento e a produção do sistema podem custar. Por quanto tempo o sistema deve continuar funcionando (em média)? Um mês, um ano ou uma década?

Confiabilidade

Quão confiável deve ser o produto final. Se for um brinquedo de criança, pode não ter que funcionar adequadamente 100 por cento do tempo, mas se for um sistema de freio antibloqueio para um carro, com certeza é melhor fazer o que deve fazer todas as vezes.

Além desses requisitos gerais, cada sistema possui requisitos funcionais detalhados. São essas coisas que dão ao sistema embarcado sua identidade única como forno de microondas, marca-passo ou pager.

A Tabela 1-1 ilustra a faixa de valores típicos para cada um dos requisitos de projeto anteriores. Os rótulos “baixo”, “médio” e “alto” destinam-se a fins ilustrativos e não devem ser tomados como delimitações estritas. Um produto real tem uma seleção de cada linha. Em alguns casos, dois ou mais critérios estão vinculados. Por exemplo, aumentos na capacidade de processamento exigida podem levar ao aumento dos custos de produção. Por outro lado, podemos imaginar que o mesmo aumento no poder de processamento teria o efeito de diminuir os custos de desenvolvimento - reduzindo a complexidade do design de hardware e software. Portanto, os valores em uma determinada coluna fazem não necessariamente andam juntos.

Tabela 1-1. Requisitos de design comuns para sistemas embarcados

Critério	Baixo	Médio	Alto
Processador	4 ou 8 bits	16 bits	32 ou 64 bits
Memória	<64 KB	64 KB a 1 MB	> 1 MB
Custo de desenvolvimento	<\$ 100.000	\$ 100.000 a \$ 1.000.000	> \$ 1.000.000
Custo de produção	<\$ 10	\$ 10 a \$ 1.000	> \$ 1.000
Número de unidades	<100	100 a 10.000	> 10.000
Consumo de energia	> 10 mW / MIPS	1 a 10 mW / MIPS	<1 mW / MIPS
Tempo de vida	Dias, semanas ou meses	Anos	Décadas
Confiabilidade	podem falhar ocasionalmente	Deve funcionar de forma confiável	Deve ser à prova de falhas

Exemplos de design incorporado

Para demonstrar a variação nos requisitos de design de um sistema embarcado para o próximo, bem como os possíveis efeitos desses requisitos no hardware, agora vamos levar algum tempo para descrever três sistemas embarcados com alguns detalhes. Nosso objetivo é colocá-lo no lugar do designer de sistema por alguns momentos antes de restringir nossa discussão ao desenvolvimento de software embarcado.

Relógio digital

No pico atual do caminho evolutivo que começou com relógios de sol, relógios de água e ampulhetas está o relógio digital. Entre seus muitos recursos estão a apresentação da data e hora (geralmente até o segundo mais próximo), a medição da duração de um evento até o centésimo de segundo mais próximo e a geração de um pequeno som irritante no início de cada hora. Acontece que essas são tarefas muito simples que não requerem muito poder de processamento ou memória. Na verdade, a única razão para empregar um processador é oferecer suporte a uma variedade de modelos e recursos a partir de um único design de hardware.

O relógio digital típico contém um processador de 4 bits simples e barato. Como os processadores com registradores tão pequenos não podem endereçar muita memória, esse tipo de processador geralmente contém sua própria ROM no chip. E, se houver registros suficientes disponíveis, este aplicativo pode não requerer nenhuma RAM. Na verdade, todos os componentes eletrônicos - processador, memória, contadores e relógios em tempo real - provavelmente serão armazenados em um único chip. Os únicos outros elementos de hardware do relógio são as entradas (botões) e saídas (visor e alto-falante).

O objetivo de um designer de relógio digital é criar um produto razoavelmente confiável com um custo de produção extraordinariamente baixo. Se, após a produção, alguns relógios mantiverem o tempo mais confiável do que a maioria, eles podem ser vendidos sob uma marca com uma marcação mais alta. De resto, ainda pode ser obtido lucro com a venda do relógio através de um canal de vendas com desconto. Para versões de baixo custo, os botões do cronômetro ou alto-falante podem ser eliminados. Isso limitaria a funcionalidade do relógio, mas pode exigir poucas ou mesmo nenhuma alteração do software. E, é claro, o custo de todo esse esforço de desenvolvimento pode ser bastante alto, porque será amortizado ao longo de centenas de milhares ou até milhões de vendas de relógios.

No caso do relógio digital, vemos que o software, especialmente quando cuidadosamente projetado, permite uma enorme flexibilidade em resposta a um mercado altamente competitivo e em rápida mudança.

Player de videogame

Ao retirar o Sony PlayStation 2 do centro de entretenimento, você está se preparando para usar um sistema integrado. Em alguns casos, essas máquinas são mais potentes do que computadores pessoais da mesma geração. Ainda assim, os jogadores de videogame para o mercado doméstico são relativamente baratos em comparação com os computadores pessoais. São os requisitos concorrentes de alto poder de processamento e baixo custo de produção que mantêm os designers de videogame acordados à noite.

As empresas que produzem jogadores de videogame geralmente não se importam com quanto custa desenvolver o sistema, desde que os custos de produção do produto resultante sejam baixos - normalmente em torno de cem dólares. Eles podem até encorajar seus engenheiros a projetar processadores personalizados a um custo de desenvolvimento de milhões de dólares cada.

Portanto, embora possa haver um processador de 64 bits dentro do seu videogame, provavelmente não é o mesmo processador que seria encontrado em um computador de uso geral. Com toda a probabilidade, o processador é altamente especializado para as demandas dos videogames que pretende jogar.

Como o custo de produção é tão crucial no mercado de videogames domésticos, os designers também usam truques para mudar os custos. Por exemplo, uma tática é mover o máximo possível da memória e de outros componentes eletrônicos periféricos da placa de circuito principal para os cartuchos do jogo. * Isso ajuda a reduzir o custo do jogador, mas aumenta o preço de cada jogo. Portanto, embora o sistema possa ter um poderoso processador de 64 bits, ele pode ter apenas alguns megabytes de memória na placa de circuito principal. Isso é apenas memória suficiente para inicializar a máquina a um estado a partir do qual ela pode acessar memória adicional no cartucho do jogo.

Podemos ver, no caso do player de videogame, que em produtos de alto volume, muito esforço de desenvolvimento pode ser direcionado para o ajuste fino de cada aspecto de um produto.

Mars Rover

Em 1976, duas espaçonaves não tripuladas chegaram ao planeta Marte. Como parte de sua missão, eles deveriam coletar amostras da superfície marciana, analisar a composição química de cada uma e transmitir os resultados aos cientistas na Terra. Essas missões Viking foram incríveis. Cercados por computadores pessoais que devem ser reiniciados ocasionalmente, podemos achar notável que, há mais de 30 anos, uma equipe de cientistas e engenheiros construiu com sucesso dois computadores que sobreviveram a uma jornada de 34 milhões de milhas e funcionaram corretamente por meia década. Claramente, a confiabilidade era um dos requisitos mais importantes para esses sistemas.

E se um chip de memória tivesse falhado? Ou o software continha bugs que o fizeram travar? Ou uma conexão elétrica quebrou durante o impacto? Não há como evitar que tais problemas ocorram e, em outras missões espaciais, esses problemas se mostraram desastrosos. Portanto, todos esses pontos de falha em potencial e muitos outros tiveram que ser eliminados adicionando circuitos redundantes ou funcionalidade extra: um processador extra aqui, diagnósticos de memória especial ali, um temporizador de hardware para reinicializar o sistema se o software travar e assim por diante.

Mais recentemente, a NASA lançou a missão Pathfinder. Seu objetivo principal era demonstrar a viabilidade de chegar a Marte com um orçamento. É claro que, dados os avanços em tecnologia feitos desde meados dos anos 70, os designers não precisaram desistir de muito para conseguir isso. Eles podem ter reduzido a quantidade de redundância um pouco, mas ainda assim deram à Pathfinder mais poder de processamento e memória do que a Viking. O Mars Pathfinder era, na verdade, dois sistemas embarcados: uma nave de pouso e um rover. A nave de desembarque tinha um processador de 32 bits e 128 MB de RAM; o rover, por outro

* Por exemplo, a Atari e a Nintendo projetaram alguns de seus sistemas dessa forma.

Por outro lado, tinha apenas um processador de 8 bits e 512 KB de RAM. Essas escolhas refletem os diferentes requisitos funcionais dos dois sistemas. O custo de produção provavelmente não foi um grande problema em ambos os casos; qualquer investimento teria valido a pena uma probabilidade maior de sucesso.

A vida como desenvolvedor de software integrado

Vamos agora dar uma breve olhada em algumas das qualidades do software incorporado que diferenciam os desenvolvedores incorporados de outros tipos de desenvolvedores de software. Um desenvolvedor de software embarcado é aquele que suja as mãos ao se aproximar do hardware.

O desenvolvimento de software embutido, na maioria dos casos, requer interação próxima com o mundo físico - a plataforma de hardware. Dizemos “na maioria dos casos” porque existem sistemas embarcados muito grandes que exigem que os indivíduos trabalhem exclusivamente no software de camada de aplicativo do sistema. Esses desenvolvedores de aplicativos normalmente não interagem com o hardware. Quando projetados corretamente, os drivers de dispositivo de hardware são abstraídos do hardware real para que um desenvolvedor que esteja escrevendo um software no nível do aplicativo não saiba como uma string chega ao display, apenas que isso acontece quando uma determinada rotina é chamada com o parâmetros adequados.

Conhecimento de hardware

O desenvolvedor de software embarcado deve estar intimamente familiarizado com os circuitos integrados, as placas e barramentos e os dispositivos anexados usados para escrever software embarcado sólido (também chamado *firmware*). Os desenvolvedores integrados não devem ter medo de mergulhar nos esquemas, pegar uma ponta de prova do osciloscópio e começar a vasculhar o circuito para descobrir o que está acontecendo.

Código eficiente

Como os sistemas embarcados são normalmente projetados com o processador menos poderoso e mais econômico que atende aos requisitos de desempenho do sistema, os desenvolvedores de software embarcado devem fazer com que cada linha de código seja importante. A capacidade de escrever código eficiente é uma grande qualidade para se possuir como desenvolvedor de firmware.

Interfaces periféricas

No nível mais baixo, o firmware é muito especializado, pois cada componente ou circuito tem sua própria atividade para realizar e, além disso, sua própria maneira de realizar essa atividade. Os desenvolvedores incorporados precisam saber como se comunicar com os diferentes dispositivos ou *periféricos* para ter controle total dos dispositivos no sistema. Reagir a estímulos de periféricos externos é uma grande parte do desenvolvimento de software embarcado.

Por exemplo, em um forno de micro-ondas, o firmware pode obter os dados de um sensor de temperatura lendo um registro de 8 bits em um conversor analógico-digital externo; em outro sistema, os dados podem ser extraídos controlando um barramento serial que faz interface com o circuito do sensor externo por meio de um único fio.

Código robusto

Há expectativas de que os sistemas embarcados funcionem por anos na maioria dos casos. Este não é um requisito típico para aplicativos de software escritos para um PC ou Mac. Agora, existem exceções. No entanto, se você tivesse que desligar continuamente o micro-ondas para que ele esquentasse o seu almoço pelo tempo adequado, provavelmente seria a última vez que você comprou um produto daquela empresa.

Recursos mínimos

Na mesma linha de criação de um sistema mais robusto, outro grande diferencial entre o software embarcado e outros tipos de software são as restrições de recursos. As regras para escrever firmware são diferentes das regras para escrever software para um PC. Considere a alocação de memória, por exemplo. Um aplicativo para um PC moderno pode presumir que terá acesso a recursos praticamente ilimitados. Mas, em um sistema embarcado, você ficará sem memória se não planejar com antecedência e projetar o software de maneira adequada.

Um desenvolvedor de software embarcado deve gerenciar de perto os recursos, desde a memória até a capacidade de processamento, para que o sistema opere de acordo com as especificações e não ocorram falhas. Por exemplo, o uso de funções de alocação de memória dinâmica padrão pode causar fragmentação e, eventualmente, o sistema pode parar de operar. Isso requer uma reinicialização, pois você não tem onde armazenar os dados recebidos.

Frequentemente, em software embarcado, um desenvolvedor aloca toda a memória necessária ao sistema no momento da inicialização. Isso é mais seguro do que usar a alocação de memória dinâmica, embora nem sempre possa ser feito.

Software reutilizável

Como mencionamos antes, *portabilidade de código* ou *reutilização de código* — Escrever software para que ele possa ser movido de plataforma de hardware para plataforma de hardware — é muito útil para ajudar na transição para novos projetos. Isso nem sempre pode ser feito; vimos como cada sistema embarcado é individual. Ao longo deste livro, examinaremos métodos básicos para garantir que seu código incorporado possa ser movido mais facilmente de um projeto para outro. Portanto, se o seu próximo projeto usar um LCD para o qual você já desenvolveu um driver, você pode inserir o código antigo e economizar um tempo precioso na programação.

Ferramentas de desenvolvimento

As ferramentas que você usará ao longo de sua carreira como desenvolvedor embarcado variam de empresa para empresa e, frequentemente, de projeto para projeto. Isso significa que você precisará aprender novas ferramentas à medida que avança em sua carreira. Normalmente, essas ferramentas não são tão poderosas ou fáceis de usar quanto as usadas no desenvolvimento de software para PC.

As ferramentas de depuração que você pode encontrar podem variar de um simples LED a um completo *emulador no circuito* (GELO). Isso requer que você, como desenvolvedor de firmware e responsável por depurar seu código, seja muito engenhoso e tenha um conjunto de técnicas que possa recorrer quando o ambiente de depuração estiver faltando. Ao longo do livro, apresentaremos diferentes “ferramentas de software de baixo nível” que você pode implementar com pouco impacto no design de hardware.

Essas são apenas algumas qualidades que separam os desenvolvedores de software embarcado do resto do pacote. Investigaremos essas e outras técnicas específicas para o desenvolvimento de software embarcado à medida que continuarmos.

A linguagem C: o menor denominador comum

Uma das poucas constantes na maioria dos sistemas embarcados é o uso da linguagem de programação C. Mais do que qualquer outra, C se tornou a linguagem de programadores embarcados. Nem sempre foi assim e não continuará a ser para sempre. No entanto, neste momento, C é a coisa mais próxima que existe de um padrão no mundo incorporado. Nesta seção, explicaremos por que C se tornou tão popular e por que o escolhemos como o idioma principal deste livro.

Como o desenvolvimento de software bem-sucedido freqüentemente depende da seleção da melhor linguagem para um determinado projeto, é surpreendente descobrir que uma linguagem provou ser apropriada para processadores de 8 e 64 bits; em sistemas com bytes, kilobytes e megabytes de memória; e para equipes de desenvolvimento que variam de uma a doze ou mais pessoas. No entanto, esta é precisamente a gama de projetos em que C prosperou.

A linguagem de programação C tem muitas vantagens. É pequeno e bastante simples de aprender, os compiladores estão disponíveis para quase todos os processadores em uso hoje e há um corpo muito grande de programadores C experientes. Além disso, C tem o benefício de independência de processador, o que permite que os programadores se concentrem em algoritmos e aplicativos, em vez de nos detalhes de uma arquitetura de processador particular. No entanto, muitas dessas vantagens se aplicam igualmente a outras linguagens de alto nível. Então, por que C teve sucesso onde tantas outras linguagens falharam amplamente?

Talvez o maior ponto forte do C - e o que o diferencia de outras linguagens como Pascal e FORTRAN - seja o fato de ser uma linguagem de alto nível de “baixo nível”. Como veremos ao longo do livro, C oferece aos programadores embarcados um grau extraordinário de controle direto do hardware sem sacrificar os benefícios das linguagens de alto nível. A natureza de “baixo nível” do C foi uma intenção clara dos criadores da linguagem. Na verdade, Brian W. Kernighan e Dennis M. Ritchie incluíram o seguinte comentário nas páginas iniciais de seu livro *A linguagem de programação C* (Prentice Hall):

C é uma linguagem relativamente de “nível baixo”. Essa caracterização não é pejorativa; significa simplesmente que C lida com o mesmo tipo de objetos que a maioria dos computadores. Eles podem ser combinados e movidos com os operadores aritméticos e lógicos implementados por máquinas reais.

Poucas linguagens de alto nível populares podem competir com C na produção de código compacto e eficiente para quase todos os processadores. E, desses, apenas C permite que os programadores interajam com o hardware subjacente com tanta facilidade.

Outras linguagens incorporadas

Claro, C não é a única linguagem usada por programadores embarcados. Pelo menos quatro outras linguagens - assembly, C ++, Forth e Ada - valem a pena mencionar com mais detalhes.

No início, o software embarcado era escrito exclusivamente na linguagem assembly do processador de destino. Isso deu aos programadores o controle total do processador e de outro hardware, mas a um preço. As linguagens assembly têm muitas desvantagens, sendo a menor delas os custos mais altos de desenvolvimento de software e a falta de portabilidade do código. Além disso, encontrar programadores de montagem qualificados tornou-se muito mais difícil nos últimos anos. O assembly agora é usado principalmente como um complemento para a linguagem de alto nível, geralmente apenas para código de sistema de inicialização ou aqueles pequenos pedaços de código que devem ser extremamente eficientes ou ultracompactos, ou não podem ser escritos de outra maneira.

Forth é eficiente, mas de nível extremamente baixo e incomum; aprender a fazer o trabalho leva mais tempo do que com C.

C ++ é um superconjunto orientado a objetos de C que está cada vez mais popular entre os programadores embarcados. Todos os recursos básicos da linguagem são iguais ao C, mas C ++ adiciona novas funcionalidades para melhor abstração de dados e um estilo de programação mais orientado a objetos. Esses novos recursos são muito úteis para desenvolvedores de software, mas alguns deles reduzem a eficiência do programa executável. Portanto, C ++ tende a ser mais popular com grandes equipes de desenvolvimento, onde os benefícios para os desenvolvedores superam a perda de eficiência do programa.

Ada também é uma linguagem orientada a objetos, embora substancialmente diferente de C ++. Ada foi originalmente projetado pelo Departamento de Defesa dos Estados Unidos para o desenvolvimento de software militar de missão crítica. Apesar de ser duas vezes aceito como um padrão internacional (Ada83 e Ada95), ele não ganhou muito espaço fora das indústrias de defesa e aeroespacial. E aí vem perdendo terreno nos últimos anos. Isso é lamentável porque a linguagem Ada tem muitos recursos que simplificariam o desenvolvimento de software embarcado se usada em vez de C ou C ++.

Escolhendo um idioma para o livro

Uma das principais questões que os autores de um livro como este enfrentam é que linguagem de programação ou linguagens discutir. Tentar cobrir muitos idiomas pode confundir o leitor ou prejudicar pontos mais importantes. Por outro lado, o foco muito estreito pode tornar a discussão desnecessariamente acadêmica ou (pior para os autores e a editora) limitar o mercado potencial para o livro.

Certamente, C deve ser a peça central de qualquer livro sobre programação embarcada, e este livro não é exceção. Todo o código de amostra é escrito em C, e a discussão se concentrará em problemas de programação relacionados a C. Claro, tudo o que é dito

sobre a programação C aplica-se igualmente ao C ++. Usaremos a linguagem assembly apenas quando uma tarefa de programação específica não puder ser realizada de outra maneira.

Achamos que esse foco em C com uma breve introdução à montagem reflete com mais precisão a maneira como o software embarcado é realmente desenvolvido hoje e como continuará a ser desenvolvido no curto prazo. É por isso que os exemplos nesta edição não usam C ++. Esperamos que essa escolha mantenha a discussão clara, forneça informações úteis para as pessoas que estão desenvolvendo sistemas reais e inclua o maior público possível. No entanto, cobrimos o impacto do C ++ no software embarcado no Capítulo 14.

Práticas de codificação consistentes

Qualquer que seja o idioma selecionado para um determinado projeto, é importante instituir algumas diretrizes básicas de codificação ou estilos a serem seguidos por todos os desenvolvedores em um projeto. As diretrizes de codificação podem facilitar a leitura do código, tanto para você quanto para o próximo desenvolvedor que terá que herdar seu código. Entender exatamente o que uma determinada rotina de software está fazendo é difícil o suficiente sem ter que lutar contra várias mudanças no estilo de codificação que surgiram porque vários desenvolvedores diferentes tocaram a mesma rotina ao longo dos anos, cada um deixando sua própria marca exclusiva. Questões estilísticas, como como as variáveis são nomeadas ou onde a chave deve residir, podem ser muito pessoais para alguns desenvolvedores.

Existem vários padrões de codificação decentes circulando na Internet. Um padrão de que gostamos está localizado online em <http://www.ganssle.com> e foi desenvolvido por Jack Ganssle. Outro de que gostamos, de Miro Samek, está localizado online em <http://www.quantum-leaps.com>.

Esses padrões fornecem orientações sobre tudo, desde estruturas de diretório a nomes de variáveis e são um ótimo ponto de partida; você pode incorporar a eles os estilos que achar necessários e úteis. Se um padrão de codificação para toda a equipe não for algo que você possa vender para sua empresa, use um você mesmo e cumpra-o.

Algumas palavras sobre hardware

É da natureza da programação que os livros sobre o assunto incluam exemplos. Normalmente, esses exemplos são selecionados para que os leitores interessados possam experimentá-los facilmente. Isso significa que os leitores devem ter acesso às mesmas ferramentas de desenvolvimento de software e plataformas de hardware usadas pelos autores. Infelizmente, não faz sentido executar nenhum dos programas de exemplo nas plataformas disponíveis para a maioria dos leitores - PCs, Macs e estações de trabalho Unix.

Inteiros FixedWidth: às vezes o tamanho é importante

Os programadores de computador nem sempre se importam com a largura de um inteiro quando segurado pelo processador. Por exemplo, quando escrevemos:

```
int i;  
  
para (i = 0; i < N; i++) {  
  
    ...  
}
```

geralmente esperamos que nosso compilador gere o código mais eficiente possível, seja isso torna o contador de loop uma quantidade de 8, 16, 32 ou mesmo 64 bits.

Desde que o número inteiro seja largo o suficiente para conter o valor máximo (N, no exemplo mostrado acima), queremos que o processador seja usado da maneira mais eficiente. E é exatamente isso que os padrões ISO C e C++ dizem ao redator do compilador para fazer: escolher o tamanho inteiro mais eficiente que atenderá à solicitação específica. Por causa do tamanho variável dos inteiros em diferentes processadores e a flexibilidade correspondente dos padrões de linguagem, o código anterior pode resultar em um inteiro de 32 bits com um compilador, mas um inteiro de 16 bits com outro - possivelmente mesmo quando o mesmo processador é o alvo.

Mas em muitas outras situações de programação, o tamanho do inteiro é importante. A programação incorporada, em particular, muitas vezes envolve uma manipulação considerável de dados inteiros de larguras fixas.

Em retrospecto, com certeza teria sido bom se os autores do padrão C tivessem definido alguns nomes padrão e tornassem os provedores de compilador responsáveis por fornecer o typedef apropriado para cada tipo de inteiro de tamanho fixo em um arquivo de cabeçalho de biblioteca. Alternativamente, o padrão C poderia ter especificado que cada um dos tipos curto, int, e grande tem largura padrão em todas as plataformas; mas isso pode ter um impacto no desempenho, particularmente em processadores de 8 bits que devem implementar adições de 16 e 32 bits em sequências de várias instruções.

Curiosamente, verifica-se a atualização de 1999 do padrão C da Organização Internacional de Padronização (ISO) (também conhecido como *C99*) fez exatamente isso. A ISO finalmente colocou o peso de seu padrão por trás de um conjunto preferido de nomes para tipos de dados inteiros de tamanho fixo assinados e não assinados. Os nomes de tipo recém-definidos são:

```
8 bits: int8_t, uint8_t  
16 bits: int16_t, uint16_t  
32 bits: int32_t, uint32_t  
64 bits: int64_t, uint64_t
```

De acordo com o padrão atualizado, este conjunto obrigatório de typedefs (junto com alguns outros) deve ser definido pelos fornecedores do compilador e incluído no novo arquivo de cabeçalho *stdint.h*.

- *contínuo* -

Se você já estiver usando um compilador compatível com C99, esse novo recurso de linguagem torna a declaração de uma variável inteira de largura fixa ou um registro tão simples quanto usar um dos novos nomes de tipo.

Mesmo se você não tiver um compilador atualizado, a inclusão desses nomes no padrão C99 sugere que é hora de atualizar seus padrões e práticas de codificação. Ame-os ou odeie-os, pelo menos esses novos nomes fazem parte de um padrão aceito internacionalmente. Como resultado direto, será muito mais fácil no futuro portar programas C que requerem inteiros de largura fixa para outros compiladores e plataformas de destino. Além disso, os módulos que são reutilizados ou vendidos com o código-fonte podem ser mais facilmente compreendidos quando estão em conformidade com as convenções de nomenclatura e digitação padrão, como as do C99.

Se você ainda não tem um compilador compatível com C99, você terá que escrever seu próprio conjunto de `typedefs`, usando o conhecimento específico do compilador do `char`, `short`, e grande larguras primitivas.

Para os exemplos neste livro, usamos o estilo C99 para tipos de variáveis que requerem larguras específicas. Nós geramos nosso próprio `stdint.h` que é específico para o `gcc` variante visando o processador ARMXScale. Nosso arquivo pode não funcionar em outros ambientes de construção.

Até mesmo selecionar uma plataforma embarcada padrão é difícil. Como você já aprendeu, não existe um sistema embarcado “típico”. Qualquer que seja o hardware selecionado, a maioria dos leitores não terá acesso a ele. Mas, apesar desse problema bastante significativo, sentimos que é importante selecionar uma plataforma de hardware de referência para uso nos exemplos. Ao fazer isso, esperamos tornar os exemplos consistentes e, assim, toda a discussão mais clara - quer você tenha o hardware escolhido na sua frente ou não.

Ao escolher uma plataforma de exemplo, nosso primeiro critério foi que a plataforma deveria ter uma combinação de periféricos para suportar vários exemplos no livro. Além disso, buscamos uma plataforma que permitisse aos leitores continuar seus estudos de desenvolvimento de software embarcado, expandindo nossos exemplos com projetos mais avançados. Outro critério era encontrar uma placa de desenvolvimento que oferecesse suporte às ferramentas de desenvolvimento de software GNU; com seu licenciamento de código aberto e cobertura em uma ampla variedade de processadores embutidos, as ferramentas de desenvolvimento GNU foram a escolha ideal.

O hardware escolhido consiste em um processador de 32 bits (o XScale ARM), * uma grande quantidade de memória (64 MB de RAM e 16 MB de ROM) e alguns tipos comuns de entradas, saídas e componentes periféricos. A placa que escolhemos chama-se VIPER-Lite e é fabricada e vendida pela Arcom. Uma imagem da placa de desenvolvimento Arcom VIPERLite (junto com o módulo adicional e outro hardware de suporte) é mostrada na Figura 1-4. Informações adicionais sobre a placa Arcom e instruções para obtê-la podem ser encontradas no Apêndice A.

* O processador na placa VIPER-Lite é o processador PXA255 XScale, que é baseado na arquitetura ARM v.5TE. O processador XScale foi desenvolvido por uma divisão de sistemas embarcados da Intel Corporation que foi vendida ao Marvell Technology Group em julho de 2006.

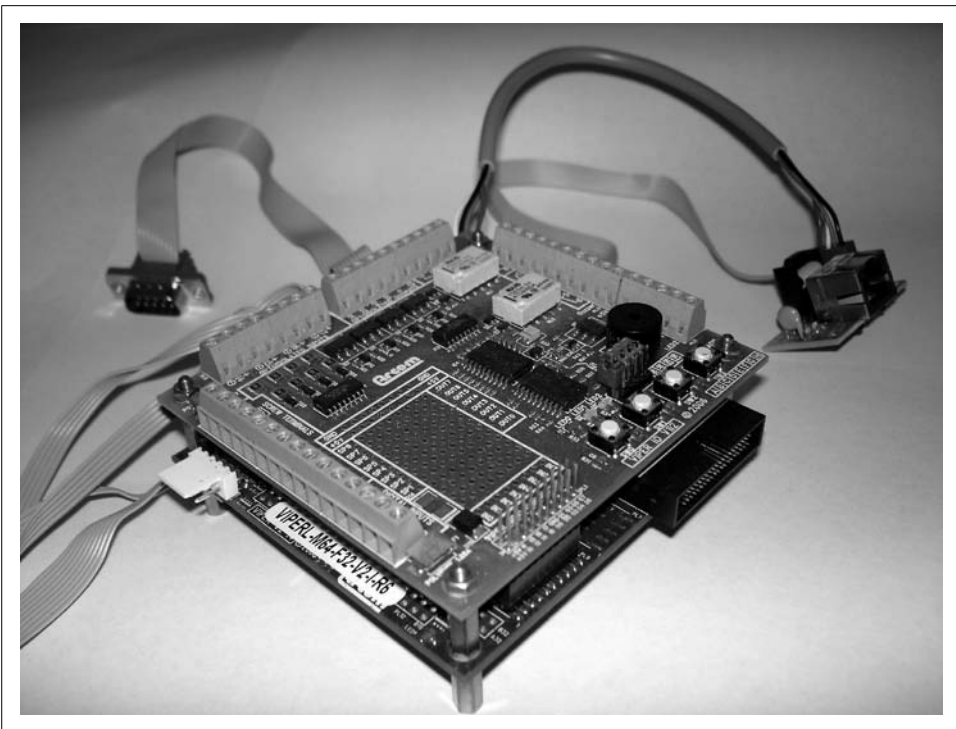


Figura 1-4. As placas de desenvolvimento Arcom VIPER-Lite

Se você tiver acesso ao hardware de referência, poderá trabalhar com os exemplos do livro à medida que são apresentados. Caso contrário, você precisará portar o código de exemplo para uma plataforma incorporada à qual você tenha acesso. Para esse fim, fizemos todos os esforços para tornar os programas de exemplo o mais portáteis possível. No entanto, o leitor deve ter em mente que o hardware é diferente em cada sistema embarcado e que alguns dos exemplos podem não fazer sentido em um hardware diferente do que escolhemos aqui. Por exemplo, não faria sentido portar nosso driver de memória flash para uma placa que não tivesse dispositivos de memória flash.

Embora entremos em alguns detalhes básicos sobre hardware, o foco principal deste livro é o software embarcado. Recomendamos que você dê uma olhada em *Projetando Sistemas Embarcados* por John Catsoulis (O'Reilly). John tem uma vasta experiência no assunto e faz um trabalho maravilhoso apresentando materiais muitas vezes difíceis de uma forma muito compreensível. É um ótimo companheiro para este livro.

Conhecendo o Hardware

duro · porcelana *n.* A parte de um sistema de computador que pode ser chutado.

Como engenheiro de software embarcado, você terá a oportunidade (e o desafio) de trabalhar com muitas peças de hardware diferentes em sua carreira. Neste capítulo, começaremos dando uma olhada nos fundamentos para a compreensão de um esquema. Também ensinaremos a você um procedimento simples que usamos para nos familiarizar com qualquer nova placa. No processo, o guiaremos pela criação de um arquivo de cabeçalho em linguagem C que descreve os recursos mais importantes da placa e um software que inicializa o hardware para um estado conhecido.

Compreendendo o quadro geral

Antes de escrever software para um sistema embarcado, você deve primeiro estar familiarizado com o hardware no qual ele será executado. Em primeiro lugar, você só precisa entender o funcionamento geral do sistema, como a função principal da placa e quais são as entradas e saídas. Inicialmente, você não precisa entender cada pequeno detalhe do hardware - como cada componente ou periférico opera e quais registros precisam ser programados para funções específicas.

Sempre que receber uma nova placa, você deve reservar um tempo para ler os principais documentos fornecidos com ela. Se a placa for um produto de prateleira, ela pode vir com um “Guia do usuário” ou “Manual do programador” que foi escrito para desenvolvedores de software. (O kit de desenvolvimento da Arcom, por exemplo, inclui essas informações, bem como fichas técnicas para todos os principais componentes da placa.) No entanto, se a placa foi projetada de forma personalizada para o seu projeto, a documentação pode ser mais enigmática ou pode ter sido escrita principalmente para a referência dos designers de hardware. De qualquer forma, este é o melhor lugar para começar.

Enquanto você lê a documentação, coloque a placa de lado. Isso ajudará você a se concentrar no quadro geral. Haverá muito tempo para examinar o real

embarque mais de perto quando terminar de ler. Antes de pegar o quadro, você deve ser capaz de responder a duas perguntas básicas sobre ele:

- Qual é o propósito geral do conselho administrativo?
- Como os dados fluem por ele?

Por exemplo, imagine que você seja um desenvolvedor de software em uma equipe de design que está construindo um servidor de impressão. Você acaba de receber um protótipo de placa dos designers de hardware. O objetivo da placa é compartilhar uma impressora entre vários computadores. O hardware lê os dados de uma conexão de rede e os envia a uma impressora para saída. O servidor de impressão deve mediar entre os computadores e decidir qual computador da rede enviará dados para a impressora. As informações de status também fluem na direção oposta aos computadores da rede.

Embora a finalidade da maioria dos sistemas seja autoexplicativa, o fluxo dos dados pode não ser. Frequentemente achamos que um diagrama de blocos é útil para alcançar uma compreensão rápida. Se você tiver sorte, a documentação fornecida com seu hardware conterá um diagrama de blocos. No entanto, você também pode achar útil criar seu próprio diagrama de blocos. Dessa forma, você pode deixar de fora os componentes de hardware que não estão relacionados ao fluxo básico de dados através do sistema.

No caso da placa Arcom, o hardware foi projetado para fins de demonstração e não com um aplicativo específico em mente. No entanto, vamos imaginar que tem um propósito. O usuário do dispositivo conecta os computadores à porta Ethernet e uma impressora à porta paralela. Qualquer computador na rede pode enviar documentos para a impressora, embora apenas um deles possa fazer isso por vez.

O diagrama na Figura 2-1 ilustra o fluxo de dados por meio do servidor de impressão. (Apenas os dispositivos de hardware envolvidos nesta aplicação da placa Arcom são mostrados.) Olhando o diagrama de blocos, você deve ser capaz de visualizar rapidamente o fluxo dos dados através do sistema. Os dados a serem impressos são aceitos do controlador Ethernet, mantidos na RAM até que a impressora esteja pronta para mais dados e entregues à impressora por meio da porta paralela. As informações de status são enviadas de volta para os vários computadores que solicitam saída na impressora. O software que faz tudo isso acontecer está armazenado na ROM. Observe que o barramento PC / 104 inclui sinais de buffer dos barramentos de endereço e dados, além de outros sinais.

Para ter uma ideia melhor de como o diagrama de blocos se relaciona ao hardware real na placa Arcom para nosso dispositivo de servidor de impressão, examine a Figura 2-2, que mostra o diagrama sobreposto na parte superior da placa Arcom. Esta figura dá uma ideia melhor dos ICs envolvidos no dispositivo do servidor de impressão e como os dados são roteados pelo hardware real.

Recomendamos a criação de um caderno de projeto ou fichário. Depois de criar um diagrama de blocos, coloque-o como a primeira página do caderno do projeto. Você precisa dele à mão para que possa consultá-lo ao longo do projeto. À medida que você continua trabalhando com esta peça de hardware, anote tudo o que aprender sobre ela em seu caderno. Se você conseguir um

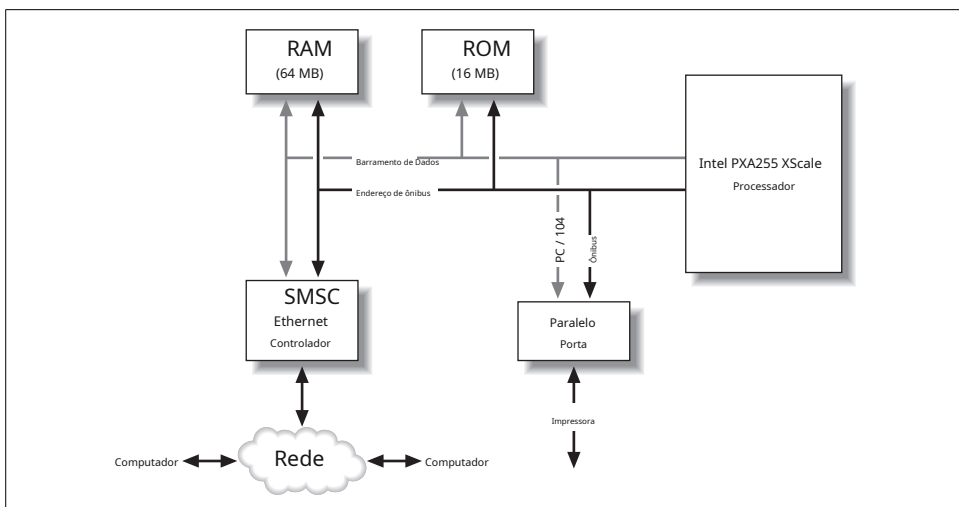


Figura 2-1. Diagrama de blocos para o servidor de impressão

apostila útil em uma reunião, coloque-a em seu caderno. Coloque guias lá para que você possa pular rapidamente para informações importantes que você consulta o tempo todo. Você também pode fazer anotações sobre o design e a implementação do software. É muito útil consultar suas anotações para refrescar a memória sobre o motivo de uma decisão específica ter sido tomada para o software. Um caderno de projeto é valioso não apenas enquanto você está desenvolvendo o software, mas também quando o projeto é concluído. Você apreciará o esforço extra que coloca para manter um notebook no caso de precisar fazer alterações em seu software ou trabalhar com hardware semelhante, meses ou anos depois.

Se você ainda tiver dúvidas gerais após ler os documentos de hardware, peça ajuda a um engenheiro de hardware. Se você ainda não conhece o designer do hardware, reserve alguns minutos para se apresentar. Se tiver algum tempo, leve-o para almoçar ou compre-lhe uma cerveja depois do trabalho. (Você nem precisa falar sobre o projeto o tempo todo!) Descobrimos que muitos engenheiros de software têm dificuldade em se comunicar com engenheiros de hardware e vice-versa. No desenvolvimento de sistemas embarcados, é especialmente importante que as equipes de hardware e software possam se comunicar umas com as outras. Este capítulo lhe dará uma base sólida para que você possa falar a linguagem do engenheiro de hardware.

Antes de prosseguirmos, vamos fazer um breve desvio para entender melhor os conceitos básicos de hardware e esquemas.

Noções básicas de hardware

A próxima etapa para entender o hardware é dar uma olhada no *esquemático*. Um esquema é um desenho composto de símbolos padronizados para representar todos os componentes e conexões de um circuito. O esquema fornece os detalhes do hardware,

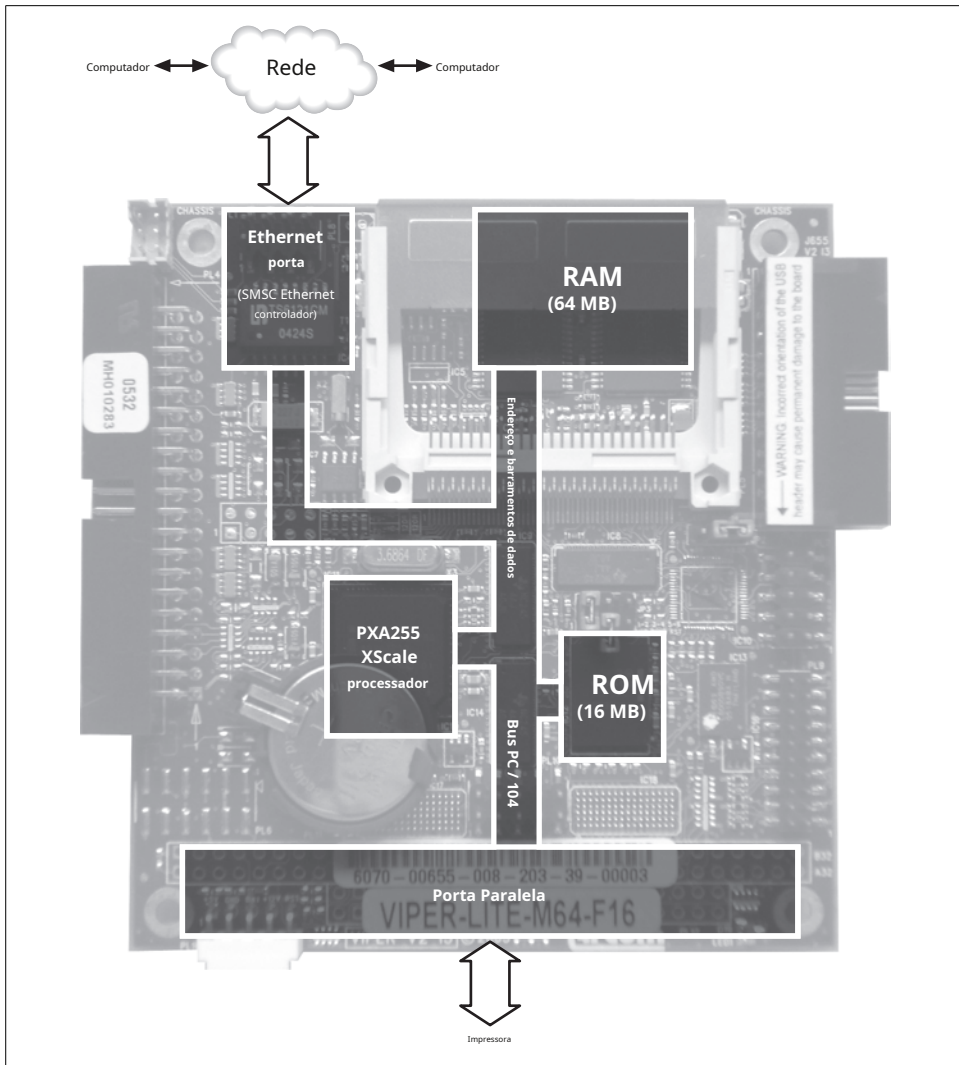


Figura 2-2. Diagrama de blocos para o servidor de impressão na placa Arcom

mostrando os componentes individuais representados no diagrama de blocos, como os componentes estão interconectados e, o mais importante, onde colocar a ponta de prova do osciloscópio para ver o que está acontecendo com um circuito específico. Na maioria dos projetos, não é necessário entender como cada circuito elétrico da placa funciona, mas você precisa entender a operação básica do hardware.

Junto com os guias do usuário ou manuais para a placa, também é útil coletar o *folhas de dados* para todos os principais componentes da placa. A folha de dados é uma especificação completa de um determinado componente de hardware, incluindo parâmetros elétricos, de temporização e de interface.

Freqüentemente, o engenheiro de hardware já coletou as planilhas de dados; em caso afirmativo, seu trabalho está parcialmente concluído. Você pode querer dar uma olhada nas outras informações disponíveis para um determinado componente, porque geralmente existem documentos separados de hardware e software, especialmente para dispositivos mais complexos. Por exemplo, um processador geralmente inclui um Guia do Programador, além de outras literaturas. Esses documentos podem fornecer informações valiosas para o uso de vários recursos do processador; Ocasionalmente, eles até fornecem trechos de código.

Existem também notas de aplicação que tratam de problemas específicos associados a um componente específico. É uma boa ideia procurar qualquer *errata* documentos para todos os dispositivos. A errata do dispositivo lhe dará um alerta sobre quaisquer problemas relacionados à forma como o dispositivo opera e, mais importante, soluções alternativas para esses problemas.

É uma boa ideia verificar periodicamente se há atualizações das informações dos componentes da placa. Isso o evitará da frustração de perseguir um problema que foi corrigido na folha de dados mais recente. Todas essas informações são uma vantagem quando você está tentando entender uma nova peça de hardware. Geralmente, você pode encontrar esses documentos no site do fabricante.

Fundamentos esquemáticos

Antes de dar uma olhada em um esquema, vamos examinar alguns dos fundamentos do esquema. A Figura 2-3 mostra alguns dos símbolos esquemáticos básicos que você encontrará, embora possa haver algumas variações nos símbolos de esquemático para esquemático. A primeira coluna fornece o nome do componente específico; a segunda coluna mostra o prefixo do designador de referência ou o nome do componente; e a terceira coluna mostra os símbolos do componente relacionado.

Você pode notar que dois símbolos são mostrados para o componente de diodo. O símbolo à direita é para um diodo emissor de luz (LED), que veremos em breve.

Os símbolos para aterramento e poder também podem variar de esquemático para esquemático; dois símbolos para alimentação e aterramento estão incluídos na Figura 2-3. Além do VCC, o designador de referência comumente usado para energia é VDD. Como muitos circuitos usam vários níveis de tensão, você também pode encontrar símbolos de energia que são rotulados com a tensão real, como +5 ou +3,3, em vez de VCC ou VDD. Os símbolos de energia e aterramento são normalmente colocados verticalmente, conforme mostrado, enquanto os outros símbolos na Figura 2-3 podem aparecer em qualquer orientação.

UMA *designador de referência* é uma combinação de letras, números ou ambos, que são usados para identificar componentes em um esquema. Os designadores de referência normalmente incluem um número para ajudar na identificação de uma parte específica no esquema. Por exemplo, três resistores em um esquema podem ter designadores de referência R4, R21 e R428. Os designadores de referência são normalmente serigrafados (uma sobreposição pintada) na placa de circuito para identificação da peça.

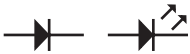
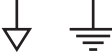
Componente	Referência Prefixo do Designador	Símbolo
Resistor	R	
Capacitor	C	
Diodo	D	
Cristal	X	
Indutor	eu	
Poder	VCC	
Chão	GND	

Figura 2-3. Símbolos esquemáticos básicos

Junto com os designadores de referência, certos componentes (como capacitores, indutores e resistores) são marcados por seus valores associados. Por exemplo, na Figura 2-4, o resistor R39 tem um valor de 680 Ω .



Os valores de alguns componentes em um esquema são escritos de forma a auxiliar no esclarecimento. Por exemplo, um resistor com um valor de 4,7 k Ω tem seu valor escrito como 4K7. Um resistor com um valor de 12,4 Ω é escrito como 12R4. Usando esse método, é mais fácil entender o valor do componente caso o decimal não seja impresso corretamente.

Você também notará que os símbolos de circuito integrado não estão incluídos nesta figura. Isso ocorre porque as representações esquemáticas de IC variam amplamente. Normalmente, um engenheiro de hardware precisa criar seu próprio símbolo esquemático para os ICs usados no projeto. Cabe então ao engenheiro de hardware usar o método mais claro possível para capturar e representar o símbolo do IC.

O designador de referência para ICs também pode variar. Normalmente, a letra U é usada. O esquema da placa Arcom, no entanto, usa o designador de referência IC.

Os símbolos IC também incluem um *tipo de componente* ou número de peça usado pelo fabricante. O tipo de componente costuma ser útil quando você precisa procurar as planilhas de dados das partes de um projeto específico. O texto descritivo pode poupar o trabalho de decifrar as marcações e códigos na parte superior de um IC específico.

Agora que temos uma introdução aos símbolos usados em um esquema, vamos dar uma olhada em um esquema. A Figura 2-4 é um esquema parcial de um tabuleiro fictício. Nesta figura, mostramos algumas das conexões no processador.

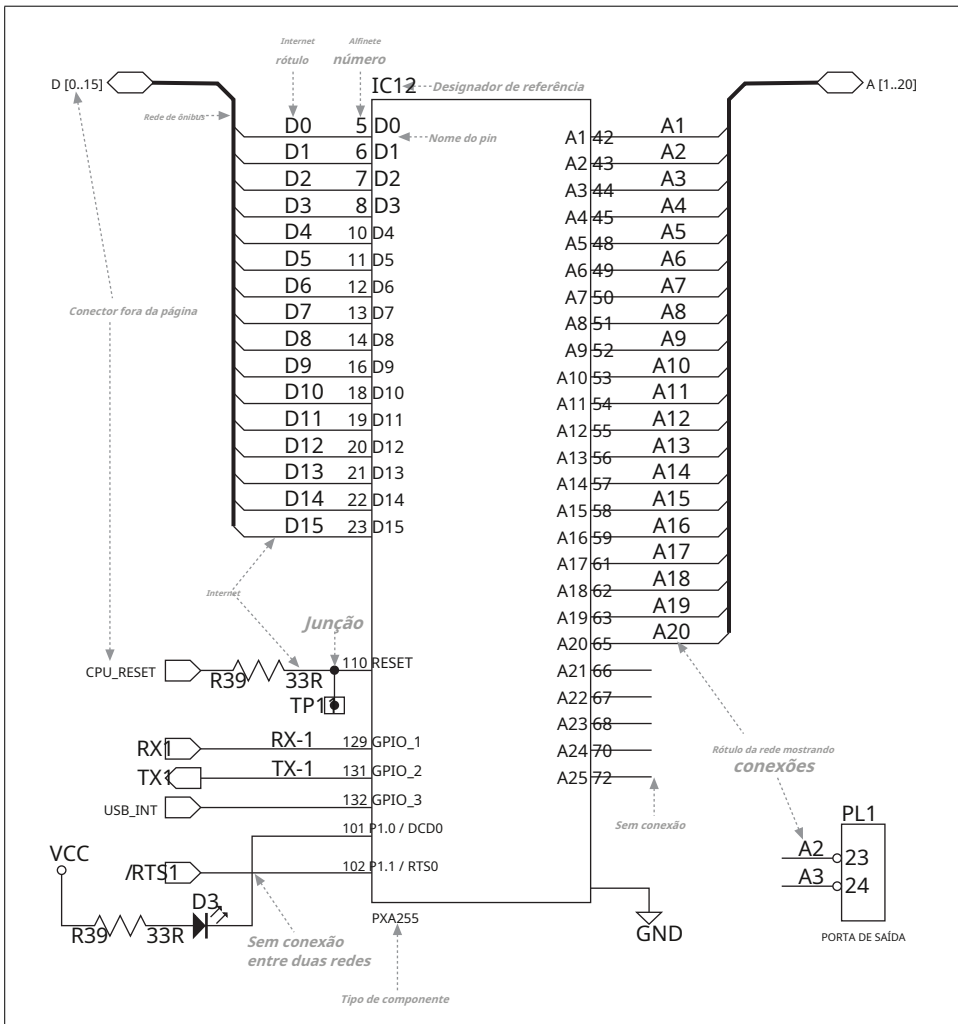


Figura 2-4. Esquema de exemplo



Os rótulos em itálico e setas associadas não fazem parte do esquema original. Estes são incluídos para apontar aspectos particulares do esquema. Queríamos observar isso porque muitas vezes as notas de texto são incluídas no esquema para esclarecimento ou questões de layout. Uma dessas notas de esclarecimento na Figura 2-4 é a etiqueta OUTPUT PORT na porta PL1.

O processador é o principal componente neste esquema. O símbolo para o processador tem um designador de referência IC12, que está localizado na parte superior do símbolo neste esquema. O tipo de componente do processador é PXA255 e está localizado na parte inferior do símbolo.

Os pinos do processador e seus associados *números de pinos* correr ao longo dos lados do símbolo. Por exemplo, o bit 0 do barramento de dados é denominado D0 e está localizado no pino número 5 do processador.

Você também notará que alguns pinos, como P1.1 / $\overline{\text{RTS0}}$ pino número 102, têm uma barra sobre o *nome do pino*. Isso indica que o sinal é *baixo ativo*. Isso significa que um nível lógico de 0 ativará a funcionalidade desse sinal, enquanto um nível lógico de 1 desativará a função. O tipo oposto de operação é *alto ativo*.

A baixa funcionalidade ativa também pode ser indicada por uma barra (/) ou um til (~) colocado antes ou depois do nome do sinal. O sinal é, então, normalmente pronunciado "não RTS0" ou "barra RTS0". Alguns fabricantes de componentes representam um sinal ativo baixo com o prefixo "n" na frente do nome do sinal, como nRESET.

O fio que conecta os diferentes componentes é chamado de *líquido*. Duas redes que se conectam criam um *junção*. No esquema, um ponto de junção é indicado por um ponto, como você pode ver na Figura 2-4 no pino RESET do processador. Em contraste, quando duas redes se cruzam, mas não estão conectadas, não há junção. Isso é mostrado onde a rede conectada ao LED D3 cruza a rede / RTS1.

Dizemos que os pinos não conectados a nenhuma rede são *não conecta* ou *abrir*. Frequentemente, nenhuma conexão é representada em um esquema com uma pequena cruz no final da rede. Exemplos de pinos sem conexão são mostrados nos pinos do processador A21 a A25. Às vezes, os fabricantes de IC não rotulam nenhum pino de conexão como NC.

Sinais relacionados, como sinais de dados ou sinais de endereço, são representados em um esquema por uma linha mais espessa chamada de *rede de ônibus*. Por exemplo, o barramento de dados é rotulado como D [0..15] (em outros esquemas o barramento de dados pode ser rotulado como [D0: D15]), o que significa que a rede do barramento de dados é composta pelos sinais D0 a D15. Os sinais individuais são mostrados conectando-se aos pinos de dados do processador. Da mesma forma, a rede de barramento de endereço é identificada como A [1..20] e é composta pelos sinais A1 a A20. As redes conectadas a uma rede de barramento ainda precisam ser etiquetadas individualmente.

Se cada rede em um esquema estivesse conectada ao local desejado, depois de um tempo, ela poderia criar um ninho de rato e tanto. * Normalmente, evitar-se que as redes se cruzem para manter a clareza do esquema. Para facilitar essa clareza, o engenheiro de hardware pode usar *rótulos de rede* para atribuir nomes às redes. A convenção é que as redes

* A propósito, "ninho de rato" é o termo usado para descrever a conexão de redes feita durante o layout. Depois de ver o estágio inicial de um layout, você entenderá como esse nome foi derivado.

marcados com o mesmo rótulo de rede estão conectados, mesmo que o engenheiro não tenha desenhado uma linha conectando-os.

Por exemplo, na Figura 2-4, uma parte do conector com o designador de referência PL1 é mostrada. (A propósito, os conectores e jumpers costumam usar o designador de referência J.) Como a rede conectada ao pino número 23 do conector PL1 é rotulada como A2, e a rede conectada ao pino número 43 do processador é rotulada como A2, eles estão conectados, embora o o engenheiro de hardware não executou uma linha para representar a rede A2 conectada do processador ao PL1.

A fim de ajudar no teste de hardware e software, os engenheiros de hardware costumam incluir *pontos de teste*. Um ponto de teste é uma área metálica na placa acabada que fornece acesso a um sinal específico para ajudar na depuração ou monitoramento desse sinal. Um ponto de teste, com o designador de referência TP11, é mostrado na Figura 2-4 no pino RESET do processador. Com a mudança para pacotes de IC cada vez menores e pinos menores, os pontos de teste são uma necessidade para depuração e também ajudam nos testes de produção. Além disso, é impossível sondar em qualquer pino de uma parte do pacote BGA (ball grid array), porque todos os pinos estão contidos sob o IC. Nesse caso, um ponto de teste ajuda muito.

Nos casos em que um esquema não pode caber em uma única página, deve haver uma maneira de interconectar as redes de uma página para outra. Este é o trabalho do *conector fora da página*.

Os conectores fora da página podem ser usados para redes individuais ou redes de barramento. Por exemplo, o conector off-page do barramento de dados é D [0..15]. Este é exatamente o mesmo nome de conector fora da página usado na página de memória para conectar o barramento de dados do processador ao

Barramento de dados da RAM.



Descobrimos algumas idéias que podem ser úteis para conectores fora da página. Eles podem ser úteis principalmente para o engenheiro de hardware que trabalha no esquema, mas outras pessoas da equipe também devem saber sobre eles.

Primeiro, é útil se o tipo de sinal (entrada, saída ou bidirecional) for representado corretamente pelo conector off-page apropriado. Assim, na Figura 2-4, o conector off-page do barramento de dados indica que esses são sinais bidirecionais; o conector fora da página CPU_RESET indica que este sinal é uma entrada para o processador; e o conector TX1 fora da página indica que este sinal é uma saída do processador.

Outra ideia útil é adicionar uma pequena nota de texto ao lado de cada conector fora da página com o (s) número (s) de página em que essa rede específica é usada. Isso pode não fazer sentido para um esquema de 5 páginas, mas folhear 20 páginas de esquemas pode ser um pesadelo.

Dicas adicionais podem ser encontradas em dezembro de 2002 *Programação de sistemas embarcados* * artigo “Design for Debugability,” que pode ser encontrado online em <http://www.embedded.com>.

* *Programação de sistemas embarcados* revista mudou seu nome para *Projeto de sistemas embarcados*.

Quando você der uma olhada no conjunto completo de esquemas, notará que há um bloco no canto inferior direito de cada página. Este é o bloco de título; cada esquema que encontramos tem alguma versão disso. O bloco de título contém informações úteis sobre o esquema, como a data, o nome do designer, a revisão, o número da página e a descrição do esquema nessa página e, muitas vezes, uma lista das alterações feitas.

Neste ponto, temos um conhecimento sólido dos componentes do sistema que compõem nossa plataforma e como eles estão interconectados. A seguir, veremos como conhecer o hardware.

Examine a paisagem

Geralmente é útil se colocar no lugar do processador por um tempo. Imagine como é ser o processador. Como é o mundo do processador? Quem está conectado a ele? Como ele se comunica com esses outros dispositivos?

Se você pensar dessa perspectiva, uma coisa que você rapidamente percebe é que o processador tem muitos compatriotas. Essas são as outras peças de hardware da placa, com as quais o processador se comunica. O processador possui diferentes métodos de comunicação com essas outras peças de hardware. Nesta seção, você aprenderá a reconhecer seus nomes e endereços.

A primeira coisa a notar é que existem dois tipos básicos de hardware aos quais os processadores se conectam: memórias e periféricos. As memórias são para o armazenamento e recuperação de dados e código. *Periféricos* são dispositivos de hardware especializados que coordenam a interação com o mundo externo ou executam uma função de hardware específica. Por exemplo, dois dos periféricos mais comuns em sistemas embarcados são portas seriais e temporizadores.

Os membros do 80x86 da Intel e algumas outras famílias de processadores têm dois espaços de endereços distintos por meio dos quais podem se comunicar com essas memórias e periféricos. O primeiro espaço de endereço é chamado de *espaço de memória* e se destina principalmente a dispositivos de memória; o segundo é reservado exclusivamente para periféricos e é chamado de *Espaço de E / S*. No entanto, os periféricos também podem ser localizados no espaço da memória, a critério do projetista do hardware. Quando isso acontece, dizemos que esses periféricos são mapeados na memória e que o sistema *E / S mapeada na memória*.

Alguns processadores suportam apenas um espaço de memória, caso em que todos os periféricos são mapeados na memória.

Do ponto de vista do processador, os periféricos mapeados na memória se parecem e atuam muito como dispositivos de memória. No entanto, a função de um periférico é bastante diferente da função de um dispositivo de memória. Em vez de simplesmente armazenar os dados fornecidos a ele, um periférico pode interpretá-lo como um comando ou como dados a serem processados de alguma forma.

Os projetistas de hardware embarcado geralmente preferem usar E / S com mapeamento de memória exclusivamente, porque isso tem vantagens para os desenvolvedores de hardware e software. É atraente para o desenvolvedor de hardware porque ele pode eliminar completamente o espaço de E / S e alguns de seus fios associados. Isso pode não reduzir significativamente o custo de produção da placa, mas pode reduzir a complexidade do design do hardware. Os periféricos mapeados por memória facilitam a vida do programador, que pode usar ponteiros, estruturas e uniões da linguagem C para interagir com os periféricos mais facilmente.

Mapa de Memória

Todos os processadores armazenam seus programas e dados na memória. Essa memória pode residir no mesmo chip do processador ou em chips de memória externos. A memória está localizada no espaço de memória do processador e o processador se comunica com ela por meio de dois conjuntos de fios elétricos chamados de barramento de endereço e barramento de dados. Para ler ou escrever um determinado local na memória, o processador primeiro escreve o endereço desejado no barramento de endereços. Alguma lógica (no processador ou em um circuito externo), conhecida como um *decodificador de endereço*, interpreta os bits de endereço superiores neste barramento e seleciona a memória apropriada ou o chip periférico. Os dados são então transferidos pelo barramento de dados. O decodificador de endereço pode ser um IC externo, mas hoje muitos processadores incluem essa lógica no chip.

Existem também sinais de controle para leitura e gravação em vários dispositivos no espaço de memória de um processador, comumente chamados de *barramento de controle*. Esses sinais de barramento de controle incluem leitura, gravação e seleção de chip (ou habilitação de chip). Alguns processadores combinam os sinais de leitura e gravação em um único sinal de leitura / gravação. Nesses processadores, uma operação de leitura é realizada definindo o sinal para um nível e uma operação de gravação é executada definindo o sinal para o nível oposto. Por exemplo, se o nome do sinal for RD / WR (pronuncia-se “barra de leitura e gravação”), o sinal é definido como 1 para uma operação de leitura e definido como 0 para uma operação de gravação.

O sinal de seleção de chip é definido para seu nível ativo quando o endereço de um determinado dispositivo cai dentro da faixa de endereço do dispositivo. Por exemplo, digamos que um dispositivo RAM ocupe a faixa de endereço de 0x0000 a 0x0FFF. Quando a instrução do software acessa a variável localizada no endereço 0x01F2, a seleção do chip para a RAM é definida em seu nível ativo.

Os sinais de leitura e gravação são configurados para seus níveis ativos pelo processador com base no tipo de transação de memória. A Figura 2-5 é um exemplo de um *diagrama de tempo*, uma representação gráfica da relação de tempo entre os vários sinais para uma determinada operação. Todo o diagrama na Figura 2-5 mostra uma leitura *ciclo*. Nesse caso, o ciclo está lendo 16 bits de dados da memória. Normalmente, uma mesa de *requisitos de tempo* acompanha um diagrama de tempo. Os requisitos de temporização detalham os tempos mínimos e máximos aceitáveis para cada um dos vários sinais e as relações de temporização entre os sinais.

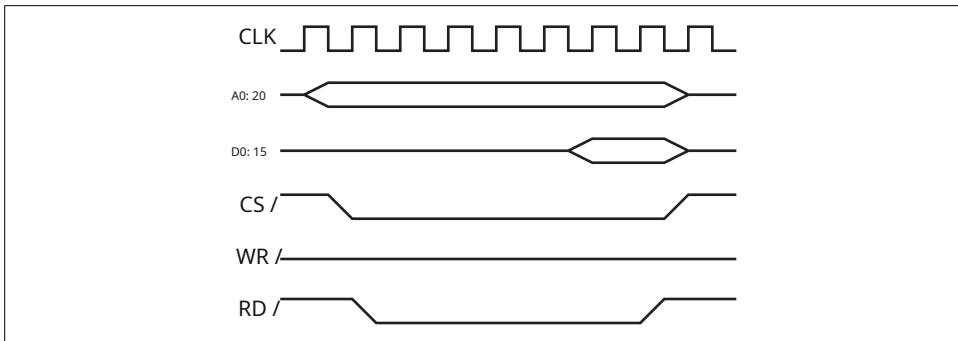


Figura 2-5. Diagrama de tempo de exemplo

O sinal de clock (CLK) é a base para todas as operações do processador e é mostrado como o sinal superior no diagrama de temporização da Figura 2-5. O processador *relógio* é geralmente uma onda quadrada que sequencia todas as operações do processador.

O próximo grupo de sinais é o barramento de endereço, A [0:20], seguido pelo barramento de dados, D [0:15]. Esses barramentos são representados em diagramas de tempo, conforme mostrado na Figura 2-5, onde uma única entrada representa toda a gama de sinais, em vez de cada sinal ter sua própria entrada. Um barramento é normalmente estável (o que significa que contém um endereço ou dados válidos) durante o período de tempo em que uma única linha se divide em duas. Em termos de hardware, o barramento passa de estar triestado (linha única) para ter informações reais presentes (linha dupla), e então volta a ser triestado novamente.

O próximo sinal é a seleção de chip baixo ativo (CS /), e depois disso é o sinal de gravação (WR /), que também é baixo ativo. Como este é um diagrama de tempo para uma operação de leitura, o sinal de gravação permanece inativo durante todo o ciclo. O último sinal é o lido (RD /). Ele fica baixo (ativo) depois que o endereço é definido pelo processador.

Exemplos adicionais de diagramas de temporização para o processador PXA255 podem ser encontrados na seção Controlador de memória do *Manual do desenvolvedor do processador PXA255* também Enquanto o *Especificações elétricas, mecânicas e térmicas do processador PXA255* Ficha de dados.



Alguns processadores também podem incluir outros tipos de sinais de controle para ajudar a acessar vários tipos de dispositivos periféricos. Esses sinais podem ser chamados de Pronto, Espera, Espera, Reconhecer, Esperar e outras coisas. Um engenheiro de hardware pode usar esses sinais para acessar uma ampla gama de dispositivos, principalmente aqueles que operam mais lentamente do que o processador. Por exemplo, uma ROM mais lenta pode usar o sinal Hold do processador para dizer ao processador que ele precisa de mais tempo para completar a leitura de um endereço de memória específico. O processador pode então esperar até que a ROM consiga terminar de obter os dados para o processador.

Enquanto você está lendo sobre uma nova placa, crie uma tabela que mostre o nome e a faixa de endereço de cada dispositivo de memória e periférico que está localizado no espaço da memória. Esta tabela é chamada de *mapa de memória*. Organize a mesa de forma que o mais baixo

o endereço está na parte inferior e o endereço mais alto está na parte superior. Cada vez que você adiciona um dispositivo ao mapa de memória, coloque-o em seu local apropriado na memória e rotule os endereços inicial e final em hexadecimal. Depois de inserir todos os dispositivos no mapa de memória, certifique-se de rotular todas as regiões não utilizadas da memória como tal.

Se você olhar novamente para o diagrama de blocos da placa Arcom na Figura 2-1, verá que há três dispositivos conectados aos barramentos de endereço e dados. (O barramento PC / 104 é conectado aos barramentos de endereço e dados por meio de buffers.) Esses dispositivos são o controlador Ethernet de RAM, ROM e SMSC. A RAM está localizada na parte inferior da faixa de endereços de memória. A ROM está localizada na parte superior da faixa.



Às vezes, um único intervalo de endereços, especialmente para dispositivos de memória, é composto de vários ICs. Por exemplo, o engenheiro de hardware pode usar dois chips ROM, cada um com capacidade de armazenamento de 1 MB. Isso dá ao processador um total de 2 MB de ROM. Além disso, o engenheiro de hardware é capaz de configurar os dois chips ROM individuais de forma que o processador não saiba qual chip está realmente acessando; a divisão dos dois chips é transparente para o processador e ele vê a memória como um bloco contíguo.

O mapa de memória na Figura 2-6 mostra a aparência dos dispositivos de memória na Figura 2-1 para o processador. Também incluídos na Figura 2-6 estão os registros periféricos internos do processador, rotulados como Periféricos PXA255, que são mapeados no espaço de memória do processador. Em certo sentido, este é o "catálogo de endereços" do processador. Assim como você mantém uma lista de nomes e endereços em sua vida pessoal, deve manter uma lista semelhante para o processador. O mapa de memória contém uma entrada para cada uma das memórias e periféricos que são acessíveis a partir do espaço de memória do processador. Este diagrama é sem dúvida a informação mais importante sobre o sistema para um engenheiro de software embarcado e deve ser mantido atualizado e mantido como parte dos registros permanentes associados ao projeto.

Para cada nova placa, você deve criar um arquivo de cabeçalho em linguagem C que descreve seus recursos mais importantes. Este arquivo fornece uma interface abstrata para o hardware. Na verdade, ele permite que você se refira aos vários dispositivos na placa pelo nome, e não pelo endereço. Isso tem o benefício adicional de tornar seu software de aplicativo mais portátil. Se o mapa de memória mudar - por exemplo, se os 64 MB de RAM forem movidos - você só precisa alterar as linhas afetadas do arquivo de cabeçalho específico da placa e recompilar seu aplicativo.

Abstrair o hardware em um arquivo (para projetos menores) ou um diretório de arquivos (para projetos maiores) também permite que você reutilize certas partes do seu código conforme você passa de um projeto para outro. Como o engenheiro de hardware provavelmente reutilizará componentes em novos designs, você também pode reutilizar drivers para esses componentes nesses novos designs.

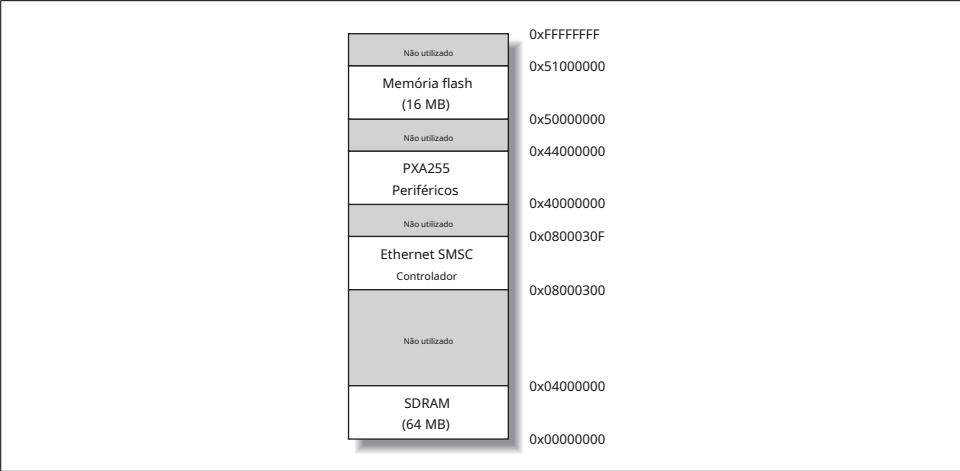


Figura 2-6. Mapa de memória para a placa Arcom

À medida que este capítulo avança, mostraremos como criar um arquivo de cabeçalho para a placa Arcom; o código a seguir é a primeira seção deste arquivo. Esta parte do arquivo de cabeçalho descreve o mapa de memória:

```
/ *****
*
* Mapa de Memória
*
*      Endereço Base      Descrição do tamanho
*      -----
*      0x00000000 64M SDRAM
*      0x08000300 N / A controlador Ethernet
*      0x50000000 16M Flash
*
* ***** /

# define SDRAM_BASE          (0x00000000)
# define ETHERNET_BASE      (0x08000300)
# define FLASH_BASE         (0x50000000)
```

Aprenda a se comunicar

Agora que você conhece os nomes e endereços da memória e dos periféricos conectados ao processador, é hora de aprender como se comunicar com os periféricos. Existem duas técnicas básicas de comunicação: *votação* e *interrupções*. Em ambos os casos, o processador geralmente emite algum tipo de comando para o dispositivo, escrevendo - por meio da memória ou espaço de E / S - valores de dados específicos para endereços específicos dentro do dispositivo e, em seguida, espera que o dispositivo conclua a tarefa atribuída . Por exemplo, o processador pode pedir a um cronômetro para fazer uma contagem regressiva de 1.000 a 0. Assim que o

a contagem regressiva começa, o processador está interessado em apenas uma coisa: o cronômetro já terminou de contar?

Se a pesquisa for usada, o processador verifica repetidamente se a tarefa foi concluída. Isso é análogo à criança pequena que pergunta repetidamente: "Já chegamos?" ao longo de uma longa viagem. Como a criança, o processador gasta uma grande quantidade de tempo útil fazendo perguntas e obtendo uma resposta negativa. Para implementar o polling no software, você só precisa criar um loop que leia o registro de status do dispositivo em questão. Aqui está um exemplo:

```
Faz
{
    / * Jogar, ler, ouvir música, etc. * /
    . . .

    / * Faça uma pesquisa para ver se já chegamos.
    * / status = areWeThereYet ();

} enquanto (status == NÃO);
```

A segunda técnica de comunicação usa interrupções. Uma interrupção é um sinal elétrico assíncrono de um periférico para o processador. As interrupções podem ser geradas a partir de periféricos externos ou internos ao processador, bem como por software.

Quando interrupções são usadas, o processador emite comandos para o periférico exatamente como antes, mas então espera por uma interrupção para sinalizar a conclusão do trabalho atribuído. Enquanto o processador aguarda a chegada da interrupção, ele está livre para continuar trabalhando em outras coisas. Quando o sinal de interrupção é declarado, o processador termina sua instrução atual, temporariamente deixa de lado seu trabalho atual e executa um pequeno pedaço de software chamado de *rotina de serviço de interrupção (ISR)* ou *manipulador de interrupções*. Quando o ISR é concluído, o processador retorna ao trabalho que foi interrompido.

Claro, isso não é tudo automático. O próprio programador deve escrever o ISR e “instalar” e habilitá-lo para que seja executado quando ocorrer a interrupção relevante. Nas primeiras vezes que você fizer isso, será um desafio significativo. Mas, mesmo assim, o uso de interrupções geralmente diminui a complexidade do código geral de uma pessoa, dando-lhe uma estrutura melhor. Em vez de a votação do dispositivo ser incorporada a uma parte não relacionada do programa, as duas partes do código permanecem apropriadamente separadas.

No geral, as interrupções são um uso muito mais eficiente do processador do que o polling. O processador é capaz de usar uma porcentagem maior de seu tempo de espera realizando um trabalho útil. No entanto, há alguma sobrecarga associada a cada interrupção. Leva um bom tempo - em relação ao tempo que leva para executar um opcode - para deixar de lado o trabalho atual do processador e transferir o controle para a rotina de serviço de interrupção. Muitos dos registros do processador devem ser salvos na memória.

Na prática, as interrupções e as pesquisas são usadas com frequência. As interrupções são usadas quando a eficiência é fundamental ou quando vários dispositivos devem ser monitorados simultaneamente. O polling é normalmente usado quando o processador deve responder a algum evento

mais rapidamente do que é possível usando interrupções ou quando grandes quantidades de dados devem chegar em intervalos específicos, como durante a aquisição de dados em tempo real. Analisaremos mais detalhadamente as interrupções no Capítulo 8.

Conhecendo o processador

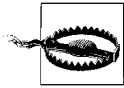
Se você nunca trabalhou com o processador da placa antes, reserve um tempo para se familiarizar com ele agora. Isso não deve demorar muito se você fizer toda a programação em uma linguagem de alto nível, como C. Você precisa se aprofundar e descobrir como funcionam determinados periféricos do processador. Geralmente, para o usuário de uma linguagem de alto nível, a maioria dos processadores se parece e age da mesma forma. No entanto, se for fazer qualquer programação em linguagem assembly, você precisa se familiarizar com a arquitetura de registro do processador e o conjunto de instruções.

Tudo o que você precisa saber sobre o processador pode ser encontrado nos databooks fornecidos pelo fabricante. Se você ainda não tem um databook ou guia do programador para o seu processador, você deve obter um imediatamente. Se você pretende ser um programador de sistemas embarcados de sucesso, deve ser capaz de ler os livros de dados e extrair algo deles. Os databooks do processador são geralmente bem escritos - no que diz respeito aos databooks - então eles são um lugar ideal para começar. Comece folheando o livro de dados e observando as seções mais relevantes para as tarefas em questão. Em seguida, volte e comece a ler a seção de visão geral do processador.

As coisas que você deseja aprender sobre o processador em seu livro de dados são:

- Para qual endereço o processador salta após uma reinicialização?
- Qual é o estado dos registros e periféricos do processador na reinicialização?
- Qual é a sequência adequada para programar os registros de um periférico?
- Onde a tabela de vetores de interrupção deve ser localizada? Ele precisa estar localizado em um endereço específico na memória? Se não, como o processador sabe onde encontrá-lo?
- Qual é o formato da tabela de vetores de interrupção? É apenas uma tabela de indicadores para funções ISR?
- Existe alguma interrupção especial - às vezes chamada *armadilhas* —Que são gerados dentro do próprio processador? Deve ser escrito um ISR para lidar com cada um deles?
- Como as interrupções são habilitadas e desabilitadas? Globalmente e individualmente?
- Como as interrupções são reconhecidas ou eliminadas?

Além do databook do processador, a Internet contém uma riqueza de informações para desenvolvedores de software embarcado. O site do fabricante é um ótimo lugar para começar. Além disso, a busca por um processador específico pode render grande quantidade de informações úteis de outros desenvolvedores, incluindo trechos de código que fornecem detalhes exatos sobre como escrever seu software. Vários grupos de notícias também são direcionados ao desenvolvimento de software integrado e a processadores específicos.



Você precisa ter o cuidado de compreender totalmente quaisquer problemas de licenciamento do software que encontrar na Internet, caso decida usar o código de outra pessoa. Pode ser necessário envolver o departamento jurídico de sua empresa para evitar problemas.

Outra ferramenta útil para entender o processador é uma placa de desenvolvimento. Assim que o processador for selecionado, você pode pesquisar suas opções para uma placa de desenvolvimento. Você precisa considerar os periféricos e ferramentas de software incluídos na placa de desenvolvimento. Por exemplo, se seu aplicativo vai incluir uma porta Ethernet, seria uma boa ideia selecionar uma placa de desenvolvimento que também inclua uma porta Ethernet. Normalmente, há um exemplo de software incluído na placa de desenvolvimento também. Se o projeto usar um processador com o qual você não trabalhou antes, o software de exemplo pode ajudá-lo a subir a curva de aprendizado muito mais rápido. A placa de desenvolvimento o ajudará a dar um passo inicial no desenvolvimento de software embarcado.

Outro benefício de uma placa de desenvolvimento é que se você estiver vendo algumas peculiaridades relacionadas ao hardware do seu projeto, você sempre pode voltar para a placa de desenvolvimento (onde o hardware deve ser estável) e executar alguns testes para ver se o problema é específico do novo design.

Processadores em geral

Muitos dos processadores mais comuns são membros de famílias de dispositivos relacionados. Em alguns casos, os membros dessa família de processadores representam pontos ao longo de um caminho evolutivo. O exemplo mais óbvio é a família 80x86 da Intel, que vai do 8086 original ao Pentium 4 e além. Na verdade, a família 80x86 teve tanto sucesso que gerou toda uma indústria de imitadores.

Como é usado neste livro, o termo *processador* refere-se a qualquer um dos três tipos de dispositivos conhecido como *microprocessadores*, *microcontroladores*, e *processadores de sinais digitais*. O nome microprocessador geralmente é reservado para um chip que contém um poderoso *unidade central de processamento* (CPU) que não foi projetado com nenhum cálculo específico em mente. Esses chips geralmente são a base de computadores pessoais e estações de trabalho de última geração, embora microprocessadores também sejam usados em sistemas embarcados. Outros microprocessadores amplamente conhecidos são os membros do Freescale * 68K - encontrado em computadores Macintosh mais antigos - e as onipresentes famílias 80x86.

Um microcontrolador é muito parecido com um microprocessador, exceto que foi projetado especificamente para uso em sistemas embarcados. Os microcontroladores normalmente incluem uma CPU, memória (uma pequena quantidade de RAM, ROM ou ambos) e outros periféricos no mesmo circuito integrado. Se você comprar todos esses itens em um único chip, é possível reduzir substancialmente o custo de um sistema embarcado. Entre as mais

* Esta é a nova divisão de semicondutores da Motorola.

microcontroladores populares são o 8051 e seus muitos imitadores e a série 68HCxx. Também é comum encontrar versões de microcontroladores de microprocessadores populares. Por exemplo, o 386 EX da Intel é uma versão microcontrolada do microprocessador 80386.

O tipo final de processador é um *processador de sinal digital*, ou DSP. A CPU em um DSP é especialmente projetada para realizar cálculos de processamento de sinal em tempo discreto - como aqueles necessários para comunicações de áudio e vídeo - extremamente rápido. Como os DSPs podem realizar esses tipos de cálculos muito mais rápido do que outros processadores, eles oferecem uma alternativa de microprocessador poderosa e de baixo custo para projetistas de telefones celulares e outros equipamentos de telecomunicações e multimídia. Dispositivos analógicos, Freescale e TI são fornecedores de dispositivos DSP comuns.

O processador PXA255 XScale

O processador da placa Arcom é um PXA255 que incorpora o núcleo XScale. O XScale é baseado na arquitetura ARM Versão 5TE. Para saber mais sobre o processador ARM, um ótimo livro é o de David Seal *Manual de referência de arquitetura ARM* (Addison-Wesley); este livro é comumente referido como "ARM ARM".

Além da CPU, o PXA255 contém uma unidade de controle de interrupção, um controlador de memória, vários pinos de E / S de uso geral, quatro temporizadores / contadores, um I² Unidade de interface de barramento C, quatro portas seriais, 16 canais de acesso direto à memória (DMA), um controlador de memória que suporta vários tipos de memória, incluindo DRAM, um cliente USB, um controlador LCD, dois moduladores de largura de pulso, um relógio em tempo real, um watchdog unidade de temporizador e uma unidade de gerenciamento de energia. Esses dispositivos de hardware extras estão localizados no mesmo chip e são chamados de *periféricos no chip*. A CPU é capaz de se comunicar e controlar os periféricos on-chip diretamente, por meio de barramentos internos.

Embora os periféricos on-chip sejam dispositivos de hardware distintos, eles agem como pequenas extensões da CPU PXA255. O software pode controlá-los lendo e gravando nos registros específicos do periférico. Os registros de controle e status para cada um dos periféricos no chip estão localizados em endereços fixos no espaço da memória. Os endereços exatos de cada registro podem ser encontrados no *Manual do desenvolvedor do processador PXA255*. Para isolar esses detalhes de seu software aplicativo e ajudar na legibilidade de seu software, é uma boa prática incluir os endereços de todos os registros que você usará no arquivo de cabeçalho de sua placa. Você pode ver no seguinte snippet de código que é mais difícil entender o que está acontecendo quando os endereços são usados diretamente:

```
if (bLedEnable == TRUE) {  
    *((uint32_t *) 0x40E00018) = 0x00400000;  
}
```

Embora os comentários possam esclarecer o que está acontecendo no código acima, é melhor usar nomes mais descritivos em seu software que farão seu código se autodocumentar, mas adicione comentários também para ajudar no entendimento.

Além de um arquivo de cabeçalho que descreve os recursos da placa, um arquivo de cabeçalho em linguagem C que descreve os registros do processador também deve ser criado. A seguir, um exemplo de nomes descritivos para alguns dos registros no processador PXA255. Isso é também útil para definir nomes descritivos para bits específicos em um registro, se eles quiserem ser endereçado individualmente. *

```
/ *****  
 * Periféricos no chip do processador PXA255 XScale ARM  
 ***** /  
  
/ * Timer Registers * /  
# define TIMER_0_MATCH_REG      (*(uint32_t volatile *) 0x40A00000) (*(uint32_t  
# define TIMER_1_MATCH_REG      volatile *) 0x40A00004) (*(uint32_t volatile *)  
# define TIMER_2_MATCH_REG      0x40A00008) (*(uint32_t volatile *) 0x40A0000C)  
# define TIMER_3_MATCH_REG      (*(uint32_t volatile *) 0x40A00008) (*(uint32_t  
# define TIMER_COUNT_REG        volatile *) 0x40A0000C) (*(uint32_t volatile *)  
# define TIMER_STATUS_REG        0x40A00010) (*(uint32_t volatile *) 0x40A00014)  
# define TIMER_INT_ENABLE_REG    (*(uint32_t volatile *) 0x40A0001C)  
  
/ * Interrupção do temporizador para habilitar descrições de bits de registro * /  
# define TIMER_0_INTEN (0x01)  
# define TIMER_1_INTEN (0x02)  
# define TIMER_2_INTEN (0x04)  
# define TIMER_3_INTEN (0x08)  
  
/ * Descrições de bits de registro de status do temporizador * /  
# define TIMER_0_MATCH (0x01)  
# define TIMER_1_MATCH (0x02)  
# define TIMER_2_MATCH (0x04)  
# define TIMER_3_MATCH (0x08)  
  
/ * Registros do controlador de interrupção * /  
# define INTERRUPT_PENDING_REG  (*(uint32_t volatile *) 0x40D00000) (*(  
# define INTERRUPT_ENABLE_REG   ((uint32_t volatile *) 0x40D00004) (*(  
# define INTERRUPT_TYPE_REG     ((uint32_t volatile *) 0x40D00008))  
  
/ * Descrições de bits de registro de habilitação de interrupção * /  
# define GPIO_0_ENABLE (0x00000100)  
# define UART_ENABLE (0x00400000)  
# define TIMER_0_ENABLE (0x04000000)  
# define TIMER_1_ENABLE (0x08000000)  
# define TIMER_2_ENABLE (0x10000000)  
# define TIMER_3_ENABLE (0x20000000)
```

* Discutiremos o uso da palavra-chave volátil no Capítulo 7.

```

/* Registros de E / S de uso geral (GPIO) */
#define GPIO_0_LEVEL_REG (*(uint32_t volatile *) 0x40E00000)
#define GPIO_1_LEVEL_REG (*(uint32_t volatile *) 0x40E00004)
#define GPIO_2_LEVEL_REG (*(uint32_t volatile *) 0x40E00008)
#define GPIO_0_DIRECTION_REG (*(uint32_t volatile *) 0x40E0000C)
#define GPIO_1_DIRECTION_REG (*(uint32_t volatile *) 0x40E00010)
#define GPIO_2_DIRECTION_REG (*(uint32_t volatile *) 0x40E00014)
#define GPIO_0_SET_REG (*(uint32_t volatile *) 0x40E00018)
#define GPIO_1_SET_REG (*(uint32_t volatile *) 0x40E0001C)
#define GPIO_2_SET_REG (*(uint32_t volatile *) 0x40E00020)
#define GPIO_0_CLEAR_REG (*(uint32_t volatile *) 0x40E00024)
#define GPIO_1_CLEAR_REG (*(uint32_t volatile *) 0x40E00028)
#define GPIO_2_CLEAR_REG (*(uint32_t volatile *) 0x40E0002C)
#define GPIO_0_FUNC_LO_REG (*(uint32_t volatile *) 0x40E00054)
#define GPIO_0_FUNC_HI_REG (*(uint32_t volatile *) 0x40E00058)

```

Vamos dar uma olhada no snippet de código anterior escrito para usar uma definição de registro do arquivo de cabeçalho de exemplo:

```

if (bLedEnable == TRUE) {

    GPIO_0_SET_REG = 0x00400000;
}

```

Este código é muito mais fácil de ler e entender, mesmo sem um comentário. Definir registros em um arquivo de cabeçalho, como mostramos no código anterior, também evita que você ou outro membro da equipe vá até o livro de dados a cada dois minutos para consultar um endereço de registro.

Estude os periféricos externos

Neste ponto, você estudou todos os aspectos do novo hardware, exceto os periféricos externos. Esses são os dispositivos de hardware que residem fora do chip do processador e se comunicam com ele por meio de interrupções e registros de E / S ou mapeados na memória.

Comece fazendo uma lista dos periféricos externos. Dependendo da sua aplicação, esta lista pode incluir LCD ou controladores de teclado, conversão de analógico para digital (A / D) *ers*, chips de interface de rede ou *circuitos integrados específicos de aplicação* (ASICs). No caso da placa Arcom, a lista contém apenas dois itens: o controlador Ethernet SMSC e a porta paralela.

Você deve obter uma cópia do manual do usuário ou folha de dados para cada dispositivo em sua lista. Neste estágio inicial do projeto, seu objetivo ao ler esses documentos é entender as funções básicas do dispositivo. O que o dispositivo faz? Quais registros são usados para emitir comandos e receber os resultados? O que significam os vários bits e campos maiores dentro desses registros? Quando, se alguma vez, o dispositivo gera interrupções? Como as interrupções são reconhecidas ou eliminadas no dispositivo?

Ao projetar o software integrado, você deve tentar dividir o programa ao longo das linhas de dispositivo. Normalmente, é uma boa ideia associar um software

módulo chamado de *driver do dispositivo* com cada um dos periféricos externos. Um driver de dispositivo nada mais é do que uma coleção de rotinas de software que controlam a operação de um periférico específico e isolam o software aplicativo dos detalhes desse dispositivo de hardware específico. Teremos muito mais a dizer sobre drivers de dispositivo mais tarde.

Inicialize o Hardware

A etapa final para conhecer seu novo hardware é escrever um software de inicialização. Esta é a sua melhor oportunidade de desenvolver uma relação de trabalho próxima com o hardware, especialmente se for desenvolver o restante do software em uma linguagem de alto nível.

Durante a inicialização do hardware, pode ser impossível evitar o uso da linguagem assembly. No entanto, após concluir esta etapa, você estará pronto para começar a escrever pequenos programas. *



Se você for um dos primeiros engenheiros de software a trabalhar com uma nova placa - especialmente um protótipo - o hardware pode não funcionar como anunciado. Todas as placas baseadas em processador requerem alguns testes de software para confirmar a exatidão do design do hardware e o funcionamento adequado dos vários periféricos. Isso o coloca em uma posição estranha quando algo não está funcionando corretamente. Como saber se o hardware ou software está causando o problema? Se você for bom com hardware ou tiver acesso a um simulador, poderá construir alguns experimentos para responder a essa pergunta. Caso contrário, você provavelmente deve pedir a um engenheiro de hardware para se juntar a você no laboratório para uma sessão de depuração conjunta.

A inicialização do hardware deve ser executada antes do código de inicialização descrito no Capítulo 4. O código descrito presume que o hardware já foi inicializado e se preocupa apenas com a criação de um ambiente de tempo de execução adequado para programas de linguagem de alto nível. A Figura 2-7 fornece uma visão geral de todo o processo de inicialização, desde a redefinição do processador até a inicialização do hardware e código de inicialização C

para a Principal.

O primeiro estágio do processo de inicialização é o código de redefinição. Este é um pequeno pedaço de linguagem assembly (geralmente apenas duas ou três instruções) que o processador executa imediatamente após ser ligado ou reiniciado. O único propósito deste código é transferir o controle para a rotina de inicialização do hardware. A primeira instrução do código de reset deve ser colocada em um local específico na memória, geralmente chamado de *Redefinir*

* Para tornar o exemplo do Capítulo 3 um pouco mais fácil de entender, não mostramos nenhum código de inicialização ali. No entanto, é necessário fazer com que o código de inicialização do hardware funcione antes de escrever programas simples, como LED piscando. A placa Arcom inclui um monitor de depuração que lida com toda a inicialização do hardware de linguagem assembly.

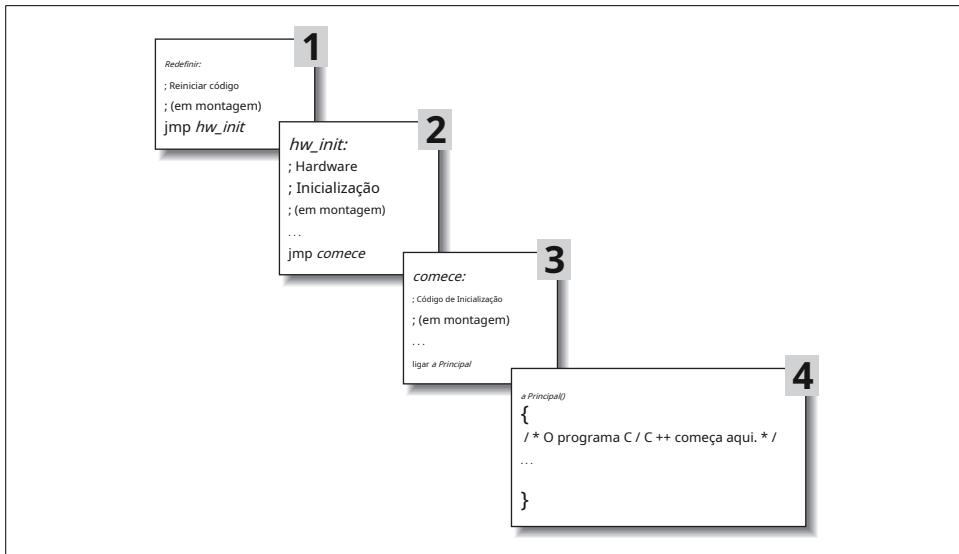


Figura 2-7. O processo de inicialização de hardware e software

Morada ou *redefinir vetor*, que é especificado no livro de dados do processador. O endereço de reinicialização do PXA255 é 0x00000000.

A maior parte da inicialização real do hardware ocorre no segundo estágio. Neste ponto, precisamos informar o processador sobre seu ambiente. Este também é um bom lugar para inicializar o controlador de interrupção e outros periféricos críticos. Dispositivos de hardware menos críticos podem ser inicializados quando o driver de dispositivo associado é iniciado, geralmente de dentro a *Principal*.

O PXA255 possui pinos de seleção de inicialização que permitem especificar o tipo e a largura do dispositivo de memória a partir do qual o processador tenta executar as instruções iniciais. O dispositivo de memória a partir do qual o processador inicializa normalmente contém o código para programar vários registros internos do PXA255 que devem ser programados antes que qualquer trabalho útil possa ser feito com o processador. Esses registros internos são responsáveis por configurar o mapa de memória e fazem parte do controlador de memória interna do processador. Ao programar os registros de configuração da interface de memória, você essencialmente está ativando cada uma das memórias e dispositivos periféricos que estão conectados ao processador.

O PXA255 contém seis chips selecionados para interface com vários tipos de memória externa. Essas seleções de chips são ativas para acessos em intervalos de memória específicos. Cada seleção de chip está associada a um único fio de “chip habilitado” que vai do processador a algum outro chip. Em muitos sistemas, o circuito para fazer isso é externo ao processador. A associação entre determinados chips selecionados e dispositivos de hardware deve ser estabelecida pelo designer de hardware. Tudo que você precisa fazer é obter uma lista de configurações de seleção de chip dela e carregar essas configurações nos registros de configuração de memória.

Após o reset, o PXA255 salta para o endereço 0x00000000, que ativa a seleção de chip 0 (o pino nCS [0]). Essa é a “posição fetal” do processador e implica que essa seleção de chip seja usada para acessar algum tipo de memória não volátil, como flash no caso da placa Arcom. Como não há outros chips selecionados configurados neste ponto, o software também não deve exigir o uso de nenhuma RAM. Isso inclui fazer uma chamada de sub-rotina, pois isso exigirá acesso à pilha (discutida a seguir), que reside na RAM; chamadas de função são proibidas. Até que a RAM esteja ativa e o ponteiro da pilha inicializado, seu código deve ser linear.

A rotina de inicialização de hardware, `hw_init`, deve começar inicializando os registros de configuração da interface de memória para informar ao processador sobre a outra memória e dispositivos periféricos que estão instalados na placa. Quando esta tarefa for concluída, toda a gama de endereços de ROM e RAM estará habilitada, de forma que o restante do software possa ser localizado em qualquer endereço conveniente na ROM ou RAM.

O terceiro estágio de inicialização contém o *código de inicialização*. Seu trabalho é preparar o caminho para o código escrito em uma linguagem de alto nível. Uma das tarefas importantes deste código é configurar a pilha para o sistema. a *pilha* é a área da RAM que o processador usa para armazenamento temporário durante a execução. A pilha opera no *ultimo a entrar primeiro a sair* (LIFO) princípio em que os itens de dados são colocados e retirados da pilha: normalmente, variáveis locais (também conhecidas como *variáveis automáticas*) e endereços de retorno de chamadas de função durante a execução do programa. Depois de inicializar a pilha, o código de inicialização chama a Principal. Desse ponto em diante, todos os outros softwares podem ser escritos em uma linguagem de alto nível.

Felizmente, você está começando a entender como o software embarcado vai da reinicialização do processador para o seu programa principal. É certo que a primeira vez que você tentar puxar todos esses componentes (código de redefinição, inicialização de hardware, código de inicialização de linguagem de alto nível e aplicativo) juntos em uma nova placa, haverá problemas. Portanto, espere gastar algum tempo depurando cada um deles. Veremos a depuração no Capítulo 5.

Com uma nova plataforma de hardware, alguns problemas de hardware também podem surgir. Isso pode levar a um problema em que o processador simplesmente não faz nada. Às vezes, o problema é um problema básico que foi esquecido. Algumas das coisas básicas a fazer são:

- Certifique-se de que o processador e o ROM estejam recebendo a voltagem adequada necessária para operar as peças.
- Verifique se o sinal do relógio está funcionando. O processador não fará nada sem um relógio.
- Verifique se o processador está saindo da reinicialização corretamente. Você pode verificar o endereço que um processador está buscando usando um analisador lógico. Isso validará se o processador está tentando buscar a primeira instrução do local que você espera.
- Certifique-se de que um cronômetro de watchdog não esteja reiniciando o processador.
- Certifique-se de que os pinos de entrada no processador sejam puxados para cima ou para baixo. Isso é particularmente importante para pinos de interrupção. Um pino de entrada em um estado desconhecido (comumente chamado de *flutuando* pin) pode causar todos os tipos de estragos para um processador.

O engenheiro de hardware pode lidar com essas tarefas para você, mas não tenha medo de pular e examinar os esquemas você mesmo. Ou melhor ainda, veja se você pode sentar-se no laboratório com o engenheiro de hardware enquanto ele faz a verificação inicial da placa.

Espere que a apresentação inicial do hardware seja a parte mais difícil do projeto. Você logo verá que, depois de ter um programa básico operando ao qual possa recorrer, o trabalho fica cada vez mais fácil - ou pelo menos mais semelhante a outros tipos de programação de computador.

Seu primeiro programa integrado

*ACHTUNG! Das machine is nicht fur gefingerpoken
und mittengrabben. Ist easy schnappen der
springenwerk, blowenfusen und corkenpoppen mit
spitzensparken. Ist nicht fur gewerken por das
dummkopfen. Das rubberneckn sightseeren keepen
mãos nos bolsos. Relaxen und vatch das
luzes piscantes!*

- Sinal de Laboratório de Eletrônica

Neste capítulo, vamos mergulhar direto na programação embarcada por meio de um exemplo. Nosso exemplo é semelhante em espírito ao “Hello, World!” exemplo encontrado no início da maioria dos outros livros de programação. Discutiremos por que escolhemos este programa específico e apontaremos as partes dele que dependem do hardware de destino. Este capítulo contém apenas o código-fonte do programa. Discutiremos como criar o executável e como realmente executá-lo nos capítulos a seguir.

Olá Mundo!

Parece que todos os livros de programação já escritos começam com o mesmo exemplo - um programa que imprime “Hello, World!” na tela do usuário. Um exemplo usado em demasia como este pode parecer um pouco enfadonho. Entre outras coisas, o exemplo ajuda os leitores a avaliar rapidamente a facilidade ou dificuldade com que programas simples podem ser escritos no ambiente de programação em questão. Nesse sentido, “Olá, mundo!” serve como uma referência útil para usuários de linguagens de programação e plataformas de computador.

Baseado no “Hello, World!” benchmark, os sistemas embarcados estão entre as plataformas de computador mais desafiadoras para os programadores trabalharem. Em alguns sistemas embarcados, pode até ser impossível implementar o “Hello, World!” programa. E nos sistemas que são capazes de suportá-lo, a impressão de strings de texto costuma ser mais um ponto final do que um começo.

Uma suposição principal do “Hello, World!” Um exemplo é que existe algum tipo de dispositivo de saída no qual cadeias de caracteres podem ser impressas. Uma janela de texto no

o monitor do usuário muitas vezes serve a esse propósito. Mas a maioria dos sistemas embarcados carece de um monitor ou dispositivo de saída analógico. E aqueles que têm um normalmente requerem um software embarcado especial, chamado driver de vídeo, para ser implementado primeiro - uma maneira bastante desafiadora de começar a carreira de programação embarcada.

Seria muito melhor começar com um programa embarcado pequeno, facilmente implementado e altamente portátil, no qual há pouco espaço para erros de programação. Afinal, a razão pela qual nossos colegas escritores de livros continuam a usar o “Hello, World!” exemplo é que implementá-lo é um acéfalo. Isso elimina uma das variáveis - caso o programa do usuário não funcione corretamente na primeira vez: não é um bug em seu código; em vez disso, é um problema com as ferramentas de desenvolvimento ou processo que ele usou para criar o programa executável.

Os programadores incorporados devem ser autossuficientes. Eles devem sempre começar cada novo projeto com a suposição de que nada funciona - que tudo em que podem confiar é a sintaxe básica de sua linguagem de programação. Mesmo as rotinas de biblioteca padrão podem não estar disponíveis para eles. Estas são as funções auxiliares, como `printf` e `memcpy` —Que a maioria dos outros programadores dá como certo. Na verdade, as rotinas da biblioteca costumam fazer parte do padrão da linguagem C tanto quanto a sintaxe básica. No entanto, a parte da biblioteca do padrão é mais difícil de suportar em todas as plataformas de computação possíveis e, ocasionalmente, é ignorada pelos fabricantes de compiladores para sistemas embarcados.

Portanto, você não encontrará um verdadeiro “Hello, World!” programa neste capítulo. Em vez disso, escreveremos o programa em linguagem C mais simples que pudermos, sem presumir que você tenha hardware especializado (o que exigiria um driver de dispositivo) ou qualquer biblioteca com funções como `printf`. À medida que avançamos no livro, adicionaremos gradualmente rotinas de biblioteca padrão e o equivalente a um dispositivo de saída de caracteres ao nosso repertório. Nesse momento, você estará no caminho certo para se tornar um especialista na área de programação de sistemas embarcados.

O Programa de LED Piscando

Quase todos os sistemas embarcados que encontramos em nossas respectivas carreiras tiveram pelo menos um LED que pode ser controlado por software. Se o designer do hardware planeja deixar o LED fora do circuito, faça um lobby forte para obter um conectado a um pino de E / S de uso geral (GPIO). Como veremos mais tarde, esta pode ser a ferramenta de depuração mais valiosa que você possui.

Um substituto popular para o “Hello, World!” programa é aquele que pisca um LED a uma taxa de 1 Hz (um ciclo liga-desliga completo por segundo). * Normalmente, o código necessário para

* Claro, a taxa de intermitência é completamente arbitrária. Mas uma das coisas boas sobre a taxa de 1 Hz é que é fácil de confirmar com um cronômetro. Simplesmente inicie o cronômetro, conte algumas piscadas, pare o cronômetro e veja se o número de segundos decorridos é igual ao número de piscadas que você contou. Precisa de maior precisão? Simplesmente conte mais piscadas.

ligar e desligar um LED é limitado a algumas linhas de código, portanto, há muito pouco espaço para a ocorrência de erros de programação. E como quase todos os sistemas embarcados possuem LEDs, o conceito subjacente é extremamente portátil.

Nosso primeiro passo é aprender como controlar o LED verde que queremos alternar. Na placa Arcom, o LED verde está localizado no módulo complementar mostrado na Figura 3-1. O LED verde é identificado como “LED2” no módulo complementar. O conselho da Arcom

VIPER-Lite Techni
L do módulo adicional
usado para rastrear o
método que você precisa

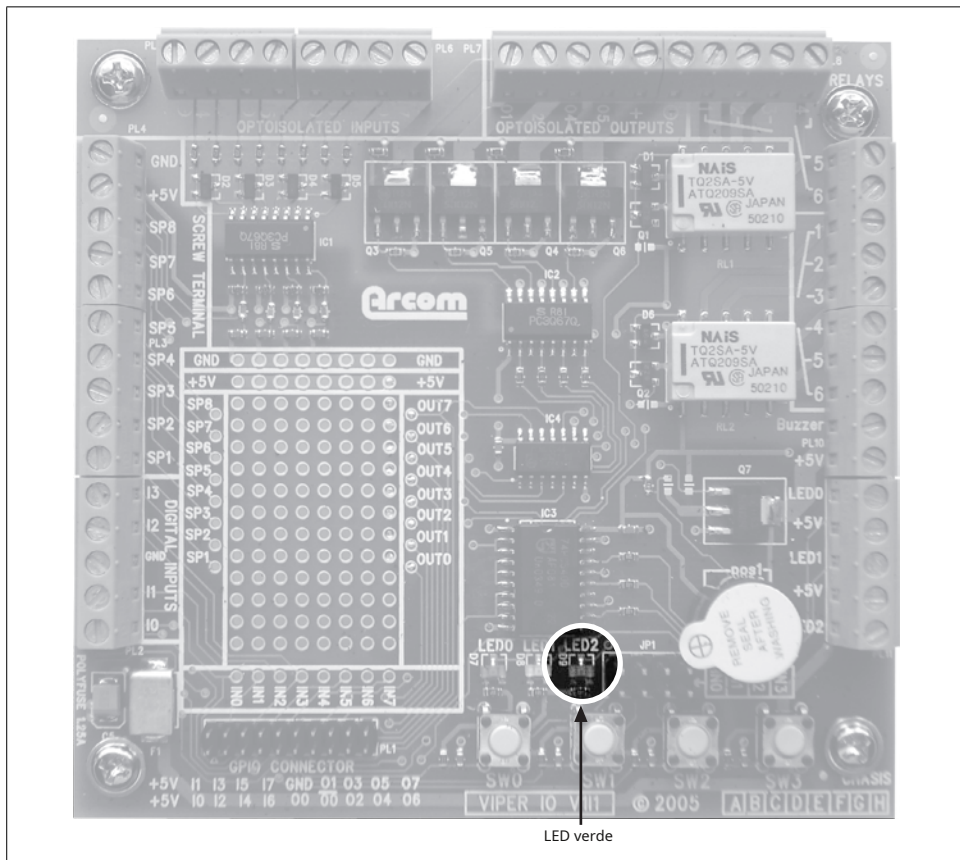


Figura 3-1. Módulo adicional da placa Arcom contendo o LED verde

LED2 é controlado pelo sinal OUT2, conforme descrito na seção de LEDs da placa Arcom *Manual Técnico VIPER-I / O*. Este texto também nos informa que os sinais para os LEDs estão invertidos; portanto, quando a saída é alta, os LEDs ficam desligados e vice-versa. A seção de E / S de uso geral do *Manual Técnico VIPER*

mostra que o sinal OUT2 é controlado pelo pino 22 GPIO do processador. Portanto, precisaremos ser capazes de definir o pino 22 GPIO alternadamente alto e baixo para que nosso programa intermitente funcione corretamente.

A superestrutura do programa de LED piscando é mostrada a seguir. Esta parte do programa é independente do hardware. No entanto, depende das funções dependentes de hardware `ledInit`, `ledToggle`, e `delay_ms` para inicializar o pino GPIO que controla o LED, altere o estado do LED e controle o tempo, respectivamente. Essas funções são descritas nas seções a seguir, onde realmente teremos uma noção de como é fazer programação de sistemas embarcados.

```
# inclui "led.h"

/ *****
*
* Função:          a Principal
*
* Descrição:  Pisca o LED verde uma vez por segundo.
*
* Notas:
*
* Retorna:      Esta rotina contém um loop infinito.
*
***** /
int main (vazio)
{
    /* Configure o pino de controle do LED verde. */
    ledInit ();

    enquanto (1)
    {
        /* Altera o estado do LED verde. */
        ledToggle ();

        /* Pausa por 500 milissegundos. */
        delay_ms (500);
    }

    return 0;
}
```

A função `ledInit`

Antes de começarmos a usar um periférico específico, primeiro precisamos entender o hardware usado para controlar aquele periférico específico. * Como o LED que queremos piscar está conectado a um dos 85 pinos GPIO bidirecionais do processador PXA255, precisamos

* Toda a documentação da placa Arcom está contida no CD-ROM VIPER-Lite Development Kit. Isso inclui fichas técnicas e manuais do usuário para os componentes da placa.

concentre-se naqueles. Frequentemente, como é o caso do processador PXA255, os pinos de E / S dos processadores incorporados têm várias funções. Isso permite que os mesmos pinos sejam usados como E / S controláveis pelo usuário ou para oferecer suporte a uma funcionalidade periférica específica dentro do processador. Os registros de configuração são usados para selecionar como o aplicativo usará cada pino de porta específico.

No PXA255, cada pino de porta pode ser configurado para uso pelo periférico interno (chamado de pino de função alternativa) ou pelo usuário (chamado de pino de uso geral). Para cada pino GPIO, existem vários registros de 32 bits. Esses registros permitem a configuração e o controle de cada pino GPIO. A descrição dos registros para a porta GPIO que contém o pino para o LED verde é mostrada na Tabela 3-1. Esses registros são localizado dentro do chip PXA255.

Tabela 3-1. Registros GPIO PXA255

Nome de registro	Modelo	Endereço	Nome	Propósito
GPLR0	Somente leitura	0x40E00000	GPIO Pin-Level Registro	Reflete o estado de cada pino GPIO. 0 = O estado do pino está baixo. 1 = O estado do pino está alto.
GPDR0	Ler escrever	0x40E0000C	Direção do pino GPIO Registro	Controla se um pino é uma entrada ou saída. 0 = O pino está configurado como uma entrada. 1 = O pino está configurado como uma saída.
GPSR0	Somente escrita	0x40E00018	Saída de pino GPIO Definir registro	Para pinos configurados como saída, o pino é definido como alto escrevendo 1 no bit apropriado neste registro. 0 = O pino não é afetado. 1 = Se configurado como saída, o nível do pino é definido alto.
GPCR0	Somente escrita	0x40E00024	Saída de pino GPIO Registro limpo	Para pinos configurados como saída, o pino é definido como baixo, escrevendo 1 no bit apropriado neste registro. 0 = O pino não é afetado. 1 = Se configurado como saída, o nível do pino é definido como baixo.
GAFR0_U	Ler escrever	0x40E00058	GPIO Alternate Registro de função (Alto)	Configura pinos GPIO para E / S geral ou funcionalidade alternativa. 00 = O pino GPIO é usado como E / S de propósito geral. 01 = O pino GPIO é usado para a função alternativa 1. 10 = O pino GPIO é usado para a função alternativa 2. 11 = O pino GPIO é usado para a função alternativa 3.

uma Você pode ver informações adicionais sobre os pinos GPIO do processador PXA255 no *Manual do desenvolvedor do processador PXA255*.

o *Manual do desenvolvedor do processador PXA255* afirma que a configuração dos pinos GPIO para os LEDs são controlados pelos bits 20 (vermelho), 21 (amarelo) e 22 (verde) no registro GPDR0 de 32 bits. A Figura 3-2 mostra a localização do bit para GPIO pino 22 no registro GPDR0; este bit configura a direção do pino 22 do GPIO que controla o LED verde.

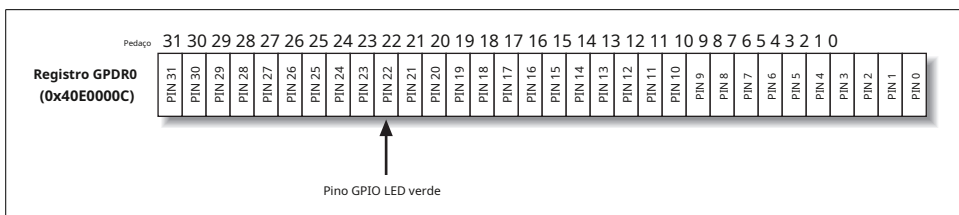


Figura 3-2. Registro GPDR0 do processador PXA255

Os registros de controle periférico PXA255 estão localizados no espaço da memória, conforme mostrado na Figura 2-6 no Capítulo 2. Os endereços desses registros são fornecidos na Tabela 3-1. Como os registradores são mapeados na memória, eles são facilmente acessados em C da mesma maneira que qualquer local da memória é lido ou escrito.

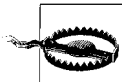


Você pode notar enquanto lê o *Manual do desenvolvedor do processador PXA255* que certos registros contêm bits que são designados como reservados. Isso é típico de muitos registradores em um processador. O manual do processador indicará como esses bits devem ser lidos ou escritos. No caso do processador PXA255, o manual afirma que os bits reservados devem ser escritos como zeros e ignorados quando lidos. É importante que você não use para outros fins quaisquer bits rotulados como reservados.

Acesso de registro de espaço de E / S

Se os registros de pino GPIO estiverem localizados no espaço de E / S, eles podem ser acessados apenas usando a linguagem assembly. As instruções em linguagem de montagem 80x86 para acessar o espaço de E / S são no e Fora. A linguagem C não tem suporte embutido para essas operações. Funções Wrapper C chamadas porto e outport fazem parte de alguns pacotes de biblioteca padrão específicos do 80x86.

A maioria dos registros dentro de uma CPU tem uma configuração padrão após o reset. Isso significa que antes que possamos controlar a saída em qualquer pino de E / S, precisamos ter certeza de que o pino está configurado corretamente. Após a reinicialização, todos os pinos GPIO no PXA255 são configurados como entradas. Além disso, eles funcionam como pinos de E / S de uso geral, em vez de pinos de função alternativa.



Embora os pinos GPIO que controlam os LEDs sejam configurados como pinos de E / S de uso geral na reinicialização, precisamos garantir que o outro software em execução não alterou a funcionalidade desses pinos GPIO.

É uma boa prática sempre inicializar o hardware que você vai usar, mesmo se você achar que o comportamento padrão é adequado.

Em nosso caso, precisamos configurar o pino 22 do GPIO como uma saída via bit 22 no registro GPCR0. Além disso, o pino GPIO que controla o LED verde deve ser configurado para funcionar como um pino de E / S de uso geral por meio do mesmo bit no registro GAFR0_U.

A máscara de bits para o pino GPIO que controla o LED verde na placa Arcom é definida em nosso programa como:

```
# define LED_GREEN (0x00400000)
```

Uma técnica fundamental usada pelo ledInit rotina é uma leitura-modificação-gravação de um registro de hardware. Primeiro, leia o conteúdo do registro, depois modifique o bit que controla o LED e, finalmente, escreva o novo valor de volta na localização do registro. O código em ledInit executa duas operações de leitura-modificação-gravação - uma no registro GAFR0_U e uma no GPCR0, nessa ordem. Essas operações são feitas usando os operadores da linguagem C & = e | =, respectivamente; o efeito de $x \& = y$ é o mesmo de $x = x \& y$. Vamos examinar mais de perto esses operadores e a manipulação de bits no Capítulo 7.

o ledInit A função configura o processador PXA255 na placa Arcom para controlar o LED verde localizado no módulo complementar. No código a seguir, você pode notar que limpamos o pino GPIO no registro GPCR0 para garantir que a tensão de saída no pino GPIO seja primeiro definida como zero, conforme sugerido no manual do desenvolvedor.

```
# define PIN22_FUNC_GENERAL (0xFFFFCFFF)

/ *****
*
* Função:      ledInit
*
* Descrição: Inicialize o pino GPIO que controla o LED.
*
* Notas:      Esta função é específica da placa Arcom.
*
* Retorna:    Nenhum.
*
***** /
void ledInit (void)
{
    / * Desligue a tensão do pino GPIO, o que acenderá o LED. Isto deveria
    * ser feito antes de os pinos serem configurados. * /
    GPIO_0_CLEAR_REG = LED_GREEN;

    / * Certifique-se de que o pino de controle do LED está definido para executar geral
    * funções de propósito. O RedBoot pode ter alterado a operação do pino. * /
    GPIO_0_FUNC_HI_REG &= PIN22_FUNC_GENERAL;

    / * Define o pino de controle do LED para operar como saída. * /
    GPIO_0_DIRECTION_REG |= LED_GREEN;
}
```

A função ledToggle

Essa rotina é executada em um loop infinito e é responsável por alterar o estado do LED. O estado deste LED é controlado gravando no Registro de definição de pino GPIO (GPSR) ou no Registro de limpeza de pino GPIO (GPCR). O registro GPSR0 nos permite definir o nível do pino GPIO de controle de LED alto; o registro GPCR0 nos permite definir o nível do pino GPIO de controle do LED baixo. Escrever no bit 22 desses registros altera a tensão no pino externo e, portanto, o estado do LED verde. Como o pino GPIO para o LED está invertido, quando o bit 22 do registro GPSR0 é definido, o LED verde está desligado, enquanto que quando o bit 22 do registro GPCR0 é definido, o LED verde está ligado. O estado do LED é determinado pelo GPIO Pin Level Register (GPLR).

Conforme descrito anteriormente, o processador PXA255 possui registros separados de somente gravação para configurar (GPSR0) e limpar (GPCR0) o bit que controla o pino GPIO. Portanto, uma leitura-modificação-gravação não pode ser usada para alternar o estado do LED. O algoritmo real do ledToggle A rotina é simples: determine o estado atual do LED de interesse e escreva no registro GPIO o bit que controla aquele LED para definir o novo estado do LED.

```
/ *****
*
* Função:      ledToggle
*
* Descrição:  Alterna o estado de um LED.
*
* Notas:      Esta função é específica da placa Arcom.
*
* Retorna:    Nenhum.
*
***** /
void ledToggle(void)
{
    /* Verifique o estado atual do pino de controle do LED. Então mude o
     * estado em conformidade. */
    if (GPIO_0_LEVEL_REG & LED_GREEN)
        GPIO_0_CLEAR_REG = LED_GREEN;
    outro
        GPIO_0_SET_REG = LED_GREEN;
}
```

A função delay_ms

Também precisamos implementar um atraso de 500 ms entre os alternadores de LED. Fazemos isso esperando ocupados no seguinte delay_ms rotina. Essa rotina aceita a duração do atraso solicitado, em milissegundos, como seu único parâmetro. Em seguida, multiplica esse número pela constante CYCLES_PER_MS para obter o número total de iterações while-loop que são necessárias para atrasar o período de tempo solicitado:

```

/* Número de ciclos de decremento e teste. */
#define CYCLES_PER_MS (9000)

/ *****
*
* Função:      delay_ms
*
* Descrição: Espera ocupado pelo número solicitado de milissegundos.
*
* Notas:      O número de ciclos de decremento e teste por milissegundo
*             foi determinado por tentativa e erro. Este valor depende do
*             tipo de processador, velocidade, compilador e nível de
*             otimização.
*
* Retorna:    Nenhum.
*
***** /
void delay_ms (int milissegundos) {

    ciclos voláteis longos = (milissegundos * CYCLES_PER_MS);

    enquanto (ciclos! = 0)
        ciclos--;
}

```

A constante específica do hardware `CYCLES_PER_MS` representa o número de vezes que o processador pode passar pelo loop `while` em um milissegundo. Para determinar esse número, usamos tentativa e erro. Veremos mais tarde como usar um contador de hardware para obter melhor precisão de tempo.

As quatro funções `main`, `ledInit`, `ledToggle`, e `delay_ms` faça todo o trabalho do programa de LED piscando. Claro, ainda precisamos conversar sobre como construir e executar este programa. Examinaremos esses tópicos nos próximos dois capítulos. Mas, primeiro, temos algo a dizer sobre os loops infinitos e seu papel nos sistemas embarcados.

O papel do loop infinito

Uma das diferenças mais fundamentais entre os programas desenvolvidos para sistemas incorporados e aqueles escritos para outras plataformas de computador é que os programas incorporados quase sempre têm um loop infinito. Normalmente, esse loop envolve uma parte significativa da funcionalidade do programa, como acontece no programa de LED piscando. O loop infinito é necessário porque o trabalho do software embarcado nunca termina. Ele deve ser executado até que o mundo chegue ao fim ou o tabuleiro seja reiniciado, o que acontecer primeiro.

Além disso, a maioria dos sistemas embarcados executa apenas um software. Embora o hardware seja importante, o sistema não é um relógio digital ou um telefone celular ou um forno de micro-ondas sem esse software. Se o software parar de funcionar, o hardware se tornará inútil. Portanto, as partes funcionais de um programa embarcado são quase sempre cercadas por um loop infinito que garante que serão executadas para sempre.

Se tivéssemos esquecido o loop infinito no programa de LED piscando, o LED teria simplesmente mudado de estado uma vez.

Compilando, vinculando e localizando

Considero que a regra de ouro exige que, se gosto de um programa, devo compartilhá-lo com outras pessoas que gostem. Os vendedores de software querem dividir os usuários e conquistar eles, fazendo com que cada usuário concorde em não compartilhar com os outros. Eu me recuso a quebrar a solidariedade com outros usuários em

Por aqui. Não posso, em sã consciência, assinar um contrato de sigilo ou um contrato de licença de software. Para poder continuar a usar computadores sem desonra, decidi reunir um corpo de software livre suficiente para que eu seja capaz de viver sem nenhum software que não seja gratuito.

- Richard Stallman, fundador do Projeto GNU
O Manifesto GNU

Neste capítulo, examinaremos as etapas envolvidas na preparação de seu software para execução em um sistema embarcado. Também discutiremos as ferramentas de desenvolvimento associadas e veremos como construir o programa de LED piscando mostrado no Capítulo 3.

Mas antes de começarmos, queremos deixar claro que a programação de sistemas embarcados não é substancialmente diferente da programação que você fez antes. A única coisa que realmente mudou é que você precisa entender a plataforma de hardware de destino. Além disso, cada plataforma de hardware de destino é única - por exemplo, o método de comunicação por meio de uma interface serial pode variar de processador para processador e de plataforma para plataforma. Infelizmente, essa singularidade entre as plataformas de hardware leva a uma grande complexidade de software adicional e é também a razão pela qual você precisará estar mais ciente do processo de construção de software do que nunca.

Nós nos concentramos no uso de ferramentas de software de código aberto nesta edição do livro. É maravilhoso que os desenvolvedores de software tenham sistemas operacionais poderosos e ferramentas totalmente gratuitas e disponíveis para exploração e alteração. As soluções de código aberto são muito populares e oferecem forte concorrência para suas contrapartes comerciais.

O processo de construção

Quando as ferramentas de construção são executadas no mesmo sistema que o programa que produzem, elas podem fazer muitas suposições sobre o sistema. Isso normalmente não é o caso no desenvolvimento de software embutido, onde as ferramentas de compilação são executadas em um *hospedeiro* computador que difere do *alvo* plataforma de hardware. Existem muitas coisas que as ferramentas de desenvolvimento de software podem fazer automaticamente quando a plataforma de destino está bem definida. * Essa automação é possível porque as ferramentas podem explorar recursos do hardware e do sistema operacional nos quais o programa será executado. Por exemplo, se todos os seus programas forem executados em PCs compatíveis com IBM que executam o Windows, seu compilador pode automatizar - e, portanto, ocultar da sua visão - certos aspectos do processo de construção do software. As ferramentas de desenvolvimento de software embarcado, por outro lado, raramente podem fazer suposições sobre a plataforma de destino. Em vez disso, o usuário deve fornecer um pouco de seu próprio conhecimento do sistema às ferramentas, dando-lhes instruções mais explícitas.

O processo de conversão da representação do código-fonte do seu software incorporado em uma imagem binária executável envolve três etapas distintas:

1. Cada um dos arquivos de origem deve ser compilado ou montado em um arquivo de objeto.
2. Todos os arquivos-objeto resultantes da primeira etapa devem ser vinculados para produzir um único arquivo-objeto, denominado programa relocável.
3. Os endereços de memória física devem ser atribuídos aos deslocamentos relativos dentro do programa relocável em um processo denominado relocação.

O resultado da etapa final é um arquivo contendo uma imagem binária executável que está pronta para ser executada no sistema embarcado.

O processo de desenvolvimento de software embarcado que acabamos de descrever é ilustrado na Figura 4-1. Nesta figura, as três etapas são mostradas de cima para baixo, com as ferramentas que executam as etapas mostradas em caixas com cantos arredondados. Cada uma dessas ferramentas de desenvolvimento pega um ou mais arquivos como entrada e produz um único arquivo de saída. Informações mais específicas sobre essas ferramentas e os arquivos que elas produzem são fornecidas nas seções a seguir.

Cada uma das etapas do processo de construção do software embarcado é uma transformação realizada pelo software executado em um computador de uso geral. Para distinguir este computador de desenvolvimento (geralmente um PC ou estação de trabalho Unix) do sistema embarcado de destino, é referido como o computador host. O compilador, montador, vinculador e localizador são executados em um computador host, e não no próprio sistema embarcado. Ainda,

* Usado dessa forma, o termo "plataforma de destino" é mais bem compreendido para incluir não apenas o hardware, mas também o sistema operacional que forma o ambiente de tempo de execução básico para o seu software. Se nenhum sistema operacional estiver presente, como às vezes é o caso em um sistema embarcado, a plataforma de destino é simplesmente o processador no qual seu programa é executado.

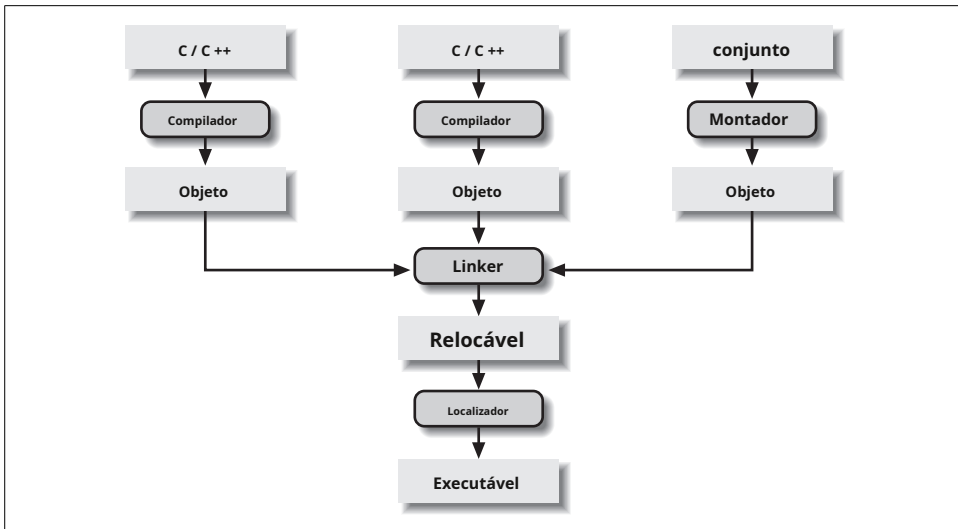


Figura 4-1. O processo de desenvolvimento de software embarcado

essas ferramentas combinam seus esforços para produzir uma imagem binária executável que será executada corretamente apenas no sistema embarcado de destino. Esta divisão de responsabilidades é mostrado na Figura 4-2.

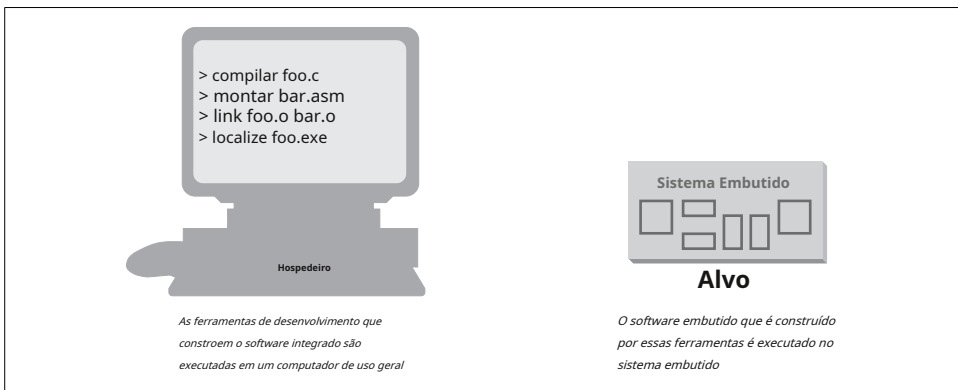


Figura 4-2. A divisão entre host e destino

Neste livro, usaremos as ferramentas GNU (compilador, montador, vinculador e depurador) para nossos exemplos. Essas ferramentas são extremamente populares entre os desenvolvedores de software embarcado porque estão disponíveis gratuitamente (até mesmo o código-fonte é gratuito) e oferecem suporte a muitos dos processadores embutidos mais populares. Usaremos recursos dessas ferramentas específicas como ilustrações para os conceitos gerais discutidos. Uma vez compreendidos, esses mesmos conceitos básicos podem ser aplicados a qualquer ferramenta de desenvolvimento equivalente. Os manuais para todas as ferramentas de desenvolvimento de software GNU podem ser encontrados online em <http://www.gnu.org/manual>.

Compilando

O trabalho de um *compilador* é principalmente para traduzir programas escritos em alguma linguagem legível por humanos em um conjunto equivalente de opcodes para um processador específico. Nesse sentido, um *montador* também é um compilador (você pode chamá-lo de “compilador de linguagem assembly”), mas que executa uma tradução um-para-um muito mais simples de uma linha de mnemônicos legíveis para o opcode equivalente. Tudo nesta seção se aplica igualmente a compiladores e montadores. Juntas, essas ferramentas constituem a primeira etapa do processo de construção do software embarcado.

Obviamente, cada processador possui sua própria linguagem de máquina exclusiva, portanto, você precisa escolher um compilador que produza programas para seu processador de destino específico. No caso de sistemas embarcados, esse compilador quase sempre é executado no computador host. Simplesmente não faz sentido executar o compilador no próprio sistema embarcado. Um compilador como este - que roda em uma plataforma de computador e produz código para outra - é chamado de *compilador cruzado*. O uso de um compilador cruzado é um dos recursos definidores do desenvolvimento de software embarcado.

O compilador GNU C (*gcc*) e montador (*Como*) podem ser configurados como compiladores nativos ou compiladores cruzados. Essas ferramentas oferecem suporte a um conjunto impressionante de combinações de host-destino. O *gcc* O compilador será executado em todos os sistemas operacionais PC e Mac comuns. O suporte ao processador de destino é extenso, incluindo AVR, Intel x86, MIPS, PowerPC, ARM e SPARC. Informações adicionais sobre *gcc* pode ser encontrado online no <http://gcc.gnu.org>.

Independentemente da linguagem de entrada (C, C ++, assembly ou qualquer outra), a saída do compilador cruzado será um arquivo de objeto. Este é um arquivo binário especialmente formatado que contém o conjunto de instruções e dados resultantes do processo de tradução do idioma. Embora partes desse arquivo contenham código executável, o arquivo-objeto não pode ser executado diretamente. Na verdade, a estrutura interna de um arquivo de objeto enfatiza a incompletude do programa maior.

O conteúdo de um arquivo de objeto pode ser considerado uma estrutura de dados muito grande e flexível. A estrutura do arquivo é geralmente definida por um formato padrão, como o Common Object File Format (COFF) ou o Executable and Linkable Format (ELF). Se você estiver usando mais de um compilador (ou seja, estará escrevendo partes do seu programa em diferentes linguagens de origem), você precisa se certificar de que cada compilador é capaz de produzir arquivos-objeto no mesmo formato; *gcc* suporta ambos os formatos de arquivo mencionados anteriormente. Embora muitos compiladores (particularmente aqueles executados em plataformas Unix) suportem formatos de arquivo de objeto padrão, como COFF e ELF, alguns outros produzem arquivos de objeto apenas em formatos proprietários. Se você estiver usando um dos compiladores do último grupo, poderá descobrir que precisa obter todas as outras ferramentas de desenvolvimento do mesmo fornecedor.

A maioria dos arquivos de objeto começa com um cabeçalho que descreve as seções a seguir. Cada uma dessas seções contém um ou mais blocos de código ou dados originados no

arquivo de origem que você criou. No entanto, o compilador reagrupou esses blocos em seções relacionadas. Por exemplo, em *gcc* todos os blocos de código são coletados em uma seção chamada *texto*, variáveis globais inicializadas (e seus valores iniciais) em uma seção chamada *dados*, e variáveis globais não inicializadas em uma seção chamada *bss*.

Geralmente, também há uma tabela de símbolos em algum lugar do arquivo de objeto que contém os nomes e localizações de todas as variáveis e funções referenciadas no arquivo de origem. Partes desta tabela podem estar incompletas, entretanto, porque nem todas as variáveis e funções são sempre definidas no mesmo arquivo. Estes são os símbolos que se referem a variáveis e funções definidas em outros arquivos de origem. É cabe ao vinculador resolver essas referências não resolvidas.

Linking

Todos os arquivos-objeto resultantes da compilação na etapa um devem ser combinados. Os próprios arquivos-objeto são individualmente incompletos, principalmente porque algumas das variáveis internas e referências de função ainda não foram resolvidas. O trabalho do vinculador é combinar esses arquivos-objeto e, no processo, resolver todos os símbolos não resolvidos.

A saída do vinculador é um novo arquivo de objeto que contém todo o código e dados dos arquivos de objeto de entrada e está no mesmo formato de arquivo de objeto. Ele faz isso mesclando o *texto*, *dados*, e *bss* seções dos arquivos de entrada. Quando a execução do vinculador for concluída, todo o código de linguagem de máquina de todos os arquivos de objeto de entrada estará no *texto* seção do novo arquivo, e todas as variáveis inicializadas e não inicializadas residirão no novo *dados* e *bss* seções, respectivamente.

Enquanto o vinculador está no processo de mesclar o conteúdo da seção, ele também está à procura de símbolos não resolvidos. Por exemplo, se um arquivo de objeto contém uma referência não resolvida a uma variável chamada *foo*, e uma variável com o mesmo nome é declarada em um dos outros arquivos de objeto, o vinculador irá combiná-los. A referência não resolvida será substituída por uma referência à variável real. Por exemplo, se *foo* está localizado no deslocamento 14 da seção de dados de saída, sua entrada na tabela de símbolos agora conterá esse endereço.

O vinculador GNU (*ld*) roda em todas as mesmas plataformas host que o compilador GNU. É uma ferramenta de linha de comando que leva os nomes de todos os arquivos-objeto, e possivelmente bibliotecas, para serem vinculados como argumentos. Com o software incorporado, um arquivo de objeto especial que contém o código de inicialização compilado, que é abordado posteriormente nesta seção, também deve ser incluído nesta lista. O vinculador GNU também possui uma linguagem de script que pode ser usada para exercer um controle mais rígido sobre o arquivo-objeto que é gerado.

Se o mesmo símbolo for declarado em mais de um arquivo de objeto, o vinculador não poderá prosseguir. Provavelmente reclamará com o programador (exibindo uma mensagem de erro) e sairá.

Por outro lado, se uma referência de símbolo permanecer não resolvida depois que todos os arquivos de objeto tiverem sido mesclados, o vinculador tentará resolver a referência por conta própria. A referência pode ser a uma função, como `memcpy`, `strlen`, ou `malloc`, isso faz parte da biblioteca C padrão, portanto, o vinculador abrirá cada uma das bibliotecas descritas a ele na linha de comando (na ordem fornecida) e examinará suas tabelas de símbolos. Se o vinculador descobrir uma função ou variável com esse nome, a referência será resolvida incluindo o código associado e as seções de dados dentro do arquivo de objeto de saída. * Observe que o vinculador GNU usa *ligação seletiva*, que mantém outras funções não referenciadas fora da imagem de saída do vinculador.

Infelizmente, as rotinas de biblioteca padrão geralmente requerem algumas mudanças antes de serem usadas em um programa embutido. Um problema é que as bibliotecas padrão fornecidas com a maioria dos conjuntos de ferramentas de desenvolvimento de software chegam apenas na forma de objeto. Você raramente tem acesso ao código-fonte da biblioteca para fazer as alterações necessárias. Felizmente, uma empresa chamada Cygnus (que agora faz parte da Red Hat) criou uma versão freeware da biblioteca C padrão para uso em sistemas embarcados. Este pacote é chamado *newlib*. Você só precisa baixar o código-fonte para esta biblioteca da Web (atualmente localizado em <http://sourceware.org/newlib>), implementar algumas funções específicas de destino e compilar o lote inteiro. A biblioteca pode então ser vinculada ao seu software embutido para resolver quaisquer chamadas de biblioteca padrão não resolvidas anteriormente.

Depois de mesclar todas as seções de código e dados e resolver todas as referências de símbolo, o vinculador produz um arquivo de objeto que é uma cópia especial “relocável” do programa. Em outras palavras, o programa está completo, exceto por uma coisa: nenhum endereço de memória foi ainda atribuído ao código e às seções de dados dentro dele. Se você não estava trabalhando em um sistema embarcado, você teria terminado de construir seu software agora.

Mas os programadores incorporados nem sempre concluem o processo de construção neste ponto. Os endereços dos símbolos no processo de vinculação são relativos. Mesmo que seu sistema embarcado inclua um sistema operacional, você provavelmente ainda precisará de uma imagem binária absolutamente localizada. Na verdade, se houver um sistema operacional, o código e os dados em que ele consiste provavelmente também estarão no programa realocável. Todo o aplicativo incorporado - incluindo o sistema operacional - é frequentemente vinculado estaticamente e executado como uma única imagem binária.

Código de inicialização

Uma das coisas que as ferramentas tradicionais de desenvolvimento de software fazem automaticamente é inserir *código de inicialização*: um pequeno bloco de código em linguagem assembly que prepara o caminho para a execução de software escrito em uma linguagem de alto nível. Cada linguagem de alto nível tem seu próprio conjunto de expectativas sobre o ambiente de tempo de execução. Por exemplo,

* Estamos falando apenas de links estáticos aqui. Quando a vinculação dinâmica de bibliotecas é usada, o código e os dados associados à rotina da biblioteca não são inseridos diretamente no programa.

programas escritos em C usam uma pilha. O espaço para a pilha deve ser alocado antes que o software escrito em C possa ser executado adequadamente. Essa é apenas uma das responsabilidades atribuídas ao código de inicialização para programas C.

A maioria dos compiladores cruzados para sistemas embarcados inclui um arquivo de linguagem assembly chamado *startup.asm*, *crt0.s* (abreviação de tempo de execução C) ou algo semelhante. A localização e o conteúdo deste arquivo são geralmente descritos na documentação fornecida com o compilador.

O código de inicialização para programas C geralmente consiste na seguinte série de ações:

1. Desative todas as interrupções.
2. Copie todos os dados inicializados da ROM para a RAM.
3. Zere a área de dados não inicializada.
- 4. Aloque espaço para e inicialize a pilha.**
5. Inicialize o ponteiro da pilha do processador.
6. Ligue a Principal.

Normalmente, o código de inicialização também inclui algumas instruções após a chamada para a Principal. Essas instruções serão executadas apenas no caso de o programa de linguagem de alto nível sair (ou seja, a chamada para a Principal retorna). Dependendo da natureza do sistema embarcado, você pode querer usar essas instruções para parar o processador, redefinir todo o sistema ou transferir o controle para uma ferramenta de depuração.

Como o código de inicialização geralmente não é inserido automaticamente, o programador geralmente deve montá-lo sozinho e incluir o arquivo objeto resultante na lista de arquivos de entrada para o vinculador. Ele pode até precisar dar ao vinculador uma opção de linha de comando especial para impedi-lo de inserir o código de inicialização normal. O código de inicialização de trabalho para uma variedade de processadores alvo pode ser encontrado em um pacote GNU chamado *libgloss*.

Localizando

A ferramenta que realiza a conversão de programa relocável em imagem binária executável é chamada de *localizador*. Ele assume a responsabilidade pela etapa mais fácil do processo de construção. Na verdade, você mesmo deve fazer a maior parte do trabalho nesta etapa, fornecendo informações sobre a memória na placa de destino como entrada para o localizador. O localizador usa essas informações para atribuir endereços de memória física a cada uma das seções de código e dados dentro do programa relocável. Em seguida, ele produz um arquivo de saída que contém uma imagem de memória binária que pode ser carregada no destino.

Esteja você escrevendo um software para um computador de uso geral ou um sistema embarcado, em algum ponto as seções de seu programa relocável devem receber endereços reais. Às vezes, o software que já está no destino faz isso por você, como o RedBoot faz na placa Arcom.

Em alguns casos, existe uma ferramenta de desenvolvimento separada, chamada de *localizador*, para atribuir endereços. No entanto, no caso das ferramentas GNU, esse recurso é integrado ao vinculador (*ld*).

Monitores de depuração

Em alguns casos, um *monitor de depuração* (ou *Monitor ROM*) é o primeiro código executado quando a placa é ligada. No caso da placa Arcom, existe um monitor de depuração chamado RedBoot. uma RedBoot, cujo nome é um acrônimo para o programa Embedded Debug and Bootstrap da RedHat, é um monitor de depuração que pode ser usado para fazer download de software, executar operações de memória básica e gerenciar memória não volátil. Este software na placa Arcom contém o código de inicialização e executa as tarefas listadas anteriormente para inicializar o hardware para um estado conhecido. Por causa disso, os programas baixados para rodar no RAMvia RedBoot não precisam estar vinculados ao código de inicialização e devem ser vinculados, mas não localizados.

Depois que o hardware foi inicializado, o RedBoot envia um prompt para uma porta serial e espera pela entrada do usuário (você) para dizer o que fazer. O RedBoot oferece suporte a comandos para carregar software, despejar memória e realizar várias outras tarefas. Vamos dar uma olhada em como usar o RedBoot para carregar um programa de software no próximo capítulo.

uma. Informações adicionais sobre o RedBoot podem ser encontradas online em <http://ecos.sourceware.org/redboot>. O Guia do Usuário do RedBoot também está localizado neste site. Uma descrição do procedimento de inicialização do RedBoot está contida no livro *Desenvolvimento de software integrado com eCos*, por Anthony Massa (Prentice Hall PTR).

As informações de memória exigidas pelo vinculador GNU podem ser passadas para ele na forma de um *script de vinculador*. Esses scripts às vezes são usados para controlar a ordem exata das seções de código e dados dentro do programa relocável. Mas aqui, queremos fazer mais do que apenas controlar a ordem; também queremos estabelecer a localização física de cada seção na memória.

O que se segue é um exemplo de um script de vinculador para a placa Arcom. Este arquivo de script de vinculador é usado para construir o programa de LED piscando abordado no Capítulo 3:

```
ENTRADA (principal)

MEMÓRIA
{
    ram: ORIGIN = 0x00400000, LENGTH = 64M
    rom: ORIGIN = 0x60000000, LENGTH = 16M
}

SEÇÕES
{
    dados :                               /* Dados inicializados. */
    {
        _DataStart = ;
        * (.dados)
        _DataEnd   = ;
    }
}
```

```

}> ram

bss:                                /* Dados não inicializados. */
{
    _BssStart = . ;
    * (.bss)
    _BssEnd   = ;
}> ram

texto :                             /* As instruções reais. */
{
    * (.texto)
}> ram
}

```

Este script informa o localizador embutido do vinculador GNU sobre a memória na placa de destino, que contém 64 MB de RAM e 16 MB de ROM flash. * O arquivo de script do vinculador instrui o vinculador GNU a localizar o dados, bss, e texto seções na RAM começando no endereço 0x00400000. A primeira instrução executável é designada com o ENTRADA comando, que aparece na primeira linha do exemplo anterior. Neste caso, o ponto de entrada é a função a Principal.

Nomes no arquivo de comando do vinculador que começam com um sublinhado (por exemplo, `_DataStart`) pode ser referenciado de forma semelhante a variáveis comuns de dentro de seu código-fonte. O vinculador usará esses símbolos para resolver referências nos arquivos de objeto de entrada. Assim, por exemplo, pode haver uma parte do software embutido (geralmente dentro do código de inicialização) que copia os valores iniciais das variáveis inicializadas de ROM para o dados seção na RAM. Os endereços de início e parada para esta operação podem ser estabelecidos simbolicamente, referindo-se aos endereços como `_DataStart` e `_DataEnd`.

Um script de vinculador também pode usar vários comandos para direcionar o vinculador a executar outras operações. Informações e opções adicionais para arquivos de script do vinculador GNU podem ser Encontrado em <http://www.gnu.org>.

A saída desta etapa final do processo de construção é uma imagem binária contendo endereços físicos para o sistema embarcado específico. Esta imagem binária executável pode ser baixada para o sistema embarcado ou programada em um chip de memória. Você verá como baixar e executar essas imagens de memória no próximo capítulo.

Construindo o Programa de LED Piscando

Nesta seção, mostramos um exemplo de procedimento de construção para a placa de desenvolvimento Arcom VIPER-Lite. Se outra plataforma de hardware for usada, um processo semelhante deve ser seguido usando as ferramentas e convenções que acompanham esse hardware.

* Também existe uma versão do Arcomboard que contém 32 MB de flash. Se você tiver esta versão do, altere o arquivo de script do vinculador da seguinte maneira:

```
rom: ORIGIN = 0x60000000, LENGTH = 32M
```

O procedimento de instalação para as ferramentas de desenvolvimento de software é fornecido no Apêndice B. Depois que as ferramentas são instaladas, os comandos cobertos nas seções a seguir são inseridos em um shell de comando. Para usuários do Windows, o shell de comando é um shell bash Cygwin (Cygwin é um ambiente Unix para Windows); para usuários do Linux, é um shell de comando regular.



Neste e nos capítulos subsequentes, os comandos inseridos em um ambiente de shell são indicados pelo sinal de número (#). Os comandos inseridos no ambiente RedBoot são indicados pelo prompt RedBoot (RedBoot>).

A seguir, daremos uma olhada nos comandos individuais para realizar manualmente as três tarefas separadas (compilar, vincular e localizar) descritas anteriormente neste capítulo. Então, aprenderemos como automatizar o procedimento de construção com makefiles.

Compilar

Como o implementamos, o exemplo de LED piscando consiste em dois módulos de origem: *led.c* e *blink.c*. A primeira etapa do processo de construção é compilar esses dois arquivos. A estrutura básica para o *gcc* o comando do compilador é:

```
arm-elf-gcc [ opções] arquivo ...
```

As opções de linha de comando de que precisaremos são:

- g

Para gerar informações de depuração no formato padrão

- C

Para compilar e montar, mas não vincular

- Muro

Para habilitar a maioria das mensagens de aviso

- Eu ../ inclui

Para olhar no diretório *incluir* para arquivos de cabeçalho

Aqui estão os comandos reais para compilar os arquivos de origem C:

```
# arm-elf-gcc -g -c -Wall -I ../ inclui led.c  
# arm-elf-gcc -g -c -Wall -I ../ inclui blink.c
```

Dividimos a etapa de compilação em dois comandos separados, mas você pode compilar os dois arquivos com um comando. Para usar um único comando, basta colocar os dois arquivos de origem após as opções. Se você quiser opções diferentes para um dos arquivos de origem, precisará compilá-lo separadamente, conforme mostrado. Para obter informações adicionais sobre as opções do compilador, dê uma olhada em <http://gcc.gnu.org>.

Executar esses comandos será uma boa maneira de verificar se as ferramentas foram configuradas corretamente. O resultado de cada um desses comandos é a criação de um arquivo-objeto que possui o mesmo prefixo de *.c* arquivo e a extensão *.o*. Então, se tudo correr bem, agora haverá dois arquivos adicionais— *led.o* e *blink.o* —No diretório de trabalho. O procedimento de compilação é mostrado na Figura 4-3.

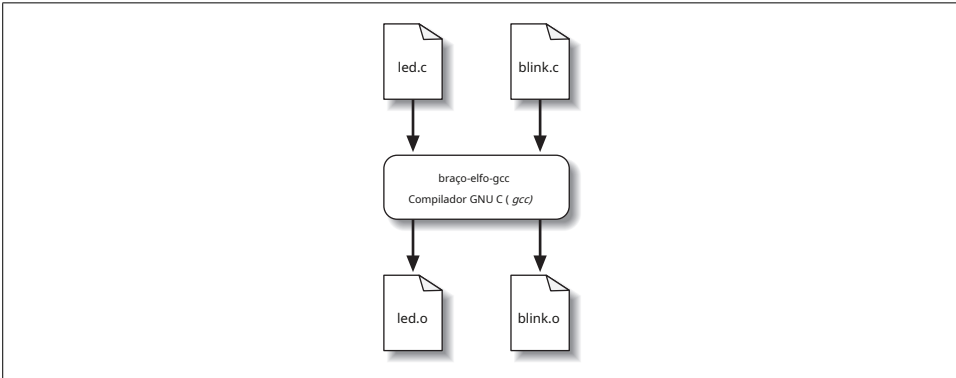


Figura 4-3. Compilando o programa de LED piscando

Ligar e localizar

Agora temos os dois arquivos de objeto— *led.o* e *blink.o* —Que precisamos para realizar a segunda etapa no processo de compilação. Como discutimos anteriormente, o vinculador GNU realiza a vinculação e localização dos arquivos-objeto.

Para a terceira etapa, localização, há um arquivo de script de vinculador denominado *viperlite.ld* que nós contribuimos para *ld* para estabelecer a localização de cada seção na memória da placa Arcom. A estrutura básica para o linker e localizador *ld* o comando é:

arm-elf-ld [opções] arquivo ...

As opções de linha de comando de que precisaremos nesta etapa são:

- **Mapa *blink.map***

Para gerar um arquivo de mapa e usar o nome de arquivo fornecido

- **T *viperlite.ld***

Para ler o script do linker

- **N**

Para definir as seções de texto e dados para serem legíveis e graváveis

- **o *blink.exe***

Para definir o nome do arquivo de saída (se esta opção não estiver incluída, *ld* usará o padrão

nome do arquivo de saída *a.out*)

O comando real para vincular e localizar é:

arm-elf-ld -Map blink.map -T viperlite.ld -N -o blink.exe led.o blink.o

A ordem dos arquivos de objeto determina sua colocação na memória. Como não estamos vinculando nenhum código de inicialização, a ordem dos arquivos-objeto é irrelevante. Se o código de inicialização foi incluído, você deseja que o arquivo de objeto seja localizado no endereço adequado. O arquivo de script do vinculador pode ser usado para especificar onde você deseja que a rotina de inicialização (e outro código) resida na memória. Além disso, você também pode usar o arquivo de script do vinculador para especificar endereços exatos para códigos ou dados, caso ache necessário fazê-lo.

Como você pode ver neste comando, os dois arquivos de objeto— *led.o* e *blink.o*— são os últimos argumentos na linha de comando para vinculação. O arquivo de script do vinculador, *viperlite.ld*, também é passado para localizar os dados e o código na memória da placa Arcom. O resultado deste comando é a criação de dois arquivos— *blink.map* e *blink.exe*—No diretório de trabalho. O procedimento de vinculação e localização é mostrado na Figura 4-4.

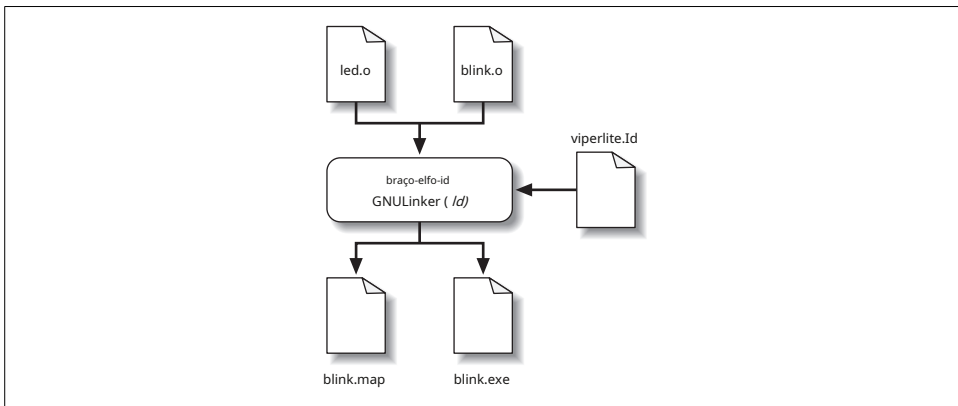


Figura 4-4. Vinculando e localizando o programa de LED piscando

O *.map* arquivo fornece uma lista completa de todos os endereços de código e dados para a imagem final do software. Se você nunca viu esse arquivo de mapa antes, certifique-se de dar uma olhada nele antes de continuar lendo. Ele fornece informações semelhantes ao conteúdo do script do vinculador descrito anteriormente. No entanto, esses são resultados em vez de instruções e, portanto, incluem os comprimentos reais das seções e os nomes e localizações dos símbolos públicos encontrados no programa realocável. Veremos mais tarde como este arquivo pode ser usado como um auxiliar de depuração.

Formate o arquivo de saída

A última etapa da seção anterior cria uma imagem do programa de LED piscando que podemos carregar na placa Arcom. Em certos casos, pode ser necessário formatar a imagem do procedimento de construção para sua plataforma de destino específica.

Uma ferramenta incluída com o conjunto de ferramentas GNU que pode ajudar na formatação de imagens é o *faixa* utilitário, que faz parte do pacote de utilitários binários chamado *binutils* (pronunciado

Outro Método de Vinculação

Você pode notar que, para exemplos posteriores no livro, *gcc* é invocado durante o processo de vinculação. o *gcc* o compilador invoca o vinculador indiretamente. Quando *gcc* compila certos programas, pode introduzir chamadas para bibliotecas de tempo de execução especiais nos bastidores. Vinculando via *gcc* garante que as versões corretas dessas bibliotecas (chamadas *multilibs*) estão vinculados para a configuração especificada.

Se o vinculador, *ld*, fossem chamados diretamente, o conjunto correto de multilibs também precisaria ser especificado na linha de comando para garantir que a imagem fosse vinculada corretamente. Para evitar isso, vamos usar *gcc* para invocar o vinculador.

"Bin-you-tills"). o *faixa* utilitário pode remover seções específicas de um arquivo de objeto. A estrutura básica de comando para o *faixa* utilitário é:

braço-elfo-tira [*opções*] *arquivo de entrada* ... [-o *arquivo de saída*]

O procedimento de construção para os capítulos subsequentes do livro gera dois arquivos executáveis: um com informações de depuração e outro sem. O executável que contém as informações de depuração inclui *dbg* em seu nome de arquivo. A imagem de depuração deve ser usada com *gdb*. Se uma imagem for baixada com o RedBoot, a imagem sem depuração deve ser usada.

O comando usado para retirar as informações do símbolo é:

arm-elf-strip --remove-section = .comment blinkdbg.exe -o blink.exe

Isso remove a seção chamada. Comente da imagem *blinkdbg.exe* e cria o novo arquivo de saída *blink.exe*.

Pode haver outro momento em que você precise de um arquivo de imagem que pode ser gravado em ROM ou flash. O conjunto de ferramentas GNU tem exatamente o que você precisa para esta tarefa. A utilidade *objcopy* (cópia de objeto) é capaz de copiar o conteúdo de um arquivo de objeto em outro arquivo de objeto. A estrutura básica para o *objcopy* utilitário é:

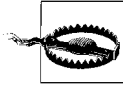
braço-elfo-objcopy [*opções*] *arquivo de entrada* [*arquivo de saída*]

Por exemplo, vamos supor que queremos converter nosso programa de LED piscando do formato ELF em um arquivo Intel Hex Format. * A linha de comando que usamos para isso é:

arm-elf-objcopy -O ihex blink.exe blink.hex

Este comando usa o *-O ihex* opção para gerar um arquivo Intel Hex Format. O arquivo de entrada é *blink.exe* (a *objcopy* utilitário determina o tipo de arquivo de entrada). Finalmente, o arquivo de saída é nomeado *blink.hex*.

* O formato Intel Hex é um formato de arquivo ASCII desenvolvido pela Intel para armazenar e baixar imagens binárias.



Se nenhum nome de arquivo de saída for fornecido, o *faixa* e *objcopia* utilitários sobrescrevem o arquivo de entrada original com o arquivo gerado.

Algumas das outras ferramentas GNU são úteis para fornecer outras informações sobre a imagem que você construiu. Por exemplo, o *Tamanho* utilitário, que faz parte do *binutils* pacote, lista os tamanhos de seção e tamanho total para um determinado arquivo de objeto. Aqui está o comando para usar o utilitário de tamanho:

```
# arm-elf-size blink.exe
```

A saída resultante é:

texto	dados	bss	dezembro	nome de arquivo hexadecimal
328	0	0	328	148 blink.exe

A linha superior consiste em títulos de coluna e mostra as seções texto, dados, e bss. O programa de LED piscando contém 328 bytes no texto seção, nenhum byte no dados seção, e nenhum byte no bss seção. o dezembro coluna mostra o tamanho total da imagem em decimal, e o hex coluna mostra em hexadecimal (decimal 328 = hexadecimal 0x148). Esses tamanhos totais estão em bytes. A última coluna, nome do arquivo, contém o nome do arquivo do arquivo objeto.

Você notará que o tamanho da seção, 328 bytes, é muito menor do que o tamanho de arquivo de aproximadamente 3 KB de nosso *blink.exe*. Isso ocorre porque as informações de depuração também estão localizadas no *blink.exe* Arquivo.

Informações adicionais sobre o outro GNU *binutils* pode ser encontrado online em <http://www.gnu.org>.

Agora estamos prontos para baixar o programa para nossa placa de desenvolvimento, o que faremos no próximo capítulo. Para encerrar nossa discussão sobre a construção de programas, vamos dar uma olhada rápida em outra ferramenta útil no processo de construção.

Uma rápida olhada nos arquivos Make

Você pode imaginar como o processo de construção poderia ser tedioso se você tivesse um grande número de arquivos de código-fonte para um projeto específico. Inserir manualmente os comandos individuais do compilador e do vinculador na linha de comando torna-se cansativo muito rapidamente. Para evitar isso, um *makefile* pode ser usado. Um makefile é um script que diz ao *faço* utilitário como construir um programa específico. (O *faço* utilitário normalmente é instalado com as outras ferramentas GNU.) *faço* O utilitário segue as regras do makefile para gerar automaticamente arquivos de saída a partir de um conjunto de arquivos de origem de entrada.

Makefiles pode ser um pouco difícil de configurar, mas eles podem economizar muito tempo e uma ferramenta muito poderosa ao construir arquivos de projeto repetidamente (e continuamente) novamente. Ter uma amostra disponível pode reduzir a dor de configurar um makefile.

O layout básico para uma regra de compilação de makefile é:

```
alvo:      pré-requisito
          comando
```

o alvo é o que será construído, o pré-requisito é um arquivo que deve existir antes do alvo pode ser criado, e o comando é um comando shell usado para criar o alvo. Pode haver vários pré-requisitos na linha de destino (separada por um espaço em branco) e / ou várias linhas de comando. Mas certifique-se de colocar uma tabulação, não espaços, no início de cada linha contendo um comando.

Aqui está um makefile para construir nosso programa de LED piscando:

```
XCC      = arm-elf-gcc
LD       = arm-elf-ld
CFLAGS  = -g -c -Wall \
          -Eu ../includo
LDFLAGS = -Map blink.map -T viperlite.ld -N

todos: blink.exe

led.o: led.c led.h
    $(XCC) $(CFLAGS) led.c

blink.o: blink.c led.h
    $(XCC) $(CFLAGS) blink.c

blink.exe: blink.o led.o viperlite.ld
    $(LD) $(LDFLAGS) -o $@ led.o blink.o

limpar:
    - rm -f blink.exe *.o blink.map
```

As primeiras quatro instruções neste makefile contêm variáveis para uso no makefile. Os nomes das variáveis estão no lado esquerdo do sinal de igual. Neste makefile, as respectivas variáveis fazem o seguinte:

XCC
Define o programa executável do compilador

LD
Define o programa executável do vinculador

CFLAGS
Define os sinalizadores para o compilador

LDFLAGS
Define os sinalizadores para o vinculador

As variáveis em um makefile são usadas para eliminar parte da duplicação de texto e também para facilitar a portabilidade. Para usar uma variável no código, a sintaxe \$ () é usada com o nome da variável entre parênteses.

Observe que se uma linha em um makefile ficar muito longa, você pode continuar na linha seguinte usando a barra invertida (\), como mostrado com o CFLAGS variável.

Agora, para as regras de construção. Os alvos de construção neste arquivo são todos, led.o, blink.o, e blink.exe. A menos que você especifique um destino ao invocar o *faço* utilitário, ele procura o primeiro alvo (neste caso, o primeiro alvo é tudo) e tenta construí-lo; isso, por sua vez, pode levá-lo a encontrar e construir outros alvos. o *faço* O utilitário cria (ou recria, conforme o caso) o arquivo de destino se ele não existir ou se os arquivos de pré-requisito forem mais recentes que o arquivo de destino.

Neste ponto, pode ser útil examinar o makefile de baixo para cima. Em ordem para blink.exe para ser criado, blink.o e led.o precisa ser construído conforme mostrado nos pré-requisitos. No entanto, como esses arquivos não existem, o *faço* utilitário precisará criá-los primeiro. Ele pesquisará maneiras de criar esses dois arquivos e os encontrará listados como destinos no makefile. o *faço* O utilitário pode criar esses arquivos porque os pré-requisitos (os arquivos de origem) para esses dois destinos existem.

Porque os alvos led.o e blink.o são tratados da mesma forma, vamos nos concentrar em apenas um deles. Os pré-requisitos para o alvo led.o estão led.c e led.h. Como afirmado acima, o comando diz ao *faço* utilitário como criar o destino. A primeira parte do comando para led.o é uma referência à variável XCC, conforme indicado pela sintaxe \$ (XCC), e a próxima parte do comando é uma referência à variável CFLAGS, conforme indicado pela sintaxe \$ (CFLAGS). o *faço* O utilitário simplesmente substitui as referências de variáveis pelo texto atribuído a elas no makefile. A parte final do comando é o arquivo fonte led.c. Amarrados juntos, esses elementos constroem o comando que o *faço* utilitário é executado. Isso gera um comando na linha de comando do shell da seguinte maneira:

```
arm-elf-gcc -g -c -Wall -I ../ inclui led.c
```

Este é o mesmo comando que inserimos manualmente para compilar o *led.c* arquivo anteriormente neste capítulo, na seção "Construindo o programa de LED piscando". o *faço* compila utilitários *blink.c* do mesmo jeito.

Neste ponto, o *faço* utilitário tem todos os pré-requisitos necessários para gerar o alvo blink.exe alvo padrão. O comando que o *faço* utilitário executa (o mesmo comando que inserimos manualmente para vincular e localizar o programa de LED piscando) para construir blink.exe é:

```
arm-elf-ld -Map blink.map -T viperlite.ld -N -o blink.exe led.o blink.o
```

Você pode notar que neste makefile o vinculador é invocado diretamente. Em vez de, *gcc* poderia ter sido usado para invocar o vinculador indiretamente com a seguinte linha:

```
arm-elf-gcc -Wl, -Map, blink.map -T viperlite.ld -N -o blink.exe led.o blink.o
```

Ao invocar o vinculador indiretamente, a opção especial *-Wl* é usado para que *gcc* passa a solicitação para gerar um arquivo de mapa de vinculador para o vinculador em vez de tentar analisar o próprio argumento. Embora este programa simples de LED piscando não precise se conectar usando

gcc, você deve se lembrar que programas C mais complexos podem precisar de suporte especial de biblioteca de tempo de execução da *gcc* e precisará ser vinculado dessa forma.

A última parte do *makefile* é o alvo *limpar*. No entanto, como não era necessário para o destino padrão, o comando não foi executado.

Para executar as instruções de construção do *makefile*, simplesmente mude para o diretório que contém o *makefile* e digite o comando:

*faço*

o *faço* utilitário irá pesquisar o diretório atual por um arquivo chamado *makefile*. Se o seu *makefile* tiver um nome diferente, você pode especificá-lo na linha de comando após o - *f* opção.

Com o comando anterior, o *faço* utilitário fará o primeiro alvo que encontrar. Você também pode especificar destinos na linha de comando para o *faço* Utilitário. Por exemplo, porque tudo é o destino padrão no *makefile* anterior, você pode usar o seguinte comando com a mesma facilidade:

*faça tudo*

Um alvo chamado *limpar* é normalmente incluído em um *makefile*, com comandos para remover arquivos de objetos antigos e executáveis, a fim de permitir que você crie uma nova construção. A linha de comando para executar o *limpar* alvo é:

*limpar*

Lembre-se de que apresentamos um exemplo muito básico do *faço* utilitário e *makefiles* para um projeto muito básico. o *faço* O utilitário contém ferramentas muito poderosas em seus recursos avançados que podem beneficiá-lo ao executar projetos grandes e mais complexos.



É importante manter o *makefile* atualizado conforme seu projeto muda. Lembre-se de incorporar novos arquivos de origem e manter seus pré-requisitos atualizados. Se os pré-requisitos não forem configurados corretamente, você pode alterar um arquivo de origem específico, mas esse arquivo de origem não será incorporado à construção. Essa situação pode deixá-lo coçando a cabeça.

Informações adicionais sobre o GNU *faço* utilitário pode ser encontrado online em <http://www.gnu.org> bem como no livro *Gerenciando projetos com GNU make*, por Robert Mecklenburg (O'Reilly). Esses recursos lhe darão uma compreensão mais profunda de ambos os *faço* utilitário e *makefiles* e permitem que você use seus recursos mais poderosos.

Baixando e depurando

Lembro-me do exato momento em que percebi que grande parte da minha vida a partir de então seria gasta em encontrar erros em meus próprios programas.

- Maurice Wilkes, Chefe do Laboratório de Computação da Universidade de Cambridge, 1959

Depois de ter uma imagem binária executável armazenada como um arquivo no computador host, você precisará fazer o download dessa imagem para o sistema embarcado e executá-la. A imagem binária executável geralmente é carregada em um dispositivo de memória na placa de destino e executada a partir daí. E se você tiver as ferramentas certas à sua disposição, será possível definir pontos de interrupção no programa ou observar sua execução de maneiras menos intrusivas. Este capítulo descreve várias técnicas de download, execução e depuração de software incorporado em geral, bem como enfoca as técnicas disponíveis em nosso ambiente de desenvolvimento.

Baixando o programa de LED piscando

Com a maioria dos sistemas embarcados, existem vários meios de obter uma imagem no destino e executar o programa, alguns mais desafiadores do que outros. Nesta seção, investigamos os métodos disponíveis para baixar o programa de LED piscando na placa Arcom, bem como alguns outros métodos que podem ser úteis para outros projetos.

O ciclo de desenvolvimento de software para um PC e um sistema embarcado inclui muitos dos mesmos estágios. A Figura 5-1 é um diagrama geral do ciclo de desenvolvimento de software embarcado.

Conforme mostrado na Figura 5-1, o ciclo de desenvolvimento de software começa com o design e a primeira implementação do código. Depois disso, geralmente há iterações dos estágios de construção, download e depuração e correção de bugs. Como muito tempo é gasto nessas três fases, ajuda a eliminar quaisquer torções nesse processo, de forma que a maior parte do tempo possa ser gasta na depuração e teste do software. (Este diagrama não

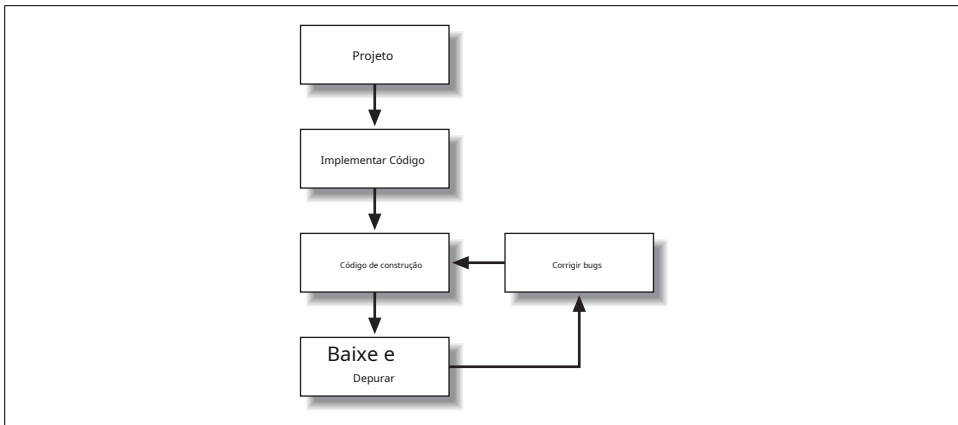


Figura 5-1. Ciclo de desenvolvimento de software

leve em consideração o manuseio do recurso creep inevitavelmente infligido pelo departamento de marketing.)

Por ser um diagrama muito básico, outras etapas que podem ser necessárias são *perfilamento* e *otimização*. A criação de perfil permite que um desenvolvedor determine várias métricas sobre um programa, como onde o processador está gastando a maior parte de seu tempo. Otimização é o processo pelo qual o desenvolvedor tenta eliminar gargalos no software usando várias técnicas, como implementação de código crítico em linguagem assembly. Muitas vezes, as técnicas de otimização são específicas do compilador, do processador ou do sistema.

Outra tarefa nesta fase é *integração*. Depois que o ciclo de desenvolvimento é concluído, o teste em nível de sistema normalmente é feito. E depois que um produto é enviado, o software entra em sua fase de manutenção durante o ciclo de vida do produto, quando o código deve ser suportado e sustentado. As técnicas de depuração mostradas neste capítulo também se aplicam ao estágio de manutenção.

Como o código deve ser testado repetidamente no hardware de destino, um método rápido e direto para carregar o software no destino é o ideal. Vamos dar uma olhada nas técnicas e ferramentas que podemos usar para essa tarefa.

Monitores de depuração

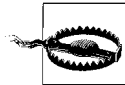
UMA *monitor de depuração*, também chamado de *Monitor ROM*, é um pequeno programa que reside na memória não volátil no hardware de destino que facilita várias operações necessárias durante o desenvolvimento. Uma das tarefas que um monitor de depuração controla é a inicialização básica do hardware. Um monitor de depuração permite que você baixe e execute software na RAM para depurar o programa.

A maioria dos monitores de depuração inclui muitos outros recursos úteis para auxiliar no ciclo de desenvolvimento. Por exemplo, a maioria dos monitores de depuração incorporam algum tipo de *interface da Linha de comando (CLI)* onde os comandos podem ser emitidos para o monitor de depuração para

execução no hardware de destino - por exemplo, via porta serial. Esses comandos incluem download de software, execução do programa, *espreitando* (lendo) e *cutucando* (escrever) registros de memória e processador, comparando ou exibindo blocos de memória e definindo configurações de inicialização para o hardware.

Em alguns sistemas, um monitor de depuração também é incorporado às unidades de produção no campo. Isso pode ser usado para atualizar o firmware para adicionar novos recursos ou corrigir bugs após a unidade ser implantada.

Alguns processadores incluem um programa semelhante a um monitor de depuração na memória do chip. Por exemplo, os DSPs da série TMS320C5000 da Texas Instruments incluem um programa chamado de *bootloader*. Este bootloader é usado para transferir software de uma fonte externa (off-chip) para a memória interna (on-chip), permitindo que o código resida em uma memória mais lenta para armazenamento e seja transferido para uma memória mais rápida antes da execução. O carregador de inicialização determina onde carregar o código com base na configuração do modo de inicialização, que é determinada pela amostragem de pinos de E / S DSP específicos durante a inicialização.



Esteja ciente de que as velocidades de execução de RAM e ROM normalmente são muito diferentes. O código executado na RAM normalmente é executado muito mais rápido do que o código executado na ROM, o que pode fazer com que o software se comporte de maneira diferente. Certos tipos de bugs, como erros de temporização, podem se revelar apenas quando executados a partir de apenas um tipo de memória.

RedBoot

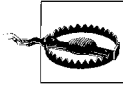
A placa Arcom inclui um monitor de depuração chamado RedBoot, que é descrito na barra lateral “Monitores de depuração” no Capítulo 4. RedBoot reside no flash do carregador de inicialização na placa Arcom e usa a porta serial COM1 da placa para sua linha de comando interface. o *Manual Técnico VIPER* e *Manual Técnico VIPER-I / O* descrever como conectar os vários módulos à placa Arcom.

Depois de conectar a placa Arcom corretamente, você precisa conectar a porta COM1 da placa Arcom a uma porta serial em seu computador usando o cabo incluído no kit de desenvolvimento.

Para se comunicar com a CLI do RedBoot, você precisa executar um programa de terminal (minicom no Linux ou HyperTerminal no Windows serve) em seu computador. Defina as configurações da porta serial da seguinte forma:

- Taxa Baud: 115200
- Bits de dados: 8
- Paridade: Nenhuma
- Bits de parada: 1
- Controle de fluxo: Nenhum

Agora você está pronto para ligar a placa Arcom.



O Redboot executa um script depois de ser executado para inicializar automaticamente o Linux embarcado (se presente na sua versão da placa Arcom). Para evitar que o script seja executado, pressione as teclas Ctrl e C juntas quando o RedBoot exibir sua mensagem de inicialização.

Se todas as conexões forem feitas corretamente, uma mensagem de inicialização é emitida da porta COM1 da placa Arcom assim que a alimentação é aplicada, junto com o prompt do RedBoot, que se parece com este:

```
Ethernet eth0: endereço MAC 00: 80: 12: 1c: 89: b6 Nenhuma
informação de IP para o dispositivo!
```

```
RedBoot (tm) bootstrap e ambiente de depuração [ROM]
Lançamento não certificado, versão W468 V3I7 - construído 10:11:20, 15 de março de 2006
```

```
Plataforma: VIPER (XScale PXA255)
Copyright (C) 2000, 2001, 2002, 2003, 2004 Red Hat, Inc.
```

```
RAM: 0x00000000-0x04000000, [0x00400000-0x03fd1000] disponível
FLASH: base 0x60000000, tamanho 0x02000000, 256 blocos de 0x00020000 bytes cada. ==
Executando script de inicialização em 1.000 segundos - digite ^ C para abortar
^ C
RedBoot>
```

Como não inserimos um endereço de protocolo da Internet (IP) para a placa Arcom, o RedBoot exibe a mensagem: Nenhuma informação de IP para o dispositivo! Esta mensagem pode ser ignorada por enquanto. Outra coisa a notar é que interrompemos a execução do script de inicialização (e o carregamento do Linux) digitando Ctrl-C (mostrado no código anterior como ^ C) quando o RedBoot é iniciado.

A mensagem de inicialização do RedBoot contém informações sobre a construção e a versão da imagem RedBoot. A memória disponível também é listada antes do prompt, incluindo os intervalos de endereço de memória RAM e flash.

Pode entrar ajuda no prompt para obter uma lista dos comandos suportados. Uma descrição dos comandos do RedBoot pode ser encontrada online em <http://ecos.sourceforge.org>.

Baixando com RedBoot

Agora que o RedBoot está instalado e funcionando, estamos prontos para baixar e executar o programa de LED piscando. RedBoot é capaz de carregar e executar arquivos ELF. Portanto, usamos o *blink.exe* arquivo criado no Capítulo 4 como a imagem do programa a ser executado na placa Arcom.

Para iniciar o download, digite o seguinte carga comando no prompt do RedBoot:

```
RedBoot> load -m xmodem
```

Isso diz ao RedBoot para carregar uma imagem usando o *xmodem* protocolo como método. Depois de pressionar a tecla Enter, o RedBoot começa a produzir o caractere C enquanto aguarda o envio do arquivo.

Para iniciar a transferência de arquivos usando o Windows HyperTerminal, selecione Transfer → Enviar arquivo ... a partir do menu (use um comando semelhante se você tiver um programa de terminal diferente). Isso abre a caixa de diálogo Enviar arquivo; selecione Xmodem para o protocolo. Navegue até a localização do *blink.exe* programa e selecione-o. Em seguida, clique em Enviar. Uma caixa de diálogo de estatísticas de transferência será exibida, mostrando o status da transferência do arquivo.

Assim que a transferência for concluída com sucesso, o RedBoot emite uma mensagem semelhante à seguinte:

```
Ponto de entrada: 0x00400110, faixa de endereço: 0x00000024-0x0040014c
xyzModem - modo CRC, 24 (SOH) / 0 (STX) / 0 (CAN) pacotes, 2 tentativas
```

Mostra o ponto de entrada do programa - neste caso, 0x00400110. Se você se referir ao arquivo de mapa gerado pelo vinculador durante o processo de construção, *blink.map*, o endereço do ponto de entrada deve ser familiar, conforme mostrado nesta parte do mapa:

Nome	Origem	Comprimento	
. texto	0x004000b0 0x00400110	0x9c	blink.o a Principal

O arquivo de mapa mostra que a rotina a Principal reside em 0x00400110, que é o ponto de entrada para a execução do programa de LED piscando. O valor 0x9C é o comprimento total do arquivo objeto *blink.o*.

Executando programas com RedBoot

Agora podemos executar o programa. Digite o seguinte comando no prompt do RedBoot:

```
RedBoot> ir
```

Depois de pressionar Enter, o RedBoot passa o controle da placa Arcom para o programa de LED piscando. Se tudo der certo, você deverá ver o LED verde piscando na placa complementar.

Você acaba de concluir com êxito sua primeira passagem pelo ciclo de desenvolvimento de software integrado.

Suporte de rede RedBoot

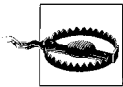
Você pode notar que a placa Arcom contém uma porta Ethernet. O RedBoot inclui uma pilha de rede para permitir a comunicação por meio desta porta. Por exemplo, você pode abrir uma sessão Telnet e se comunicar com o RedBoot pela porta Ethernet. RedBoot também suporta download de software usando o *Protocolo de transferência de arquivos trivial* (TFTP). Deixamos a investigação desses recursos avançados do RedBoot como um exercício para você.

Quando em ROM ...

Outra maneira de baixar software embarcado é carregar a imagem binária em um dispositivo ROM e inserir fisicamente esse chip em um soquete na placa de destino. Obviamente, o conteúdo de um dispositivo de memória verdadeiramente somente leitura não poderia ser substituído. No entanto, como você verá no Capítulo 6, os sistemas embarcados geralmente empregam dispositivos especiais de memória somente leitura que podem ser programados (ou reprogramados) com a ajuda de um equipamento especial chamado de *programador de dispositivo* ou *queimador*. Um programador de dispositivo é um sistema de computador que possui um ou mais soquetes de IC na parte superior - de formas e tamanhos variados - e é capaz de programar dispositivos de memória de todos os tipos.

Em um cenário de desenvolvimento ideal, o programador do dispositivo estaria conectado à mesma rede que o computador host. Dessa forma, os arquivos que contêm imagens binárias executáveis podem ser facilmente transferidos para ele para programação de ROM. Depois que a imagem binária foi transferida para o programador do dispositivo, o chip de memória é colocado em um soquete de tamanho e forma apropriados, e o tipo de dispositivo é selecionado em um menu na tela. O processo real de programação do dispositivo pode levar de alguns segundos a vários minutos, dependendo do tamanho da imagem binária, do tipo de dispositivo de memória que você está usando e da qualidade e velocidade do programador do dispositivo.

Depois de programar a ROM, ela está pronta para ser inserida em seu soquete na placa. Claro, isso não deve ser feito enquanto o sistema embarcado ainda estiver ligado. A alimentação deve ser desligada e reaplicada somente após o chip ter sido inserido com cuidado.



Deve-se ter cuidado ao remover e inserir qualquer peça que tenha soquete. Os pinos podem entortar com uma facilidade surpreendente, e um pino torto ou quebrado pode causar todos os tipos de problemas difíceis de depurar.

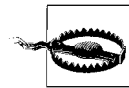
Assim que a energia for aplicada, um processador começará a buscar e executar o código armazenado dentro da ROM. No entanto, esteja ciente de que cada tipo de processador tem suas próprias regras sobre a localização de sua primeira instrução. Por exemplo, quando o processador ARM é reiniciado, ele começa buscando e executando o que quer que esteja armazenado no endereço físico 0x00000000. Isso é chamado de *redefinir o endereço*, e as instruções localizadas lá são conhecidas coletivamente como o *Reiniciar código*. No caso da placa de desenvolvimento Arcom, o código de redefinição faz parte do monitor de depuração RedBoot.

Se o seu programa não parece estar funcionando, algo pode estar errado com o seu código de redefinição. Você deve sempre garantir que a imagem binária carregada na ROM satisfaça as regras de reinicialização do processador de destino. Durante o desenvolvimento do produto, geralmente achamos útil ligar um dos LEDs da placa logo após a conclusão do código de reinicialização. Dessa forma, sabemos à primeira vista que qualquer novo código satisfaz ou não atende aos requisitos mais básicos do processador.

Gerenciando ROM com RedBoot

A placa Arcom inclui um tipo de memória chamada *clarão*, que é programável no circuito. Mesmo quando encaixada em soquetes para fácil remoção, a memória flash não precisa ser removida da placa para ser reprogramada. O monitor de depuração RedBoot inclui software que pode executar a função de programação do dispositivo.

O RedBoot também contém vários comandos para gerenciar um sistema de arquivos flash, chamados de *Sistema de Imagem Flash* (FIS). O FIS permite que você especifique regiões em flash, semelhante a um sistema de arquivos em uma unidade de disco rígido. Usando o FIS, você pode criar, gravar e apagar locais do flash com base nos “nomes de arquivo” que você selecionar.



Seja extremamente cuidadoso para não corromper as imagens existentes ou os dados de configuração que residem no flash na placa Arcom. Se isso acontecer, você pode tornar a placa inutilizável.

Para ver o que está contido no FIS, digite o seguinte comando:

RedBoot> **lista fis**

que produzirá uma lista semelhante a esta:

Nome	FLASH addr	Mem addr	Comprimento	Ponto de entrada
Diretório FIS	0x00000000	0x00000000	0x0001F000	0x00000000
Configuração do RedBoot	0x0001F000	0x00000000	0x00001000	0x00000000
sistema de arquivo	0x00020000	0x00000000	0x01FE0000	0x00000000

Esse Lista comando mostra as imagens atualmente disponíveis no RedBoot FIS. Existem alguns outros comandos FIS suportados pelo RedBoot. Para obter detalhes sobre os outros comandos e opções relacionados ao FIS, consulte o Guia do Usuário do RedBoot online em <http://ecos.sourceforge.org>.

Você tem que tomar algumas decisões ao decidir como e onde baixar um programa para o hardware. A maior desvantagem de usar memória flash para downloads é que não há uma maneira fácil de depurar software que está sendo executado fora da memória flash ou ROM. Ao executar uma única etapa ou até um ponto de interrupção, o depurador substitui a instrução subsequente por uma interrupção de software, que é usada para interromper a execução do processador. Portanto, um depurador não funciona em nenhuma forma de memória somente leitura, como flash. Obviamente, você ainda pode examinar o estado dos LEDs e de outros hardwares visíveis externamente, mas isso nunca fornecerá tantas informações e feedback quanto um depurador. Portanto, o flash pode funcionar quando você sabe que o software funciona e está pronto para implantar o sistema, mas não é muito útil durante o desenvolvimento do software.

Alguns processadores podem contornar o problema de execução fora do flash ou ROM. Em alguns casos, o processador inclui uma instrução TRACE que executa uma única instrução e, em seguida, vetores automaticamente para uma interrupção. Em outros processadores, um registro de ponto de interrupção leva você de volta ao monitor de depuração.

Depuradores Remotos

Se disponível, um *depurador remoto* pode ser usado para baixar, executar e depurar software embutido em uma porta serial ou conexão de rede entre o host e o destino (também chamado *depuração de plataforma cruzada*). O programa em execução no host de um depurador remoto tem uma interface de usuário que se parece com qualquer outro depurador que você possa ter usado. A tela principal do depurador geralmente é uma interface de linha de comando ou interface gráfica de usuário (GUI). Os depuradores de GUI normalmente contêm várias janelas menores para mostrar simultaneamente a parte ativa do código-fonte, o conteúdo do registro atual e outras informações relevantes sobre o programa em execução.

Observe que, no caso de sistemas embarcados, o depurador e o software que está sendo depurado estão sendo executados em dois sistemas de computador diferentes. O software de depurador remoto é executado no computador host e fornece a interface de usuário que acabamos de descrever. Mas também há um componente de back-end que é executado no processador de destino e se comunica com o front-end do depurador de host por meio de um link de comunicação. O back-end do depurador fornece controle de baixo nível do processador de destino. A Figura 5-2 mostra como esses dois componentes funcionam juntos.

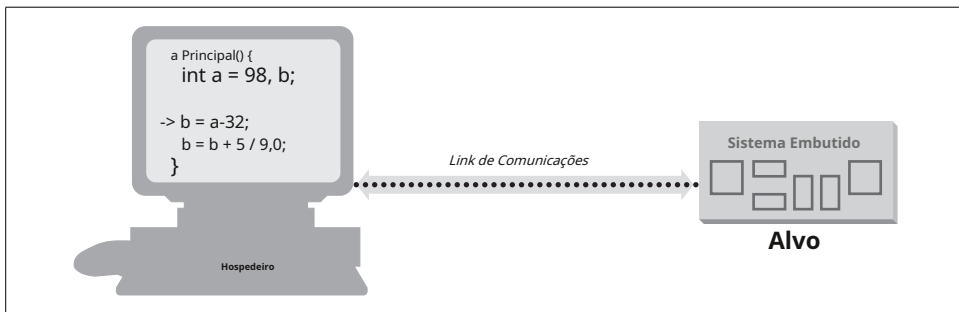


Figura 5-2. Componentes de uma sessão de depuração remota

O monitor de depuração reside na ROM - colocada lá por você ou na fábrica - e é iniciado automaticamente sempre que o processador de destino é reiniciado. Ele monitora o link de comunicação com o computador host e responde às solicitações do software host do depurador remoto. Obviamente, essas solicitações e as respostas do monitor devem estar em conformidade com algum protocolo de comunicação predefinido e normalmente são de nível muito baixo. Exemplos de solicitações que o software host pode fazer são "registro de leitura x , " "Modificar registro y , " "leitura n bytes de memória começando no endereço z , " "E" modificar os dados no endereço uma . "O depurador remoto combina sequências desses comandos de baixo nível para realizar tarefas de depuração complexas, como baixar um programa, passo único e definir pontos de interrupção.



É útil construir o programa que está sendo testado para incluir informações de depuração simbólicas, o que fizemos com o `-G` opção durante a etapa de compilação do procedimento de construção no Capítulo 4. O `-G` opção faz com que o compilador coloque informações adicionais no arquivo de objeto para uso pelo depurador. Essas informações de depuração permitem que o depurador se relacione entre o programa executável e o código-fonte.

Um desses depuradores é o depurador GNU (*`gdb`*). Como as outras ferramentas GNU, ele foi originalmente projetado para ser usado como um depurador nativo e mais tarde recebeu a capacidade de realizar depuração remota. o *`gdb`* o monitor de depuração executado no hardware de destino é chamado de *`stub do gdb`*. Informações adicionais sobre *`gdb`* pode ser encontrado online em <http://sources.redhat.com/gdb>.

As ferramentas de software GNU incluem *`gdb`*. A versão instalada é baseada em CLI, portanto, há alguns comandos para aprender a fim de executar o depurador corretamente. Existem várias GUIs disponíveis para *`gdb`*, como o Insight (<http://sources.redhat.com/insight>) e DataDisplay (<http://www.gnu.org/software/ddd>).

RedBoot contém um *`gdb`*- monitor de depuração compatível. Portanto, uma vez que um host tenta se comunicar com o alvo usando o *`gdb`* protocolo, o RedBoot passa o controle do alvo para o *`gdb`* stub para a sessão de depuração.

Conforme descrito anteriormente, o host e o destino usam um protocolo predefinido. Para *`gdb`*, este protocolo é o protocolo serial remoto baseado em ASCII. Para saber mais sobre o *`gdb`* Protocolo serial remoto, vá para <http://sources.redhat.com/gdb>. Outro bom recurso para informações sobre *`gdb`* é *Depurando com GDB: o GNU Source-Level Debugger*, por Richard Stallman, Roland Pesch e Stan Shebs (Fundação para o Software Livre). *

Os depuradores remotos são uma das ferramentas de download e teste mais comumente usadas durante o desenvolvimento de software embarcado. Isso se deve principalmente ao seu baixo custo. Os desenvolvedores de software embutido já possuem o computador host necessário. Além disso, o preço de um depurador remoto não aumenta significativamente o custo de um conjunto de ferramentas de desenvolvimento cruzado (compilador, vinculador, localizador etc.).

No entanto, existem algumas desvantagens em usar um monitor de depuração, incluindo a incapacidade de depurar o código de inicialização. Outra desvantagem é que o código deve ser executado na RAM. Além disso, ao usar um monitor de depuração, deve existir um canal de comunicação para fazer a interface do destino com o computador host.

Depuração no ArcomBoard

`gdb` é capaz de operar em portas seriais ou de rede TCP / IP. RedBoot também suporta *`gdb`* depurar sessões em qualquer uma dessas portas. Para o exemplo a seguir, usamos a porta serial. Em seguida, cobriremos alguns dos aspectos básicos *`gdb`* comandos que estão no exemplo.

* Este documento está incluído em formato eletrônico no CD-ROM do kit de desenvolvimento Arcom VIPER-Lite.



Para demonstrar alguns recursos adicionais de depuração, adicionamos uma variável global, `gCapítulo`, para o programa de LED piscando.

Para se preparar para os exemplos de depuração, desligue e ligue a placa Arcom e interrompa o script de inicialização do RedBoot pressionando Ctrl-C. Assim que a mensagem de inicialização do RedBoot for enviada, você está pronto para começar.

Invocar *gdb*, passando o nome do programa a depurar como argumento, usando o seguinte comando:

```
# arm-elf-gdb blink.exe
```

gdb gera uma mensagem semelhante a esta:

```
GNU gdb 6.3
Copyright 2004 Free Software Foundation, Inc.
GDB é um software livre, coberto pela GNU General Public License, e você está
convidado a alterá-lo e / ou distribuir cópias sob certas condições. Digite "show
copying" para ver as condições.
Não há absolutamente nenhuma garantia para o GDB. Digite "mostrar garantia" para obter
detalhes. Este GDB foi configurado como "--host = i686-pc-cygwin --target = arm-elf" ...
```



Se você usar o executável errado que não contém informações de depuração com *gdb*, a seguinte mensagem é gerada:

(nenhum símbolo de depuração encontrado)

Também deve haver um (*gdb*) prompt à espera de entrada. Em seguida, emita o comando para ter *gdb* conectar à placa Arcom. O comando a seguir assume que a porta serial do computador que está conectada à placa de destino é COM1 (se um diferente A porta serial do PC é usada, modifique o comando de acordo):

```
(gdb) destino remoto / dev / ttyS0
```

Problemas de conexão GDB

Porque a mesma porta serial do computador está sendo usada para *gdb* e comunicações RedBoot, certifique-se de que outro programa (como o programa de terminal que você usou para baixar o código) não abriu a porta. Se outro programa tiver controle sobre a porta serial do computador, *gdb* não será capaz de se conectar à placa Arcom de destino. Você também deve verificar se o computador host está conectado à porta serial correta na placa Arcom.

Depois de *gdb* conectar-se ao destino com sucesso, uma resposta semelhante a esta seguirá:

Depuração remota usando / dev / ttyS0

O computador host executando o *gdb* interface de linha de comando agora está conectada ao *gdb* stub residente no hardware de destino dentro do RedBoot.

Em seguida, baixe o blink.exe programa no alvo com o comando:

(gdb) **carregar blink.exe**

Quando o carregamento do programa é concluído com sucesso, uma mensagem semelhante a esta é a saída de *gdb*:

Carregando dados da seção, tamanho 0x4 lma 0x400000
Carregando texto da seção, tamanho 0x148 lma 0x400004
Endereço inicial 0x400110, tamanho da carga 332
Taxa de transferência: 2.656 bits em <1 seg, 166 bytes / gravação.

Agora você está pronto para começar a depurar!

Vamos começar definindo um *ponto de interrupção* no código. Um ponto de interrupção é um endereço ou condição em que o depurador interromperá a execução do programa. O depurador define um ponto de interrupção substituindo uma determinada instrução por uma interrupção de software (que é enviada de volta ao depurador). A remoção de um ponto de interrupção remove a interrupção do software e restaura a instrução pré-colocada.

Use o seguinte comando para definir um ponto de interrupção que é atingido quando a rotina ledToggle é chamado:

(gdb) **b ledToggle**



gdb os comandos não diferenciam maiúsculas de minúsculas (embora os símbolos sejam) e podem ser abreviados para a string exclusiva mais curta. Por exemplo, você pode definir um ponto de interrupção com qualquer um destes comandos:

(gdb) **breakpoint ledToggle**

(gdb) **quebrar ledToggle**

(gdb) **br ledToggle**

(gdb) **b ledToggle**

Cada comando atinge o mesmo objetivo - ou seja, definir um ponto de interrupção na rotina ledToggle. Os comandos da CLI do RedBoot operam de maneira semelhante.

Depois de definir o ponto de interrupção com sucesso, *gdb* responde com informações sobre o ponto de interrupção da seguinte forma:

Ponto de interrupção 1 em 0x400070: arquivo led.c, linha 66.

A resposta mostra o número do breakpoint (1 neste caso), o endereço do breakpoint (0x400070), o arquivo onde a função está localizada (*led.c*), e o número da linha naquele arquivo onde o ponto de interrupção é definido (66).

Se você precisar verificar quais pontos de interrupção são definidos dentro de um programa para um *gdb* sessão, você pode usar o comando:

```
(gdb) informação b
```

A resposta de *gdb* é algo assim:

Tipo de Num	Endereço Disp Enb	O que
1 ponto de interrupção	mantenha y 0x00400070	em ledToggle em led.c: 66

Em seguida, execute o programa inserindo o Prosseguir comando:

```
(gdb) c
```

Uma vez *gdb* atingir o ponto de interrupção, ele interromperá a execução e produzirá a linha de código-fonte que será executada a seguir:

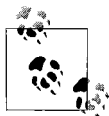
```
Ponto de interrupção 1, ledToggle () em led.c: 66
66 if (GPIO_0_LEVEL_REG & LED_GREEN)
```

A primeira linha de saída mostra o ponto de interrupção atingido, junto com a descrição. A segunda linha mostra o número da linha no arquivo, 66 neste caso, com o código-fonte dessa linha. O LED verde deve estar aceso agora porque esse é seu estado inicial no programa.

Ter *gdb* mostrar o código-fonte onde o programa atual é interrompido, use o Lista comando:

```
(gdb) eu
```

Isso irá despejar, por padrão, 10 linhas de código-fonte. Para descartar as próximas 10 linhas de código-fonte, basta inserir o Lista comando novamente.



Para repetir o último comando em um *gdb* sessão, simplesmente pressione Enter.

Um recurso útil do depurador remoto é que ele pode verificar os valores dos símbolos. O comando para mostrar o valor do gCapítulo variável é:

```
(gdb) imprimir / x gCapítulo
```

O / x opção formata a saída em hexadecimal. A resposta de *gdb* é:

```
$ 1 = 0x5
```

O \$ 1 é um número de história especial que *gdb* atribui, e *gdb* permite referência de volta a este valor em um momento posterior. O valor atual de gCapítulo é 5.

o imprimir comando também pode ser usado para alterar o valor de um símbolo. No comando a seguir, a variável gCapítulo é alterado para 12:

```
(gdb) p / x gCapítulo = 12
```

A resposta de *gdb* é:

```
$ 2 = 0xc
```

Isso mostra que o novo valor de *gCapítulo* é 0xC (12 decimais).

Dica de depuração: usando o mapa de memória para pesquisa de valor de símbolo

Símbolos de depuração associam nomes de variáveis e funções a seus endereços, bem como incluir informações de tipo sobre o símbolo. Isso permite que você faça referência a uma variável específica usando seu nome de símbolo.

Pode haver um momento em que você não consiga usar esses símbolos por um motivo ou outro; por exemplo, talvez você precise depurar código lançado por outra parte e não há informações de depuração contidas nesse arquivo de programa.

Quando as informações do símbolo não estão disponíveis, não se perde a esperança de obter informações adicionais usando um depurador. O arquivo de mapa pode ser usado para pesquisar manualmente os endereços dos símbolos. Por exemplo, o *blink.map* arquivo mostra o endereço da variável

gCapítulo como 0x00400000 com um comprimento de 4 bytes:

.dados	0x00400000	0x4 blink.o
	0x00400000	gCapítulo

Dado o endereço de um símbolo, você pode usar *gdb* para descobrir o valor atual desse símbolo usando o comando:

```
(gdb) x / d 0x400000
```

O comando *x* significa “examinar”. Pode ser seguido imediatamente por uma opção especificando o formato no qual exibir os dados (aqui, */ d* para decimal) e então o endereço de memória para ler (aqui 0x400000). Nesse caso, *gdb* responde com a seguinte saída:

```
0x400000 <gCapítulo>: 12
```

Isso mostra o valor atual de *gCapítulo* como 12 (este valor foi definido no comando anterior). Isso permite que você peek e poke variáveis sem ter as informações do símbolo.

Outro recurso muito útil de um depurador remoto é a capacidade de percorrer as linhas do código-fonte, uma ação comumente chamada *passo único*. Existem vários tipos diferentes de comandos de uma única etapa, como avançar em uma única instrução de máquina e avançar em uma única linha de código-fonte. O seguinte comando, *próximo*, segue uma única linha de código-fonte:

```
(gdb) n
```



Ao depurar, também é importante perceber que o uso da otimização do compilador pode afetar o comportamento do código na sessão de depuração. A maioria dos compiladores tem uma opção para ativar a otimização. Por exemplo, com *gcc*, a *-O* opção invoca otimização. Esta opção tem vários níveis, que são indicados por um número (0, 1, 2 ou 3) que segue a opção de troca - por exemplo, *-O2*. A chave de otimização *-Os* otimiza para tamanho. Por padrão, nenhuma otimização é selecionada. Detalhes sobre as otimizações específicas para cada nível podem ser encontrados no *gcc* documentação localizada online em <http://gcc.gnu.org/onlinedocs>.

O motivo pelo qual a otimização do compilador precisa ser considerada durante a depuração é porque o compilador pode reordenar o código e remover seções inteiras do código e / ou variáveis sem avisar você. É por esse motivo que a maior parte da depuração é feita com a otimização desativada. Lembre-se disso ao analisar o código-fonte de uma única etapa.

A linha de código-fonte a ser executada a seguir é produzida por *gdb* como mostrado aqui:

```
69          GPIO_0_SET_REG = LED_GREEN;
```

gdb fornece dois comandos para avançar uma linha de cada vez: Passo e próximo. A diferença entre eles é que quando você atinge o início de uma chamada de função, Passo entra na função e executa a primeira instrução dentro da função, enquanto próximo executa toda a função.

Agora execute o programa novamente com o Prosseguir comando:

```
(gdb) c
```

Você pode notar que o LED agora está desligado. A execução irá parar mais uma vez quando *gdb* encontra o ponto de interrupção em *ledToggle*.

gdb permite que você verifique quais funções foram executadas usando o *backtrace* ou *bt* comando. o *backtrace* comando mostra como seu programa chegou onde está atualmente. Introduzir o *backtrace* comando da seguinte forma:

```
(gdb) bt
```

o *gdb* a saída é parecida com isto:

```
# 0 ledToggle () em led.c: 66
# 1 0x00400140 em main () em blink.c: 75
```

A resposta de *gdb* mostra a função executada mais recentemente (indicada por # 0), seguido pela função que o chamou (indicada por # 1), e assim por diante. O precedente resposta mostra que a rotina Principal chamou a rotina *ledToggle*.

Com *gdb* você também pode ver os valores de registro do processador:

```
(gdb) registros de informação
```

Para imprimir o valor de um registro específico, use o comando:

```
(gdb) p / x $ pc
```

Este comando produz o valor atual do registro do contador do programa em formato hexadecimal.

Agora você deve ter um bom entendimento de como usar *gdb*. Examinamos os comandos mais importantes, mas pode ser útil brincar com alguns outros neste ponto.

Para remover o ponto de interrupção definido anteriormente, use o *excluir* comando:

```
(gdb) d
```

Em seguida, confirme a remoção de todos os pontos de interrupção. Agora você pode continuar executando o programa e observar o LED piscar.

Para interromper a execução, pressione Ctrl-C. No entanto, o programa pode não responder a este comando. Se esse é o caso, *gdb* perguntará se você deseja interromper a depuração do programa.

Uma desvantagem de um depurador de interface de linha de comando é que os comandos precisam ser aprendidos. No entanto, depois de pegar o jeito, você descobrirá que é muito fácil manobrar.

Adicional *gdb* comandos podem ser encontrados usando o *gdb* ajuda comando. A assistência com um comando específico pode ser obtida digitando ajuda seguido pelo nome do comando (por exemplo, ajuda ponto de interrupção). Há também um *gdb* guia de referência rápida disponível em vários sites online.

Emuladores

Um *emulador no circuito (ICE)* fornece muito mais funcionalidade do que um depurador remoto. Além de fornecer os recursos disponíveis com um depurador remoto, um ICE permite depurar o código de inicialização e os programas em execução na ROM, definir pontos de interrupção para o código em execução na ROM e até mesmo executar testes que exigem mais RAM do que o sistema contém.

Um ICE normalmente toma o lugar de - ou emula - o processador em sua placa de destino. (Alguns emuladores vêm com um adaptador que se conecta ao processador no destino.) O próprio ICE é um sistema integrado, com sua própria cópia do processador de destino, RAM, ROM e software integrado. Emuladores em circuito geralmente são muito caros. Mas são ferramentas poderosas e, em uma situação difícil, nada mais o ajudará a realizar melhor o trabalho de depuração.

Como um monitor de depuração, um emulador usa um depurador remoto para sua interface de host. Em alguns casos, é até possível usar o mesmo front-end do depurador para ambos. Mas, como o emulador tem sua própria cópia do processador de destino, é possível monitorar e controlar o estado do processador em tempo real. Isso permite que o emulador

suporta recursos de depuração poderosos, como pontos de interrupção de hardware e rastreamento em tempo real. Informações adicionais sobre emuladores em circuito podem ser encontradas em novembro de 2001 *Programação de sistemas embarcados* artigo “Introdução aos emuladores In-Circuit,” que pode ser encontrado online em <http://www.embedded.com>.

Com um monitor de depuração, você pode definir pontos de interrupção em seu programa. No entanto, esses pontos de interrupção de software são restritos a buscas de instruções - o equivalente ao comando “interromper a execução se esta instrução estiver prestes a ser buscada”. Emuladores, por outro lado, também suportam *pontos de interrupção de hardware*. Os pontos de interrupção de hardware permitem que você interrompa a execução em resposta a uma ampla variedade de eventos - não apenas buscas de instruções, mas também interrupções, leituras e gravações de memória. Por exemplo, você pode definir um ponto de interrupção de hardware no evento “barramento de endereço = 0x2034FF00 e barramento de dados = 0x20310000”.

Outro recurso útil de um emulador no circuito é *rastreamento em tempo real*. Normalmente, um emulador incorpora um grande bloco de RAM para fins especiais que é dedicado a armazenar informações sobre cada ciclo do processador executado. Este recurso permite que você veja exatamente em que ordem as coisas aconteceram, para que possa ajudá-lo a responder a perguntas como “A interrupção do temporizador ocorreu antes ou depois da variável Barra tornou-se 12?” Além disso, os recursos de rastreamento em tempo real geralmente são capazes de restringir as informações armazenadas ou pós-processar os dados antes de visualizá-los, para reduzir a quantidade de memória desperdiçada.

Outro tipo de ferramenta de depuração semelhante a um ICE é um *modo de depuração em segundo plano (BDM)*, ou *JTAG* (pronunciado “jay-tag”) depurador. Os depuradores JTAG são normalmente mais baratos do que os emuladores no circuito, mas oferecem muitas das mesmas funcionalidades. Essas ferramentas contam com uma interface de depuração (a interface JTAG) e circuitos de teste on-chip encontrados em processadores modernos. Informações adicionais sobre emuladores JTAG podem ser encontradas em fevereiro de 2003 *Programação de sistemas embarcados* artigo “Introdução ao On-Chip Debug,” localizado online em <http://www.embedded.com>.

O artigo “Como escolher um emulador no circuito” na edição de julho de 2002 da *Programação de sistemas embarcados* é útil para aprender como selecionar um ICE.

Outro tipo de dispositivo que emula um dispositivo de memória somente leitura é um *Emulador de ROM*. Como um ICE, é um sistema integrado que se conecta ao destino e se comunica com o host. No entanto, com este dispositivo, a conexão de destino é por meio de um soquete ROM. Para o processador embutido, ele se parece com qualquer outro dispositivo de memória somente leitura. Mas para o depurador remoto, parece um monitor de depuração.

Os emuladores de ROM têm algumas vantagens sobre os monitores de depuração. Primeiro, ninguém precisa portar o código do monitor de depuração para seu hardware de destino específico. Em segundo lugar, o emulador de ROM fornece sua própria conexão serial ou de rede ao host, portanto, não é necessário usar os próprios recursos do destino, geralmente limitados. E, finalmente, o emulador de ROM é um verdadeiro substituto para a ROM original, portanto, nenhuma memória do destino é usada pelo código do monitor de depuração.

Algumas desvantagens de usar um emulador de ROM são que ele não fornece recursos gerais de depuração (ou seja, leitura de valores de registro do processador) e é inútil em sistemas que não possuem memória externa.

Outras ferramentas úteis

Esta seção é uma introdução a outras ferramentas que muitos desenvolvedores de software consideram úteis.

Simuladores

UMA *simulador* é um programa totalmente baseado em host que simula (daí o nome atraente) a funcionalidade e o conjunto de instruções do processador de destino. A interface do usuário geralmente é a mesma ou semelhante à do depurador remoto. Na verdade, pode ser possível usar um host depurador também para o destino do simulador, conforme mostrado na Figura 5-3. Embora os simuladores tenham muitas desvantagens, eles são muito valiosos nos estágios iniciais de um projeto, quando ainda não há nenhum hardware real para os programadores experimentarem. Se você não consegue colocar as mãos em uma placa de desenvolvimento, um simulador é a melhor ferramenta para iniciar o desenvolvimento de software.

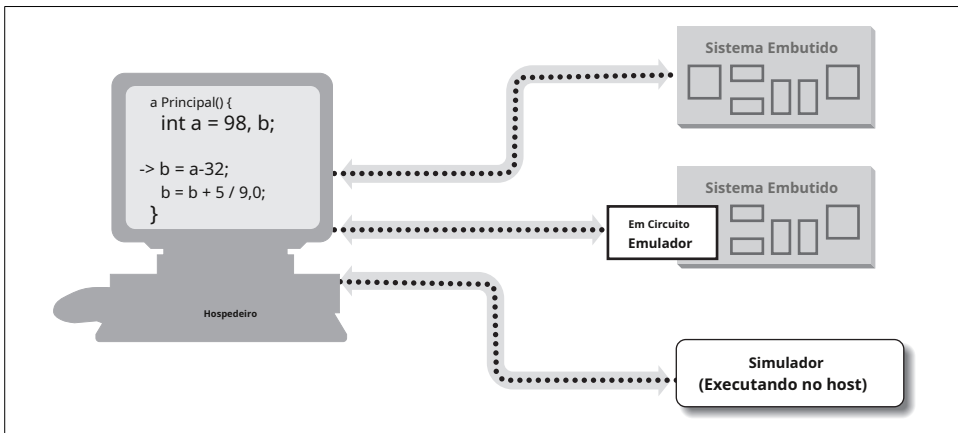


Figura 5-3. Uma interface de depurador comum

De longe, a maior desvantagem de um simulador é que ele simula apenas o processador. Os sistemas incorporados frequentemente contêm um ou mais periféricos importantes. A interação com esses dispositivos pode, às vezes, ser imitada com scripts de simulador ou outras soluções alternativas, mas essas soluções alternativas costumam ser mais problemáticas do que a simulação vale a pena. Portanto, você provavelmente não fará muito com o simulador quando o hardware embarcado real estiver disponível.

Dica de depuração: verificação de hardware usando um simulador

Se você alguma vez se deparar com uma situação na qual - depois de ler o databook - acha que o processador de destino está se comportando de maneira diferente do que deveria, tente executar o mesmo software em um simulador. Se o seu programa funcionar bem aí, você saberá que é um problema relacionado ao seu hardware. Mas se o simulador exibir a mesma estranheza do chip real, você saberá que interpretou mal a documentação do processador o tempo todo.

Ferramentas de Hardware

Como mencionado antes, um dos principais aspectos que diferencia o desenvolvedor embarcado do desenvolvedor de software típico é a “proximidade” com o hardware. Várias ferramentas estão disponíveis para ajudá-lo a descobrir o que está acontecendo com o hardware. Uma compreensão básica de como usar essas ferramentas é essencial para desenvolver boas habilidades de depuração, principalmente porque essas mesmas ferramentas são muito úteis para depuração de software de baixo nível.

Assim que você tiver acesso ao seu hardware de destino - e especialmente durante a depuração de hardware -, os analisadores lógicos e osciloscópios podem ser ferramentas de depuração indispensáveis. Um analisador lógico e um osciloscópio são mais úteis para depurar as interações entre o processador e outros chips na placa. Como podem visualizar apenas sinais externos ao processador, no entanto, essas ferramentas não podem controlar o fluxo de execução do software. Essa falta de controle de execução de software os torna significativamente menos úteis por si próprios, mas, juntamente com uma ferramenta de depuração de software, como um depurador remoto ou um emulador, eles podem ser extremamente valiosos.

UMA *analisador lógico* é um equipamento de laboratório projetado especificamente para solucionar problemas de hardware digital. Ele pode ter dezenas ou até centenas de entradas, cada uma capaz de detectar apenas uma coisa: se o sinal elétrico ao qual está conectado está atualmente no nível lógico 1 ou 0. Qualquer subconjunto das entradas que você selecionar pode ser exibido em uma linha do tempo como ilustrado na Figura 5-4. A maioria dos analisadores lógicos também permite que você comece a capturar dados, ou *desencadear*, em um padrão particular. Por exemplo, você pode fazer esta solicitação: “Exiba os valores dos sinais de entrada de 1 a 10, mas não comece a registrar o que acontece até que as entradas 2 e 5 sejam zero ao mesmo tempo.”

Um *osciloscópio* é outra peça de equipamento de laboratório para depuração de hardware. Mas este é usado para examinar qualquer sinal elétrico - analógico ou digital - em qualquer peça de hardware. Os osciloscópios às vezes são úteis para observar rapidamente a tensão ou a forma de onda do sinal em um pino específico ou, na ausência de um analisador lógico, para algo um pouco mais complexo. No entanto, o número de entradas é muito menor (geralmente há de duas a quatro) e a lógica de disparo avançada nem sempre está disponível.

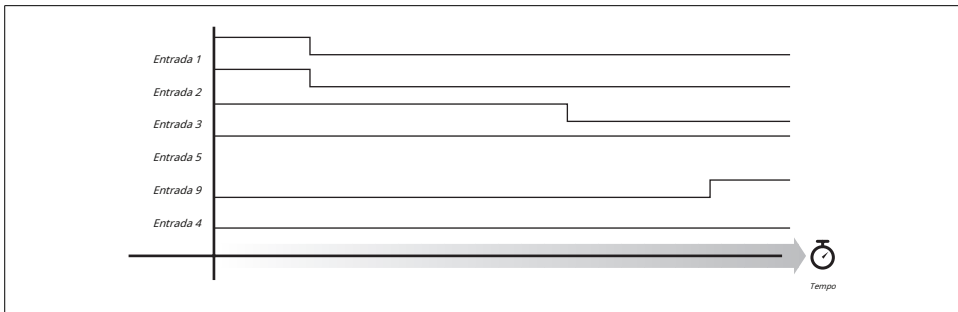
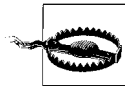


Figura 5-4. Um visor de analisador lógico típico

Dica de depuração: acionamento externo

Ocasionalmente, é desejável coordenar sua observação de algum conjunto de sinais elétricos no alvo com o software embutido que está sendo executado lá. Por exemplo, você pode querer observar a interação do barramento entre o processador e um dos periféricos conectados a ele. Um truque útil é adicionar uma instrução de saída ao software antes do início da interação em que você está interessado. Essa instrução de saída deve fazer com que um padrão lógico exclusivo apareça em um ou mais pinos do processador. Por exemplo, você pode fazer com que um pino de E / S sobressalente mude de zero para um. Um analisador lógico pode então ser configurado para disparar na ocorrência desse evento e capturar tudo o que se segue.



Ao usar um osciloscópio, certifique-se de conectar o cabo de aterramento da ponta de prova ao aterramento do hardware de destino. Deixar de fazer isso lhe dará uma imagem incorreta do que está acontecendo.

Uma das técnicas de depuração mais primitivas disponíveis é o uso de um LED como um indicador de sucesso ou falha. A ideia básica é percorrer lentamente o código de ativação do LED através do programa maior. Em outras palavras, comece primeiro com o código de ativação do LED no endereço de reinicialização. Se o LED acender, você pode editar o programa - movendo o código de ativação do LED para logo após o próximo marco de execução - e então reconstruir e testar. Como essa técnica fornece muito poucas informações sobre o estado do processador, ela é mais apropriada para programas muito simples e executados linearmente, como o código de inicialização. Mas se você não tiver acesso a um depurador remoto ou a qualquer outra ferramenta de depuração, esse tipo de depuração pode ser sua única opção.

Se um LED não estiver presente em sua plataforma de hardware, você ainda pode usar esta técnica de depuração com um sinal de E / S e um osciloscópio. Nesse caso, defina o sinal de E / S para um nível específico assim que alcançar um marco de execução específico. Usando o osciloscópio, você pode sondar esse pino de E / S para determinar se o código o definiu

adequadamente. Nesse caso, você sabe que o código foi executado com êxito até aquele ponto e agora pode mover o código do sinal de E / S para a próxima etapa.

O método de usar um sinal de E / S e um osciloscópio também pode ser usado como uma ferramenta básica de medição de desempenho. Um pino de E / S pode ser usado para medir quanto tempo um programa está gastando em uma determinada rotina ou quanto tempo leva para executar um fragmento de código específico. Isso pode mostrar gargalos potenciais no programa.

Por exemplo, para medir com precisão a duração do tempo gasto no `delay_ms` rotina (quando passado em um parâmetro de 1), podemos definir um pino de I / O alto quando entrarmos na rotina e, em seguida, definir o mesmo pino de I / O baixo antes de sair. Poderíamos então conectar um osciloscópio a este pino de I / O para medir a quantidade de tempo que o pino de I / O fica alto, que é o tempo gasto no `delay_ms` rotina. A tela do osciloscópio deve ser semelhante à imagem da Figura 5-5.

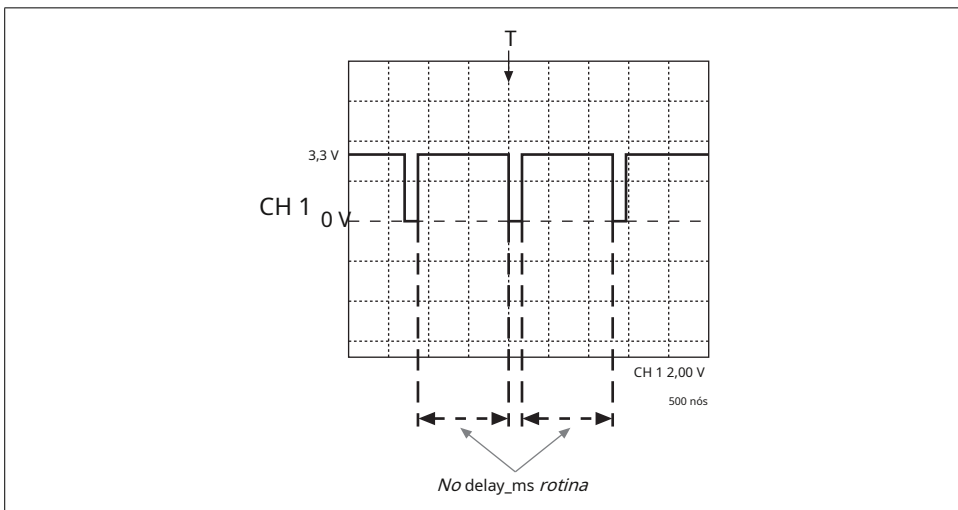


Figura 5-5. Usando sinais de I / O para depuração e medições de desempenho

Conforme mostrado na Figura 5-5, o canal 1 (CH 1) é a ponta de prova que capturou o sinal. As linhas horizontais pontilhadas indicam incrementos de tensão - neste caso, 2 volts por divisão; as linhas verticais pontilhadas indicam incrementos de tempo - neste caso, 500 microssegundos por divisão. O "T" na parte superior da tela, junto com a seta, indica quando o osciloscópio disparou na borda descendente do pino de E / S. O sinal de E / S vai de (aproximadamente) 0 a 3,3 volts.

Aliás, este teste mostra que o real `delay_ms` rotina que deveria atrasar 1 milissegundo está um pouco fora, porque o tempo entre definir o pino I / O alto e definir o pino I / O baixo é um pouco mais longo do que duas divisões.

Se os pinos de E / S estão disponíveis e várias entradas são suportadas pelo osciloscópio, você pode usar vários sinais de E / S simultaneamente para obter um instantâneo de todo o

sistema. Em sistemas mais complexos, você pode mover as chamadas do conjunto de E / S nas várias rotinas e medir o desempenho de cada rotina.

Localizando o pino 1

Antes de (cuidadosamente) sondar a placa de circuito, vamos aprender como identificar pinos específicos em um IC. A Figura 5-6 mostra vários métodos comuns usados para identificar o pino 1 em um IC. Conforme mostrado nesta figura, uma almofada quadrada é frequentemente usada para o pino 1, enquanto as outras almofadas são normalmente círculos. Este é normalmente o caso quando o IC está em um *pacote dual inline* (MERGULHAR). Outro indicador do pino 1 é um círculo serigrafado ao lado do pino 1.

Os fabricantes de IC também indicam o pino 1 colocando um recorte circular próximo a ele ou um recorte de arco no topo do CI, caso em que o pino 1 está localizado à esquerda deste recuo. Em alguns chips menores, o lado do pino 1 é *chanfrado* (um bosque é cortado). Os outros números dos pinos quase sempre aumentam à medida que você se move no sentido anti-horário a partir do pino 1 ao redor do chip.

Uma combinação destes pode ser usada em alguns casos. Você pode querer dar uma olhada na placa da Arcom para ter uma ideia melhor de como são alguns desses indicadores do pino 1.

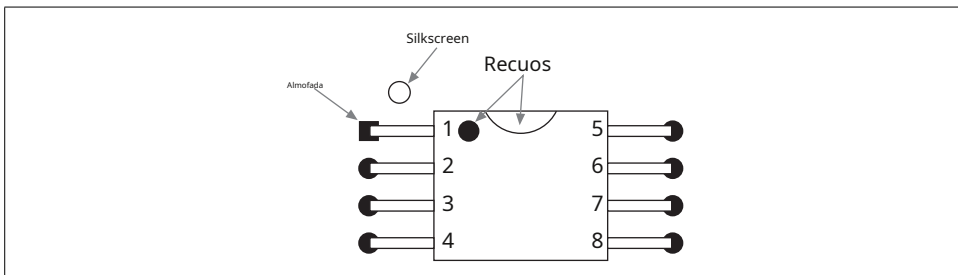


Figura 5-6. Identificando o pino 1 em um IC

Fiapo

UMA *fiapo* O programa é uma ferramenta para verificar estaticamente o código-fonte em busca de problemas de portabilidade e erros comuns de sintaxe de codificação, como valores de retorno ignorados e inconsistências de tipo. Um compilador fornece parte dessa verificação de erro, mas um programa lint verifica essas áreas de um programa com muito mais cuidado e, portanto, ajuda no desenvolvimento de um software mais robusto.

Configurando *fiapo* é semelhante a configurar um compilador, onde diferentes opções são passadas para o programa para controlar o tipo de saída produzida. Na verdade, você pode aumentar seu procedimento de construção para incluir uma verificação de lint que envia sua saída para um arquivo para você revisar posteriormente.

Uma boa introdução ao uso *fiapo* é o artigo “Introdução ao Lint” dos arquivos de *Programação de sistemas embarcados*. Este artigo pode ser encontrado online em <http://www.embedded.com>. Para obter informações adicionais, pegue *Verificando Programas C com Lint*, por Ian F. Darwin (O'Reilly).

Vários comerciais e de código aberto *fiapo* programas estão disponíveis. Um desses programas é chamado *Tala*. Informações adicionais sobre o Splint, bem como a versão mais recente dele pode ser encontrado online em <http://www.splint.org>.

Controle de versão

O gerenciamento do código-fonte é uma parte importante de qualquer projeto de desenvolvimento. Isso é especialmente verdadeiro quando vários desenvolvedores estão trabalhando no mesmo código-fonte ao mesmo tempo.

Controle de versão O software permite o armazenamento do código-fonte em um repositório que pode ser atualizado à medida que o projeto avança nos diferentes estágios de desenvolvimento. Vários programas de controle de versão oferecem vários recursos, como registro, comparações de arquivos e lançamentos de marcação, bem como rastreamento de correções de bugs e atualizações de código para novos recursos. O software de controle de versão também pode ajudar a encontrar bugs introduzidos após as alterações feitas em um lançamento de código estável.

O software de controle de versão pode ser útil não apenas para o código-fonte, mas para todos os arquivos associados a um projeto ou produto específico, incluindo programas, as ferramentas usadas para construir o software e a documentação. Existem vários programas de controle de versão de código aberto disponíveis. Aqui estão alguns:

Sistema de versões simultâneas (CVS) (<http://ximbiot.com/cvs/cvshome>)

Um sistema que permite que muitas pessoas trabalhem em grandes conjuntos de arquivos simultaneamente e pode até mesmo combinar alterações de diferentes pessoas em um único arquivo. *CVS essencial*, por Jennifer Vesperman (O'Reilly) cobre CVS. Existem também vários aplicativos hospedeiros gráficos para CVS, como o programa baseado em Windows WinCVS (<http://www.wincvs.org>). *Subversion*

(<http://subversion.tigris.org>)

Uma continuação do CVS que resolve alguns dos problemas do CVS para grandes projetos e está ganhando adeptos. *Controle de versão com Subversion*, por Ben Collins-Sussman, Brian W. Fitzpatrick e C. Michael Pilato (O'Reilly) cobre o software de controle de versão Subversion. *

Sistema de controle de revisão (RCS) (<http://www.gnu.org/software/rcs>)

Uma versão de software livre (projeto GNU) de um programa de controle de fonte Unix tradicional adequado para pequenos projetos.

* O livro do Subversion também está disponível online em <http://svnbook.red-bean.com>.

Explore o hardware

A maioria das ferramentas de depuração descritas neste capítulo será usada em um ponto ou outro em cada projeto incorporado. Osciloscópios e analisadores lógicos são usados com mais frequência para depurar problemas de hardware; simuladores para testar o software antes que o hardware esteja disponível; e depurar monitores e emuladores durante a integração e depuração de software. *Fiapo* e software de controle de versão são normalmente usados em todo o projeto. Para ser mais eficaz, você deve entender para que serve cada ferramenta, quando e onde aplicá-la para obter o maior impacto.

Em muitas ocasiões, os engenheiros de software não querem nada com o hardware, mas essa atitude diminui a utilidade do engenheiro de software. A maioria dos projetos é bem-sucedida porque os membros da equipe têm uma variedade de habilidades e podem ajudar em outras áreas além da disciplina em que foram treinados. Não olhe para um problema de hardware como algo que só pode ser resolvido por um engenheiro de hardware. Veja isso como algo com que você pode aprender e ajudar a resolver.

Não tenha medo de entrar lá - sozinho ou com o engenheiro de hardware - e descobrir o que está acontecendo. Se você não entender o problema, sente-se no laboratório com o engenheiro de hardware por um tempo para ter uma ideia melhor de qual é o problema. Você pode até ser capaz de escrever um trecho de código que agrava o problema e, como resultado, descubra sua causa.

Memória

Tyrell: *Se lhes dermos um passado, criamos uma almofada para suas emoções e, conseqüentemente, podemos controlá-los Melhor.*

Deckard: *Recordações. Você está falando sobre memórias.*
- o filme *Blade Runner*

Neste capítulo, você aprenderá tudo o que precisa saber sobre memória em sistemas embarcados. Em particular, você aprenderá sobre os tipos de memória que provavelmente encontrará, como testar os dispositivos de memória para ver se estão funcionando corretamente e como usar a memória flash.

Tipos de Memória

Muitos tipos de dispositivos de memória estão disponíveis para uso em sistemas de computador modernos. Como engenheiro de software embarcado, você deve estar ciente das diferenças entre eles e entender como usar cada tipo de maneira eficaz. Em nossa discussão, abordaremos esses dispositivos do ponto de vista do software. Enquanto você está lendo, tente ter em mente que o desenvolvimento desses dispositivos levou várias décadas. Os nomes dos tipos de memória freqüentemente refletem a natureza histórica do processo de desenvolvimento.

A maioria dos desenvolvedores de software pensa na memória como sendo RAM ou ROM. Mas, na verdade, existem subtipos de cada classe e até mesmo uma terceira classe de memórias híbridas que exibem algumas das características da RAM e da ROM. Em um dispositivo RAM, os dados armazenados em cada local da memória podem ser lidos ou gravados, conforme desejado. Em um dispositivo ROM, os dados armazenados em cada local da memória podem ser lidos à vontade, mas nunca gravados. Os dispositivos híbridos oferecem permanência semelhante à ROM, mas sob algumas condições é possível sobrescrever seus dados. A Figura 6-1 fornece um sistema de classificação para os dispositivos de memória comumente encontrados em sistemas embarcados.

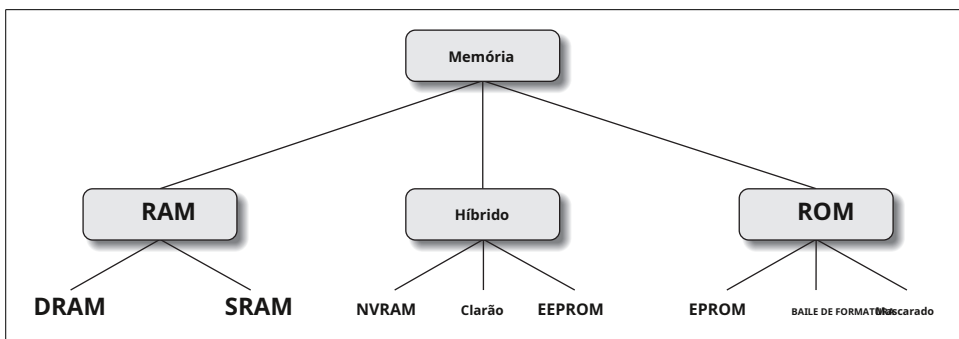


Figura 6-1. Tipos de memória comuns em sistemas embarcados

Tipos de RAM

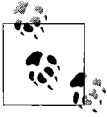
Existem dois dispositivos de memória importantes na família RAM: SRAM e DRAM. A principal diferença entre eles é a duração dos dados armazenados. *RAM estática* (*SRAM*) retém seu conteúdo enquanto a energia elétrica for aplicada ao chip. No entanto, se a alimentação for desligada ou perdida temporariamente, seu conteúdo será perdido para sempre. *RAM dinâmica* (*DRAM*), por outro lado, tem uma vida útil de dados extremamente curta - geralmente menos de um quarto de segundo. Isso é verdadeiro mesmo quando a alimentação é aplicada continuamente.

Resumindo, SRAM tem todas as propriedades da memória em que você pensa quando ouve a palavra RAM. Comparado com isso, DRAM parece meio inútil. Para que serve um dispositivo de memória que retém seu conteúdo por apenas uma fração de segundo? Por si só, essa memória volátil é realmente inútil. No entanto, uma peça simples de hardware chamada de *Controlador DRAM* pode ser usado para fazer com que a DRAM se comporte mais como a SRAM (consulte a barra lateral “Controladores DRAM”, nesta seção para obter mais informações). O trabalho do controlador DRAM, frequentemente incluído no processador, é atualizar periodicamente os dados armazenados no DRAM. Ao atualizar os dados várias vezes por segundo, o controlador DRAM mantém o conteúdo da memória vivo pelo tempo que for necessário. Portanto, a DRAM é tão útil quanto a SRAM, afinal.

Ao decidir qual tipo de RAM usar, um designer de sistema deve considerar o tempo de acesso e o custo. Os dispositivos SRAM oferecem tempos de acesso extremamente rápidos (aproximadamente quatro vezes mais rápidos do que DRAM), mas são muito mais caros de produzir. Geralmente, SRAM é usado apenas onde a velocidade de acesso é crucial. No entanto, se um sistema requer apenas uma pequena quantidade de memória, SRAM pode fazer mais sentido porque você pode evitar o custo de um controlador DRAM.

Um custo por byte muito mais baixo torna a DRAM atraente sempre que grandes quantidades de RAM são necessárias. A DRAM também está disponível em capacidades muito maiores do que a SRAM. Muitos sistemas embarcados incluem os dois tipos: um pequeno bloco de SRAM (algumas centenas de kilobytes) ao longo de um caminho de dados crítico e um bloco muito maior de DRAM (em megabytes)

para todo o resto. Alguns pequenos sistemas embarcados sobrevivem sem nenhuma memória adicional: eles usam apenas a memória no chip do microcontrolador.



Existem algumas variações de DRAM que você pode encontrar, incluindo DRAM síncrona (SDRAM), SDRAM de taxa de dados dupla (SDRAM DDR) e DRAM Rambus (RDRAM).

Controladores DRAM

Se o seu sistema embarcado incluir DRAM, provavelmente também haverá um controlador DRAM integrado (ou no chip). O PXA255 possui um controlador DRAM no chip. O controlador DRAM é uma peça extra de hardware colocada entre o processador e os chips de memória. Seu principal objetivo é realizar as operações de atualização necessárias para manter seus dados vivos na DRAM. No entanto, ele não pode fazer isso corretamente sem alguma ajuda sua.

Uma das primeiras coisas que seu software deve fazer é inicializar o controlador DRAM. Se você não tiver nenhuma outra RAM no sistema, deve fazer isso antes de criar a pilha ou heap, porque essas áreas de memória estariam localizadas na DRAM. Este código de inicialização é geralmente escrito em linguagem assembly e colocado dentro do módulo de inicialização de hardware.

Quase todos os controladores DRAM requerem uma curta sequência de inicialização que consiste em um ou mais comandos de configuração. Os comandos de configuração informam ao controlador sobre a interface de hardware para a DRAM e com que frequência os dados devem ser atualizados. Para determinar a sequência de inicialização de seu sistema específico, consulte o projetista da placa ou leia os livros de dados que descrevem o DRAM e o controlador DRAM. Se a DRAM em seu sistema não parece estar funcionando corretamente, pode ser que o controlador DRAM não tenha sido inicializado ou tenha sido inicializado incorretamente.

Tipos de ROM

As memórias na família ROM são diferenciadas pelos métodos usados para gravar novos dados nelas (geralmente chamados *programação* ou *queimando*) e o número de vezes que eles podem ser reescritos. Essa classificação reflete a evolução dos dispositivos ROM de hardwired para one-time programável para apagável e programável. Uma característica comum a todos esses dispositivos é a capacidade de reter dados e programas para sempre, mesmo quando a energia é removida.

Os primeiros ROMs eram dispositivos com fio que continham um conjunto pré-programado de dados ou instruções. O conteúdo da ROM teve que ser especificado antes da produção do chip, de modo que os dados reais pudessem ser usados para organizar os transistores dentro do chip! Memórias conectadas ainda são usadas, embora agora sejam chamadas *ROMs mascaradas* para distingui-los de outros tipos de ROM. A principal vantagem de uma ROM mascarada é um

baixo custo de produção. Infelizmente, o custo é baixo apenas quando centenas de milhares de cópias da mesma ROM são necessárias e nenhuma alteração é necessária.

Outro tipo de ROM é o *ROM programável (PROM)*, que é comprado em um estado não programado. Se você olhar o conteúdo de uma PROM não programada, verá que todos os bits são 1s. O processo de gravação de seus dados no PROM envolve um equipamento especial denominado *programador de dispositivo*, que grava dados no dispositivo aplicando uma voltagem mais alta do que o normal aos pinos de entrada especiais do chip. Uma vez que uma PROM tenha sido programada desta forma, seu conteúdo nunca poderá ser alterado. Caso seja necessário alterar o código ou dados armazenados na PROM, o chip deve ser descartado e substituído por um novo. Como resultado, os PROMs também são conhecidos como *programável uma vez* (Dispositivos OTP). Muitos pequenos microcontroladores embutidos também são considerados programáveis uma vez, porque contêm PROM embutido.

Um *ROM apagável e programável (EPROM)* é programado exatamente da mesma maneira que um PROM. No entanto, as EPROMs podem ser apagadas e reprogramadas repetidamente. Para apagar uma EPROM, basta expor o dispositivo a uma forte fonte de luz ultravioleta. (Há uma "janela" na parte superior do dispositivo para permitir que a luz ultravioleta alcance o silício. Você pode comprar uma borracha EPROM contendo essa luz.) Ao fazer isso, você essencialmente redefine o chip inteiro para seu estado inicial - não programado. O tempo de apagamento de uma EPROM pode ser de 10 a 45 minutos, o que pode tornar a depuração de software um processo lento.

Embora mais caro do que os PROMs, sua capacidade de serem reprogramados tornou os EPROMs um recurso comum do processo de desenvolvimento e teste de software embarcado por muitos anos. Agora é relativamente raro ver EPROMs usados em sistemas embarcados, já que foram suplantados por tecnologias mais novas.

Tipos Híbridos

À medida que a tecnologia de memória amadureceu nos últimos anos, a linha entre os dispositivos de RAM e ROM ficou turva. Existem agora vários tipos de memória que combinam os melhores recursos de ambos. Esses dispositivos não pertencem a nenhum dos grupos e podem ser chamados coletivamente de dispositivos de memória híbrida. As memórias híbridas podem ser lidas e gravadas conforme desejado, como a RAM, mas mantêm seu conteúdo sem energia elétrica, assim como a ROM. Os ciclos de gravação em memórias híbridas são semelhantes à RAM, mas demoram significativamente mais do que gravações em uma RAM, portanto, você não gostaria de usar esse tipo para a memória principal do sistema. Dois dos dispositivos híbridos, EEPROM e flash, são descendentes de dispositivos ROM; a terceira, NVRAM, é uma versão modificada da SRAM.

Um *ROM eletricamente apagável e programável (EEPROM)* é internamente semelhante a uma EPROM, mas com a operação de apagamento realizada eletricamente. Além disso, um único byte em uma EEPROM pode ser apagado e reescrito. Depois de gravados, os novos dados permanecerão no dispositivo para sempre - ou pelo menos até serem apagados eletricamente. 1

a compensação para essa funcionalidade aprimorada é o custo mais alto; outra é que normalmente a EEPROM é boa para 10.000 a 100.000 ciclos de gravação.

EEPROMs estão disponíveis em uma interface paralela padrão (barramento de endereço e dados), bem como em uma interface serial. Em muitos projetos, o Inter-IC (I² C) ou barramentos de interface periférica serial (SPI) são usados para se comunicar com dispositivos EEPROM seriais. Vamos dar uma olhada no 2 Barramentos C e SPI no Capítulo 13.

Clarão é o avanço recente mais importante na tecnologia de memória. Ele combina todos os melhores recursos dos dispositivos de memória descritos até agora. Os dispositivos de memória flash são de alta densidade, baixo custo, não voláteis, rápidos (para ler, mas não para escrever) e eletricamente reprogramáveis. Essas vantagens são esmagadoras e o uso de memória flash aumentou dramaticamente em sistemas embarcados como resultado direto.

Apagar e gravar dados em uma memória flash requer uma sequência específica de gravações usando certos valores de dados. Do ponto de vista do software, as tecnologias flash e EEPROM são muito semelhantes. A principal diferença é que os dispositivos flash podem ser apagados apenas um setor por vez, não byte a byte. Os tamanhos de setor típicos variam de 8 KB a 64 KB. Apesar dessa desvantagem, o flash é muito mais popular do que o EEPROM e também está substituindo rapidamente muitos dos dispositivos ROM.

O terceiro membro da classe de memória híbrida é *RAM não volátil* (NVRAM). A não-volatilidade também é uma característica da ROM e das memórias híbridas discutidas anteriormente. No entanto, uma NVRAM é fisicamente muito diferente desses dispositivos. Normalmente, uma NVRAM é apenas uma SRAM com bateria de reserva. Quando a energia está ligada, a NVRAM opera como qualquer outra SRAM. Mas quando a energia está desligada, a NVRAM consome apenas energia elétrica suficiente da bateria para reter seu conteúdo atual. NVRAM às vezes é encontrada em sistemas embarcados para armazenar informações críticas do sistema. A propósito, o “CMOS” em um PC compatível com IBM era historicamente um NVRAM.

A Tabela 6-1 resume as características dos diferentes tipos de memória.

Tabela 6-1. Características do dispositivo de memória

Tipo de memória	Volátil?	Gravável?	Apagar/ reescrever o tamanho	Apagar / reescrever ciclos	Custo relativo	Velocidade relativa
SRAM	sim	sim	Byte	Ilimitado	Caro	Rápido
DRAM	sim	sim	Byte	Ilimitado	Moderado	Moderado
ROM mascarada	Não	Não	N / D	N / D	Barato (em quantidade)	Devagar
BALILE DE FORMATURA	Não	Uma vez, com programador	N / D	N / D	Moderado	Devagar
EPROM	Não	Sim com programador	Chip inteiro	Limitada (ver especificações)	Moderado	Devagar
EEPROM	Não	sim	Byte	Limitada (ver especificações)	Caro	Moderado para ler, lento para escrever

Tabela 6-1. Características do dispositivo de memória (continuação)

Tipo de memória	Volátil?	Gravável?	Apagar/ reescrever o tamanho	Apagar / reescrever ciclos	Custo relativo	Velocidade relativa
Claro	Não	sim	Setor	Limitada (ver especificações)	Moderado	Rápido para ler, lento para escrever
NVRAM	Não	sim	Byte	Nenhum	Caro	Rápido

Acesso direto à memória

Já que estamos discutindo a memória, este é um bom momento para discutir uma técnica de transferência de memória chamada *acesso direto à memória* (DMA). DMA é uma técnica para transferir blocos de dados diretamente entre dois dispositivos de hardware com envolvimento mínimo da CPU. Na ausência de DMA, o processador deve ler os dados de um dispositivo e gravá-los no outro, um byte ou palavra de cada vez. Para cada byte ou palavra transferida, o processador deve buscar e executar uma sequência de instruções. Se a quantidade de dados a ser transferida for grande ou a frequência das transferências for alta, o restante do software pode nunca ter a chance de ser executado. Porém, se um controlador DMA estiver presente, ele pode realizar toda a transferência, com pouca ajuda do processador.

É assim que funciona o DMA. Quando um bloco de dados precisa ser transferido, o processador fornece ao controlador DMA os endereços de origem e destino e o número total de bytes. O controlador DMA então transfere os dados da origem para o destino automaticamente. Quando o número de bytes restantes chega a zero, a transferência do bloco termina.

Em um cenário típico de DMA, o bloco de dados é transferido diretamente para ou da memória. Por exemplo, um controlador de rede pode querer colocar um pacote de rede de entrada na memória à medida que chega, mas notificar o processador apenas quando o pacote inteiro for recebido. Ao usar o DMA, o processador pode gastar mais tempo processando os dados assim que eles chegam e menos tempo transferindo-os entre dispositivos. O processador e o controlador DMA devem usar o mesmo endereço e barramentos de dados durante esse tempo, mas isso é tratado automaticamente pelo hardware e o processador não está envolvido na transferência real. Durante uma transferência DMA, o controlador DMA arbitra o controle do barramento entre o processador e a operação DMA.

Edições Endian

Endianness é o atributo de um sistema que indica se os inteiros são representados da esquerda para a direita ou da direita para a esquerda. Por que, no mundo atual de máquinas virtuais e processadores gigaHertz, um programador se importaria com um assunto tão bobo? Bem, infelizmente, endianness deve ser escolhido toda vez que uma arquitetura de hardware ou software é projetada, e não há muito na forma de lei natural para ajudar na decisão.

Endianness vem em duas variedades: grande e pequena. UMA *big-endian* representação tem um inteiro multibyte escrito com seu byte mais significativo à esquerda; um número assim representado é facilmente lido por humanos que falam inglês. UMA *pequeno endian* a representação, por outro lado, coloca o byte mais significativo à direita. Claro, as arquiteturas de computador não têm uma "esquerda" ou "direita" intrínseca. Esses termos humanos são emprestados de nossas formas escritas de comunicação. As seguintes definições são mais precisas:

Big-endian

Significa que o byte mais significativo de qualquer campo de dados multibyte é armazenado no endereço de memória mais baixo, que também é o endereço do campo maior

Pequeno endian

Significa que o byte menos significativo de qualquer campo de dados multibyte é armazenado no endereço de memória mais baixo, que também é o endereço do campo maior

A origem dos termos estranhos big-endian e little-endian pode ser rastreada até o livro de 1726 *As Viagens de Gulliver*, por Jonathan Swift. Em uma parte da história, a resistência a um decreto imperial para quebrar ovos cozidos na "ponta pequena" leva à guerra civil. O enredo é uma sátira da ruptura do rei Henrique VIII da Inglaterra com a Igreja Católica. Algumas centenas de anos depois, em 1981, Danny Cohen aplicou os termos e a sátira à nossa situação atual em *Computador IEEE* (vol. 14, não. 10).

Endianness em dispositivos

Endianness não importa em um único sistema. Isso importa apenas quando dois computadores estão tentando se comunicar. Cada processador e cada protocolo de comunicação deve escolher um tipo de endianness ou outro. Assim, dois processadores com endianness diferente entrarão em conflito se se comunicarem por meio de um dispositivo de memória. Da mesma forma, um processador little-endian tentando se comunicar em uma rede big-endian precisará fazer o reordenamento de bytes de software.

Os processadores 80x86 da Intel e seus clones são little-endian. As famílias SPARC da Sun, 68K da Motorola e as famílias PowerPC são todas big-endian. Alguns processadores têm até um bit em um registro que permite ao programador selecionar o endianness desejado. O processador PXA255 suporta a operação big-endian e little-endian através do bit 7 no registro de controle 1 (coprocessador 15 (CP15), registro 1).

Uma diferença de endianness pode causar problemas se um computador tentar, sem saber, ler dados binários gravados no formato oposto a partir de um local ou arquivo de memória compartilhada. A Figura 6-2 (a) mostra o conteúdo da memória para os dados 0x12345678 (um longo), 0xABCD (uma palavra) e 0xEF (um byte) em uma máquina little-endian. Os mesmos dados representados em uma máquina big-endian são mostrados na Figura 6-2 (b).

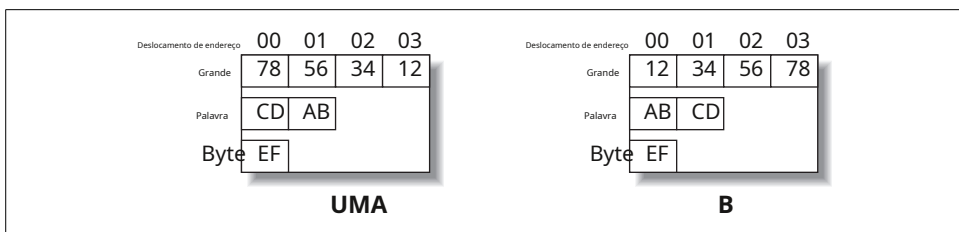


Figura 6-2. (a) memória little-endian, (b) memória big-endian

Endianness na rede

Outra área onde endianness é um problema é nas comunicações de rede. Como diferentes tipos de processadores (big-endian e little-endian) podem estar na mesma rede, eles devem ser capazes de se comunicarem entre si. Portanto, as pilhas de rede e os protocolos de comunicação também devem definir seu endianness. Caso contrário, dois nós de endianness diferente seriam incapazes de se comunicar. Este é um exemplo mais substancial de endianness afetando o programador embarcado.

Acontece que todas as camadas de protocolo no conjunto TCP / IP são definidas como bigendian. Em outras palavras, qualquer valor de 16 ou 32 bits nos vários cabeçalhos de camada (por exemplo, um endereço IP, um comprimento de pacote ou uma soma de verificação) deve ser enviado e recebido com seu byte mais significativo primeiro.

Digamos que você deseja estabelecer uma conexão de soquete TCP a um computador cujo endereço IP é 192.0.1.2. O IPv4 usa um número inteiro exclusivo de 32 bits para identificar cada host de rede. O endereço IP decimal pontuado deve ser convertido em um número inteiro.

A representação de inteiro multibyte usada pelos protocolos TCP / IP é às vezes chamada *ordem de bytes da rede*. Mesmo que os computadores em cada extremidade sejam little-endian, os inteiros multibyte passados entre eles devem ser convertidos para a ordem de bytes da rede antes da transmissão pela rede e, em seguida, convertidos de volta para little-endian na extremidade receptora.

Suponha que um PC little endian baseado em 80x86 esteja se comunicando com um servidor big-endian baseado em SPARC pela Internet. Sem manipulação adicional, o processador 80x86 converterá 192.0.1.2 no inteiro little-endian 0x020100C0 e transmitirá os bytes na seguinte ordem: 0x02, 0x01, 0x00, 0xC0. O SPARC receberia os bytes na seguinte ordem: 0x02, 0x01, 0x00, 0xC0. O SPARC reconstruiria os bytes em um inteiro big-endian 0x020100c0 e interpretaria incorretamente o endereço como 2.1.0.192.

Evitar esse tipo de confusão leva a um pequeno detalhe irritante de implementação para desenvolvedores de pilha TCP / IP. Se a pilha for executada em um processador little-endian, ela terá que reordenar (em tempo de execução) os bytes de cada campo de dados multibyte dentro dos cabeçalhos das várias camadas. Se a pilha for executada em um processador big-endian, não há nada com que se preocupar. Para a pilha ser portátil (ou seja, ser capaz de rodar em processadores de ambos

tipos), terá que decidir se deseja ou não fazer esse reordenamento. A decisão normalmente é feita em tempo de compilação.

Uma solução comum para o problema de endianness é definir um conjunto de quatro macros de pré-processador:

htons ()

Reordene os bytes de um valor sem sinal de 16 bits do pedido do processador para o pedido da rede. O nome da macro pode ser lido como “abreviação de host para rede”.

htonl ()

Reordene os bytes de um valor não assinado de 32 bits do pedido do processador para o pedido da rede. O nome da macro pode ser lido como “host para rede por muito tempo”.

ntohs ()

Reordene os bytes de um valor não assinado de 16 bits do pedido da rede para o pedido do processador. O nome da macro pode ser lido como “rede para hospedar curto”.

ntohl ()

Reordene os bytes de um valor não assinado de 32 bits do pedido da rede para o pedido do processador. O nome da macro pode ser lido como “rede para hospedar por muito tempo”.

A seguir está um exemplo da implementação dessas macros. Vamos dar uma olhada nos operadores shift à esquerda (<<) e shift à direita (>>) no Capítulo 7.

```
# se definido (BIG_ENDIAN) &&! definido (LITTLE_ENDIAN)

# define htons (A) (A)
# define htonl (A) (A)
# define ntohs (A) (A)
# define ntohl (A) (A)

# elif definido (LITTLE_ENDIAN) &&! definido (BIG_ENDIAN)

# define htons (A) (((uint16_t) (A) & 0xff00) >> 8) | \
    (((uint16_t) (A) & 0x00ff) << 8)
# define htonl (A) (((uint32_t) (A) & 0xff000000) >> 24) | \
    (((uint32_t) (A) & 0x00ff0000) >> 8) | \
    (((uint32_t) (A) & 0x0000ff00) << 8) | \
    (((uint32_t) (A) & 0x000000ff) << 24)) htons
# define ntohs
# define ntohl      htonl

# outro

# error BIG_ENDIAN ou LITTLE_ENDIAN deve ser #defined, mas não ambos.

# fim se
```

Se o processador no qual a pilha TCP / IP deve ser executada também for big-endian, cada uma das quatro macros será definida para não fazer nada e não haverá impacto no desempenho do tempo de execução. Se, no entanto, o processador for little-endian, as macros irão reordenar o

bytes apropriadamente. Essas macros são chamadas rotineiramente durante a construção e análise de pacotes de rede e quando as conexões de soquete são criadas.

Podem ocorrer penalidades de desempenho de tempo de execução ao usar TCP / IP em um processador little-endian. Por esse motivo, pode ser imprudente selecionar um processador little-endian para uso em um dispositivo com abundância de funcionalidade de rede, como um roteador ou gateway. Os programadores incorporados devem estar cientes do problema e estar preparados para converter entre suas diferentes representações, conforme necessário.

Teste de Memória

Uma das primeiras peças de software embarcado sério que você provavelmente escreverá é um teste de memória. Assim que o hardware do protótipo estiver pronto, o projetista gostaria de ter alguma garantia de que conectou o endereço e as linhas de dados corretamente e que os chips de memória estão funcionando corretamente. A princípio, isso pode parecer uma atribuição bastante simples, mas ao examinar o problema mais de perto, você perceberá que pode ser difícil detectar problemas sutis de memória com um teste simples. Na verdade, como resultado da ingenuidade do programador, muitos sistemas embarcados incluem testes de memória que detectariam apenas as falhas de memória mais catastróficas. Alguns deles podem nem perceber que os chips de memória foram removidos da placa!

O objetivo de um teste de memória é confirmar se cada local de armazenamento em um dispositivo de memória está funcionando. Em outras palavras, se você armazenar o número 50 em um endereço específico, espera encontrar esse número armazenado lá até que outro número seja escrito. A ideia básica por trás de qualquer teste de memória, então, é escrever algum conjunto de valores de dados para cada endereço no dispositivo de memória e verificar os dados lendo-os de volta. Se todos os valores lidos forem iguais aos que foram gravados, diz-se que o dispositivo de memória passou no teste. Como você verá, é apenas por meio da seleção cuidadosa do conjunto de valores de dados que você pode ter certeza de que um resultado aprovado é significativo.

Claro, um teste de memória como o que acabamos de descrever é inevitavelmente destrutivo. No processo de teste da memória, você deve sobrescrever seu conteúdo anterior. Como geralmente é impraticável sobrescrever o conteúdo de memórias não voláteis, os testes descritos nesta seção são geralmente usados apenas para testes de RAM. Na verdade, a execução de testes de memória abrangentes em flash ou EEPROM geralmente é uma má ideia, porque o número de gravações envolvidas pode encurtar a vida útil do dispositivo. No entanto, se o conteúdo de uma memória híbrida não for importante - como são durante o estágio de desenvolvimento do produto - esses mesmos algoritmos também podem ser usados para testar esses dispositivos. Abordamos o problema de validação do conteúdo de uma memória não volátil na seção “Validando o conteúdo da memória”, posteriormente neste capítulo.

Problemas Comuns de Memória

Antes de aprender sobre algoritmos de teste específicos, você deve estar familiarizado com os tipos de problemas de memória que podem ocorrer. Um equívoco comum entre

engenheiros de software é que a maioria dos problemas de memória ocorre dentro dos próprios chips. Embora já tenha sido um grande problema (algumas décadas atrás), problemas desse tipo são cada vez mais raros. Os fabricantes de dispositivos de memória realizam uma variedade de testes de pós-produção em cada lote de chips. Se houver um problema com um determinado lote, é improvável que um dos chips defeituosos chegue ao seu sistema.

O único tipo de problema de chip de memória que você pode encontrar é uma falha catastrófica. Isso geralmente é causado por algum tipo de dano físico ou elétrico ao chip após a fabricação. Falhas catastróficas são incomuns e geralmente afetam grandes partes do chip. Como uma grande área é afetada, é razoável supor que uma falha catastrófica será detectada por qualquer algoritmo de teste decente.

Em nossa experiência, uma fonte mais comum de problemas de memória é a placa de circuito. Os problemas típicos da placa de circuito são:

- Problemas com a fiação entre o processador e o dispositivo de memória
- Chips de memória ausentes
- Chips de memória inseridos incorretamente

Esses são os problemas que um bom algoritmo de teste de memória deve ser capaz de detectar. Esse teste também deve ser capaz de detectar falhas de memória catastróficas sem procurá-las especificamente. Portanto, vamos discutir os problemas da placa de circuito com mais detalhes.

Problemas de fiação elétrica

Um problema de fiação elétrica pode ser causado por um erro no projeto ou na produção da placa ou como resultado de danos recebidos após a fabricação. Cada um dos fios que conectam o dispositivo de memória ao processador é um dos três tipos:

- Sinal de endereço
- Sinal de dados
- Sinal de controle

Os sinais de endereço e dados selecionam o local da memória e transferem os dados, respectivamente. Os sinais de controle informam ao dispositivo de memória se o processador deseja ler ou gravar a localização e precisamente quando os dados serão transferidos. Infelizmente, um ou mais desses fios podem ser encaminhados incorretamente ou danificados de tal forma que estão *em curto* (ou seja, conectado a outro fio na placa) ou *abrir* (não conectado a nada). O curto é geralmente causado por um pouco de respingo de solda, enquanto um fio aberto pode ser causado por um traço quebrado. Ambos os casos são ilustrados na Figura 6-3.

Problemas com as conexões elétricas do processador farão com que o dispositivo de memória se comporte incorretamente. Os dados podem ser corrompidos quando são armazenados, armazenados no endereço errado ou nem mesmo armazenados. Cada um desses sintomas pode ser explicado por problemas de fiação nos sinais de dados, endereço e controle, respectivamente.

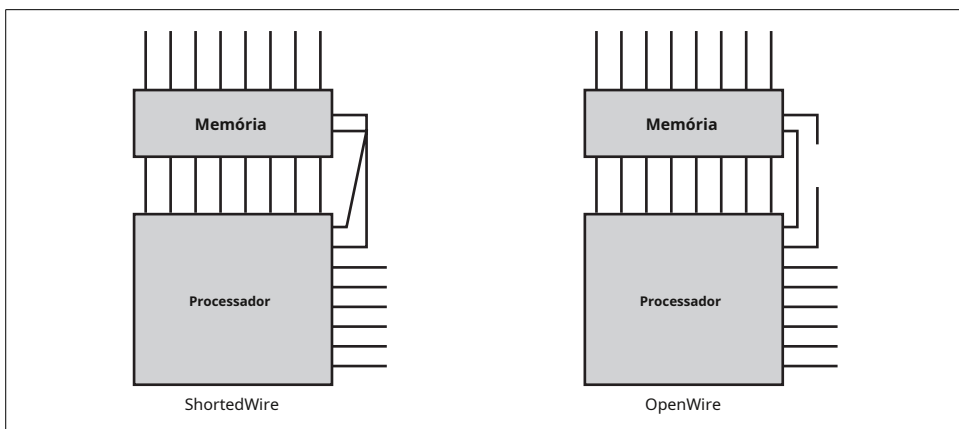


Figura 6-3. Possíveis problemas de fiação

Se o problema for com um sinal de dados, vários bits de dados podem parecer “grudados” (ou seja, dois ou mais bits sempre contêm o mesmo valor, independentemente dos dados transmitidos). Da mesma forma, um bit de dados pode estar “preso alto” (sempre 1) ou “preso baixo” (sempre 0). Esses problemas podem ser detectados escrevendo uma sequência de valores de dados projetada para testar se cada pino de dados pode ser definido como 0 e 1, independentemente de todos os outros.

Se um sinal de endereço tiver um problema de fiação, o conteúdo de duas localizações de memória pode parecer sobrepor-se. Em outras palavras, os dados gravados em um endereço substituirão o conteúdo de outro endereço. Isso acontece porque um bit de endereço em curto ou aberto faz com que o dispositivo de memória veja um endereço diferente daquele selecionado pelo processador.

Outra possibilidade é que um dos sinais de controle esteja em curto ou aberto. Embora seja teoricamente possível desenvolver testes específicos para problemas de sinais de controle, não é possível descrever um teste geral que cubra todas as plataformas. A operação de muitos sinais de controle é específica do processador ou da arquitetura da memória. Felizmente, se houver um problema com um sinal de controle, a memória provavelmente não funcionará e isso será detectado por outros testes de memória. Se você suspeitar de um problema com um sinal de controle, é melhor consultar o projetista da placa antes de construir um teste específico.

Fichas de memória perdidas

Um chip de memória ausente é claramente um problema que deve ser detectado. Infelizmente, devido à natureza capacitiva dos fios elétricos não conectados, alguns testes de memória não detectam esse problema. Por exemplo, suponha que você decidiu usar o seguinte algoritmo de teste: escreva o valor 1 no primeiro local na memória, verifique o valor lendo-o de volta, escreva 2 no segundo local, verifique o valor, escreva 3 no terceiro local, verifique e assim por diante. Porque cada leitura ocorre imediatamente após o correspondente

escrever, é possível que os dados lidos representem nada mais do que a tensão restante no barramento de dados da escrita anterior. Se os dados forem lidos rapidamente, parecerá que os dados foram armazenados corretamente na memória, embora não haja nenhum chip de memória na outra extremidade do barramento!

Para detectar um chip de memória ausente, um teste melhor deve ser usado. Em vez de realizar a leitura de verificação imediatamente após a gravação correspondente, execute várias gravações consecutivas seguidas pelo mesmo número de leituras consecutivas. Por exemplo, escreva o valor 1 no primeiro local, escreva o valor 2 no segundo local, escreva o valor 3 no terceiro local e, em seguida, verifique os dados no primeiro local, no segundo local e assim por diante. Se os valores dos dados forem únicos (como são no teste que acabamos de descrever), o chip ausente será detectado: o primeiro valor lido corresponderá ao último valor escrito (3) ao invés do primeiro (1).

Chips inseridos incorretamente

Se um chip de memória estiver presente, mas inserido incorretamente, alguns pinos do chip de memória não serão conectados à placa de circuito ou serão conectados no lugar errado. Esses pinos farão parte do barramento de dados, barramento de endereço ou fiação de controle. O sistema geralmente se comporta como se houvesse um problema de fiação ou um chip ausente. Portanto, desde que você teste problemas de fiação e chips ausentes, todos os chips inseridos incorretamente serão detectados automaticamente.

Antes de continuar, vamos revisar rapidamente os tipos de problemas de memória que devemos ser capazes de detectar. Os chips de memória raramente apresentam erros internos, mas se tiverem, eles são tipicamente catastróficos por natureza e devem ser detectados por qualquer teste. Uma fonte mais comum de problemas é a placa de circuito, onde pode ocorrer um problema de fiação ou um chip de memória pode estar ausente ou inserido incorretamente. Outros problemas de memória podem ocorrer, mas os descritos aqui são os mais comuns e também os mais simples de testar de forma genérica.

Desenvolvendo uma Estratégia de Teste

Selecionando cuidadosamente os dados de teste e a ordem em que os endereços são testados, você pode detectar todos os problemas de memória descritos anteriormente. Geralmente, é melhor dividir o teste de memória em partes pequenas e de propósito único. Isso ajuda a melhorar a eficiência do teste geral e a legibilidade do código. Testes mais específicos também podem fornecer informações mais detalhadas sobre a origem do problema, se algum for detectado.

Descobrimos que é melhor ter três testes de memória individuais, que devem ser executado na seguinte ordem:

1. Teste de barramento de dados
2. Teste de barramento de endereço
3. Teste do dispositivo

Os dois primeiros testam problemas de fiação elétrica e chips inseridos incorretamente; o terceiro destina-se a detectar chips ausentes e falhas catastróficas. Como uma consequência não intencional, o teste do dispositivo também descobrirá problemas com a fiação do barramento de controle, embora não possa fornecer informações úteis sobre a origem de tal problema.

A razão pela qual a ordem é importante é que o teste de barramento de endereço assume um barramento de dados em funcionamento, e os resultados do teste de dispositivo são insignificantes, a menos que os barramentos de endereço e de dados estejam corretos. Se algum dos testes falhar, você deve trabalhar com um engenheiro de hardware para localizar a origem do problema. Observando o valor dos dados ou endereço no qual o teste falhou, ela deve ser capaz de isolar rapidamente o problema na placa de circuito.

Teste de barramento de dados

A primeira coisa que queremos testar é a fiação do barramento de dados. Precisamos confirmar se qualquer valor colocado no barramento de dados pelo processador é recebido corretamente pelo dispositivo de memória na outra extremidade. A maneira mais óbvia de testar isso é escrever todos os valores de dados possíveis e verificar se o dispositivo de memória armazena cada um com sucesso. No entanto, esse não é o teste mais eficiente disponível. Um método mais rápido é testar o barramento um bit de cada vez. O barramento de dados passa no teste se cada bit de dados puder ser definido como 0 e 1, independentemente dos outros bits de dados.

Uma boa maneira de testar cada bit independentemente é realizar o chamado *andando 1's* teste. A Tabela 6-2 mostra os padrões de dados usados em uma versão de 8 bits deste teste. O nome *andando 1's* vem do fato de que um único bit de dados é definido como 1 e "percorreu" toda a palavra de dados. O número de valores de dados a serem testados é igual à largura do barramento de dados. Isso reduz o número de padrões de teste de 2^n para n , onde n é a largura do barramento de dados.

Tabela 6-2. Valores de dados consecutivos para um teste de caminhada 1 de 8 bits

00000001
00000010
00000100
00001000
00010000
00100000
01000000
10000000

Como estamos testando apenas o barramento de dados neste ponto, todos os valores de dados podem ser gravados no mesmo endereço. Qualquer endereço dentro do dispositivo de memória servirá. No entanto, se o barramento de dados se dividir ao passar por mais de um chip de memória, você precisará realizar o teste de barramento de dados em vários endereços - um em cada chip.

Para realizar o teste de caminhada 1, simplesmente escreva o primeiro valor de dados na tabela, verifique-o lendo-o de volta, escreva o segundo valor, verifique e assim por diante. Quando você chega ao final da tabela, o teste está concluído. Desta vez, não há problema em fazer a leitura imediatamente após a gravação correspondente, porque ainda não estamos procurando por chips ausentes. Na verdade, este teste pode fornecer resultados significativos mesmo se os chips de memória não estiverem instalados!

A função `memtestDataBus` mostra como implementar o teste de caminhada 1. Ele assume que o chamador irá selecionar o endereço de teste e testar todo o conjunto de valores de dados naquele endereço. Se o barramento de dados estiver funcionando corretamente, a função retorna 1 e o parâmetro `ppFailAddr` está configurado para NULO. Caso contrário, retorna 0 e o endereço em que o teste falhou é retornado no parâmetro `ppFailAddr`.

```

/* Defina a largura do barramento de dados para 32
bits. */ typedef uint32_t datum;

/ *****
*
* Função:      memtestDataBus
*
* Descrição: Teste a fiação do barramento de dados em uma região de memória por
*             realizando um teste de caminhada 1 em um endereço fixo
*             dentro dessa região. O endereço (e, portanto, o
*             região da memória) é selecionado pelo chamador.
*
* Notas:
*
* Retorna:     0 se o teste falhar. O endereço da falha é retornado no
*             parâmetro ppFailAddr.
*             1 se o teste for bem-sucedido. O parâmetro ppFailAddr é definido
*             como NULL.
*
***** /
int memtestDataBus (datum * pAddress, datum ** ppFailAddr) {

    padrão de dados;

    * ppFailAddr = NULL;

    /* Execute um teste de caminhada 1 no endereço fornecido.
    * / para (padrão = 1; padrão! = 0; padrão <= 1) {

        /* Escreva o padrão de teste. */
        * pAddress = padrão;

        /* Leia de volta (imediatamente está tudo bem para este teste). */
        if (* pAddress! = padrão)
        {
            * ppFailAddr = pAddress;
            return 0;
        }

    }

    return 1;
}

```

Depois de confirmar que o barramento de dados funciona corretamente, você deve testar o barramento de endereço. Problemas de barramento de endereço levam a locais de memória sobrepostos. Existem muitos endereços possíveis que podem se sobrepor. No entanto, não é necessário verificar todas as combinações possíveis. Em vez disso, você deve seguir o exemplo do teste de barramento de dados anterior e tentar isolar cada pino de endereço durante o teste. Você simplesmente precisa confirmar se cada um dos pinos de endereço pode ser definido como 0 e 1 sem afetar nenhum dos outros.

O menor conjunto de endereços que cobrirá todas as combinações possíveis é o conjunto de *potência de dois* endereços. Esses endereços são análogos ao conjunto de valores de dados usados no teste de caminhada 1. Os locais de memória correspondentes são 0x00000001, 0x00000002, 0x00000004, 0x00000008, 0x00000010, 0x00000020 e assim por diante. Além disso, o endereço 0x00000000 deve ser testado. A possibilidade de locais sobrepostos torna o teste de barramento de endereço mais difícil de implementar. Depois de escrever para um dos endereços, você deve verificar se nenhum dos outros foi sobrescrito.

É importante observar que, em alguns casos, nem todos os sinais de endereço podem ser testados dessa maneira. Parte do endereço - os bits mais significativos na extremidade esquerda - seleciona o próprio chip de memória. Outra parte - um ou dois bits menos significativos à direita - pode não ser relevante se o barramento de dados for mais largo do que 8 bits. Esses bits extras devem permanecer constantes durante todo o teste de barramento de endereço e, portanto, reduzirão o número de endereços de teste. Por exemplo, se o processador tem 20 bits de endereço, ele pode endereçar para 1 MB de memória. Se você quiser testar um bloco de memória de 128 KB, ou seja, 1/8 do espaço total de endereço de um megabyte - os 3 bits de endereço mais significativos permanecem constantes. Nesse caso, apenas os 17 bits menos significativos do barramento de endereço podem realmente ser testados.

Para confirmar que não há dois locais de memória sobrepostos, você deve primeiro escrever algum valor de dados inicial em cada deslocamento de potência de dois dentro do dispositivo. Em seguida, escreva um novo valor - uma cópia invertida do valor inicial é uma boa escolha - no primeiro deslocamento de teste e verifique se o valor de dados inicial ainda está armazenado em cada outro deslocamento de potência de dois. Se você encontrar um local (diferente daquele que acabou de escrever) que contém o novo valor de dados, você encontrou um problema com o bit de endereço atual. Se nenhuma sobreposição for encontrada, repita o procedimento para cada um dos deslocamentos restantes.

A função `memtestAddressBus` mostra como isso pode ser feito na prática. A função aceita três parâmetros. O primeiro parâmetro é o endereço base do bloco de memória a ser testado, o segundo é o seu tamanho (em bytes) e o terceiro é utilizado para retornar o endereço da falha, caso ocorra. O tamanho é usado para determinar quais bits de endereço devem ser testados. Para melhores resultados, o endereço base deve conter um 0 em cada um desses bits. Se o teste de barramento de endereço falhar, 0 é retornado e o endereço no qual o primeiro erro foi detectado é retornado no parâmetro `ppFailAddr`. Caso contrário, a função retorna 1 para indicar sucesso e conjuntos `ppFailAddr` para NULO.

```

/ *****
* Função: memtestAddressBus
*
* Descrição: Teste a fiação do barramento de endereço em uma região de memória por
*             realizar um teste de caminhada 1 nos bits relevantes
*             do endereço e verificando se há aliasing. O teste
*             encontrará falhas de endereço de bit único, como travamento
*             pinos altos, presos baixos e em curto. O endereço de base
*             e o tamanho da região são selecionados pelo chamador.
*
* Notas:      Para melhores resultados, o endereço de base selecionado deve ter
*             LSB 0 suficiente para garantir mudanças de bit de endereço único.
*             Por exemplo, para testar uma região de 64 KB, selecione um
*             endereço base em um limite de 64 KB. Além disso, o número de
*             bytes deve descrever um tamanho de região de potência de dois.
*
* Retorna:    0 se o teste falhar. O endereço da falha é retornado no
*             parâmetro ppFailAddr.
*             1 se o teste for bem-sucedido. O parâmetro ppFailAddr é definido
*             como NULL.
*
***** /
int memtestAddressBus (datum * pBaseAddress, uint32_t numBytes, datum ** ppFailAddr)
{
    uint32_t addressMask = (numBytes - 1);
    uint32_t offset;
    uint32_t testOffset;
    padrão de dados = (dato) 0xAAAAAAA;
    datum antipadrão = (datum) ~ padrão;

    * ppFailAddr = NULL;

    / * Escreva o padrão padrão em cada um dos deslocamentos de potência de
    dois. * / for (offset = sizeof (datum); (offset & addressMask)! = 0; offset <= 1)
        pBaseAddress [deslocamento] = padrão;

    / * Verifique se há bits de endereço presos alto.
    * / pBaseAddress [0] = antipadrão;

    para (deslocamento = sizeof (datum); deslocamento & addressMask; deslocamento <= 1)
    {
        if (pBaseAddress [deslocamento]! = padrão) {

            * ppFailAddr = & pBaseAddress
            [deslocamento]; return 0;

        }
    }

    pBaseAddress [0] = padrão;

```

```

/* Verifique se os bits de endereço estão travados ou em curto. */
para (testOffset = sizeof (datum); testOffset & addressMask; testOffset <= 1) {

    pBaseAddress [testOffset] = antipadrão;

    para (deslocamento = sizeof (datum); deslocamento & addressMask; deslocamento <= 1)
    {
        if ((pBaseAddress [offset] != pattern) && (offset != testOffset)) {

            * ppFailAddr = & pBaseAddress
            [deslocamento]; return 0;

        }

        pBaseAddress [testOffset] = padrão;

    }

    return 1;

}

```

Teste de aparelho

Depois de saber que o endereço e a fiação do barramento de dados estão corretos, é necessário testar a integridade do próprio dispositivo de memória. O objetivo é testar se cada bit no dispositivo é capaz de conter 0 e 1. Este teste é bastante simples de implementar, mas leva muito mais tempo para ser executado do que os dois testes anteriores.

Para um teste completo do dispositivo, você deve gravar e verificar cada local de memória duas vezes. Você é livre para escolher qualquer valor de dados para a primeira passagem, contanto que inverta esse valor durante a segunda. E como existe a possibilidade de chips de memória faltarem, é melhor selecionar um conjunto de dados que muda com (mas não é equivalente a) o endereço. Um exemplo simples é um *incremento* teste.

Os valores dos dados para o teste de incremento são mostrados nas duas primeiras colunas da Tabela 6-3. A terceira coluna mostra os valores de dados invertidos usados durante a segunda passagem deste teste. A segunda passagem representa um teste de decremento. Existem muitas outras opções de dados possíveis, mas o padrão de dados de incremento é adequado e fácil de calcular.

Tabela 6-3. Valores de dados para um teste de incremento de 8 bits

Deslocamento de memória	Valor binário	Valor invertido
0x00	00000001	11111110
0x01	00000010	11111101
0x02	00000011	11111100
0x03	00000100	11111011
...	...	...
0xFE	11111111	00000000
0xFF	00000000	11111111

A função `memtestDevice` implementa apenas um teste de incremento / decremento de duas passagens. Ele aceita três parâmetros do chamador. O primeiro parâmetro é o endereço inicial, o segundo é o número de bytes a serem testados e o terceiro é usado para retornar o endereço da falha, se ocorrer. Os primeiros dois parâmetros fornecem ao usuário controle máximo sobre quais áreas da memória são sobrescritas. A função retorna 1 em caso de sucesso, e o parâmetro `ppFailAddr` está configurado para NULO. Caso contrário, 0 é retornado e o primeiro endereço que contém um valor de dados incorreto é retornado no parâmetro `ter ppFailAddr`.

```

/ *****
*
* Função:      memtestDevice
*
* Descrição: Teste a integridade de um dispositivo de memória física por
*             realizando um teste de incremento / decremento sobre o
*             região inteira. No processo, cada bit de armazenamento
*             no dispositivo é testado como um zero e um. o
*             endereço de base e o tamanho da região são
*             selecionado pelo chamador.
*
* Notas:
*
* Retorna:     0 se o teste falhar. O endereço da falha é retornado no
*             parâmetro ppFailAddr.
*             1 se o teste for bem-sucedido. O parâmetro ppFailAddr é definido
*             como NULL.
*
***** /
int memtestDevice (datum * pBaseAddress, uint32_t numBytes, datum ** ppFailAddr) {

    uint32_t offset;
    uint32_t numWords = numBytes / sizeof (datum);
    padrão de dados;

    * ppFailAddr = NULL;

    / * Preencha a memória com um padrão conhecido. * /
    para (padrão = 1, deslocamento = 0; deslocamento < numWords; padrão ++, deslocamento ++)
        pBaseAddress [deslocamento] = padrão;

    / * Verifique cada local e inverta-o para a segunda passagem. * /
    para (padrão = 1, deslocamento = 0; deslocamento < numWords; padrão ++, deslocamento ++) {

        if (pBaseAddress [deslocamento] != padrão) {

            * ppFailAddr = & pBaseAddress
            [deslocamento]; return 0;
        }

        pBaseAddress [deslocamento] = ~ padrão;
    }
}

```

```

/* Verifique cada local para o padrão invertido e zere-o. */
para (padrão = 1, deslocamento = 0; deslocamento < numWords; padrão ++, deslocamento ++) {

    if (pBaseAddress [offset]! = ~ padrão) {

        * ppFailAddr = & pBaseAddress
        [deslocamento]; return 0;

    }

    pBaseAddress [deslocamento] = 0;

}

return 1;

}

```

Juntando tudo

Para tornar nossa discussão mais concreta, vamos considerar um exemplo prático. Suponha que queiramos testar um pedaço de 64 KB da DRAM começando no endereço 0x00500000 na placa Arcom. Para fazer isso, chamaríamos cada uma das três rotinas de teste separadamente. Em cada caso, o primeiro parâmetro é o endereço básico do bloco de memória. A largura do barramento de dados é de 32 bits e há um total de 64 KB a serem testados (correspondendo aos 16 bits mais à direita do barramento de endereço).

Se alguma das rotinas de teste de memória retornar zero, ligaremos imediatamente o LED vermelho para indicar visualmente o erro. Caso contrário, depois que todos os três testes forem concluídos com sucesso, ligaremos o LED verde. Novas funções de LED foram adicionadas, o que permite que os LEDs sejam ligados ou desligados.

Em caso de erro, a rotina de teste que falhou retornará algumas informações sobre o problema encontrado no parâmetro pFailAddr. Essas informações podem ser úteis ao se comunicar com um engenheiro de hardware sobre a natureza do problema. No entanto, as informações retornadas pela função são visíveis apenas se estivermos executando o programa de teste em um depurador ou emulador. Posteriormente, veremos um driver serial que permitirá a entrada e a saída de uma porta serial da placa. Essa pode ser uma ferramenta inestimável para obter saída de depuração de um programa.

A melhor maneira de proceder é assumir o melhor, baixar o programa de teste e deixá-lo ser executado até o fim. Então, se e somente se o LED vermelho acender, você deve examinar os códigos de retorno e o conteúdo da memória para ver qual teste falhou e por quê.

A seguir está o programa a Principal função, que realiza algumas inicializações de LED e, em seguida, executa as funções de teste de memória definidas anteriormente:

```

# inclui "memtest.h"
# inclui "led.h"

# define BASE_ADDRESS          (dado *) (0x00500000)
# define NUM_BYTES             (0x10000)

```



```

/ *****
*
* Função:          a Principal
*
* Descrição: Teste um bloco de DRAM de 64 KB.
*
* Notas:
*
* Retorna:         0 em caso de falha.
*                 1 em sucesso.
*
***** /

int main (vazio)
{
    datum * pFailAddr;

    /* Configure os pinos de controle do LED.
    * / ledInit ();

    /* Certifique-se de que todos os LEDs estejam desligados antes de iniciar o teste de
    memória. * / ledOff (LED_GREEN | LED_YELLOW | LED_RED);

    if ((memtestDataBus (BASE_ADDRESS, & pFailAddr)!= 1) ||
        (memtestAddressBus (BASE_ADDRESS, NUM_BYTES, & pFailAddr)!= 1) ||
        (memtestDevice (BASE_ADDRESS, NUM_BYTES, & pFailAddr)!= 1))
    {
        ledOn (LED_RED);
        return 0;
    }
    outro
    {
        ledOn (LED_GREEN);
        return 1;
    }
}

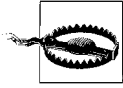
```

Infelizmente, nem sempre é possível escrever testes de memória em uma linguagem de alto nível. Por exemplo, C requer o uso de uma pilha. Mas uma pilha em si requer memória operacional. Isso pode ser razoável em um sistema que possui mais de um dispositivo de memória. Por exemplo, você pode criar uma pilha em uma área da RAM que já esteja funcionando, enquanto testa outro dispositivo de memória. Em uma situação comum, uma pequena SRAM pode ser testada a partir da montagem e a pilha pode ser criada nesta SRAM posteriormente. Em seguida, um bloco maior de DRAM poderia ser testado usando um algoritmo de teste implementado em uma linguagem de alto nível, como a que acabamos de mostrar. Se você não puder assumir RAM de trabalho suficiente para as necessidades de pilha e dados do programa de teste, será necessário reescrever essas rotinas de teste de memória inteiramente em linguagem assembly.



Pode ser possível usar o cache do processador para a pilha. Ou se o processador usa um registrador de link e as variáveis são mantidas em registradores, ainda pode ser possível escrever testes em C sem a necessidade de uma pilha.

Outra opção é executar o programa de teste de memória a partir de um emulador no circuito. Nesse caso, você pode escolher colocar a pilha em uma área da memória interna do emulador. Ao mover a memória interna do emulador no mapa de memória de destino, você pode testar sistematicamente cada dispositivo de memória no destino.



Executar um emulador antes de ter certeza de que seu hardware está funcionando envolve riscos. Se houver uma falha física (elétrica / barramento) em seu sistema, a falha pode destruir seu ICE caro.

Você também precisa ter cuidado para que o cache do processador não o leve a pensar que os testes de memória foram erroneamente bem-sucedidos. Por exemplo, imagine que o processador armazena os dados que você pretendia gravar em um determinado local da memória em seu cache. Quando você lê o local da memória de volta, o processador fornece o valor em cache. Nesse caso, você obtém um resultado válido independentemente de haver um erro de memória real. É melhor executar os testes de memória com o cache (pelo menos o cache de dados) desabilitado.

A necessidade de teste de memória talvez seja mais aparente durante o desenvolvimento do produto, quando a confiabilidade do hardware e seu design ainda não foram comprovados. No entanto, a memória é um dos recursos mais críticos em qualquer sistema embarcado, portanto, também pode ser desejável incluir um teste de memória na versão final do software. Nesse caso, o teste de memória e outros testes de confiança de hardware devem ser executados toda vez que o sistema for ligado ou reiniciado. Juntos, esse conjunto de testes inicial forma um conjunto de diagnósticos de hardware. Se um ou mais diagnósticos falharem, um técnico de reparo pode ser chamado para diagnosticar o problema e reparar ou substituir o hardware defeituoso.

Validando o Conteúdo da Memória

Normalmente não faz sentido realizar o tipo de teste de memória descrito anteriormente ao lidar com ROM ou dispositivos de memória híbrida. Os dispositivos ROM não podem ser gravados de forma alguma, e os dispositivos híbridos geralmente contêm dados ou programas que você não pode sobrescrever porque perderia as informações. No entanto, deve ficar claro que os mesmos tipos de problemas de memória podem ocorrer com esses dispositivos. Um chip pode estar faltando, inserido incorretamente ou danificado física ou eletricamente, ou pode haver um problema na fiação elétrica. Em vez de apenas presumir que esses dispositivos de memória não volátil estão funcionando corretamente, seria melhor ter alguma forma de confirmar se o dispositivo está funcionando e se os dados que ele contém são válidos. É aí que entram as somas de verificação e as verificações de redundância cíclica.

Checksums

Como podemos saber se os dados ou programas armazenados em um dispositivo de memória não volátil ainda são válidos? Uma das maneiras mais fáceis é calcular um *soma de verificação* dos dados quando é

conhecido como válido - antes de programar a ROM, por exemplo. Então, cada vez que você quiser confirmar a validade dos dados, você só precisa recalcular a soma de verificação e comparar o resultado com o valor calculado anteriormente. Se as duas somas de verificação corresponderem, os dados serão considerados válidos. Selecionando cuidadosamente o algoritmo de checksum, podemos aumentar a probabilidade de que tipos específicos de erros sejam detectados, enquanto mantemos o tamanho do checksum e o tempo necessário para verificá-lo em um tamanho razoável.

O algoritmo de soma de verificação mais simples é somar todos os bytes de dados (ou - se você preferir uma soma de verificação de 16 bits - palavras), o descarte é válido. Uma fraqueza notável desse algoritmo é que se todos os dados (incluindo a soma de verificação armazenada) forem substituídos acidentalmente por 0s, essa corrupção de dados será indetectável; a soma de um grande bloco de zeros também é zero. A maneira mais simples de superar essa fraqueza é adicionar uma etapa final ao algoritmo de soma de verificação: inverter o resultado. Dessa forma, se os dados e a soma de verificação forem de alguma forma substituídos por 0s, o teste falhará porque a soma de verificação adequada seria 0xFF.

Infelizmente, uma soma de verificação simples de soma de dados como esta falha em detectar muitos dos erros de dados mais comuns. Obviamente, se um bit de dados estiver corrompido (alterado de 1 para 0 ou vice-versa), o erro será detectado. Mas e se dois bits da mesma “coluna” estivessem corrompidos alternadamente (o primeiro muda de 1 para 0, o outro de 0 para 1)? A soma de verificação adequada não muda e o erro não seria detectado. Se erros de bit podem ocorrer, você provavelmente desejará usar um algoritmo de soma de verificação melhor. Veremos um deles na próxima seção.

Depois de calcular a soma de verificação esperada, você precisará de um local para armazená-la. Uma opção é calcular a soma de verificação antecipadamente e defini-la como uma constante na rotina que verifica os dados. Este método é atraente para o programador, mas tem várias deficiências. É possível que os dados - e, como resultado, a soma de verificação esperada - mudem durante a vida útil do produto. Isso é particularmente provável se os dados testados forem softwares integrados que serão atualizados periodicamente à medida que bugs são corrigidos ou novos recursos são adicionados.

Uma ideia melhor é armazenar a soma de verificação em algum local fixo da memória não volátil. Por exemplo, você pode decidir usar o último local do dispositivo de memória que está sendo verificado. Isso facilita a inserção da soma de verificação: apenas calcule a soma de verificação e insira-a na imagem da memória antes de programar o dispositivo de memória. Ao recalcular a soma de verificação, simplesmente pule o local que contém o resultado esperado e compare a soma de verificação do tempo de execução com o valor armazenado lá. Outro bom lugar para armazenar a soma de verificação é em outro dispositivo de memória não volátil. Ambas as soluções funcionam muito bem na prática.

Verificações de redundância cíclica

UMA *verificação de redundância Cíclica (CRC)* é um algoritmo de soma de verificação específico projetado para detectar os erros de dados mais comuns. A teoria por trás do CRC é bastante matemática e está além do escopo deste livro. No entanto, os códigos de redundância cíclicos são freqüentemente úteis em aplicativos incorporados que requerem o armazenamento ou transmissão de grandes blocos de dados. O que se segue é uma breve explicação da técnica CRC e algum código-fonte que mostra como ela pode ser implementada em C. Felizmente, você não precisa entender por que os CRCs detectam erros de dados - ou mesmo como eles são implementados - para tirar proveito de sua capacidade de detectar erros.

Aqui está uma breve explicação da matemática. Ao calcular um CRC, pense no conjunto de dados como uma sequência muito longa de 1s e 0s (chamada de *mensagem*). Esta string binária é dividida - de uma forma bastante peculiar - por uma string binária fixa menor chamada de *polinômio gerador*. o *restante* desta divisão longa binária é a soma de verificação CRC. Selecionando cuidadosamente o polinômio gerador para certas propriedades matemáticas desejáveis, você pode usar a soma de verificação resultante para detectar a maioria (mas nunca todos) os erros na mensagem. O mais forte desses polinômios geradores é capaz de detectar todos os erros de bit único e duplo, e todas as sequências de comprimento ímpar de bits de erro consecutivos. Além disso, mais de 99,99 por cento de todos os erros de burst - definidos como uma sequência de bits que tem um erro em cada extremidade - podem ser detectados. Juntos, esses tipos de erros são responsáveis por uma grande porcentagem dos erros possíveis em qualquer mensagem binária armazenada ou transmitida.

Polinômios geradores com os melhores recursos de detecção de erros são freqüentemente adotados como padrões internacionais. Dois desses padrões são descritos na Tabela 6-4. Associado a cada padrão está sua largura (em bits), o polinômio gerador, uma representação binária do polinômio (chamado de *divisor*), um valor inicial para o resto e um valor para a operação OU exclusiva (XOR) com o resto final. *

Tabela 6-4. Parâmetros CRC padrão internacional

Parâmetros	CRC16	CRC32
Tamanho do checksum (largura)	16 bits	32 bits
Polinômio gerador	$X^{16} + X^{15} + X^2 + 1$	$X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1$
Divisor (polinomial)	0x8005	0x04C11DB7
Resto inicial	0x0000	0xFFFFFFFF
Valor XOR final	0x0000	0xFFFFFFFF

* O divisor é simplesmente uma representação binária dos coeficientes do polinômio gerador, cada um dos quais sendo 0 ou 1. Para tornar isso ainda mais confuso, o coeficiente de ordem mais alta do polinômio gerador (sempre 1) é deixado de fora a representação binária. Por exemplo, o polinômio em CRC16 tem quatro coeficientes diferentes de zero. Mas a representação binária correspondente tem apenas três 1s (bits 15, 2 e 0).

O código a seguir pode ser usado para calcular qualquer fórmula CRC que tenha um conjunto semelhante de parâmetros. Para tornar isso o mais fácil possível, definimos todos os parâmetros CRC como constantes. Para selecionar os parâmetros CRC de acordo com o padrão desejado, defina uma (e apenas uma) das macros CRC16 ou CRC32.

```

/* Os parâmetros CRC. Atualmente configurado para CRC16. */
#define CRC_NAME "CRC16"
#define POLYNOMIAL 0x8005
#define INITIAL_REMAINDER 0x0000
#define FINAL_XOR_VALUE 0x0000
#define REFLECT_DATA TRUE
#define REFLECT_REMAINDER TRUE
#define CHECK_VALUE 0xBB3D

/* A largura do cálculo e resultado CRC. */ typedef
uint16_t crc_t;

#define WIDTH (8 * sizeof (crc_t))
#define TOPBIT (1 << (LARGURA - 1))

```

A função `crcCompute` pode ser chamado repetidamente de seu aplicativo para comp
calcule e verifique as somas de verificação CRC.

```

/ *****
*
* Função:      crcCompute
*
* Descrição: Calcula o CRC de uma determinada mensagem.
*
* Notas:
*
* Retorna:      O CRC da mensagem.
*
***** /
crc_t crcCompute (uint8_t const message [], uint32_t numBytes) {

    crc_t resto = INITIAL_REMAINDER;
    uint32_t byte;
    int nBit;

    /* Executa a divisão módulo-2, um byte de cada vez. */
    para (byte = 0; byte < numBytes; byte++)
    {
        /* Traga o próximo byte para o restante. */
        resto ^= (REFLECT_DATA (mensagem [byte]) << (WIDTH - 8));

        /* Executa a divisão módulo-2, um pouco de cada vez.
        * / para (nBit = 8; nBit > 0; nBit--)
        {
            /* Tenta dividir o bit de dados atual. */ if
            (resto > TOPBIT)
                resto = (resto << 1) ^ POLINOMIAL;
            outro
            resto = (resto << 1);
        }
    }
}

```

```

    }
}

/* O resto final é o resultado CRC. */
retornar (REFLECT_REMAINDER (resto) ^ FINAL_XOR_VALUE);
}

```

Uma função chamada `crcFast` que usa uma tabela de pesquisa para calcular um CRC de forma mais eficiente está incluído no site deste livro (<http://www.oreilly.com/catalog/embsys2>). Pré-computando os restos de todos os 256 bytes de dados possíveis com antecedência (`crcInit`) reduz substancialmente a quantidade de processamento feito para cada bit. Esses resultados intermediários são armazenados em uma tabela de pesquisa. Fazendo isso dessa forma, o CRC de uma mensagem grande pode ser calculado um byte por vez, em vez de bit a bit. Isso reduz significativamente o tempo de cálculo do CRC.

Um benefício adicional de dividir o cálculo entre `crcInit` e `crcFast` é esse o `crcInit` função não precisa ser executada no sistema embarcado. Na prática, o `crcInit` A função pode ser chamada durante a sequência de inicialização do destino (colocando assim a tabela CRC na RAM), ou pode ser executada antecipadamente em seu computador de desenvolvimento com os resultados armazenados na ROM do dispositivo de destino. Os valores na tabela são então referenciados continuamente por `crcFast`.

Usando FlashMemory

A memória flash oferece vantagens em relação a outros tipos de memória. Os sistemas com memória flash podem ser atualizados em campo para incorporar novos recursos ou correções de bugs descobertos após o produto ter sido enviado. Isso pode eliminar a necessidade de enviar a unidade de volta ao fabricante para atualizações de software. Existem vários problemas que precisam ser considerados ao atualizar o software para unidades no campo.

Limite o tempo de inatividade

O tempo de atualização deve ocorrer durante o tempo de inatividade. Uma vez que a unidade provavelmente não será capaz de funcionar em sua capacidade total durante a atualização, você precisa se certificar de que a unidade não está realizando uma tarefa crítica. O cliente terá que ditar o horário mais conveniente.

Falha de energia

Como a unidade se recuperará se a energia for removida (intencionalmente ou não) enquanto a atualização estiver ocorrendo? Se apenas alguns bytes da imagem do aplicativo foram programados para flash quando a energia é removida, você precisa encontrar uma maneira de determinar se ocorreu um erro e evitar que o código seja executado. Uma solução pode ser incluir um carregador (semelhante a um monitor de depuração) que não pode ser apagado porque reside em setores flash protegidos. Uma das tarefas de inicialização do carregador é verificar a memória flash para uma imagem de aplicativo válida (ou seja, para uma soma de verificação válida). Se uma imagem válida não estiver presente, o carregador precisa saber como colocar uma imagem válida na placa, via porta serial, rede ou algum outro meio.

Outra solução para falhas de energia pode ser incluir um dispositivo de memória flash grande o suficiente para armazenar duas imagens de aplicativos: a imagem atual e a imagem antiga. Quando um novo firmware está disponível, a imagem antiga é substituída pelo novo software; a imagem atual é deixada sozinha. Somente depois que a imagem for programada e verificada adequadamente, ela se tornará a imagem atual. Essa técnica garante que a unidade sempre tenha uma imagem de aplicativo válida para executar caso algo de ruim aconteça durante o procedimento de atualização.

Upgrade de execução de código

De qual chip de memória o software será executado durante o apagamento e a programação do novo software? O software que baixa a imagem pode ser executado a partir da memória flash; entretanto, o código para apagar e reprogramar um chip flash pode precisar ser executado a partir de outro dispositivo de memória.

Requisitos de tempo do dispositivo

É importante compreender os requisitos de tempo do programa e ciclos de apagamento para o dispositivo flash específico. É melhor certificar-se de que todos os dados estão presentes (e validados) antes de iniciar o ciclo de programação. Você não gostaria de iniciar a programação do dispositivo e, em seguida, ser pego esperando que o resto do novo software chegue por meio de uma conexão de rede. O dispositivo pode ter limites de tempo para programar e apagar ciclos que fazem com que o dispositivo volte ao modo de leitura se esses limites forem excedidos. O driver de dispositivo flash falhará ao gravar os dados se isso ocorrer.

Validade da imagem do software

É importante validar a imagem gravada no flash. Isso garantirá que o software seja recebido na unidade corretamente. O algoritmo CRC apresentado anteriormente neste capítulo pode ser suficiente para satisfazer a validade do software de atualização.

Segurança

Se a segurança da imagem for um problema, talvez você precise encontrar um algoritmo para assinar digitalmente e / ou criptografar o novo software. A validação e descryptografia do software seriam então realizadas antes de programar o novo software na memória flash.

Trabalhando com FlashMemory

Do ponto de vista do programador, o flash é indiscutivelmente o dispositivo de memória mais complicado já inventado. A interface de hardware melhorou um pouco desde que os dispositivos originais foram introduzidos em 1988, mas ainda há um longo caminho a percorrer. Ler da memória flash é rápido e fácil, como deveria ser. Na verdade, ler dados de um flash não é muito diferente de ler de qualquer outro dispositivo de memória. O processador simplesmente fornece o endereço e o dispositivo de memória retorna os dados armazenados naquele local. A maioria dos dispositivos flash entra neste tipo de modo de "leitura" automaticamente sempre que o sistema é reinicializado; nenhuma sequência de inicialização especial é necessária para habilitar a leitura.

Gravar dados em um flash não é tão simples. Dois fatores tornam a gravação difícil. Em primeiro lugar, cada local da memória deve ser apagado antes de ser reescrito. Se os dados antigos não forem apagados, o resultado da operação de gravação será uma combinação matemática dos valores antigos e novos.

A segunda coisa que dificulta a gravação em um flash é que pelo menos um setor, ou bloco, do dispositivo deve ser apagado; é impossível apagar um único byte. O tamanho de um setor individual varia de dispositivo para dispositivo, mas cada setor geralmente é da ordem de vários kilobytes. Além disso, dentro do mesmo dispositivo, diferentes tamanhos de setor podem ser usados.

Uma outra pequena diferença é digna de nota: os ciclos de apagamento e gravação demoram mais do que o ciclo de leitura.

Drivers de Flash

O processo de apagar os dados antigos e gravar os novos varia de um fabricante para outro e geralmente é bastante complicado. Essas interfaces de programação de dispositivo são tão complicadas que geralmente é melhor adicionar uma camada de software para tornar a memória flash mais fácil de usar. Se implementado, esta camada específica de hardware de software é geralmente chamada de *driver de flash*.

O objetivo de um driver de dispositivo em geral é ocultar os detalhes de um dispositivo específico do software aplicativo. Nesse caso, o driver flash contém o método específico para gravar e apagar um dispositivo flash específico. O driver flash deve apresentar um simples *interface de programação do aplicativo* (API) que consiste nas operações de apagar e gravar. Partes do software aplicativo que precisam modificar os dados armazenados na memória flash simplesmente chamam o driver e deixam que ele cuide dos detalhes. Isso permite que o programador do aplicativo faça solicitações de alto nível, como “Apagar o bloco no endereço 0xD0000000” ou “Gravar um bloco de dados, começando no endereço 0xD4000000”. Rotinas de driver distintas também mantêm o código específico do dispositivo separado, para que possa ser facilmente modificado se o dispositivo flash de outro fabricante for usado posteriormente.

Os fabricantes de dispositivos Flash geralmente incluem drivers de dispositivos em seus sites. Se você está procurando algum código de exemplo, esses sites são um ótimo lugar para começar. Parte do código de exemplo pode cobrir as operações básicas. Em particular, essas implementações podem não lidar com nenhum dos possíveis erros do chip. E se a operação de apagamento nunca for concluída? Você vai querer pensar nos problemas que podem surgir ao implantar suas rotinas e adicionar verificação de erros, se necessário. Implementações mais robustas podem usar um tempo limite de software como backup. Por exemplo, se o dispositivo flash não responder dentro de duas vezes o tempo máximo esperado (conforme declarado na folha de dados do dispositivo), a rotina pode interromper a pesquisa e indicar o erro ao chamador (ou usuário) de alguma forma.

Outro aprimoramento de recurso é incluir código para novas tentativas. Se um apagamento ou ciclo do programa falhar, o código pode repetir automaticamente a operação antes de retornar uma falha ao aplicativo de chamada.

Outra coisa que as pessoas às vezes fazem com a memória flash é implementar um pequeno sistema de arquivos (semelhante à parte FIS do RedBoot). Como a memória flash fornece armazenamento não volátil que também pode ser regravado, ela pode ser considerada semelhante a qualquer outro sistema de armazenamento secundário, como um disco rígido. No entanto, você também deve ter em mente a limitação do ciclo de gravação do flash. No caso do sistema de arquivos, as funções fornecidas pelo driver seriam mais orientadas a arquivos. Funções padrão do sistema de arquivos, como abrir, fechar, ler, e escrever fornecem um bom ponto de partida para a API do driver. A estrutura do sistema de arquivos subjacente pode ser tão simples ou complexa quanto seu sistema exigir. No entanto, um formato bem conhecido, como a estrutura File Allocation Table (FAT), usado pelo DOS, é bom o suficiente para a maioria dos projetos incorporados.

Periféricos

Cada pizza desliza em um slot como uma placa de circuito em um computador, se encaixando no lugar enquanto a caixa inteligente faz interface com o sistema de bordo do carro. O endereço do cliente é comunicado ao carro, que calcula e projeta a rota ideal em um display heads-up.

- Neal Stephenson

Queda de neve

Além do processador e da memória, a maioria dos sistemas incorporados contém vários outros dispositivos de hardware. Alguns desses dispositivos são específicos para cada domínio de aplicativo do sistema embarcado, enquanto outros - como temporizadores / contadores e portas seriais - são úteis em uma ampla variedade de sistemas. Os dispositivos mais comumente usados são frequentemente incluídos no mesmo chip do processador e são chamados *interno*, ou *onchip*, periféricos. Os dispositivos de hardware que residem fora do chip do processador são, portanto, chamados de periféricos externos. Neste capítulo, discutiremos os problemas de software mais comuns que surgem durante a interface com um periférico de qualquer tipo.

Registros de controle e status

Um processador embutido interage com um dispositivo periférico por meio de um conjunto de registros de controle e status. Esses registros são parte do hardware periférico e suas localizações, tamanhos e significados individuais são recursos do periférico. Por exemplo, os registros em um controlador serial são muito diferentes daqueles em um temporizador. Nesta seção, descreveremos como manipular o conteúdo desses registros de controle e status diretamente de seus programas em linguagem C.

Conforme discutido no Capítulo 2, dependendo do projeto do processador e da placa, os dispositivos periféricos estão localizados no espaço de memória do processador ou no espaço de E / S. De longe, o mais comum dos dois tipos são os periféricos mapeados na memória, que geralmente são mais fáceis de trabalhar.

O controle mapeado pela memória e os registradores de status podem ser feitos para se parecerem com variáveis comuns. Para fazer isso, você precisa simplesmente declarar um ponteiro para o registrador, ou bloco de registradores, e definir o valor do ponteiro explicitamente. O código de exemplo dos capítulos anteriores já demonstrou acesso aos registradores periféricos, mas vamos dar uma olhada no código. Os registradores GPIO no PXA255 são mapeados na memória para que os ponteiros para os registradores se pareçam com um ponteiro para qualquer outra variável inteira. O código a seguir declara a variável `pGpio0Set` como um ponteiro para um `uint32_t` - um valor de 32 bits que representa o registro do dispositivo - e inicializa explicitamente a variável com o endereço `0x40E00018`. Desse ponto em diante, o ponteiro para o registrador se parece com um ponteiro para qualquer outra variável inteira.

```
uint32_t * pGpio0Set = (uint32_t *) (0x40E00018);
```

Observe, no entanto, uma diferença muito importante entre os registros do dispositivo e as variáveis comuns na memória local. O conteúdo de um registro de dispositivo pode ser alterado sem o conhecimento ou intervenção de seu programa. Isso porque o conteúdo do registro também pode ser modificado pelo hardware periférico. Por outro lado, o conteúdo de uma variável na memória não mudará, a menos que seu programa os modifique explicitamente. Por esse motivo, dizemos que o conteúdo de um registro de dispositivo é *volátil*, ou sujeito a alteração sem aviso prévio.

A palavra-chave *volátil* deve ser usado ao declarar ponteiros para registradores de dispositivo. Isso avisa o compilador para não fazer suposições sobre os dados armazenados naquele endereço. Por exemplo, se o compilador vê uma gravação no local volátil seguido por outra gravação no mesmo local, ele não assumirá que a primeira gravação é um uso desnecessário dos ciclos do processador. E no caso de leituras, ele não assumirá que uma segunda leitura da mesma localização retornará o mesmo resultado, como faria com uma variável.

Aqui está um exemplo que usa a palavra-chave *volátil* para avisar o compilador sobre o registro GPIO Pin Output Set. O objetivo desta função é escrever o valor do registro em dois momentos diferentes, definindo assim dois pinos GPIO diferentes altos em momentos diferentes:

```
uint32_t volatile * pGpio0Set = (uint32_t volatile *) (0x40E00018);
```

```
void gpioFunction (void) {  
    /* Defina o pino 0 GPIO alto. */  
    * pGpio0Set = 1;           /* Primeira gravação. */  
  
    delay_ms (1000);  
  
    /* Definir GPIO pino 1 alto. */  
    * pGpio0Set = 2;           /* Segunda gravação. */  
}
```

Se o *volátil* palavra-chave não foi usada para declarar a variável `pGpio0Set`, o otimizador teria permissão para alterar a operação do código. Por exemplo, o compilador pode remover a configuração de `pGpio0Set` para 1 no código anterior porque o

o compilador não consegue ver nenhum propósito para esta configuração. Se o compilador intervisse dessa maneira, os pinos GPIO não funcionariam como o desenvolvedor de software pretendia. Então o volátil palavra-chave instrui a fase de otimização da compilação para deixar todas as alterações em uma variável no local e assumir que o conteúdo da variável não pode ser previsto por estados anteriores.

Seria errado interpretar a declaração de declaração de `pGpio0Set` para significar que o próprio ponteiro é volátil. Na verdade, o valor da variável `pGpio0Set` permanecerá `0x40E00018` durante a duração do programa (a menos que seja alterado em outro lugar, é claro). Os dados apontados, em vez disso, estão sujeitos a alterações sem aviso prévio. Este é um ponto muito sutil e pensar muito sobre isso pode confundi-lo. Lembre-se de que a localização de um registro é fixa, embora seu conteúdo possa não ser. E se você usar o volátil palavra-chave, o compilador assumirá o mesmo.

Você também pode notar que o ponteiro para o registro `GPSR0` é declarado como um inteiro sem sinal. Os registros às vezes consistem em vários subcampos e quase todos os valores são positivos por definição. Por essas razões, os programadores incorporados geralmente usam não assinado tipos inteiros para registros periféricos.



Inteiros com sinal podem ser necessários ao ler amostras de um conversor analógico-digital (conversor A / D ou ADC).

Manipulação de bits

Os operadores bit a bit da linguagem C podem ser usados para manipular o conteúdo dos registradores. Esses operadores são `&` (AND), `|` (OU), `~` (NÃO), `^` (XOR), `<<` (deslocamento para a esquerda) e `>>` (deslocamento para a direita). O código de exemplo nas seções a seguir mostra como testar, definir, limpar e alternar bits individuais por meio de um ponteiro para um registro de status do temporizador chamado `pTimerStatus`.

Nesta seção, numeraremos os bits da maneira que você precisa considerá-los ao criar máscaras. O bit menos significativo é denominado bit 0 e pode ser representado em uma máscara hexadecimal como `0x01`; o bit mais significativo em um byte é chamado de bit 7 e pode ser representado em uma máscara hexadecimal como `0x80`.

Bits de teste

O código a seguir testa se o bit 3 está definido no registro de status do temporizador usando o operador `&`:

```
if (*pTimerStatus & 0x08) {  
    /* Faça algo aqui ... */  
}
```

Neste caso, vamos imaginar que o valor no registrador de status do temporizador, contido na variável `pTimerStatus`, é `0x4C`; o operador `&` executa uma operação AND com `0x08`. A operação é semelhante a esta:

```

      0 1 0 0 1 1 0 0 (0x4C)
E (&)
      0 0 0 0 1 0 0 0 (0x08)
      =====
      0 0 0 0 1 0 0 0 (0x08)

```

Como o bit adequado é definido no registro, o código entra no E se demonstração.

Bits de configuração

Para definir o bit 4, o | operador é usado conforme mostrado no seguinte código:

```
* pTimerStatus |= 0x10;
```

resultando em:

```

      0 1 0 0 1 1 0 0 (0x4C)
OU (|)
      0 0 0 1 0 0 0 0 (0x10)
      =====
      0 1 0 1 1 1 0 0 (0x5C)

```

Limpando bits

O código para limpar o bit 2 usa os operadores & e ~ da seguinte maneira:

```
* pTimerStatus &= ~ (0x04);
```

Para esta operação, o inverso de 0x04 é igual a 0xFB. O operador &= define o bit 2 do registrador de status do temporizador como 0, enquanto deixa todos os outros bits inalterados. A operação é semelhante a esta:

```

      0 1 0 1 1 1 0 0 (0x5C)
E (&)
NOT (~) 1 1 1 1 1 0 1 1 (0xFB)
      =====
      0 1 0 1 1 0 0 0 (0x58)

```

Observe que todos os bits no registro permanecem os mesmos, exceto para o bit que queremos limpar.

Bits alternados

Às vezes é útil mudar um pouco para frente e para trás. Por exemplo, você pode querer acender e apagar um LED. Você também pode querer alternar um pouco para frente e para trás, sem ter que verificar primeiro, e definir ou limpar explicitamente. A alternância é feita em C com o ^ operador. Aqui está o código para alternar o bit 7 no registro de status do temporizador:

```
* pTimerStatus ^= 0x80;
```

Isso resulta na seguinte operação:

```

      0 1 0 1 1 0 0 0 (0x58)
XOR (^)
      1 0 0 0 0 0 0 0 (0x80)
      =====
      1 1 0 1 1 0 0 0 (0xD8)

```

Mudando os bits

Outro tipo de operação bit a bit útil é um deslocamento. Por exemplo, considere o que acontece com o valor do inteiro não assinado de 8 bits `bitCount` que contém `0xAC` e é deslocado para a direita em 1 bit. O código que demonstra uma mudança para a direita segue:

```
bitCount >> = 1;
```

Isto resulta em:

```
      1 0 1 0 1 1 0 0 (0xAC)
    >> por 1
    =====
      0 1 0 1 0 1 1 0 (0x56)
```

Neste caso, um 0 é deslocado da esquerda. No entanto, o padrão C também permite que o bit mais significativo seja repetido quando a variável é assinada. Recomendamos que você use números inteiros não assinados para variáveis nas quais executa operações de bits, de forma que não tenha que se preocupar com os resultados diferentes em compiladores diferentes.

Assuma o valor do inteiro sem sinal de 8 bits `bitCount` é novamente `0xAC` e é deslocado para a esquerda em 2 bits:

```
bitCount << = 2;
```

Isto resulta em:

```
      1 0 1 0 1 1 0 0 (0xAC)
    << por 2
    =====
      1 0 1 1 0 0 0 0 (0xB0)
```

Uma razão para usar um deslocamento é se você deseja realizar uma operação em cada bit de um registrador por vez; você pode criar uma máscara de bits (discutida na próxima seção) com 1 bit definido ou limpo e deslocá-lo para que possa operar nos bits individuais do registrador.

Bitmasks

UMA *bitmask* é uma constante freqüentemente usada junto com operadores bit a bit para manipular um ou mais bits em um campo inteiro maior. Uma máscara de bits é um padrão binário constante, como o literal hexadecimal de 16 bits `0x00FF`, que pode ser usado para mascarar bits específicos. Bitmasks podem ser usados com operadores bit a bit para definir, testar, limpar e alternar bits. A seguir estão exemplos de bitmasks para o registro de status do cronômetro:

```
# define TIMER_COMPLETE      (0x08)
# define TIMER_ENABLE       (0xC0)
```

As máscaras de bits `TIMER_COMPLETE` e `TIMER_ENABLE` são nomes descritivos que correspondem a bits específicos no registro de um periférico. Usando um símbolo (por exemplo, `# definir`) bitmask permite que você escreva um código mais descritivo e quase comentado por si mesmo. Substituindo literais hexadecimais por palavras, a definição torna mais fácil para você

(ou outra pessoa) para entender o código posteriormente. Aqui está um exemplo de operação bit a bit envolvendo uma máscara de bits:

```
if (*pTimerStatus & TIMER_COMPLETE) {  
  
    /* Faça algo aqui ... */  
}
```

Bitmask Macros

Aqui está uma macro útil que o ajudará a evitar erros de digitação em literais hexadecimais longos:

```
# define BIT (X)                (1 << (X))
```

Para definir um bit de registro específico em uma máscara de bits, como o bit 22, use a macro da seguinte maneira:

```
# define TIMER_STATUS           BIT (22)
```

Campos de bits

UMA *bitfield* é um campo de um ou mais bits dentro de um valor inteiro maior. Bitfields são úteis para manipulações de bits e são suportados em uma estrutura por compiladores da linguagem C.

```
estrutura  
{  
    uint8_t bit0      : 1;  
    uint8_t bit1      : 1;  
    uint8_t bit2      : 1;  
    uint8_t bit3      : 1;  
    nibble uint8_t: 4; } foo;
```

Os bits dentro de um campo de bits podem ser definidos, testados, apagados e alternados individualmente sem afetando o estado dos outros bits fora do campo de bits.

Para testar bits usando o campo de bits, use o código como o seguinte:

```
if (foo.bit0)  
{  
    /* Faça outras coisas. */  
}
```

Veja como testar um campo mais amplo (como dois bits) usando um campo de bits:

```
if (foo.nibble == 0x03) {  
  
    /* Faça outras coisas. */  
}
```

Para definir um bit usando um campo de bits, use este código:

```
foo.bit1 = 1;
```

E use um código como o seguinte para definir vários bits em um campo de bits:

```
foo.nibble = 0xC;
```

Para limpar um bit usando o campo de bits, use este código:

```
foo.bit2 = 0;
```

E para alternar um pouco usando o campo de bits, use isto:

```
foo.bit3 = ~ foo.bit3; / * ou! foo.bit3 * /
```

Existem alguns problemas dos quais você deve estar ciente caso decida usar bitfields. Bitfields não são portáteis; alguns compiladores partem do bit menos significativo, enquanto outros partem do bit mais significativo. Em alguns casos, o compilador pode exigir que o campo de bits seja encerrado em uma união; fazer isso torna o código de campo de bits portátil entre compiladores ANSI C.

No exemplo a seguir, usamos uma união para conter o campo de bits. Além de tornar o código de bitfield portátil, o sindicato fornece um acesso mais amplo ao registro.

```
União
{
    uint8_t byte;
    estrutura
    {
        uint8_t bit0      : 1;
        uint8_t bit1      : 1;
        uint8_t bit2      : 1;
        uint8_t bit3      : 1;
        nibble uint8_t: 4; } bits;
} foo;
```

Em vez de acessar apenas bits individuais, o registro pode ser gravado como um todo. Por exemplo, a união de bitfield, junto com bitmasks, pode ser útil ao inicializar um registro, conforme mostrado aqui:

```
foo.byte = (TIMER_COMPLETE | TIMER_ENABLE);
```

enquanto os bits individuais ainda estão acessíveis, conforme mostrado aqui:

```
foo.bits.bit2 = 1;
```

Bitmasks são mais eficientes do que bitfields em certos casos. Especificamente, uma máscara de bits é geralmente a melhor maneira de inicializar vários bits em um registro. Por exemplo, o código a seguir inicializa o registro de status do temporizador, definindo os dois bits denotados pelas macros e apagando todos os outros:

```
* pTimerStatus = (TIMER_COMPLETE | TIMER_ENABLE);
```

Definir e limpar bits usando um campo de bits não é mais rápido do que usar uma máscara de bits; com alguns compiladores, pode ser mais lento usar um campo de bits. Uma vantagem de usar campos de bits é que campos de bits individuais podem ser declarados volátil ou const. Isso é útil quando um registro é gravável, mas contém alguns bits somente leitura.

Registros Únicos

Alguns registradores (ou bits dentro de um registrador) podem ser somente leitura ou somente gravação. Para registros somente de gravação, as operações de leitura-modificação-gravação (como `| =`, `& =` e `^ =`) não podem ser usadas. Nesse caso, uma cópia de sombra do conteúdo do registro deve ser mantida em uma variável na RAM para manter o estado atual do registro somente de gravação. Um exemplo de um registro somente gravação usando uma cópia de sombra `timerRegValue` segue:

```
/* Inicializa o registro somente gravação do temporizador.  
* / timerRegValue = TIMER_INTERRUPT;  
* pTimerReg = timerRegValue;
```

Depois que a cópia de sombra e o registro do temporizador foram inicializados, as gravações subsequentes no registro são realizadas modificando-se primeiro a cópia de sombra `timerRegValue` e então escrever o novo valor no registrador. Por exemplo:

```
timerRegValue | = TIMER_ENABLE;  
* pTimerReg = timerRegValue;
```

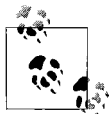
Sobreposições estruturais

Em sistemas embarcados com dispositivos de E / S mapeados em memória, às vezes é útil sobrepor uma estrutura C nos registros de controle e status de cada periférico. Os benefícios das sobreposições de estrutura são que você pode ler e escrever por meio de um ponteiro para a estrutura, o registro é bem descrito pela estrutura, o código pode ser mantido limpo e o compilador faz a construção do endereço no momento da compilação.

O código de exemplo a seguir mostra uma sobreposição de estrutura para um periférico de temporizador. Se os registradores de um periférico não se alinham corretamente, os membros reservados podem ser incluídos na estrutura. Assim, no exemplo a seguir, um campo extra ao qual você nunca se referirá é incluído no deslocamento 4 para que o ao controle campo encontra-se corretamente no deslocamento 6.

```
estrutura de typedef  
{  
    uint16_t count;           /* Offset 0 */  
    uint16_t maxCount;       /* Offset 2 */  
    uint16_t _reserved1;     /* Offset 4 */  
    controle uint16_t;        /* Offset 6 */  
} temporizador volátil_t;
```

```
timer_t * pTimer = (timer_t *) (0xABCD0123);
```



Observe que os campos individuais de uma estrutura, bem como a estrutura inteira, podem ser declarados voláteis.

Quando você usa uma sobreposição de estrutura para acessar registradores, o compilador constrói os endereços de E / S reais mapeados na memória. Os membros da `timer_t struct` definido no exemplo anterior tem os deslocamentos de endereço mostrados na Tabela 7-1.

Tabela 7-1. Deslocamentos de endereço de estrutura periférica do temporizador

Membro da estrutura	Desvio
contar	0x00
maxCount	0x02
_reserved1	0x04
ao controle	0x06

É muito importante ter cuidado ao criar uma sobreposição de estrutura para garantir que os tamanhos e endereços dos registros do periférico subjacente sejam mapeados corretamente.

Os operadores bit a bit mostrados anteriormente para testar, definir, limpar e alternar bits também podem ser usados com uma sobreposição de estrutura. O código a seguir mostra como acessar os registros do periférico do temporizador usando a sobreposição de estrutura. Aqui está o código para testar os bits:

```
if (pTimer-> control & 0x08) {  
  
    /* Faça algo aqui ... */  
}
```

Aqui está o código para definir bits:

```
pTimer-> controle | = 0x10;
```

Aqui está o código para limpar bits:

```
pTimer-> controle & = ~ (0x04);
```

E aqui está o código para alternar bits:

```
pTimer-> controle ^ = 0x80;
```

A filosofia do driver de dispositivo

Quando se trata de projetar drivers de dispositivo, sempre concentre-se em um objetivo facilmente declarado: ocultar o hardware completamente. Esta ocultação do hardware às vezes é chamada *abstração de hardware*. Quando terminar, você deseja que o módulo do driver de dispositivo seja a única parte do software em todo o sistema que lê e / ou grava os registros de controle e status de um determinado dispositivo diretamente. Além disso, se o dispositivo gerar quaisquer interrupções, a rotina de serviço de interrupção que responde a elas deve ser parte integrante do driver do dispositivo. O driver de dispositivo pode então apresentar uma interface genérica para níveis de software mais altos para acessar o dispositivo. Isso elimina a necessidade do software aplicativo incluir qualquer software específico do dispositivo. Nesta seção, explicaremos por que essa filosofia é universalmente aceita e como pode ser alcançada.

As tentativas de ocultar completamente o hardware são difíceis. Qualquer interface de programação que você selecionar refletirá os amplos recursos do dispositivo. Isso era de se esperar. O objetivo deve ser criar uma interface de programação que não precise ser alterada se o periférico subjacente for substituído por outro em sua classe geral. Por exemplo, todos os dispositivos de memória flash compartilham os conceitos de setores (embora o tamanho do setor possa ser diferente entre os chips). A seguinte interface de programação fornecida para um driver flash deve funcionar com qualquer dispositivo de memória flash:

```
void flashErase (setor uint32_t);  
void flashWrite (uint32_t offset, uint8_t * pSrcAddr, uint32_t numBytes);
```

Essas duas chamadas se parecem muito com a maneira como todos os chips flash funcionam em relação a leituras e gravações. Uma operação de apagamento pode ser realizada apenas em um setor inteiro. Uma vez apagados, bytes ou palavras individuais podem ser regravados. Mas as interfaces aqui escondem os recursos específicos do dispositivo flash e suas funções de níveis de software superiores, conforme desejado.

Os drivers de dispositivo para sistemas embarcados são bastante diferentes de seus equivalentes de PC. Em um computador de uso geral, o núcleo do sistema operacional é diferente dos drivers de dispositivo, que geralmente são escritos por outras pessoas que não os desenvolvedores de aplicativos. O sistema operacional oferece uma interface à qual os drivers devem aderir, enquanto o resto do sistema e os aplicativos dependem dos drivers para fazer isso. Por exemplo, os sistemas operacionais da Microsoft impõem requisitos rígidos na interface do software para uma placa de rede. O driver de dispositivo para uma placa de rede específica deve estar em conformidade com esta interface de software, independentemente dos recursos e capacidades do hardware subjacente. Os programas aplicativos que desejam usar a placa de rede são forçados a usar a API de rede fornecida pelo sistema operacional e não têm acesso direto à própria placa.

Em contraste, o software aplicativo em um sistema embarcado pode acessar facilmente o hardware. Na verdade, como todo o software é geralmente vinculado em uma única imagem binária, pouca distinção é feita entre o software do aplicativo, o sistema operacional e os drivers de dispositivo. Traçar essas linhas e impor restrições de acesso ao hardware são responsabilidades puramente dos desenvolvedores de software. Ambos são decisões de design que os desenvolvedores devem tomar conscientemente. Em outras palavras, os implementadores de software integrado podem trapacear com mais facilidade no design do software do que seus pares não integrados.

Os benefícios de um bom design de driver de dispositivo são três:

- Devido à modularidade, a estrutura geral do software é mais fácil de entender. Além disso, é mais fácil adicionar ou modificar recursos do aplicativo geral conforme ele evolui e amadurece, mesmo em unidades implantadas.
- Como há apenas um módulo que interage diretamente com os registros do periférico, o estado do dispositivo de hardware pode ser rastreado com mais precisão.
- As alterações de software que resultam de alterações de hardware são localizadas no driver do dispositivo, tornando o software mais portátil.

Cada um desses benefícios pode e vai ajudar a reduzir o número total de bugs em seu software embarcado e aumentar a capacidade de reutilização de seu código nos sistemas. Mas você tem que estar disposto a colocar um pouco mais de esforço no início, no momento do design, para realizar a economia.

A Figura 7-1 mostra as camadas básicas de software para um sistema embarcado. Conforme mostrado nesta figura, um driver de dispositivo fica logicamente logo acima do hardware e contém o “conhecimento” de como operar aquela parte específica do hardware.

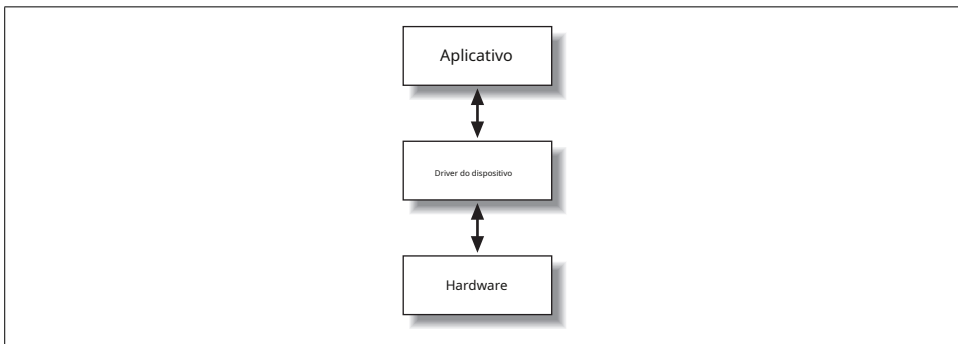


Figura 7-1. Camadas de software de sistema embarcado

Como o driver do dispositivo contém o código para operar o hardware, o software do aplicativo não precisa ser complicado por esses detalhes. Por exemplo, olhando para o programa de LED piscando, o arquivo *led.c* é o driver do dispositivo LED. Este arquivo contém todo o conhecimento sobre como inicializar e operar o LED na placa Arcom. O driver do dispositivo LED fornece uma API que consiste em *ledInit* e *ledToggle*. O aplicativo em *blink.c* usa esta API para alternar o LED. A aplicação não precisa se preocupar com o funcionamento dos registros GPIO no PXA255 para que o LED execute determinada tarefa.

A filosofia de ocultar todas as especificações e interações de hardware dentro do driver de dispositivo geralmente consiste nos cinco componentes da lista a seguir. Para tornar a implementação do driver o mais simples e incremental possível, esses elementos devem ser desenvolvidos na ordem em que são apresentados.

- 1 Uma interface para os registros de controle e status.

Para uma E / S mapeada por memória comumente usada, a primeira etapa no processo de desenvolvimento do driver é criar uma representação dos registros mapeados por memória de seu dispositivo. Isso geralmente envolve o estudo do livro de dados do periférico e a criação de uma tabela dos registros de controle e status e seus deslocamentos. O método para representar os registros de controle e status pode ser qualquer estilo que você se sinta confortável em implementar.

- 2 Variáveis para rastrear o estado atual dos dispositivos físicos (e lógicos).

A segunda etapa no processo de desenvolvimento de driver é descobrir quais variáveis de estado você precisará. Por exemplo, provavelmente precisaremos definir variáveis para

lembre-nos se o hardware foi inicializado. Registradores somente gravação também são bons candidatos para variáveis de estado.

Alguns drivers de dispositivo criam mais de um dispositivo de software para o hardware subjacente. A instância adicional é puramente *dispositivo lógico* que é implementado por cima do hardware periférico básico. Pense em um cronômetro, por exemplo. É fácil imaginar que mais de um temporizador de software poderia ser criado a partir de uma única unidade de temporizador / contador de hardware. A unidade de temporizador / contador seria configurada para gerar um tique de relógio periódico, digamos, a cada milissegundo, e o driver de dispositivo, então, gerenciaria um conjunto de temporizadores de software de vários comprimentos, mantendo informações de estado para cada um.

3. Uma rotina para inicializar o hardware para um estado conhecido.

Depois de saber como rastrear o estado do dispositivo físico (e lógico), você pode começar a escrever as funções que realmente interagem e controlam o hardware. Provavelmente, é melhor começar com a rotina de inicialização do hardware. Você precisará desse primeiro de qualquer maneira, e é uma boa maneira de se familiarizar com a interação do dispositivo.

4. Uma API para usuários do driver de dispositivo.

Depois de inicializar o dispositivo com sucesso, você pode começar a adicionar outras funcionalidades ao driver. Um primeiro passo no design do driver de dispositivo é definir os nomes e objetivos das várias rotinas, bem como seus respectivos parâmetros e valores de retorno. Após essa etapa, tudo o que resta fazer é implementar e testar cada função da API. Veremos exemplos de tais rotinas na próxima seção.

5. Interromper rotinas de serviço.

É melhor projetar, implementar e testar a maioria das rotinas de driver de dispositivo antes de habilitar interrupções pela primeira vez. Localizar a origem dos problemas relacionados à interrupção pode ser bastante desafiador. Se você adicionar possíveis bugs presentes em outros módulos de driver à mistura, pode até se tornar impossível. É muito melhor usar a votação para fazer as entranhas do motorista funcionarem. Dessa forma, você saberá como o dispositivo funciona (e se está realmente funcionando) quando começar a procurar a origem de seus problemas de interrupção - e é quase certo que haverá problemas.

Um driver de dispositivo serial

O exemplo de driver de dispositivo que vamos discutir foi projetado para controlar uma porta serial. O hardware para este driver de dispositivo usa um UART (que é pronunciado “você-arte” e significa *Transmissor receptor assíncrono universal*) periférico contido no processador PXA255. Um UART é um componente que recebe e transmite dados seriais assíncronos. *Assíncrono* significa que os dados podem chegar em intervalos inesperados, semelhantes à entrada de um teclado. Um UART aceita um byte paralelo do processador. Este byte é serializado e cada bit é transmitido no momento apropriado. A recepção funciona ao contrário.

O processador PXA255 possui quatro UARTs no chip. Para este exemplo, usaremos o Full Function UART (FFUART), que está conectado à porta COM1 da placa Arcom. (Observe que esta é a mesma porta COM usada na placa Arcom pelo RedBoot, eliminando a necessidade de trocar cabos. Os registros FFUART começam no endereço 0x40100000.)

Antes de escrever qualquer software para o driver de dispositivo serial, você deve entender o diagrama de blocos do hardware - isto é, como os sinais vão do periférico para o mundo externo e vice-versa. Isso normalmente inclui examinar a parte relevante do esquema e reunir as planilhas de dados para os diferentes ICs. Um diagrama de blocos para o a porta serial é mostrada na Figura 7-2.

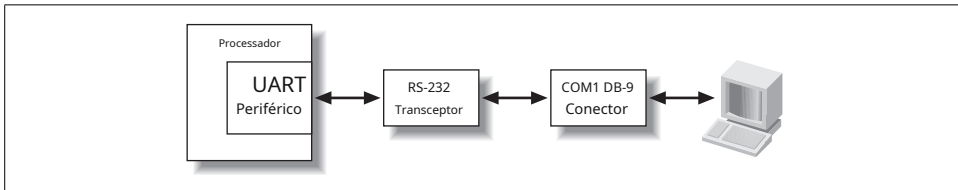


Figura 7-2. Diagrama de blocos da porta serial da placa Arcom

Conforme mostrado na Figura 7-2, o PXA255 UART se conecta ao transceptor RS-232, que então se conecta ao conector COM1 DB-9 na placa Arcom. O transceptor converte o nível de tensão que o processador da placa Arcom usa para níveis de tensão RS-232. Isso permite que o UART da placa Arcom se comunique com um PC por meio de sua porta serial.

A próxima etapa é entender como funciona o periférico específico. Quais ICs precisam ser programados para controlar o periférico? Para este driver serial, precisamos apenas nos concentrar nos registros de periféricos UART no processador. As informações sobre esses registros estão contidas na documentação do processador.

Para obter informações sobre os UARTs do processador PXA255, verifique o *Manual do desenvolvedor do processador PXA255* —Especificamente, Seção 10: UARTs. As informações sobre interrupções estão contidas na Seção 4.2: Controlador de interrupção. Ao ler esta documentação, o objetivo é obter uma compreensão de vários conceitos diferentes, incluindo:

- A estrutura de registro para controlar o periférico, ou seja, como configurar as comunicações e como obter dados para dentro e para fora do periférico
- Os endereços dos registros de controle e status
- O método que será usado para a operação do periférico (ou seja, polling ou interrupções)
- Se estiver usando interrupções, quais condições podem causar interrupções, como o driver do software é informado quando ocorre uma interrupção e como a interrupção é reconhecida

Tenha uma ideia firme do que o driver do dispositivo precisará fazer para que o periférico execute sua tarefa dentro do sistema. Depois que essas etapas iniciais forem concluídas, você pode prosseguir para a tarefa de escrever o software do driver de dispositivo.

Interface de registro

O primeiro passo para o driver de dispositivo serial é definir a interface de registro. Para este exemplo, usamos uma sobreposição de estrutura para os registradores UART, que são mapeados na memória. A sobreposição de estrutura, `uart_t`, é mostrado aqui:

```
estrutura de typedef
{
    uint32_t data;
    uint32_t interruptEnable;
    uint32_t interruptStatus;
    uint32_t uartConfig;
    uint32_t pinConfig;
    uint32_t uartStatus;
    uint32_t pinStatus;
} volatile uart_t;
```

A variável `pSerialPort` é usado para acessar os registros UART no endereço `0x40100000` e é definido como:

```
uart_t * pSerialPort = (uart_t *) (0x40100000);
```

Variáveis de Estado

Em seguida, definimos variáveis para rastrear o estado atual do hardware. Uma estrutura de parâmetros de driver serial chamada `serialparams_t` é definido. A variável global `gSerialParams` é usado no driver de dispositivo serial para encapsular e ajudar a organizar os parâmetros de configuração.

A variável `bInicializado` é usado na rotina de inicialização serial para controlar o estado de configuração do hardware. Você pode notar que no código a seguir, os tipos enumerados são definidos para a paridade (`parity_t`), bits de dados (`databits_t`), e bits de parada (`stopbits_t`). Os enumeradores são definidos para valores de bitmask no registro de configuração UART. As enumerações simplificam a programação de configuração UART e ajudam a tornar o código mais legível.

```
/* Descrições de bits do UART Config Register (LCR) */
#define DATABITS_LENGTH_0 (0x01)
#define DATABITS_LENGTH_1 (0x02)
#define STOP_BITS (0x04)
#define PARITY_ENABLE (0x08)
#define EVEN_PARITY_ENABLE (0x10)

typedef enum {PARITY_NONE, PARITY_ODD = PARITY_ENABLE,
             PARITY_EVEN = (PARITY_ENABLE | EVEN_PARITY_ENABLE)} parity_t;
```

```
typedef enum {DATA_5, DATA_6 = DATABITS_LENGTH_0, DATA_7 = DATABITS_LENGTH_1,
             DATA_8 = (DATABITS_LENGTH_0 | DATABITS_LENGTH_1)} databits_t;
```

```
typedef enum {STOP_1, STOP_2 = STOP_BITS} stopbits_t;
```

estrutura de typedef

```
{
    uint32_t dataBits;
    uint32_t stopBits;
    uint32_t baudRate;
    parity_t parity;
} serialparams_t;
```

```
serialparams_t gSerialParams;
```

Rotina de inicialização

A rotina de inicialização serialInit configura os parâmetros de comunicação padrão para o driver de dispositivo serial. Os registros UART são programados na rotina serialConfig, que é passado no gSerialParams variável. A variável bInicializado é usado para garantir que a porta serial seja configurada apenas uma vez.

```
/ *****
*
* Função:      serialInit
*
* Descrição: Inicialize a porta serial UART.
*
* Notas:      Esta função é específica da placa Arcom. Os
*              parâmetros de comunicação padrão são definidos
*              nesta função.
*
* Retorna:    Nenhum.
*
***** /
void serialInit (void)
{
    static int bInitialized = FALSE;

    /* Inicializa o UART apenas uma vez. */
    if (bInitialized == FALSE)
    {
        /* Defina os parâmetros de comunicação.
        * / gSerialParams.baudRate = 115200;
        gSerialParams.dataBits = DATA_8;
        gSerialParams.parity = PARITY_NONE;
        gSerialParams.stopBits = STOP_1;

        serialConfig (& gSerialParams);

        bInicializado = VERDADEIRO;
    }
}
```


Agora, funcionalidades adicionais podem ser adicionadas definindo outras funções API do driver de dispositivo serial. Uma API de driver de dispositivo serial deve ter funções para enviar e receber caracteres. Para enviar personagens, a função `serialPutChar` é usado; para receber personagens, `serialGetChar` é usado.

A função API do driver de dispositivo serial `serialPutChar` aguarda até que o transmissor esteja pronto e, em seguida, envia um único caractere pela porta serial. A transmissão é feita gravando no registrador de dados UART. O código a seguir mostra o `serialPutChar` função.

```
# define TRANSMITTER_EMPTY          (0x40)

/ *****
*
* Função:      serialPutChar
*
* Descrição:  Envie um personagem através da porta serial.
*
* Notas:      Esta função é específica da placa Arcom.
*
* Retorna:    Nenhum.
*
***** /
void serialPutChar (char outputChar) {

    /* Espere até que o transmissor esteja pronto para o próximo caractere. */
    while ((pSerialPort-> uartStatus & TRANSMITTER_EMPTY) == 0)
        ;

    /* Envia o caractere pela porta serial. */
    pSerialPort-> dados = outputChar;
}
```

A função API do driver de dispositivo serial `serialGetChar` espera até que um caractere seja recebido e então lê o caractere da porta serial. Para determinar se um caractere foi recebido, o bit de dados prontos é verificado no registro de status UART. O caractere recebido é retornado à função de chamada. Aqui está o `serialGetChar` função:

```
# define DATA_READY                (0x01)

/ *****
*
* Função:      serialGetChar
*
* Descrição:  Pegue um caractere da porta serial.
*
* Notas:      Esta função é específica da placa Arcom.
*
* Retorna:    O caractere recebido da porta serial.
*
***** /
char serialGetChar (void)
```

```

{
    /* Aguarde a chegada do próximo caractere. */
    while ((pSerialPort-> uartStatus & DATA_READY) == 0)
        ;

    retornar pSerialPort-> dados;
}

```

Como esse driver de dispositivo serial não usa interrupções, a etapa final na filosofia do driver de dispositivo - implementar rotinas de serviço de interrupção de driver de dispositivo - é ignorada.

Testando o driver de dispositivo serial

Agora que o driver de dispositivo serial está implementado, precisamos verificar se ele funciona corretamente. É importante verificar as funções individuais de sua nova API antes de integrar o driver ao software do sistema.

Para testar o driver de dispositivo serial, a porta COM1 da placa Arcom deve ser conectada à porta serial de um PC. Depois de fazer essa conexão, inicie um programa de terminal, como HyperTerminal ou minicom, no PC. (Os parâmetros da porta serial não precisam ser alterados, porque os parâmetros do driver de dispositivo serial padrão são os mesmos usados pelo RedBoot.)

o a Principal função demonstra como exercitar a funcionalidade do driver de dispositivo serial. Você pode notar que este software tem o início de uma interface de linha de comando - uma ferramenta indispensável comumente implementada em sistemas embarcados.

Primeiro, o driver de dispositivo serial é inicializado chamando serialInit. Em seguida, vários caracteres são produzidos na porta serial do PC para testar o serialPutChar função. Se o driver do dispositivo serial estiver funcionando corretamente, você deverá ver a mensagem começar saída na tela do terminal do seu PC.

Em seguida, um enquanto é inserido um loop que verifica se um caractere foi recebido chamando serialGetChar. Se um caractere entrar na porta serial, ele será ecoado de volta. Se o usuário inserir q no programa do terminal do PC, o programa sai; caso contrário, o loop continua e verifica se há outro caractere de entrada.

```

# inclui "serial.h"

/ *****
*
* Função:          a Principal
*
* Descrição: exercite o driver de dispositivo serial.
*
* Notas:
*
* Retorna:        Essa rotina contém um loop infinito, que pode ser
*                  encerrado inserindo q.
*
***** /

```

```

int main (vazio)
{
    char rcvChar = 0;

    /* Configure o UART para o driver serial. */ serialInit
    ();

    serialPutChar ('s');
    serialPutChar ('t');
    serialPutChar ('a');
    serialPutChar ('r');
    serialPutChar ('t');
    serialPutChar ('\ r');
    serialPutChar ('\ n');

    while (rcvChar != 'q') {

        /* Aguarde um caractere de entrada. */
        rcvChar = serialGetChar ();

        /* Echo o caractere de volta junto com um retorno de carro e avanço de linha. */
        serialPutChar (rcvChar);
        serialPutChar ('\ r');
        serialPutChar ('\ n');
    }

    return 0;
}

```

Estendendo a funcionalidade do driver de dispositivo serial

Embora o driver serial seja muito básico, ele possui funcionalidades centrais que podem ser desenvolvidas para desenvolver um programa mais robusto (e mais útil). Este driver de dispositivo fornece uma plataforma para aprender sobre a operação de UARTs. A seguir está uma lista de extensões possíveis que você pode usar para expandir a funcionalidade deste driver. Lembre-se disso para outros drivers que você também desenvolver.

Configuração selecionável

Você pode mudar `serialInit` para obter um parâmetro que permite à função de chamada especificar os parâmetros de comunicação iniciais, como taxa de transmissão, para a porta serial.

Verificação de erros

É importante que o driver de dispositivo faça a verificação de erros adequada. Outro aprimoramento seria definir códigos de erro (como erro de parâmetro, erro de hardware, etc.) para a API do driver de dispositivo. As funções no driver de dispositivo usariam esses códigos de erro para retornar o status da operação tentada. Isso permite que o software de nível superior anote as falhas e / ou tente novamente.

APIs adicionais

Adicionando `serialGetStr` e `serialPutStr` (que exigiria o armazenamento em buffer dos dados de recepção e transmissão) pode ser útil. A implementação das funções de string pode fazer uso do `serialGetChar` e `serialPutChar` funções, se fosse razoavelmente eficiente fazê-lo.

Uso FIFO

Normalmente, os UARTs contêm FIFOs para os dados recebidos e transmitidos. O uso desses FIFOs adiciona buffer aos canais de recepção e transmissão, tornando o driver UART mais robusto.

Interrupções

Implementar interrupções UART para recepção e transmissão geralmente é melhor do que usar polling. Por exemplo, na função `serialGetChar`, usar interrupções eliminaria a necessidade de o driver ficar sentado em um loop esperando por um caractere de entrada. O software aplicativo é, portanto, capaz de realizar outro trabalho enquanto aguarda o recebimento dos dados.

Design de driver de dispositivo

A maioria dos sistemas embarcados possui mais de um driver de dispositivo. Na verdade, às vezes pode haver dezenas. Conforme sua experiência cresce, você precisa entender a maneira como diferentes dispositivos no sistema interagem uns com os outros. Você também terá que considerar como o software do aplicativo usará o driver do dispositivo para que você possa fornecer uma API adequada.

Você precisará ter um bom entendimento do design geral do software e estar ciente dos possíveis problemas no sistema. Obter informações de várias fontes pode levar a um design melhor. Aqui estão algumas áreas a serem consideradas ao projetar uma arquitetura de software que inclui vários drivers de dispositivo:

Interromper prioridades

Se interrupções forem usadas para os drivers de dispositivo em um sistema, você precisará determinar e definir os níveis de prioridade apropriados.

Requisitos completos

Você precisa estar ciente dos requisitos dos vários periféricos do sistema. Você não deseja projetar e implementar o seu software de uma maneira que, sem saber, impeça o periférico de operar conforme pretendido. Isso pode causar problemas graves que o produto pode não ser utilizável até que o sistema funcione conforme especificado. É uma boa ideia usar revisões de design de software para eliminar quaisquer problemas potenciais que possam ter sido negligenciados por um desenvolvedor individual.

Uso de recursos

É importante entender quais recursos são necessários para cada driver de dispositivo. Por exemplo, imagine projetar um driver de dispositivo Ethernet em um sistema com uma quantidade muito limitada de memória. Essa limitação afetaria o esquema de buffer implementado no driver Ethernet. O driver pode acomodar o armazenamento de apenas alguns pacotes de entrada, o que afetaria a taxa de transferência da interface Ethernet.

Compartilhamento de recursos

Cuidado com as possíveis situações em que vários drivers de dispositivo precisam acessar hardware comum (como pinos de E / S) ou memória comum. Isso pode dificultar o rastreamento de bugs se o esquema de compartilhamento não for cuidadosamente pensado com antecedência. Veremos os mecanismos de compartilhamento de recursos nos Capítulos 8 e 10.

Interrupções

E, como a Srta. [Florence] Nightingale reclamava com veemência - "as mulheres nunca têm meia hora ... que possam chamar de sua" -, ela sempre era interrompida.

- Virgínia Woolf
Um quarto próprio

Neste capítulo, daremos uma olhada nas interrupções - uma maneira sofisticada de gerenciar relacionamentos com dispositivos externos. As interrupções são um aspecto importante do desenvolvimento de software embarcado e que os programadores precisam estudar cuidadosamente para criar aplicativos eficientes. O início deste capítulo fornece uma introdução às interrupções e às diferentes características associadas a elas. É importante entender o que acontece quando ocorre um evento de interrupção e como a interrupção é processada. Embora a implementação de interrupções seja específica do processador, muito do material neste capítulo se aplica a todos os processadores. Por fim, expandiremos o exemplo do LED piscando usando uma interrupção encontrada em praticamente todos os processadores.

Visão geral

Uma interrupção pode ser usada para sinalizar o processador para todos os tipos de eventos - por exemplo, porque os dados chegaram e podem ser lidos, um usuário acionou um botão ou um determinado período de tempo passou.

As interrupções permitem que os desenvolvedores separem as operações de tempo crítico do programa principal para garantir que sejam processadas de maneira priorizada. Como as interrupções são eventos assíncronos, elas podem acontecer a qualquer momento durante a execução do programa principal.

A Figura 8-1 mostra dois diagramas de fiação alternativos de periféricos conectados aos pinos de interrupção do processador.

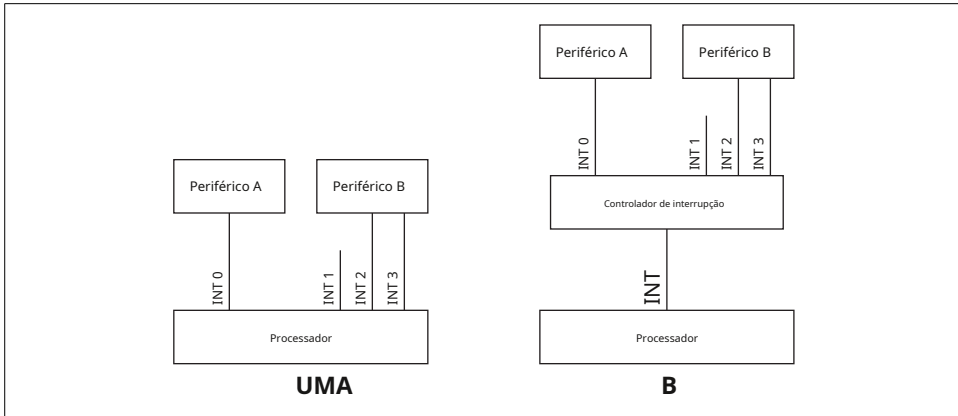


Figura 8-1. Interromper a fixação

Interrupções e eventos relacionados

O termo *interromper* é usado para cobrir vários recursos de hardware diferentes. Todos esses mecanismos são usados para desviar o processamento do programa principal para lidar com um evento, mas eles são chamados em circunstâncias diferentes e precisam ser tratados por um programador de maneiras diferentes, portanto, uma breve lista é útil para distingui-los. Observe que certos processadores podem definir esses termos de maneira diferente.

Exceção

Uma condição de erro detectada às vezes chamada de interrupção de software. Por exemplo, realizar uma divisão por zero causa uma exceção. Uma interrupção de software também é usada por um programa para executar várias funções de depuração, como pontos de interrupção. As exceções são eventos síncronos.

Interromper

Um sinal elétrico assíncrono declarado por um periférico para o processador.

Armadilha

Interrupção de um programa causada pelo hardware interno do processador. Traps são eventos síncronos.

Assíncrono os eventos não estão relacionados a tempo a nenhum outro evento conhecido no sistema. *Síncrono* eventos ocorrem como resultado de outro evento dentro do sistema. Por exemplo, um sinal indicando que alguém abriu a porta da sala é assíncrono, enquanto uma exceção causada pela divisão por zero é síncrona.

Neste capítulo, enfocamos as interrupções no sentido mais restrito.

A Figura 8-1 (a) mostra periféricos conectados diretamente aos pinos de interrupção do processador. Nesse caso, o processador contém um controlador de interrupção no chip para gerenciar e processar as interrupções de entrada. O PXA255 possui um controlador de interrupção interno.

A Figura 8-1 (b) mostra os dois periféricos conectados a um dispositivo controlador de interrupção externo. Um *controlador de interrupção* multiplexa várias interrupções de entrada em uma única interrupção de saída. O controlador também permite controlar essas interrupções de entrada individuais para desabilitá-las, priorizá-las e mostrar quais estão ativas.

Como muitos processadores incorporados contêm periféricos no chip, as interrupções desses periféricos também são roteadas para o controlador de interrupção dentro do processador principal. Às vezes, são necessárias mais interrupções em um sistema do que pinos de interrupção no processador. Para essas situações, os periféricos podem compartilhar uma interrupção. O software deve então determinar qual dispositivo causou a interrupção.

As interrupções podem ser *maskável* ou *não maskável*. As interrupções maskáveis podem ser desabilitadas e habilitadas pelo software. Interrupções não maskáveis (*NMI*) são interrupções críticas, como falha de energia ou reinicialização, que não podem ser desabilitadas pelo software.

Uma lista completa das interrupções em seu sistema pode ser construída a partir das informações nos manuais de referência do processador e da placa. Por exemplo, as interrupções suportadas pelo processador PXA255 são detalhadas no *Manual do desenvolvedor do processador PXA255*. Uma lista parcial das interrupções com suporte para o PXA255 é mostrada na Tabela 8-1. Vamos dar uma olhada no que o número de interrupção significa em breve.

Tabela 8-1. Lista de interrupção parcial para processador PXA255

Número de interrupção	Fonte de interrupção
8	GPIO Pin 0
9	GPIO Pino 1
11	USB
26	Timer 0
27	Timer 1
28	Timer 2

Prioridades

Como as interrupções são eventos assíncronos, deve haver uma maneira para o processador determinar com qual interrupção lidar primeiro, caso várias interrupções ocorram simultaneamente. O processador define um *prioridade de interrupção* para todas as interrupções e exceções que ele suporta. As prioridades de interrupção são encontradas na documentação do processador.

Por exemplo, o processador ARM suporta seis tipos de interrupções e exceções. * As prioridades dessas interrupções e exceções são mostradas na Tabela 8-2. Esta mesa é contido no *Manual do usuário da microarquitetura XScale*.

* ARMv6 também inclui uma prioridade de exceção de anulação imprecisa entre IRQ e anulação de pré-busca.

Tabela 8-2. Exceção do processador ARM e prioridades de interrupção

Prioridade	Fonte de exceção / interrupção
1 (mais alto)	Redefinir
2	Abortar dados
3	Solicitação de interrupção rápida (FIQ)
4	Solicitação de interrupção (IRQ) Abortar
5	pré-busca
6 (mais baixo)	Instrução indefinida ou interrupção de software

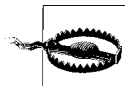
Normalmente, quando ocorre uma interrupção, um processador desativa todas as interrupções nos níveis de prioridade igual ou inferior. Se várias interrupções estão esperando para serem processadas ou *pendente*, a prioridade associada às interrupções determina a ordem em que são executadas.

O método para lidar com interrupções em diferentes prioridades é muito específico do processador. O texto a seguir descreve alguns cenários para lidar com interrupções em diferentes prioridades.

Quando uma interrupção de prioridade mais alta ocorre enquanto uma interrupção de prioridade mais baixa está sendo processada, o processamento da interrupção de prioridade mais baixa é suspenso e a interrupção de prioridade mais alta é tratada. Assim que a interrupção de prioridade mais alta for concluída, o processamento da interrupção de prioridade mais baixa continuará. Isso é chamado *interromper o aninhamento*.

Se uma interrupção de prioridade mais baixa ocorrer enquanto o processador está lidando com uma interrupção de prioridade mais alta, a interrupção de prioridade mais baixa é deixada pendente até que a interrupção de prioridade mais alta termine de ser executada.

Quando uma interrupção ocorre com a mesma prioridade da interrupção atualmente sendo processada, a interrupção atualmente sendo processada pode terminar. Em seguida, o processamento da próxima interrupção começa. Nesse caso, o aninhamento de interrupção também pode ocorrer se a interrupção atualmente em execução reativar interrupções em seu próprio nível de prioridade. Em outras palavras, uma interrupção pode permitir a si mesma ser interrompida por outras interrupções que estão no mesmo nível de prioridade.



Tenha cuidado ao aninhar interrupções. Deve haver premeditação adicional no dimensionamento da pilha, porque cada ISR interrompido deve ter seu estado de registro salvo na pilha. Isso pode levar a um estouro de pilha.

A prioridade de interrupção é definida pelo design do hardware, pelo software ou, em alguns casos, por uma combinação dos dois. Se olharmos novamente para o diagrama de fiação na Figura 8-1 (a), vemos que o processador tem quatro pinos de interrupção (INT0 a INT3). Por esta

Por exemplo, assumiremos que o processador dá a INT0 a prioridade mais alta, seguida por INT1, INT2 e INT3. O projetista do hardware deve conectar os pinos de interrupção para que as prioridades de interrupção corretas sejam atribuídas às várias interrupções periféricas. Nesse caso, a interrupção do Periférico A tem a prioridade mais alta.

Alguns controladores de interrupção permitem que as prioridades das interrupções sejam definidas no software. Nesse caso, o controlador de interrupção normalmente tem registros que definem as prioridades das várias interrupções.

Níveis e Bordas

Sensível ao nível interrupções fazem com que o processador responda enquanto o sinal de interrupção estiver no nível especificado. Essas interrupções são sensíveis a alto ou baixo nível. *Sensível às arestas* interrupções fazem com que o processador responda quando o sinal de interrupção passa por uma transição. Essas interrupções são especificadas como sensíveis a bordas ascendentes ou descendentes. A Figura 8-2 mostra sinais para uma interrupção sensível ao nível e à borda.

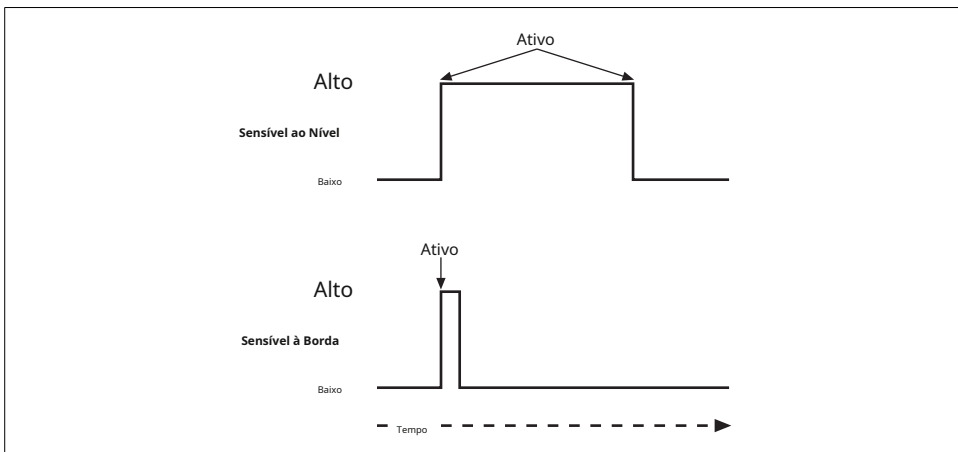


Figura 8-2. Sinais de interrupção sensíveis a nível e borda

Na Figura 8-2, a interrupção sensível ao nível está ativa alta. O tempo em que a interrupção está ativa é mostrado no diagrama de sinal, que é o tempo em que o sinal está na tensão mais alta. O sinal de interrupção na parte inferior da Figura 8-2 é uma interrupção sensível à borda ascendente. Ele está ativo quando o sinal muda de baixo para alto, é mantido alto por um certo tempo mínimo (normalmente dois ou três clocks do processador) e depois volta para baixo novamente.

Existem problemas relacionados a ambos os tipos de interrupções. Por exemplo, uma interrupção sensível à borda pode ser perdida se uma interrupção subsequente ocorrer antes que a interrupção inicial seja atendida. Por outro lado, uma interrupção sensível ao nível interrompe constantemente o processador, desde que o sinal de interrupção seja confirmado.

A maioria dos periféricos afirmam sua interrupção até que ela seja reconhecida. Alguns processadores, como o Intel386 EX, contêm registros que podem ser programados para suportar interrupções sensíveis ao nível ou à borda em interrupções individuais. Assim, a seleção de sensibilidade afeta a detecção de novas interrupções naquele sinal.

Reconhecendo uma interrupção informa ao dispositivo de interrupção que o processador recebeu a interrupção e a colocou na fila para processamento. O método de reconhecimento de uma interrupção pode variar desde a leitura de um registro do controlador de interrupção até a limpeza de um bit pendente de interrupção. Assim que a interrupção for confirmada, o periférico desativará o sinal de interrupção. Alguns processadores têm um sinal de reconhecimento de interrupção que cuida disso automaticamente no hardware.

Ativando e Desativando

As interrupções mascaráveis podem ser desabilitadas e habilitadas individualmente ou globalmente. As interrupções não mascaráveis, como o nome indica, não podem ser desabilitadas. No processador PXA255, as interrupções individuais são mascaradas no Registro de Máscara do Controlador de Interrupção (ICMR). Esse registro é mostrado na Figura 8-3.

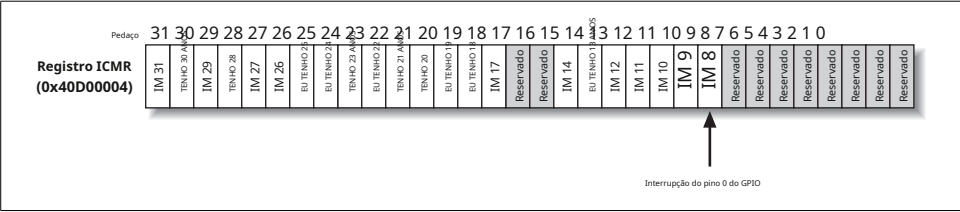
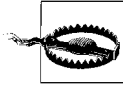


Figura 8-3. Registro de máscara de controlador de interrupção PXA255

O ICMR está localizado no endereço 0x40D00004. A Figura 8-3 mostra a máscara de interrupção (IM) para cada uma das interrupções suportadas no PXA255. Definir o bit correspondente como 1 no ICMR permite que uma fonte específica gere interrupções; definir o bit correspondente para 0 máscaras que interrompem a fonte.

Por exemplo, imagine que o pino de interrupção de um periférico está conectada ao pino GPIO 0 no processador PXA255. A Tabela 8-1 mostra que a fonte de interrupção do pino 0 GPIO é atribuída ao número de interrupção 8. Portanto, para habilitar a interrupção do pino 0 GPIO, o bit 8 do ICMR é definido como 1. Se ocorrer uma interrupção quando a interrupção do pino 0 GPIO for habilitado, ele é roteado para o controlador de interrupção para processamento. Para mascarar a fonte de interrupção do pino 0 GPIO, defina o bit 8 do ICMR como 0. Se ocorrer uma interrupção enquanto a fonte estiver desabilitada, a interrupção será ignorada.

Cada processador normalmente tem um bit de ativação / desativação de interrupção global em um de seus registradores. O PXA255 tem dois bits no Current Program Status Register (CPSR) que desabilitam globalmente todas as interrupções.



É importante lembrar de reativar as interrupções no software depois de desativá-las. Este é um problema comum que pode levar a um comportamento inexplicável no sistema. Se as interrupções forem desativadas na entrada de uma função, certifique-se de que todos os caminhos de software que saem da rotina reativem as interrupções.

Geralmente, você não pode acessar os sinalizadores de interrupção globais diretamente usando a linguagem C. Nesse caso, você precisa escrever o código assembly para habilitar e desabilitar as interrupções globais. Algumas bibliotecas de compiladores, como as da família de processadores x86, contêm funções para lidar com a ativação de interrupção global (com o permitir função) e desabilitação de interrupção global (com o desabilitar função).

Mapa de interrupção

Agora que entendemos como ocorre uma interrupção, vamos dar uma olhada em como o processador lida com uma interrupção no software. Depois que o processador é reinicializado, desligando e ligando a alimentação ou ativando o sinal de reinicialização, as interrupções são desabilitadas. Uma das tarefas do código de inicialização é habilitar interrupções globais assim que o sistema estiver pronto para elas. Parte do procedimento para garantir que o sistema está pronto para interrupções é instalar o software para lidar com elas. Uma rotina de serviço de interrupção, que realiza a ação básica necessária para lidar com a interrupção, está associada a cada interrupção.

Para que o processador execute o ISR correto, deve existir um mapeamento entre as fontes de interrupção e as funções ISR. Esse mapeamento geralmente assume a forma de um *interromper a tabela de vetores*. A tabela de vetores, localizada em um endereço de memória conhecido pelo hardware, geralmente é um array de ponteiros para funções. O processador usa o número de interrupção (um número exclusivo associado a cada interrupção) como seu índice neste array. Em alguns processadores, o valor armazenado na matriz da tabela de vetores é geralmente o endereço do ISR a ser executado. Outros processadores realmente têm instruções armazenadas no array (comumente chamado de *trampolim*) para pular para o ISR.

Para o processador ARM, os endereços na tabela de vetores de interrupção estão em locais fixos na memória. Esses endereços estão listados na Tabela 8-3.

Tabela 8-3. Tabela de vetor de interrupção ARM

Fonte de exceção / interrupção	Endereço
Redefinir	0x00000000
Instrução indefinida	0x00000004
Interrupção de software	0x00000008
Abortar pré-busca	0x0000000C
Abortar dados	0x00000010
IRQ	0x00000018
FIQ	0x0000001C

Os endereços na Tabela 8-3 são locais na memória usados pelo processador ARM para executar o ISR para uma interrupção específica. As informações sobre a tabela de vetores de interrupção estão contidas na documentação sobre o processador. Como os endereços na tabela de vetor de interrupção ARM são separados por 32 bits, o código instalado na tabela de vetor de interrupção é um salto para o ISR real.

É crítico para o programador instalar um ISR para todas as interrupções, mesmo as interrupções que não são usadas no sistema. Se um ISR não for instalado para uma interrupção específica e a interrupção ocorrer, a execução do programa pode se tornar indefinida (comumente chamada de “ir para o mato”).

Um bom procedimento é instalar um ISR padrão na tabela de vetores de interrupção para qualquer interrupção não usada. O ISR padrão garante que todas as interrupções sejam processadas e confirmadas, permitindo que o programa principal continue em execução. Faça com que seu código de inicialização inicialize todas as interrupções na tabela de vetores para o manipulador padrão para garantir que não haja interrupções não manipuladas. Em seguida, instale ISRs para interrupções específicas usadas no sistema.

A primeira parte da inicialização da tabela de vetores de interrupção é criar um mapa de interrupção que organiza as informações relevantes. Um mapa de interrupção é uma tabela que contém uma lista de números de interrupção e os dispositivos aos quais eles se referem. Essas informações devem ser incluídas na documentação fornecida com o conselho. Um mapa de interrupção parcial para a placa Arcom é mostrado na Tabela 8-4.

Tabela 8-4. Mapa de interrupção parcial para a placa Arcom

Número de interrupção	Fonte de interrupção
8	Ethernet
11	USB
21	Porta serial 2
22	Porta serial 1
26	Timer 0
27	Timer 1
28	Timer 2

Esta tabela é semelhante à lista de interrupções para o processador PXA255 mostrada na Tabela 8-1. No entanto, esta tabela mostra as fontes de interrupção específicas da placa Arcom.

Mais uma vez, nosso objetivo é traduzir as informações da tabela em um formato que seja útil para o programador. A tabela do mapa de interrupção deve ir para o caderno do projeto para referência futura. Depois de construir um mapa de interrupção, como o da Tabela 8-4, você deve adicionar uma terceira seção ao arquivo de cabeçalho específico da placa. Cada linha do mapa de interrupção torna-se um único # definir dentro do arquivo, conforme mostrado aqui:

```
/ *****
* Mapa de interrupção
***** /
```

```
# define ETHERNET_INT          (8)
# define USB_INT               (11)
# define SERIAL2_INT           (21)
# define SERIAL1_INT           (22)
# define TIMER0_INT            (26)
# define TIMER1_INT            (27)
# define TIMER2_INT            (28)
```

Interromper a rotina de serviço

Vamos dar uma olhada mais de perto no ISR. O ISR é a função chamada quando ocorre uma interrupção específica. Seu objetivo central é processar a interrupção e, em seguida, retornar o controle ao programa principal. Normalmente, as funções ISR não têm argumentos passados para eles; eles nunca podem retornar um valor.

Para reduzir ao mínimo o impacto das interrupções na execução do programa principal, é importante manter as rotinas de interrupção curtas. Se for necessário processamento adicional para uma interrupção específica, é melhor fazer isso fora do ISR. Manter os ISRs curtos também ajuda na depuração de ISR, o que pode ser difícil. Quando é feito por uma função específica, a conclusão do tratamento de interrupção fora do ISR é chamada de *rotina de serviço diferido (DSR)*.

Independentemente do processamento específico exigido pelo ISR, o ISR é responsável por fazer o seguinte:

Salvando o contexto do processador

Como o ISR e o programa principal usam os mesmos registros do processador, é responsabilidade do ISR salvar os registros do processador antes de iniciar qualquer processamento da interrupção. O processador *contexto* consiste no ponteiro de instrução, registros e quaisquer sinalizadores. Alguns processadores realizam esta etapa automaticamente.

Reconhecendo a interrupção

O ISR deve limpar a interrupção existente, o que é feito no periférico que gerou a interrupção, no controlador de interrupção ou em ambos.

Restaurando o contexto do processador

Após a interrupção do processamento, para retomar o programa principal, os valores salvos antes da execução do ISR devem ser restaurados. Alguns processadores realizam esta etapa automaticamente.

Alguns compiladores incluem a palavra-chave `interromper` ou algo semelhante. Isso permite que o compilador gere automaticamente o código usado para salvar o contexto quando o ISR é inserido e para restaurar o contexto quando o ISR é encerrado. Um exemplo de código que inclui o `interromper` a palavra-chave segue:

```
interromper void interruptServiceRoutine (void) {

    /* Processa a interrupção. */
}
```

A documentação do compilador mostra se o interromper palavra-chave é suportada. Se o compilador não suportar esta palavra-chave, um # específico do compilador pragma pode ser necessário declarar um ISR. O compilador GNU *gcc* usa uma terceira abordagem, envolvendo a palavra-chave específica do compilador `__attribute__((interrupt))`, que leva opções como argumentos, conforme mostrado aqui:

```
void interruptServiceRoutine () __attribute__((interrupt ("IRQ")));
```

Alguns processadores, como certos PICs Microchip, podem ter apenas um ISR para todas as interrupções. Este ISR deve determinar a fonte da interrupção verificando cada fonte de interrupção potencial. Nesse caso, é uma boa ideia verificar primeiro a interrupção mais importante. A técnica usada pelo ISR para determinar se a fonte de interrupção é específica do hardware.



Ao projetar seu software, normalmente é melhor incluir o ISR de um determinado dispositivo no driver desse periférico. Isso mantém todo o código específico do dispositivo para um determinado periférico isolado em um único módulo.

A Figura 8-4 é uma representação gráfica do processo de interrupção. Para este exemplo, vamos assumir que o controlador de interface de rede Ethernet gera a interrupção, embora este processo seja relevante para qualquer interrupção.

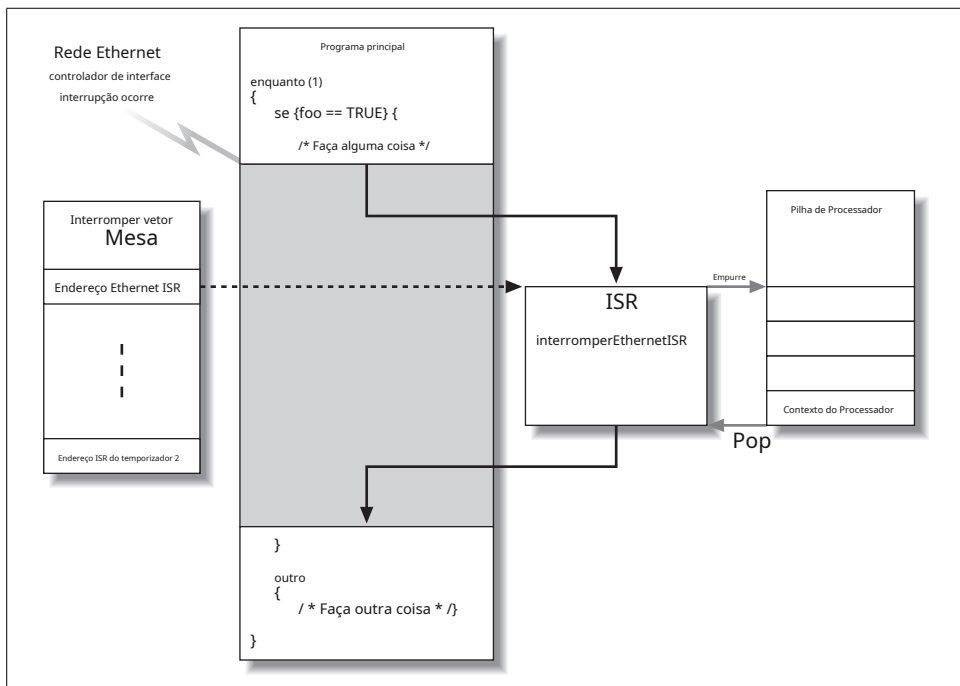


Figura 8-4. Fluxo de software durante a interrupção

Na Figura 8-4, o processador está executando o programa principal quando ocorre uma interrupção no controlador de interface de rede Ethernet. O processador termina a instrução em andamento antes de interromper a execução do programa principal. (Alguns processadores permitem a interrupção de longas instruções para que as interrupções não sejam atrasadas por longos períodos de tempo.)

Em seguida, o processador procura o endereço do ISR para o controlador de interface de rede Ethernet, `interruptEthernetISR`, na tabela de vetores de interrupção, e o processador salta para esta função. o `interruptEthernetISR` função salva o contexto do processador na pilha do processador.

O ISR então limpa a interrupção. Depois de concluído, o ISR restaura o contexto e retorna. O programa principal continua sua execução a partir do ponto em que foi interrompido. A maioria dos processadores tem uma instrução especial de “retorno da interrupção” para sair do ISR.

Um conceito importante associado a interrupções é *latência*. Latência de interrupção é a quantidade de tempo desde o momento em que ocorre uma interrupção até o momento em que o processador começa a executar a rotina de serviço de interrupção associada. A latência de interrupção é uma métrica usada por alguns engenheiros para avaliar processadores e é muito importante em sistemas de tempo real. Desativar interrupções aumenta a latência de interrupção em um sistema embarcado, porque a latência inclui o tempo entre a ocorrência da interrupção e o momento em que as interrupções são reativadas.

Embora haja um único ISR para cada interrupção, pode haver vários motivos pelos quais a interrupção ocorre. É responsabilidade do ISR determinar a causa específica da interrupção e proceder de acordo. A estrutura de código a seguir mostra como um ISR lê o registro de status de interrupção para determinar a interrupção, reconhece a interrupção escrevendo o valor de volta para o registro de status de interrupção e, em seguida, determina a causa da interrupção. É bem possível que mais de uma fonte de interrupção esteja ativa durante o ISR. Assim, o ISR deve verificar todas as fontes; deixar de fazer isso pode resultar na perda de uma interrupção.

```
interromper void interruptServiceRoutine (void) {

    uint8_t intStatus;

    /* Determine quais interrupções ocorreram. */
    intStatus = * pIntStatusReg;

    /* Confirme a interrupção. */
    * pIntStatusReg = intStatus;

    if (intStatus & INTERRUPT_SOURCE_1) {

        /* Interrompe o processamento. */
    }

    if (intStatus & INTERRUPT_SOURCE_2) {
```



```

        /* Interrompe o processamento. */
    }
}

```

Dados compartilhados e condições de corrida

Um problema comum ao projetar software que usa interrupções é como compartilhar dados entre o ISR e o programa principal. UMA *condição de corrida* é uma situação em que o resultado varia dependendo da ordem precisa em que as instruções do código principal e do ISR são executadas; isso deve ser evitado vigorosamente.

Pode ser extremamente difícil encontrar bugs de condição de corrida porque as interrupções são eventos assíncronos e, para piorar as coisas, a condição de corrida não ocorre toda vez que o código é executado. Seu software pode ser executado por dias e passar em todos os testes de produção sem exibir esse bug - mas então, uma vez que a unidade é enviada ao cliente, é certo que aparecerá.

O exemplo de código a seguir fornecerá uma melhor compreensão das condições de corrida. Imagine que a porta serial ISR `serialReceiveIsr` é invocado quando um personagem chega. Conforme os personagens são recebidos, `gIndex` é incrementado para controlar o número de caracteres armazenados no buffer de memória.

o a Principal função também usa `gIndex`, diminuindo-o quando ele processa os caracteres recebidos no buffer de memória. Aqui está o código de exemplo:

```

int gIndex = 0;

interromper void serialReceiveIsr (void) {

    /* Armazena o caractere de recebimento no buffer de memória. */

    gIndex++;
}

int main (vazio)
{
    enquanto (1)
    {
        if (gIndex)
        {
            /* Processa o caractere de recebimento no buffer de memória. */

            gIndex--;
        }
    }
}

```

Você consegue identificar o problema com este código?

Vamos assumir a variável `gIndex` tem um valor de 3 quando a linha de código que diminui essa variável é executada:

```
gIndex--;
```

Esta linha de código resulta em instruções em linguagem assembly que fazem algo assim:

```
CARREGAR gIndex em um registrador;  
REDUZA o valor do registro;  
ARMAZENE o valor do registro de volta no gIndex;
```

A primeira etapa é ler o valor de gIndex, 3, de sua localização na RAM em um registro do processador. Em seguida, o valor do registro é decrementado, resultando em um valor de 2.

Agora, suponha que uma interrupção de recebimento de porta serial ocorra antes do novo valor de gIndex é armazenado na memória. O processador para de executar a Principal e executa a porta serial ISR, serialReceiverIsr. Os incrementos ISR gIndex para um valor de 4.

O processador retoma a execução de a Principal após a saída do ISR. Neste ponto, a Principal executa a linha de código que armazena o valor do registro, 2, de volta para a variável gIndex. Agora gIndex tem um valor de 2, como se a última interrupção nunca tivesse ocorrido para incrementar gIndex.

Essa condição de corrida pode fazer com que o programa perca os caracteres de entrada. Figura 8-5 mostra a condição de corrida para este código de exemplo.

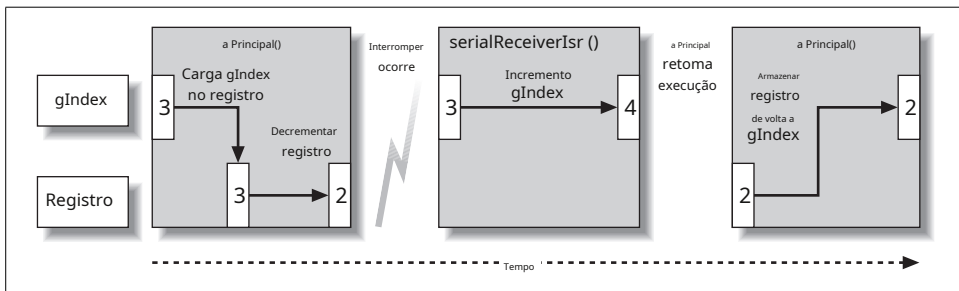


Figura 8-5. Exemplo de condição de corrida

O código de decremento no a Principal programa é chamado de *seção crítica*. Uma seção crítica é uma parte de um programa que deve ser executado em ordem e *atomicamente*, ou sem interrupção. Uma linha de código C (mesmo tão trivial quanto incremento ou decremento) não é necessariamente atômica, como vimos neste exemplo.

Então, como esse problema é corrigido? Como uma interrupção pode ocorrer a qualquer momento, a única maneira de fazer essa garantia é desabilitar as interrupções durante a seção crítica. Para proteger a seção crítica no código de exemplo anterior, as interrupções são desabilitadas antes da execução da seção crítica e habilitadas depois, conforme mostrado aqui:

```
int main (vazio)  
{  
    enquanto (1)  
    {  
        interruptDisable ();  
  
        if (gIndex)
```

```

    {
        /* Processa o caractere de recebimento no buffer de memória. */

        gIndex--;
    }

    interruptEnable ();
}
}

```

Em sistemas embarcados, e especialmente em sistemas de tempo real, é importante manter as interrupções habilitadas tanto quanto possível para evitar atrapalhar a capacidade de resposta do sistema. Tente minimizar o número de seções críticas e o comprimento do código da seção crítica.



A solução mais segura é salvar o estado do sinalizador de ativação de interrupção, desativar as interrupções, executar a seção crítica e, em seguida, restaurar o estado do sinalizador de ativação de interrupção. Habilitar interrupções no final sem garantir que elas foram habilitadas no início da seção crítica é arriscado.

Condições de corrida também podem ocorrer quando o recurso compartilhado entre um ISR e o programa principal é um periférico ou um dos registradores do periférico. Por exemplo, suponha que um programa principal e um ISR usem o mesmo registro periférico. O programa principal lê um registro e armazena o valor. Neste ponto, um ISR executa e modifica o valor nesse mesmo registro. Quando o programa principal retoma e atualiza o registro periférico, o valor do ISR é sobrescrito e perdido.

As seções críticas também são um problema ao usar um sistema operacional em tempo real (RTOS), porque as tarefas também podem compartilhar recursos, como variáveis globais ou registros periféricos. Veremos isso no Capítulo 10.

O programa de LED piscando aprimorado

Agora veremos um exemplo de interrupção usando um cronômetro. Para este exemplo, usaremos o código do LED piscando do Capítulo 3. No entanto, em vez de usar um loop para controlar o tempo do piscar do LED, usaremos um temporizador de hardware. A maioria dos microcontroladores inclui até vários temporizadores.

Existem várias vantagens em usar um temporizador em vez de um loop para o tempo: o processador está livre para lidar com outras tarefas em vez de ficar sentado em um enquanto loop sem fazer nada, um cronômetro é mais preciso para medir um loop do que um cronômetro, e você pode calcular o tempo exato em que deseja que a interrupção do cronômetro seja acionada em vez de usar uma abordagem de tentativa e erro com base no relógio do processador.

Neste programa de LED piscando aprimorado, a rotina de atraso é eliminada e um driver de dispositivo de temporizador é usado para controlar o atraso entre as alternâncias de LED. O cronômetro é usado para interromper o processador uma vez que um intervalo específico tenha decorrido.

Como funcionam os cronômetros

UMA *cronômetro* é um periférico que mede o tempo decorrido, normalmente contando os ciclos ou relógios do processador. UMA *contador*, em contraste, mede o tempo decorrido usando eventos externos. Um temporizador é configurado pela programação de um registro de intervalo no periférico do temporizador, com um valor específico calculado pelo engenheiro de software para determinar o intervalo de interrupção do temporizador. O periférico temporizador então usa um relógio para manter a contagem do número de tiques que passaram desde que o temporizador foi iniciado. O número de tiques do relógio é comparado ao valor no registro de intervalo do temporizador. Uma vez que eles são iguais, uma interrupção do temporizador é gerada (se habilitada).

O cronômetro conta os ciclos do sinal de clock principal do processador ou de um clock separado alimentado no periférico do cronômetro a partir de um pino de processador externo. Em alguns processadores, o relógio usado para o temporizador pode ser selecionado programando os registros de configuração do temporizador. Muitos processadores hoje também incluem várias fontes de relógio interno que podem ser usadas para acionar os temporizadores.

Em alguns processadores, o intervalo de tempo calculado é programado em um registro de temporizador que é decrementado a cada tique do relógio. Uma vez que o valor naquele registro atinge zero, uma interrupção do temporizador é gerada. O periférico do temporizador então recarrega o registro do temporizador com o valor do intervalo calculado (armazenado em um registro separado) e mais uma vez começa a diminuir esse valor a cada tique do relógio. Outros processadores, como o PXA255, possuem temporizadores que aumentam a contagem. Certifique-se de verificar a documentação do processador para entender como funciona o temporizador.

O processador PXA255 possui quatro temporizadores. Para este exemplo, o temporizador 0 é usado; os registros do temporizador PXA255 de 32 bits para o temporizador 0 são mostrados na Figura 8-6.

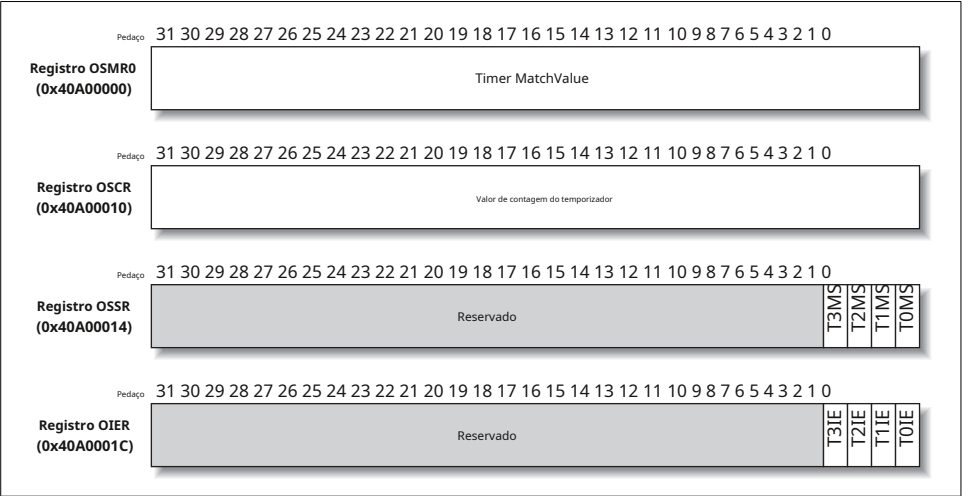


Figura 8-6. Registros do temporizador 0 do processador PXA255

No PXA255, o registrador de contagem do temporizador (OSCR) contém uma contagem que é incrementada nas bordas crescentes do relógio do temporizador, que opera a 3,6864 MHz. Em outras palavras, cada vez que o sinal do clock vai de baixo para alto, o OSCR é incrementado em um.

O registro de correspondência do cronômetro (OSMR n , Onde n é o número do cronômetro) contém os valores do cronômetro para os quatro cronômetros diferentes. Após cada borda ascendente do relógio do temporizador, o processador compara o valor no OSMR n para o OSCR. Se houver uma correspondência, uma interrupção é gerada e o bit correspondente é definido no registro de status do temporizador (OSSR). O registro de ativação de interrupção do temporizador (OIER) determina quais interrupções são ativadas para os quatro temporizadores diferentes.

Watchdog Timers

Outro tipo de cronômetro frequentemente mencionado em referência a sistemas embarcados é um *temporizador de watchdog*. Um cronômetro de watchdog é um mecanismo especial à prova de falhas de hardware que intervém se o software parar de funcionar corretamente. O cronômetro de vigilância é reiniciado periodicamente (às vezes chamado de “chutar o cachorro”) pelo software. Se o software travar ou travar, o cronômetro de watchdog expira logo, fazendo com que todo o sistema seja reiniciado automaticamente.

A inclusão e o uso de um cronômetro de watchdog é uma maneira comum de lidar com travamentos ou travamentos inesperados de software que podem ocorrer após a implantação do sistema. Por exemplo, suponha que o novo produto de sua empresa viaje para o espaço. Não importa quantos testes você faça antes da implantação, permanece a possibilidade de que haja bugs não descobertos à espreita no software e que um ou mais deles sejam capazes de travar o sistema por completo. Se o software travar, você não conseguirá se comunicar com o sistema, portanto, não será possível emitir um comando de reinicialização remotamente. Em vez disso, você deve criar um mecanismo de recuperação automática no sistema. É aí que entra o cronômetro de vigilância.

Um detalhe importante de implementação a ser lembrado ao usar um cronômetro de watchdog é que você sempre deve implementar o código que trata da redefinição do cronômetro de watchdog no loop de processamento principal. Nunca implemente o reset do cronômetro de watchdog em um ISR. A razão é que, em um sistema embarcado, o loop de processamento principal pode travar enquanto as interrupções e ISRs continuam a funcionar. Nesse caso, o cronômetro de watchdog nunca seria capaz de reinicializar o sistema e, assim, permitir que o software se recuperasse.

o a Principal rotina para a implementação de LED piscando que usa um temporizador em vez de um loop de atraso é muito semelhante ao a Principal rotina discutida no Capítulo 3. Esta parte do código é independente do hardware. o a Principal função começa com a inicialização da porta de controle de LED com o ledInit função. Uma rotina de inicialização para o driver de dispositivo do temporizador, timerInit, é chamado para inicializar e iniciar o hardware do temporizador.

O loop infinito em a Principal está vazio neste caso porque não há outro processamento necessário para esta versão do programa de LED piscando. Todo o processamento acontece

em segundo plano com a interrupção do temporizador, mas o loop infinito ainda é necessário para manter o programa em execução. Observe aqui que o `delay_ms` função foi removida:

```
# inclui "led.h"
# inclui "timer.h"

/ *****
*
* Função:          a Principal
*
* Descrição: Pisca o LED verde uma vez por segundo.
*
* Notas:
*
* Retorna:        Esta rotina contém um loop infinito.
*
***** /

int main (vazio)
{
    /* Configure o pino de controle do LED verde. */
    ledInit ();

    /* Configure e inicie o cronômetro. */
    timerInit ();

    enquanto (1)
        ;

    return 0;
}
```

o `timerInit` rotina inicializa os registros necessários para o driver de dispositivo do temporizador e, em seguida, ativa a interrupção do temporizador. A variável de estado global `bInicializado` é usado para garantir que os registros do temporizador sejam configurados apenas uma vez.

A primeira etapa para configurar o temporizador é limpar todas as interrupções pendentes. Isto é feito escrevendo o bit 0 (definido pela máscara de bits `TIMER_0_MATCH`) para o registro de status do temporizador (OSSR) (definido pela macro `TIMER_STATUS_REG`).

Para a próxima etapa, precisamos calcular o intervalo de interrupção. o *Manual do desenvolvedor do processador PXA255* afirma que os temporizadores são incrementados em um clock de 3,6864 MHz. Para alternar o LED a cada 500 ms, a seguinte equação é usada para determinar o valor para o registro de correspondência do temporizador:

$$\text{Timer Match Register Value} = \text{Timer clock} \times \text{Intervalo do}$$

cronômetro Para nosso exemplo, o cálculo é:

$$\begin{aligned} \text{Valor do registro de correspondência do temporizador} &= 3.686.400 \text{ Hz} \times 0,5 \text{ segundos} \\ &= 1.843.200 \\ &= 0x001C2000 \end{aligned}$$

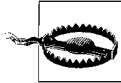
A macro `TIMER_INTERVAL_500MS` é definido com o valor de intervalo `0x001C2000`. o *Manual do desenvolvedor do processador PXA255* descreve o algoritmo para configurar um temporizador da seguinte forma:

1. Leia o valor da contagem atual no registro de contagem do cronômetro (OSCR).
2. Adicione o deslocamento do intervalo ao valor de contagem atual. Este valor corresponde à quantidade de tempo antes do próximo tempo limite.
3. Programe o novo valor de intervalo no registro de correspondência do temporizador (OSMR0).

Para definir o intervalo do temporizador, o registro OSMR0 (definido pela macro `TIMER_0_MATCH_REG`) é programado com o valor atual da contagem do temporizador mais o `CRONÔMETRO_INTERVAL_500MS` intervalo do cronômetro.

Em seguida, a interrupção do temporizador é habilitada em dois locais: o periférico do temporizador e o controlador de interrupção. No periférico do temporizador, a ativação da interrupção do temporizador é controlada pelo bit 0 (definido pelo bitmask `TIMER_O_INTEN`) no OIER de 32 bits (definido pela macro `TIMER_INT_ENABLE_REG`).

Em seguida, o controlador de interrupção é configurado para permitir interrupções do periférico do temporizador. Conforme mostrado na Tabela 8-4, a interrupção do temporizador 0 é mapeada para a interrupção número 26. Portanto, o programa define o bit de número 26 (definido pela macro `TIMER_0_ENABLE`) no Registro de Máscara do Controlador de Interrupção (ICMR) (definido pela macro `INTERROMPER_ENABLE_REG`).



Porque o `timerInit` A função habilita a interrupção do temporizador 0 no controlador de interrupção, um ISR para a interrupção do temporizador 0 deve ser instalado antes de chamar esta rotina.

Para a etapa final, inicializando o temporizador, defina a variável de estado de inicialização para `VERDADE`, Como mostrado aqui:

```
# define TIMER_INTERVAL_500MS          (0x001C2000)

/ *****
*
* Função:      timerInit
*
* Descrição:  Inicialize e inicie o cronômetro.
*
* Notas:      Esta função é específica da placa Arcom. Certifique-se de
*              que um ISR foi instalado para o cronômetro antes de
*              chamar esta rotina.
*
* Retorna:    Nenhum.
*
***** /

void timerInit (void)
{
    static int bInitialized = FALSE;
```

```

/* Inicializa o cronômetro apenas uma vez.
* / if (bInitialized == FALSE)
{
    /* Reconhece qualquer interrupção pendente do cronômetro. */
    TIMER_STATUS_REG = TIMER_0_MATCH;

    /* Inicializa o intervalo do cronômetro. */
    TIMER_0_MATCH_REG = (TIMER_COUNT_REG + TIMER_INTERVAL_500MS);

    /* Habilita a interrupção do cronômetro no periférico do cronômetro. */
    TIMER_INT_ENABLE_REG |= TIMER_0_INTEN;

    /* Habilita a interrupção do temporizador no controlador de interrupção. */
    INTERRUPT_ENABLE_REG |= TIMER_0_ENABLE;

    bInicializado = VERDADEIRO;
}
}

```

Antes de entrar no ISR, o contexto do processador é salvo. Use a seguinte declaração de função para que o compilador GNU inclua o código para salvar o contexto (e restaurar no final do ISR):

```
void timerInterrupt () __attribute__((interrupção ("IRQ")));
```

Em seguida, o ISR reconhece a interrupção do temporizador 0. A documentação do processador afirma que o reconhecimento de uma interrupção do temporizador é realizado escrevendo um 1 no bit do temporizador 0 (definido pela máscara de bits `TIMER_0_MATCH`) do OSSR de 32 bits (definido pela macro `TIMER_STATUS`). Consulte a Figura 8-6 para obter detalhes sobre o OSSR e o bit 0 de status de correspondência do temporizador 0 (TOMS).

Em seguida, o estado do LED muda com uma chamada para a função `ledToggle`. Em seguida, o valor de correspondência do temporizador é atualizado para o próximo intervalo de interrupção do temporizador. Para atualizar o registro de correspondência do cronômetro 0, primeiro leia a contagem atual do cronômetro, adicione o intervalo do cronômetro e, em seguida, escreva este resultado no registro de correspondência do cronômetro (`TIMER_0_MATCH_REG`).

Finalmente, o contexto do processador é restaurado e o ISR retorna:

```

# inclui "led.h"

/ *****
*
* Função:      timerInterrupt
*
* Descrição: Rotina de serviço de interrupção do temporizador 0.
*
* Notas:      Esta função é específica da placa Arcom.
*
* Retorna:    Nenhum.
*
***** /
void timerInterrupt (void) {

```



```

/* Reconhece a interrupção do temporizador 0. */
TIMER_STATUS = TIMER_0_MATCH;

/* Altera o estado do LED verde. */
ledToggle();

/* Definir o novo intervalo do temporizador. */
TIMER_0_MATCH_REG = (TIMER_COUNT_REG + TIMER_INTERVAL_500MS);
}

```

Agora você pode construir este código e executá-lo na placa Arcom de destino. Se construída com sucesso, esta versão do programa de LED piscando deve operar da mesma maneira que a mostrada no Capítulo 3.

Resumo dos problemas de interrupção

As interrupções são uma parte importante da maioria dos sistemas embarcados. Aqui estão alguns pontos-chave para ter em mente ao lidar com interrupções:

Obtenha a primeira interrupção

Concentre-se em fazer com que a interrupção específica na qual você está trabalhando ocorra primeiro. Em seguida, prossiga para fazer com que essa interrupção seja acionada nas vezes subsequentes.

Interrupção bloqueada

As interrupções podem ser bloqueadas em vários pontos. Certifique-se de que a interrupção específica esteja habilitada no controlador de interrupção e no dispositivo periférico de origem. Certifique-se de que as interrupções globais estão habilitadas no processador.

Instalação ISR

Verifique se o ISR está instalado na tabela de vetor de interrupção corretamente e para o vetor de interrupção correto. Compreenda o mapeamento de interrupções para o processador. Usar a técnica de depuração de LED mencionada no Capítulo 5 pode ser uma ferramenta valiosa para rastrear o caminho de execução quando ocorre uma interrupção.

Proteger contra interrupções não tratadas

Certifique-se de que haja um ISR para cada interrupção no sistema. É melhor instalar um ISR padrão para todas as interrupções no momento da inicialização para garantir que cada interrupção seja tratada.

Contexto do processador

Certifique-se de que o contexto do processador seja salvo e restaurado corretamente no ISR. Os registros podem ser pisoteados por um ISR que acabará por causar estragos em seu programa principal.

Reconheça a interrupção

A interrupção deve ser reconhecida para que o sinal seja desativado. Se isso for feito incorretamente ou não for feito, os ISRs para as interrupções de prioridade igual ou inferior não serão executados novamente. Ou, se for uma interrupção sensível ao nível, o ISR será executado repetidamente porque o sinal de interrupção permanecerá ativo.

Compartilhamento de tempo

Em alguns sistemas embarcados, você terá mais tarefas que precisam ocorrer em um intervalo específico do que temporizadores para usar em cada tarefa. Ou o processador que você deve usar terá apenas um único temporizador. Não se preocupe, existem maneiras de contornar essa situação - você pode compartilhar um cronômetro entre várias tarefas.

Por exemplo, imagine que você tem as seguintes tarefas que devem ocorrer em seu sistema embarcado:

1. Leia um sensor de temperatura a cada 5 ms.
2. Escreva um caractere em uma porta serial a cada 12 ms.
3. Alterne um pino de I / O a cada 100 ms.

Além disso, neste exemplo, o processador tem apenas um único temporizador para usar. O que estás a fazer?

O intervalo do temporizador é definido para o maior fator comum (neste caso, 1 ms) dos tempos desejados. Em seguida, o ISR conta o número de intervalos do cronômetro decorridos. Assim que o número apropriado de intervalos ocorrer para a tarefa específica, um sinalizador é definido para que o trabalho seja executado.

Existem várias maneiras de implementar esse tipo de compartilhamento de cronômetro. Uma maneira é ter uma variável de contador estática separada para cada intervalo que você precisa controlar. A seguir está um snippet de código que seria usado no ISR do temporizador para lidar com os três intervalos diferentes para os trabalhos listados:

```
timer1Count ++;
timer2Count ++;
timer3Count ++;

/* Define o sinalizador para ler o sensor de temperatura a cada 5 ms
e reinicie o contador. */ if
(timer1Count>= 5)
{
    gbReadTemperatureSensor = TRUE;
    timer1Count = 0;
}

/* Define o sinalizador para gravar o caractere na porta serial
a cada 12 ms e, em seguida, zere o contador.
*/ if (timer2Count>= 12)
{
    gbWriteSerialCharacter = TRUE;
    timer2Count = 0;
}
```

- *contínuo* -

```
/* Defina o sinalizador para alternar o pino de I / O a cada 100 ms e
   em seguida, reinicie o contador.
   */ if (timer3Count >= 100) {

       gbTogglePin = TRUE;
       timer3Count = 0;
   }
```

O programa principal deve então verificar regularmente se algum sinalizador global está definido, realizar a ação necessária e redefinir o sinalizador. Conforme discutido anteriormente neste capítulo, deve-se tomar cuidado para evitar uma condição de corrida, pois os sinalizadores globais são compartilhados entre o ISR e o programa principal.

Evite condições de corrida

É necessário muito cuidado ao projetar seu software se o sistema embarcado em que você está trabalhando usa interrupções. Os mecanismos de comunicação entre o programa principal e as rotinas de serviço de interrupção devem ser cuidadosamente pensados. As condições de corrida são erros perigosos que podem ser extremamente difíceis de encontrar.

Habilitando e desabilitando

Mantenha a desativação das interrupções ao mínimo para reduzir a latência da interrupção; isso é muito importante em sistemas de tempo real. Tenha cuidado na implementação do código de desabilitar e habilitar em torno das seções críticas. Sempre use uma variável para controlar o estado de ativação das interrupções para que as interrupções sejam restauradas corretamente e para evitar possíveis problemas de interrupção.

Juntando tudo

La commedia è finita! (A comédia acabou!)

- Ruggero Leoncavallo

Pagliacci

Neste capítulo, reuniremos todos os elementos discutidos até agora em um aplicativo embarcado completo. É necessário trabalhar muito para entender cada parte de um sistema e aplicativo, mas certos aspectos podem permanecer confusos até que todas as partes sejam combinadas. Você deve sair deste capítulo com um entendimento completo do programa de exemplo e a capacidade de desenvolver seus próprios aplicativos embarcados úteis.

Visão geral do aplicativo

O aplicativo que vamos discutir é composto dos componentes que desenvolvemos até agora, junto com algumas funcionalidades adicionais. É uma prova da complexidade do desenvolvimento de software embarcado que este exemplo venha no final deste livro, e não no seu início. Tivemos que construir gradualmente nosso caminho até a plataforma de computação que a maioria dos livros, e até mesmo compiladores de linguagem de alto nível, consideram normal.

O aplicativo Monitor and Control fornece um meio para você exercitar diferentes aspectos de um sistema embarcado (hardware e software), adicionando ao conjunto de comandos básico que fornecemos neste livro. Você verá rapidamente, conforme o seu projeto avança do design à produção, o quão valiosa esta ferramenta é para todos que trabalham no projeto.

Assim que você for capaz de escrever o programa Monitor and Control, sua plataforma embarcada começará a se parecer muito com qualquer outro ambiente de programação. As partes mais difíceis do processo de desenvolvimento de software embarcado - familiarizar-se com o hardware, estabelecer um processo de desenvolvimento de software para ele e fazer a interface com os dispositivos de hardware individuais - estão atrás de você. Você finalmente pode concentrar seus esforços nos algoritmos e interfaces de usuário que são específicos para o produto que você está

em desenvolvimento. Em muitos casos, esses aspectos de nível superior do programa podem ser desenvolvidos em outra plataforma de computador - em paralelo com o desenvolvimento de software integrado de nível inferior que estamos discutindo - e meramente transferidos para o sistema integrado quando ambos estiverem completos. Depois que o código do nível do aplicativo é depurado e robusto, você pode portar esse código para projetos futuros.

A Figura 9-1 contém uma representação de alto nível do aplicativo Monitor e Controle. Este aplicativo inclui três drivers de dispositivo e um módulo para a interface de linha de comando. Um loop infinito é usado para o processamento principal no sistema. Um RTOS pode ser incorporado a este aplicativo, caso você decida usá-lo.

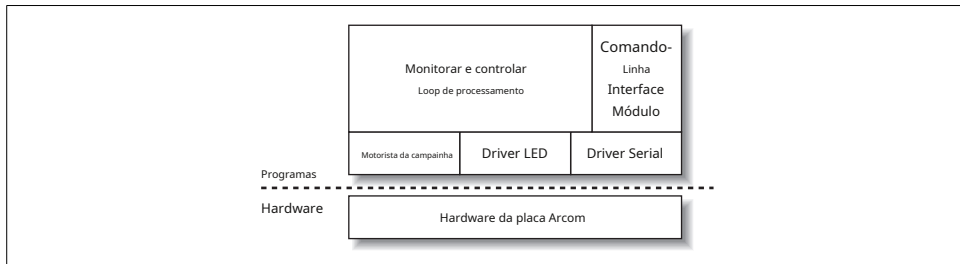


Figura 9-1. O aplicativo Monitor e Controle

Além do loop de processamento do programa Monitor e Controle e do módulo CLI, três drivers de dispositivo são mostrados na figura. Eles controlam um dos LEDs da placa Arcom, uma campainha e uma porta serial. Essa abordagem de design em camadas permite que os drivers sejam alterados quando o programa é portado para uma nova plataforma de hardware, com o software do aplicativo permanecendo inalterado.

o a Principal A função a seguir contém o loop de processamento primário para o programa Monitor e Controle, que inclui funcionalidades que exploramos nos capítulos anteriores (como enviar e receber caracteres de uma porta serial). A funcionalidade adicional inclui um driver para a campainha da placa Arcom, a capacidade de reunir caracteres de entrada em um comando e a capacidade de processar comandos. Consulte o código de exemplo online para obter detalhes sobre o driver da campainha. Porque o programa Monitor and Control aceita a entrada do usuário, um prompt (>) é enviado, o quando programa está esperando que o usuário insira um comando.

```
# inclui "serial.h"
# inclui "buzzer.h"
# inclui "led.h"
# inclui "cli.h"

/ *****
*
* Função:          a Principal
*
* Descrição: Monitore e controle o programa de interface de linha de comando.
*
```

```

* Notas:
*
* Retorna:      Esta rotina contém um loop infinito.
*
***** /
int main (vazio)
{
    char rcvChar;
    int bCommandReady = FALSE;

    /* Configure o pino de controle do LED verde. */
    ledInit ();

    /* Configure o pino de controle da campainha. */
    buzzerInit ();

    /* Configure a porta serial. */
    serialInit ();

    serialPutStr ("Programa de Monitoramento e Controle \r \n");
    serialPutChar ('>');

    enquanto (1)
    {
        /* Aguarde um caractere. */
        rcvChar = serialGetChar ();

        /* Echo o caractere de volta para a porta serial. */
        serialPutChar (rcvChar);

        /* Constrói um novo comando. */ bCommandReady
        = cliBuildCommand (rcvChar);

        /* Chame a rotina de processamento do comando CLI para verificar o comando inserido
        * e chamar a função de comando; em seguida, exiba um novo prompt. */
        if (bCommandReady == TRUE) {

            bCommandReady = FALSE;
            cliProcessCommand ();

            serialPutChar ('>');
        }
    }

    return 0;
}

```

Trabalho com portas seriais

Vimos um driver serial no Capítulo 7. O programa Monitor and Control usa o mesmo driver serial com algumas funcionalidades adicionais.

Uma mudança no driver serial é a adição de uma função "put string", serialPutStr. Essa função, que se segue, permite que as strings sejam enviadas pela porta serial por meio de uma única chamada, em vez de precisar especificar cada caractere em ordem. Isso é semelhante ao C padrão printf função, que chama o serialPutChar função repetidamente até que toda a string tenha sido transmitida pela porta serial.

```

/ *****
*
* Função:      serialPutStr
*
* Descrição: Envia uma string para a porta serial.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
void serialPutStr (char const * pOutputStr) {

    char const * pChar;

    / * Envia cada caractere da string. * /
    para (pChar = pOutputStr; * pChar != '\0'; pChar++)
        serialPutChar (* pChar);
}

```

Processamento de interface de linha de comando

O módulo de interface da linha de comandos é responsável por construir o comando de entrada, analisar um comando concluído e executar a função associada ao comando. O módulo de interface de linha de comando contém duas funções para handle estas tarefas: cliBuildCommand e cliProcessCommand.

o command_t struct tem dois membros: o nome do comando (definido pelo ponteiro nome), e a função a ser executada quando o comando é inserido (definido pelo ponteiro para uma função função).

A mesa de comando, gCommandTable, é um command_t array de tipos. O último nome de comando e função na tabela são definidos para NULO a fim de auxiliar o procedimento de consulta do comando. Comandos adicionais podem ser adicionados seguindo o formato mostrado, mas novos comandos devem ser adicionados antes do último comando de encerramento.

Você pode notar que uma macro chamada MAX_COMMAND_LEN é definido. Os nomes dos comandos devem ser menores ou iguais ao comprimento máximo do comando. O seguinte código mostra o tipo de estrutura de comando e a tabela de comando:

```

# define MAX_COMMAND_LEN      (10)
# define COMMAND_TABLE_SIZE   (4)

estrutura de typedef

```

```

{
    char const    * nome;
    vazio         (* função) (vazio);
} command_t;

command_t const gCommandTable [COMMAND_TABLE_LEN] = {

    {"AJUDA",      comandosHelp,},
    {"CONDUZIU",   comandosLed,},
    {"BUZZER",     CommandsBuzzer,},
    {NULL, NULL}

};

```

Conforme os caracteres são recebidos da porta serial, o loop de processamento principal chama o `cliBuildCommand` função, como vimos anteriormente neste capítulo. Assim que um comando completo for recebido, indicado por um retorno de carro (`\r`) caractere, o índice do buffer é reiniciado e a função retorna VERDADE. Caso contrário, o índice do buffer é incrementado e FALSO é devolvido.

`cliBuildCommand` armazena os caracteres de entrada em um buffer chamado `gCommandBuffer`. o `gCommandBuffer` tem espaço suficiente para um único comando de `MAX_COMMAND_LEN` tamanho mais um byte adicional para o caractere de terminação de string. Certos caracteres não são armazenados no buffer de comando, incluindo alimentações de linha (`\n`), guias (`\t`), e espaços.

Observe que os caracteres de entrada são convertidos em maiúsculas (com a macro `TO_UPPER`) antes da inserção no buffer. Isso torna a interface da linha de comando um pouco mais amigável; o usuário não precisa se lembrar de usar letras maiúsculas para inserir comandos válidos.

A variável estática local `idx` mantém o controle dos caracteres inseridos no buffer de comando. Conforme novos caracteres são armazenados no buffer, o índice é incrementado. Se muitos caracteres forem recebidos para um comando, o índice é definido como zero e VERDADE é retornado para iniciar o processamento do comando.

```

# define TO_UPPER(x)      (((x>= 'a') && (x<= 'z'))? ((x) - ('a' - 'A')): (x))

estático char gCommandBuffer [MAX_COMMAND_LEN + 1];

/ *****
*
* Função:      cliBuildCommand
*
* Descrição: Coloque os caracteres recebidos no buffer de comando. Uma vez
*              um comando completo é recebido retorna TRUE.
*
* Notas:
*
* Retorna:     TRUE se um comando estiver completo, caso contrário, FALSE.
*
***** /

int cliBuildCommand (char nextChar) {

```



```

estático uint8_t idx = 0;

/* Não armazene nenhum novo caractere de linha ou espaço. */
if ((nextChar == '\n') || (nextChar == '"') || (nextChar == '\t'))
    retorna falso;

/* O comando concluído foi recebido. Substitua o carro final
 * caractere de retorno com um caractere NULL para ajudar no processamento do
 * comando. */
if (nextChar == '\r') {

    gCommandBuffer [idx] = '\0';
    idx = 0;
    return TRUE;
}

/* Converte o caractere recebido em maiúsculas. Isso corresponde ao caso
 * de comandos na tabela de comandos. Em seguida, armazene o caractere recebido
 * no buffer de comando. */
gCommandBuffer [idx] = TO_UPPER (nextChar);
idx ++;

/* Se o comando for muito longo, redefina o índice e o processo
 * o buffer de comando atual. */
if (idx > MAX_COMMAND_LEN) {

    idx = 0;
    return TRUE;
}

retorna falso;
}

```

Uma vez que um comando completo é montado, o cliProcessCommand a função, que se segue, é chamada a partir do loop de processamento principal. Esta função percorre a tabela de comandos em busca de um nome de comando correspondente.

A variável idx é inicializado em zero para iniciar a pesquisa no início da tabela de comandos e, em seguida, mantém o controle do comando que está sendo verificado. A função strcmp é usado para comparar o comando do usuário com os comandos da tabela. Se o comando for encontrado, o sinalizador bCommandFound está configurado para VERDADE. Isso faz com que o loop de pesquisa saia e a função associada seja executada. Se o comando não for encontrado, um mensagem de erro é enviada pela porta serial.

```

# inclui <string.h>

/ *****
*
* Função:      cliProcessCommand
*
* Descrição: Procure o comando na tabela de comandos. Se o
*              comando for encontrado, chame a função do comando. Se o
*              comando não for encontrado, gera uma mensagem de erro.

```

```

*
* Notas:
*
* Retorna:          Nenhum.
*
***** /
void cliProcessCommand (void) {

    int bCommandFound =
    FALSE; int idx;

    /* Procure o comando na tabela de comandos até que seja encontrado ou
    * o fim da tabela é alcançado. Se o comando for encontrado, interrompa
    * fora do laço. */
    para (idx = 0; gCommandTable [idx] .name != NULL; idx++) {

        if (strcmp (gCommandTable [idx] .name, gCommandBuffer) == 0) {

            bCommandFound = TRUE;
            pausa;
        }
    }

    /* Se o comando foi encontrado, chame a função de comando. De outra forma,
    * produz uma mensagem de erro. */
    if (bCommandFound == TRUE) {

        serialPutStr ("\r \n");
        (* gCommandTable [idx] .function) ();
    }
    outro
        serialPutStr ("\r \nCommand não encontrado. \r \n");
}

```

Todas as funções na tabela de comandos estão contidas em um arquivo; isso mantém o ponto de entrada para todos os comandos em um único local. o CommandLed A função alterna o LED verde, conforme mostrado no código a seguir. Esta função usa o mesmo ledToggle função abordada no Capítulo 3.

```

# inclui "led.h"

/ *****
*
* Função:          CommandLed
*
* Descrição: Alterne a função de comando do LED verde.
*
* Notas:
*
* Retorna:          Nenhum.
*
***** /
comandos voidLed (void)
{

```

```
    ledToggle ();
}
```

o CommandsBuzzer função alterna a campainha no módulo adicional da placa Arcom chamando a função buzzerToggle, como mostrado aqui:

```
# inclui "buzzer.h"

/ *****
*
* Função:      CommandsBuzzer
*
* Descrição: Alternar a função de comando da campainha.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
void comandosBuzzer (void) {

    buzzerToggle ();
}
```

A função de comando de ajuda, comandosHelp, faz loops através do gCommandTable e envia o nome do comando pela porta serial. Isso dá ao usuário uma lista de todos os componentes mandados suportados pela interface de linha de comando.

```
# inclui "cli.h"
# inclui "serial.h"

/ *****
*
* Função:      comandosHelp
*
* Descrição: Função de comando de ajuda.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
void comandosHelp (void) {

    int idx;

    /* Loop através de cada comando na tabela e enviar o comando
     * nome para a porta serial. */
    para (idx = 0; gCommandTable [idx] .name != NULL; idx++) {

        serialPutStr (gCommandTable [idx] .name);
        serialPutStr ("\r \n");
    }
}
```

O programa Monitor and Control fornece uma linha de base de funcionalidade para o desenvolvimento de uma interface de linha de comando útil. A funcionalidade da CLI pode ser estendida adicionando novos comandos ou permitindo que os usuários insiram parâmetros para comandos. Para acomodar os parâmetros de entrada, o `command_t` a estrutura pode ser expandida para conter os valores máximo e mínimo. Ao analisar o comando, você precisará analisar os parâmetros de entrada e validar os intervalos de parâmetros com aqueles contidos na tabela de comandos.

Sistemas operacionais

*o · s · o · Pho · bi · uma n. Um medo comum entre
programadores de sistemas.*

Muitos sistemas embarcados hoje incorporam um sistema operacional. Isso pode variar de um pequeno kernel a um sistema operacional completo - normalmente chamado de *sistema operacional em tempo real* ou *RTOS* (pronunciado “are-toss”). De qualquer forma, você precisará saber quais recursos são os mais importantes e como eles são usados com o resto do software. No mínimo, você precisa entender como é um sistema operacional em tempo real por fora. Neste capítulo, daremos uma olhada detalhada nos mecanismos encontrados na maioria dos sistemas operacionais e como usá-los.

As informações neste capítulo são muito gerais e não incluem exemplos de código específicos. Isso ocorre porque os recursos e APIs que implementam as atividades neste capítulo são diferentes em cada sistema operacional. Os capítulos subsequentes mostram o que fazer no Linux e eCos, dois sistemas operacionais populares usados em ambientes integrados.

História e Propósito

Nos primeiros dias da computação, não existia um sistema operacional. Os programadores de aplicativos eram totalmente responsáveis por controlar e monitorar o estado do processador e de outros hardwares. Na verdade, o objetivo dos primeiros sistemas operacionais era fornecer uma plataforma de hardware virtual que tornasse os programas de aplicativos mais fáceis de escrever. Para atingir esse objetivo, os desenvolvedores de sistema operacional precisavam apenas fornecer uma coleção frouxa de rotinas - muito parecida com uma biblioteca de software moderna - para redefinir o hardware para um estado conhecido, ler o estado das entradas e alterar o estado das saídas.

Os sistemas operacionais modernos adicionam a isso a capacidade de executar vários softwares *tarefas* simultaneamente em um único processador - um recurso chamado *multitarefa*. Cada uma dessas tarefas (também comumente chamada de *fio*) é uma parte do software que pode ser separada do resto do sistema operacional e ser executada de forma independente. Um conjunto de requisitos de software embarcado geralmente pode ser dividido em um pequeno número de tais

peças. Por exemplo, o dispositivo servidor de impressão descrito no Capítulo 2 contém duas tarefas principais de software:

- Receber dados dos computadores conectados à porta Ethernet.
- Formatar e enviar os dados para a impressora conectada à porta paralela.

As tarefas fornecem uma abstração de software chave que torna o projeto e a implementação de software embarcado mais fácil e o código-fonte resultante mais simples de entender e manter. Ao quebrar o programa maior em partes menores, o programador pode concentrar mais facilmente sua energia e talentos nos recursos exclusivos do sistema em desenvolvimento.

Estritamente falando, um sistema operacional não é um componente obrigatório de qualquer sistema de computador - integrado ou não. Sempre é possível executar as mesmas funções de dentro do próprio programa aplicativo. Na verdade, todos os exemplos até agora neste livro fizeram exatamente isso. Existe um caminho de execução - começando em a Principal —Que é executado no sistema. Isso equivale a ter apenas uma tarefa. Mas, à medida que a complexidade do aplicativo se expande além da simples tarefa de piscar um LED, os benefícios de um sistema operacional superam em muito os custos associados.

Uma parte comum de todos os sistemas operacionais é o *núcleo*. Na maioria dos sistemas operacionais, o kernel consiste no agendador, na rotina para lidar com a alternância entre as diferentes tarefas em execução no sistema e nos mecanismos de comunicação entre as tarefas.

The Scheduler

Já falamos sobre multitarefa e a ideia de que um sistema operacional possibilita a execução de vários “programas” ao mesmo tempo. Mas o que isso significa? Como é possível executar várias tarefas simultaneamente? Na verdade, as tarefas não são executadas ao mesmo tempo. Em vez disso, eles são executados em pseudoparalelo. Eles simplesmente se revezam no uso do processador. Isso é semelhante à maneira como várias pessoas podem ler a mesma cópia de um livro. Apenas uma pessoa pode realmente usar o livro em um determinado momento, mas cada pessoa pode lê-lo se todos se revezarem.

Um sistema operacional é responsável por decidir qual tarefa usará o processador em um determinado momento. A parte do sistema operacional que decide qual das tarefas tem o direito de usar o processador em um determinado momento é chamada de *Agendador*. O planejador é o coração e a alma de qualquer sistema operacional. Alguns dos algoritmos de programação mais comuns são:

Primeiro a entrar, primeiro a sair (FIFO)

Este agendamento (também chamado *multitarefa cooperativa*) permite que cada tarefa seja executada até que seja concluída, e somente depois disso é que a próxima tarefa é iniciada.

Trabalho mais curto primeiro

Este algoritmo permite que cada tarefa seja executada até que seja concluída ou suspensa; a próxima tarefa selecionada é aquela que exigirá a menor quantidade de tempo do processador para ser concluída.

Prioridade

Esse algoritmo é normalmente usado em sistemas operacionais de tempo real. Cada tarefa recebe uma prioridade que é usada para determinar quando a tarefa é executada quando está pronta. A programação prioritária pode ser preemptiva ou não preemptiva. *Preemptivo* significa que qualquer tarefa em execução pode ser interrompida pelo sistema operacional se uma tarefa de prioridade mais alta ficar pronta.

Round robin

Neste algoritmo de agendamento, cada tarefa é executada por um determinado período de tempo chamado de *intervalo de tempo*. Depois de decorrido esse intervalo de tempo, a tarefa em execução é interrompida pelo sistema operacional e a próxima tarefa na linha tem sua chance de ser executada. A tarefa antecipada não pode ser executada novamente até que cada uma das outras tarefas naquela rodada tenha sua chance.

Agendamento em tempo real

O planejador em alguns sistemas operacionais é em tempo real. Enquanto os agendadores da seção anterior levam em consideração apenas prioridades e intervalos de tempo, um agendador em tempo real leva em consideração que algumas tarefas têm prazos. Algumas tarefas em um sistema em tempo real podem não ter prazos ou podem ter penalidades baixas para prazos perdidos. Tarefas como essas podem usar o tempo de processamento em segundo plano para fazer seu trabalho.

Nesse tipo de sistema operacional, o escalonador em tempo real deve saber os prazos de todas as tarefas. O planejador deve basear as decisões em uma comparação dos prazos de tarefas que estão na fila de prontas.

Existem vários programadores em tempo real, incluindo:

Executivo em tempo real

Cada tarefa é atribuída a um intervalo de tempo exclusivo em um padrão de repetição periódica. Um executivo em tempo real é mais estático do que um sistema operacional e assume que o programador sabe quanto tempo leva cada tarefa. Se o executivo sabe que a tarefa pode completar seu trabalho no tempo previsto, seus prazos podem ser cumpridos. Isso não funcionará se você precisar criar e sair de tarefas rapidamente ou executar tarefas em intervalos irregulares.

Prazo mais cedo primeiro

O sistema operacional rastreia o tempo do próximo prazo final para todas as tarefas. Em cada ponto de agendamento, o agendador seleciona a tarefa com o prazo mais próximo. Este é um agendador baseado em prioridade, mas com a adição de que ele calcula os prazos de tarefas em tempo real a cada tique do cronômetro e ajusta as prioridades conforme apropriado. Se novas prioridades forem atribuídas, isso pode significar que novas tarefas são executadas e, portanto, que uma troca de contexto (que discutiremos mais tarde neste capítulo na seção “Troca de contexto”) deve ocorrer. Uma desvantagem desse escalonador é que ele tem uma grande sobrecarga computacional.

Frouxidão mínima primeiro

O sistema operacional rastreia os prazos e o tempo para concluir o trabalho restante de cada tarefa. O agendador seleciona a tarefa com menos *frouxidão*, onde a frouxidão é o tempo disponível menos o tempo de trabalho restante e o tempo disponível é o tempo do prazo final menos o tempo atual. A teoria por trás desse algoritmo de escalonamento é que a tarefa com menos “tempo de sobra” pode menos render o processador. A ordem exata em que as tarefas são executadas pode ser diferente do agendador mais cedo do prazo.

Reserva de recursos

Quando uma tarefa é criada, ela declara seus requisitos em termos de prazos, uso do processador e outros recursos relevantes. O sistema operacional deve admitir a nova tarefa somente se o sistema puder garantir esses recursos sem fazer com que nenhuma outra tarefa (já admitida) falhe.

O algoritmo de agendamento que você escolhe depende de seus objetivos. Algoritmos diferentes produzem resultados diferentes. Vamos supor que você receba 10 empregos e cada um leve um dia para terminar. Em 10 dias, você terá todos eles concluídos. Mas e se um ou mais tiverem um prazo? Se a nona tarefa dada a você tem um prazo de três dias, fazer as tarefas na ordem em que as recebe fará com que você perca esse prazo.

O objetivo de um algoritmo de agendamento em tempo real é garantir que as restrições de tempo críticas, como prazos e tempo de resposta, sejam atendidas. Quando necessário, são tomadas decisões que favorecem as restrições de tempo mais críticas, mesmo ao custo de violar outras.

Para demonstrar a importância de um algoritmo de escalonamento, considere um sistema com apenas duas tarefas, que chamaremos de Tarefa 1 e Tarefa 2. Suponha que ambas sejam tarefas periódicas com períodos T_1 e T_2 , e para cada tarefa, o prazo é o início de seu próximo ciclo. A tarefa 1 tem $T_1 = 50$ ms e um tempo de execução de pior caso de $C_1 = 25$ ms. A tarefa 2 tem $T_2 = 100$ ms e $C_2 = 40$ ms.

O processador tem tempo suficiente para lidar com as duas tarefas? Podemos começar a responder a esta pergunta observando quanto tempo do processador cada tarefa precisa em seu pior caso - seu *utilização*. A utilização da tarefa eu é:

$$\text{você } i = C_{eu} / T_{eu}$$

Assim, se $U_1 = 50$ por cento e $U_2 = 40$ por cento, a utilização total solicitada é:

$$U = U_1 + U_2 = 90 \text{ por cento}$$

Parece lógico que, se a utilização for inferior a 100 por cento, deve haver tempo de CPU disponível suficiente para executar as duas tarefas.

Vamos considerar um *prioridade estática* algoritmo de agendamento, onde as prioridades das tarefas são exclusivas e não podem ser alteradas em tempo de execução. Com duas tarefas, existem apenas duas possibilidades:

- Caso 1: Prioridade (T_1) > Prioridade (T_2)
- Caso 2: Prioridade (T_1) < Prioridade (T_2)

Os dois casos são mostrados na Figura 10-1. No Caso 1, ambas as tarefas cumprem seus respectivos prazos. No caso 2, no entanto, a Tarefa 1 perde um prazo, apesar de 10 por cento do tempo ocioso, porque a prioridade mais alta da Tarefa 2 exigia que ela fosse agendada primeiro. Isso ilustra o importância da atribuição de prioridades.

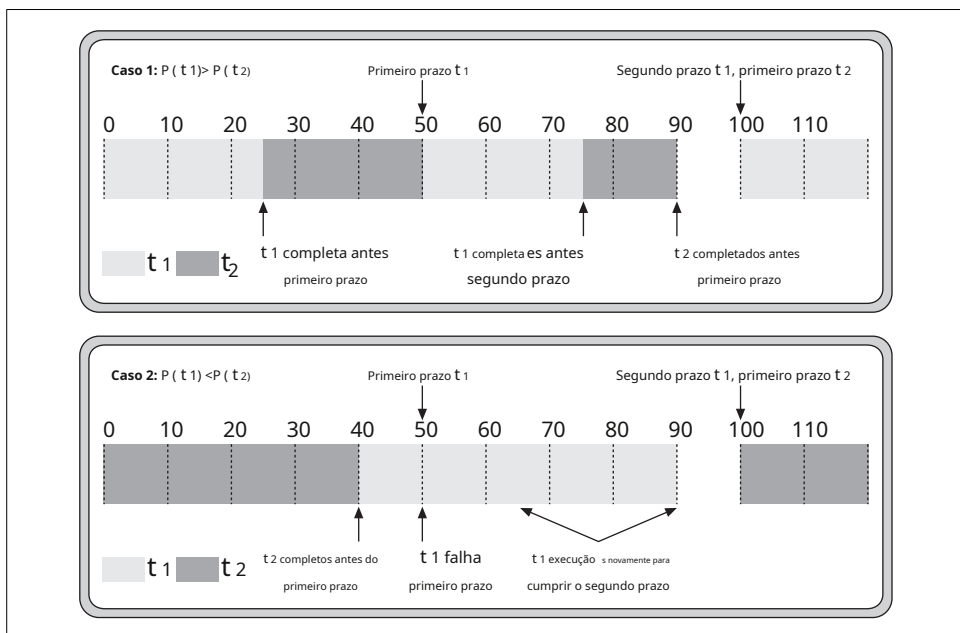


Figura 10-1. Exemplos de resultados de agendamento

Os sistemas de tempo real às vezes exigem uma maneira de compartilhar o processador que permite que as tarefas mais importantes tomem o controle do processador assim que necessário. Portanto, a maioria dos sistemas operacionais de tempo real utiliza um algoritmo de agendamento baseado em prioridade que oferece suporte à preempção. Essa é uma maneira elegante de dizer que, a qualquer momento, a tarefa que está usando o processador é garantidamente a tarefa de maior prioridade pronta para isso. As tarefas de prioridade mais baixa devem esperar até que as tarefas de prioridade mais alta sejam concluídas usando o processador antes de retomar seu trabalho. O planejador detecta tais condições por meio de interrupções ou outros eventos causados pelo software; esses eventos são chamados de pontos de agendamento.

A Figura 10-2 mostra um cenário de duas tarefas em execução em diferentes níveis de prioridade.

Existem duas tarefas na Figura 10-2: Tarefa A e Tarefa B. Nesse cenário, a Tarefa B tem maior prioridade do que a Tarefa A. No início, a Tarefa A está em execução. Quando a Tarefa B está pronta para ser executada, o agendador antecipa a Tarefa A e permite que a Tarefa B seja executada. A tarefa B é executada até concluir seu trabalho. Nesse ponto, o controle do processador retorna à Tarefa A, que continua seu trabalho de onde parou.

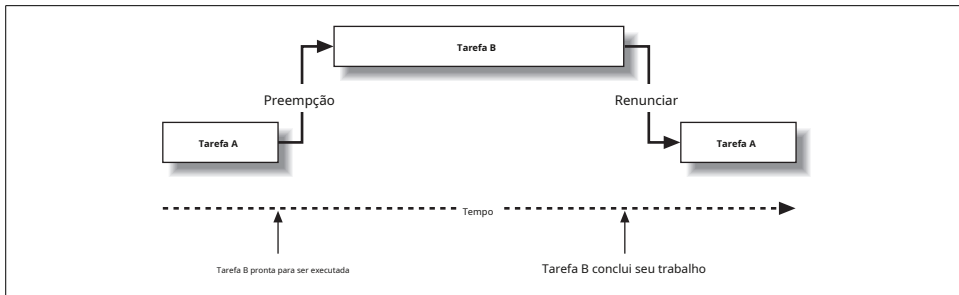


Figura 10-2. Agendamento prioritário de duas tarefas

Quando um algoritmo de agendamento baseado em prioridade é usado, também é necessário ter a mesma política de agendamento de backup para tarefas com o mesma prioridade. Uma programação comum algoritmo para tarefas com a mesma prioridade é round robin. Na Figura 10-3, mostramos um cenário no qual três tarefas estão em execução para demonstrar o agendamento round robin com uma fração de tempo. Neste exemplo, as tarefas B e C têm a mesma prioridade, que é maior do que a Tarefa A.

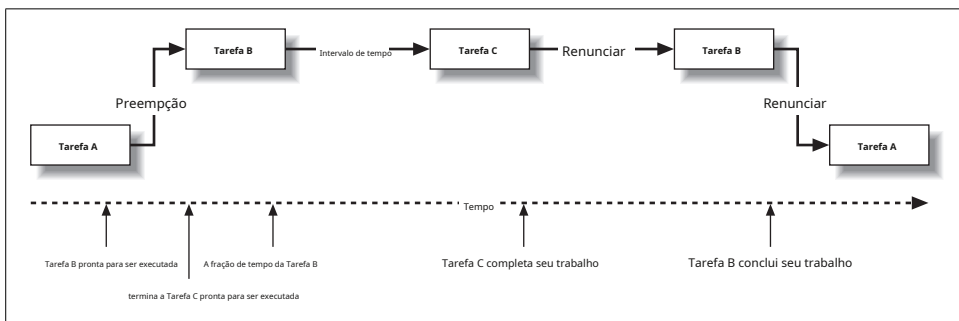


Figura 10-3. Agendamento prioritário de três tarefas

No cenário mostrado na Figura 10-3, a Tarefa A está sendo executada quando a Tarefa B está pronta para ser executada. A tarefa A é interrompida para que a tarefa B possa ser executada. Enquanto a Tarefa B está em execução, a Tarefa C também fica pronta para ser executada. O agendador, portanto, permite que a Tarefa B seja executada por um período de fração de tempo. Depois que esse período expira, o agendador dá à Tarefa C a oportunidade de ser executada. Em seguida, a Tarefa C conclui seu trabalho. Como a Tarefa B ainda tem trabalho a ser concluído e tem a prioridade mais alta de todas as tarefas prontas, o agendador permite que a Tarefa B seja executada. Assim que a Tarefa B terminar seu trabalho, a Tarefa A pode terminar.

Pontos de Agendamento

Você pode estar se perguntando como o planejador - que também é um software - obtém a oportunidade de executar e fazer seu trabalho. É onde *pontos de agendamento* entrar. Em termos simples, os pontos de agendamento são o conjunto de eventos do sistema operacional que resultam na execução do código do agendador. A seguir está uma lista de pontos de agendamento:

Criação de tarefa

Quando uma tarefa é criada, o planejador é chamado para selecionar a próxima tarefa a ser executada. Se a tarefa atualmente em execução ainda tiver a prioridade mais alta de todas as tarefas prontas, será permitido continuar usando o processador. Caso contrário, a tarefa pronta de maior prioridade será executada a seguir.

Exclusão de tarefa

Como na criação de tarefas, o agendador é chamado durante a exclusão da tarefa. No entanto, no caso de exclusão de tarefa, se a tarefa atualmente em execução está sendo excluída, uma nova tarefa é sempre selecionada em virtude do fato de que a anterior não existe mais.

Tique do relógio

O tique do relógio é a pulsação do sistema. É um evento periódico que é disparado por uma interrupção do temporizador (semelhante ao que já discutimos). O tique-taque do relógio oferece uma oportunidade para despertar tarefas que estão esperando por um certo período de tempo antes de realizar sua próxima ação.

Bloco de tarefas

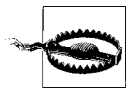
Quando a tarefa em execução faz uma chamada de sistema que bloqueia, essa tarefa não pode mais usar o processador. Assim, o planejador é chamado para selecionar uma nova tarefa a ser executada.

Desbloqueio de tarefa

Uma tarefa pode estar aguardando a ocorrência de algum evento, como o recebimento de dados. Assim que o evento ocorre, a tarefa bloqueada fica pronta para execução e, portanto, imediatamente elegível para ser considerada para execução.

Bloqueio e desbloqueio

Os sistemas operacionais em tempo real podem permitir que uma tarefa bloqueie o planejador. Bloquear o agendador proíbe sua execução e, por sua vez, impede que outras tarefas sejam executadas (uma vez que não podem ser agendadas). A tarefa que bloqueia o planejador mantém o controle do processador, independentemente de as tarefas de alta prioridade estarem prontas para serem executadas ou não. Isso permite que a tarefa com o bloqueio do planejador execute operações sem se preocupar com problemas de thread-safe com outras tarefas. As interrupções ainda funcionam e os ISRs ainda funcionam.



Deve-se ter cuidado ao travar o agendador, pois pode dificultar a capacidade de resposta do sistema. Em geral, o bloqueio do planejador deve ser evitado sempre que possível.

Cada bloqueio do planejador deve ter uma contraparte de desbloqueio, caso contrário, o sistema tem vai parar de funcionar. A seguir está um exemplo de bloqueio e desbloqueio a Agendador:

```
/ * Executa operações de tarefa. * /
```

```
os_scheduler_lock ();
```

```
/* Executar trabalho enquanto o planejador não pode executar outra tarefa. */
```

```
os_scheduler_unlock();
```

```
/* Executa outras operações de tarefa. */
```

O sistema operacional acompanha o estado de bloqueio do planejador com uma variável. Esta variável é incrementada quando são feitas chamadas para bloquear o planejador e diminuída quando são feitas chamadas para desbloquear. O planejador sabe que pode ser executado quando a variável de estado de bloqueio é definida como 0.

Tarefas

Diferentes tipos de tarefas podem ser executados em um sistema operacional. Por exemplo, uma tarefa pode ser periódica, e sai após a conclusão de seu trabalho. A tarefa pode então ser reiniciada quando houver mais trabalho a ser feito. Normalmente, no entanto, uma tarefa é executada para sempre, semelhante ao loop infinito discutido no Capítulo 3. Cada tarefa também tem sua própria pilha, que é tipicamente alocada estaticamente.

Estados da Tarefa

Lembra que dissemos que apenas uma tarefa poderia realmente usar o processador em um determinado momento? Essa tarefa é chamada de *correndo* tarefa, e nenhuma outra tarefa pode estar no mesmo estado ao mesmo tempo. Tarefas que estão prontas para serem executadas, mas não estão usando o processador no momento, estão no *pronto* estado, e as tarefas que estão esperando que algum evento externo a elas ocorram antes de continuar estão no *esperando* Estado. A Figura 10-4 mostra o relações entre esses três estados.

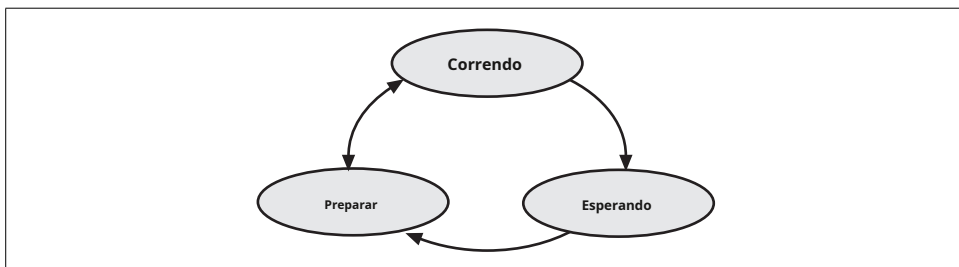


Figura 10-4. Possíveis estados de uma tarefa

Uma transição entre os estados pronto e em execução ocorre sempre que o sistema operacional seleciona uma nova tarefa para ser executada durante um ponto de agendamento. A tarefa que estava em execução anteriormente sai do estado de execução e a nova tarefa (selecionada na fila de tarefas no estado pronto) é promovida a execução. Uma vez em execução, uma tarefa sairá desse estado apenas se for encerrada, se uma tarefa de maior prioridade ficar pronta ou se precisar

esperar que algum evento externo a ele ocorra antes de continuar. Neste último caso, diz-se que a tarefa *bloquear*, ou espere, até que esse evento ocorra. Uma tarefa pode ser bloqueada esperando por outra tarefa ou por um dispositivo de E / S, ou pode ser bloqueada por hibernação (esperando que um período de tempo específico passe).

Quando a tarefa é bloqueada, ela entra no estado de espera e o sistema operacional seleciona uma das tarefas prontas para ser executada. Portanto, embora possa haver qualquer número de tarefas em cada um dos estados pronto e em espera, sempre haverá exatamente uma tarefa no estado de execução a qualquer momento.

É importante observar que apenas o planejador pode promover uma tarefa ao estado de execução. Tarefas recém-criadas e tarefas que terminaram de esperar por seu evento externo são colocadas no estado de prontas primeiro. O planejador incluirá essas novas tarefas prontas em sua futura tomada de decisão.

Para controlar as tarefas, o sistema operacional geralmente implementa filas para cada um dos estados de espera e pronto. A fila pronta geralmente é classificada por prioridade para que a tarefa de maior prioridade esteja no topo da fila. O agendador pode então escolher facilmente a tarefa de maior prioridade a ser executada em seguida.

Mudança de contexto

O processo real de mudança de uma tarefa para outra é chamado de *mudança de contexto*. Os contextos de tarefa são específicos do processador e às vezes específicos do compilador, assim como o código que implementa a troca de contexto. Isso significa que sempre deve ser escrito em linguagem assembly e é muito específico para o hardware. Os sistemas operacionais, especialmente para sistemas embarcados, enfatizam o objetivo de minimizar o tempo necessário para alternar os contextos das tarefas. A Figura 10-5 mostra um exemplo de uma operação de troca de contexto entre duas tarefas: A (na prioridade 150) e B (na prioridade mais alta, 200).

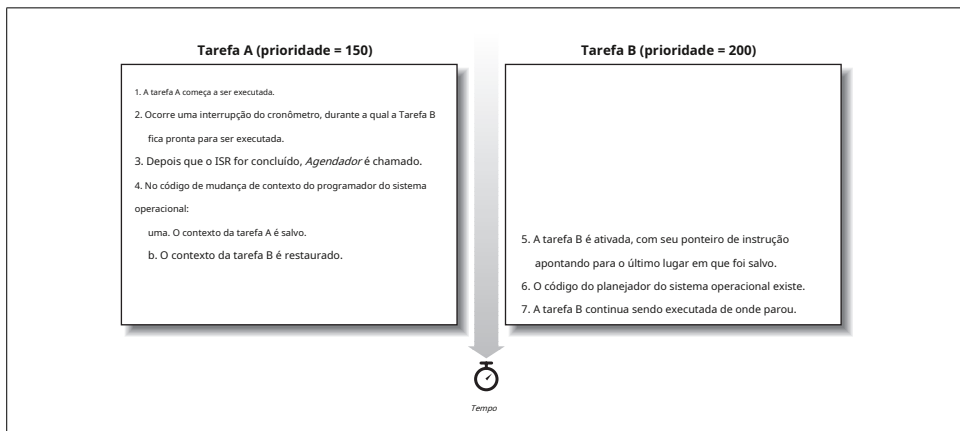


Figura 10-5. Uma mudança de contexto

A tarefa ociosa

Se não houver tarefas no estado pronto quando o agendador for chamado, a *tarefa ociosa* é executado. A tarefa ociosa parece a mesma em todos os sistemas operacionais. É simplesmente um loop infinito que não faz nada e nunca bloqueia. A tarefa ociosa é completamente escondida do desenvolvedor do aplicativo. Às vezes, no entanto, o sistema operacional atribui a ele um ID de tarefa e prioridade válidos. A tarefa ociosa é sempre considerada no estado pronto (quando não está em execução) e, por causa de sua baixa prioridade, pode ser encontrada no final da lista pronta. Outras tarefas às vezes são chamadas de *tarefas do usuário* para distingui-los da tarefa ociosa.

Contexto da Tarefa

O agendador mantém informações sobre o estado de cada tarefa. Esta informação é chamada de *contexto da tarefa* e serve a um propósito semelhante a um marcador. Na analogia anterior de múltiplos leitores, presume-se que cada leitor do livro tenha seu próprio marcador. O proprietário do marcador deve ser capaz de reconhecê-lo (por exemplo, tem seu nome escrito) e deve indicar onde ele parou de ler quando abriu mão do controle do livro pela última vez. Este é o contexto do leitor.

O contexto de uma tarefa registra o estado do processador imediatamente antes do ponto em que outra tarefa assume o controle dele. Isso geralmente consiste em um ponteiro para a próxima instrução a ser executada (o ponteiro da instrução), o endereço do topo atual da pilha (o ponteiro da pilha) e o conteúdo do sinalizador do processador e dos registradores de uso geral.

Para manter as tarefas e seus contextos organizados, o sistema operacional mantém algumas informações sobre cada tarefa. Os sistemas operacionais escritos em C costumam manter essas informações em uma estrutura de dados chamada de *bloco de controle de tarefas*. O bloco de controle de tarefa contém um ponteiro para o contexto da tarefa, o estado atual da tarefa, a prioridade da tarefa, a função de ponto de entrada da tarefa e quaisquer dados específicos da tarefa (como parâmetros e identificação da tarefa).

Prioridades de Tarefas

Definir as prioridades das tarefas em um sistema é importante. É preciso ter cuidado para que as tarefas de prioridade mais baixa façam seu trabalho, da mesma forma que as tarefas de prioridade mais alta. Caso contrário, pode ocorrer privação de uma tarefa, onde uma tarefa de baixa prioridade é impedida de realizar qualquer trabalho.

Existem vários motivos pelos quais a fome pode ocorrer em um sistema, incluindo o seguinte:

- A sobrecarga do processador ocorre quando as tarefas de alta prioridade monopolizam o processador e estão sempre em execução ou prontas para serem executadas.
- As tarefas de baixa prioridade estão sempre no final de uma fila de eventos com base em prioridade e, portanto, podem ter sua execução bloqueada permanentemente.
- Uma tarefa pode ser impedida de ser executada por um bug em outra tarefa; por exemplo, uma tarefa falha em sinalizar quando deveria.

Existem soluções para esses problemas, como:

- Usando um processador mais rápido.
- Usando uma fila FIFO de tarefas em vez de agendamento baseado em prioridade. Isso pode não ser possível, dependendo da implementação do sistema operacional. Além disso, pode ser apropriado em alguns sistemas privar as tarefas de baixa prioridade, mas esta deve ser uma decisão consciente.
- Corrigindo todos os bugs (às vezes é mais fácil falar do que fazer).

Avalie a programação monotônica

o *algoritmo monotônico de taxa (RMA)* é um procedimento para determinar a prioridade ótima de cada tarefa periódica em um sistema. Este procedimento atribui prioridades fixas às tarefas para maximizar sua "capacidade de programação". Um conjunto de tarefas é considerado agendável se todas as tarefas cumprirem todos os prazos o tempo todo. O algoritmo é direto:

Atribua a prioridade de cada tarefa de acordo com seu período, de forma que quanto menor o período, maior a prioridade.

O raciocínio por trás desse algoritmo é que a tarefa com o período mais curto tem menos tempo para fazer seu trabalho quando estiver pronta novamente. No próximo exemplo, o período da Tarefa 1 é mais curto do que o período da Tarefa 2. Seguindo a regra de RMA, a Tarefa 1 recebe a prioridade mais alta. Isso corresponde ao Caso 1 da Figura 10-1, que é a atribuição de prioridade que conseguiu cumprir os dois prazos.

RMA é um exemplo de algoritmo de prioridade estática. A alternativa para um algoritmo de prioridade estática é a classe muito mais complicada de agendadores dinâmicos, que aparecem em sistemas operacionais sofisticados de nível comercial; não recomendamos que você tente projetar seu próprio agendador dinâmico. RMA é o algoritmo de prioridade estática ideal. Se um determinado conjunto de tarefas não pode ser agendado usando o RMA, esse conjunto de tarefas não pode ser agendado usando nenhum algoritmo de prioridade estática.

Uma das principais limitações do agendamento de prioridade fixa é que nem sempre é possível utilizar totalmente o processador. Mesmo que RMA seja o esquema de prioridade fixa ideal, ele tem um limite escalonável de pior caso do seguinte:

$$C_n = n(2^{1/n} - 1)$$

Onde n é o número de tarefas em um sistema. Como você esperaria, o limite agendável de pior caso para uma tarefa é 100 por cento. Mas à medida que o número de tarefas aumenta, o limite programável diminui, eventualmente se aproximando de seu limite de cerca de 69,3 por cento ($\ln 2$, para ser preciso).

É teoricamente possível que um conjunto de tarefas exija apenas 70% da utilização da CPU no total e ainda assim não cumpra todos os seus prazos. Por exemplo, considere o caso mostrado na Figura 10-6. A única mudança em relação ao exemplo mostrado na Figura 10-1 é que tanto o período quanto o tempo de execução da Tarefa 2 foram reduzidos. Com base em RMA, Tarefa 1

é atribuída uma prioridade mais alta. Apesar da utilização de apenas 90%, a Tarefa 2 perde seu primeiro prazo. Inverter as prioridades não teria melhorado a situação.

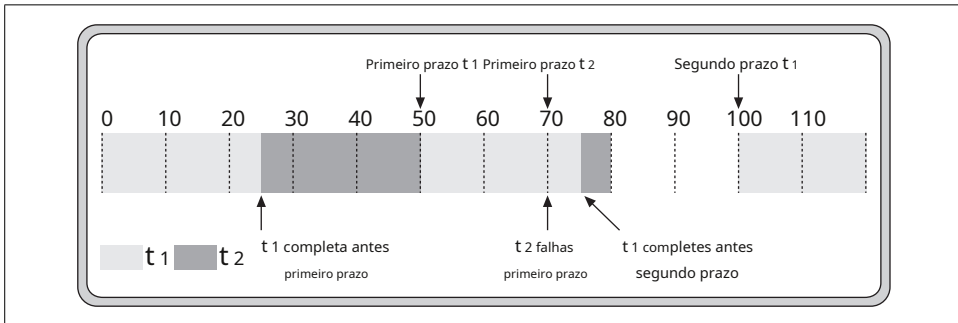


Figura 10-6. Exemplo mostrando um conjunto de tarefas não programáveis

Nesse caso, a única forma de cumprir todos os prazos é por meio de um algoritmo de escalonamento dinâmico, que, por aumentar a complexidade do sistema, não está disponível em muitos sistemas operacionais.

Às vezes, um determinado conjunto de tarefas terá utilização total acima do limite agendável do pior caso e ainda será agendável com prioridades fixas. O caso 1 na Figura 10-1 é um exemplo perfeito. A agendabilidade depende, então, das especificidades dos períodos e tempos de execução de cada tarefa do conjunto, que devem ser analisados manualmente. Somente se a utilização total for menor que C_n você pode pular essa etapa de análise manual e assumir que todas as tarefas cumprirão todos os seus prazos.

O RMA se degrada normalmente, pois se apenas um prazo for perdido, com certeza será a tarefa de menor prioridade com trabalho pendente. O “conjunto crítico” é o conjunto de tarefas que nunca perderá nenhum prazo. Portanto, o RMA funciona bem para uma mistura de tarefas em tempo real difíceis e suaves, em que os prazos rígidos correspondem às tarefas de maior frequência.

Mecânica da Tarefa

No início deste capítulo, discutimos como o sistema operacional faz parecer que as tarefas estão sendo executadas simultaneamente. Ao escrever código para tarefas, é melhor manter isso em mente e escrever tarefas como se fossem executadas em paralelo. A operação básica de uma tarefa é mostrada na Figura 10-7.

Conforme mostrado na Figura 10-7, a primeira parte inicializa quaisquer variáveis específicas da tarefa ou outros recursos usados pela tarefa. Em seguida, um loop infinito é usado para realizar o trabalho da tarefa. Primeiro, a tarefa espera que algum tipo de evento ocorra. Neste ponto, a tarefa está bloqueada. Ele está no estado de espera e é colocado na fila de espera. Existem vários tipos de eventos: um sinal de outra tarefa, um sinal de um ISR e a expiração

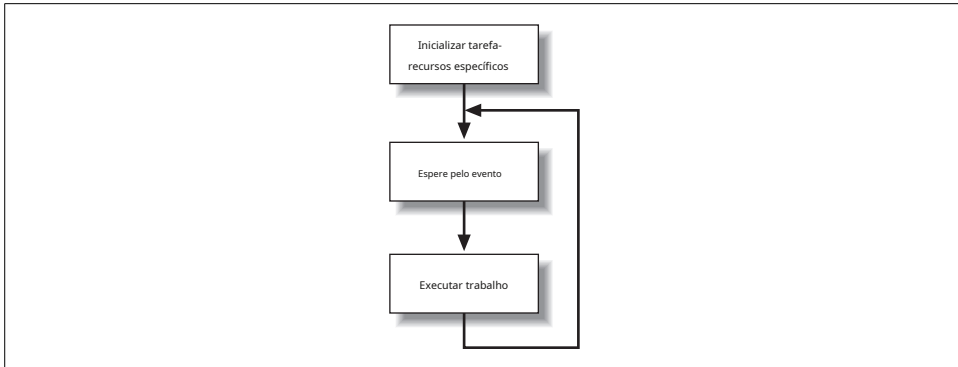


Figura 10-7. Operação de tarefa básica

de um cronômetro definido pelo aplicativo. Quando um desses eventos ocorre, a tarefa está no estado pronto e o planejador pode executar a tarefa quando o tempo do processador estiver disponível.

O código para esta tarefa básica (usando funções imaginárias) seria mais ou menos assim:

```
void taskBasicExample (void) {  
  
    /* Inicializar todos os recursos específicos da tarefa.  
    * / taskBasicInit ();  
  
    enquanto (1)  
    {  
        /* Espere o evento acontecer. * /  
        os_wait_for_event (event_type);  
  
        /* Execute o trabalho para esta tarefa. * /  
  
    }  
}
```

Sincronização de Tarefas

Embora frequentemente falemos sobre as tarefas em um sistema operacional multitarefa como entidades completamente independentes, isso não é totalmente preciso. Todas as tarefas estão trabalhando juntas para resolver um problema maior e devem ocasionalmente se comunicar umas com as outras para sincronizar suas atividades. Por exemplo, no dispositivo do servidor de impressão, a tarefa da impressora não tem nenhum trabalho a fazer até que novos dados sejam fornecidos a ela por uma das tarefas do computador. Portanto, as tarefas da impressora e do computador devem se comunicar entre si para coordenar seu acesso a buffers de dados comuns.

Os sistemas operacionais contêm vários mecanismos que ajudam na sincronização entre duas tarefas. Cada um dos mecanismos é útil para diferentes cenários. Um determinado sistema operacional pode oferecer tipos adicionais ou ligeiramente diferentes de métodos de sincronização, portanto, abordaremos aqueles comumente encontrados em sistemas operacionais de tempo real.

Interfaces de programação de aplicativos

Uma das coisas mais irritantes sobre os sistemas operacionais de tempo real é a falta de uma API comum. Este é um problema específico para empresas que desejam compartilhar o código do aplicativo entre produtos baseados em sistemas operacionais diferentes.

A funcionalidade básica de cada sistema operacional em tempo real é praticamente a mesma. Cada função representa um serviço que o sistema operacional pode executar para o programa de aplicativo. Mas não existem muitos serviços diferentes possíveis. E é frequentemente o caso que a única diferença real entre duas implementações é o nome da função.

Esse problema persistiu nas últimas décadas e não há fim à vista. Ainda durante esse mesmo tempo, as APIs Win32 e POSIX (pronuncia-se “paw-zicks”) tomaram conta de PCs e estações de trabalho Unix, respectivamente. *POSIX*, abreviação de Portable Operating System Interface (o X foi adicionado ao final para soar como uma variante do Unix), é um padrão IEEE que descreve a API de um sistema operacional de modelo de processo semelhante ao Unix.

Então, por que não surgiu um padrão semelhante para sistemas embarcados? Não foi por falta de tentativa. Na verdade, os autores do padrão POSIX original (IEEE 1003.1) também criaram um padrão para sistemas de tempo real (IEEE 1003.4b). E alguns dos sistemas operacionais de tempo real comerciais mais semelhantes ao Unix são compatíveis com esta API padrão. No entanto, para a grande maioria dos programadores de aplicativos, é necessário aprender uma nova API para cada sistema operacional.

No Capítulo 8, apresentamos o conceito de seção crítica, que é simplesmente um segmento de código que deve ser executado em sua totalidade antes que o sistema operacional tenha permissão para executar qualquer outra coisa. Frequentemente, uma seção crítica consiste em uma única instrução da linguagem C que define ou lê uma variável. Vimos alguns exemplos de interrupções no Capítulo 8; outro exemplo pode ser uma variável de status compartilhada entre uma tarefa da impressora e outras tarefas no sistema.

Lembre-se de que, em um ambiente multitarefa, você geralmente não sabe em que ordem as tarefas serão executadas no tempo de execução. Uma tarefa pode ser gravar alguns dados em um buffer de memória quando é repentinamente interrompido por uma tarefa de prioridade mais alta. Se a tarefa de prioridade mais alta modificasse essa mesma região da memória, coisas ruins poderiam acontecer. No mínimo, alguns dos dados da tarefa de prioridade mais baixa seriam substituídos. Quando um agendador round robin está em uso - como normalmente é em sistemas operacionais multitarefa - até mesmo uma tarefa com a mesma prioridade pode interromper e acessar recursos.

Uma forma de garantir que as instruções que constituem a seção crítica sejam executadas em ordem e sem interrupção é desabilitar as interrupções. No entanto, desabilitar interrupções ao usar um sistema operacional pode não ser permitido e deve ser evitado; outros mecanismos devem ser usados para executar essas operações atômicas. Exploraremos aqueles que você encontrará na maioria dos sistemas operacionais.

Mutexes e semáforos

Uma forma de sincronização é um *mutex* (abreviatura de *mut*ual *ex*clusão). Um mutex garante acesso exclusivo a variáveis ou hardware compartilhados. Um mutex é análogo a uma chave de banheiro em uma parada de descanso de alto tráfego. Apenas uma pessoa (tarefa) pode usar o banheiro (recurso compartilhado) por vez. Essa pessoa (tarefa) deve primeiro adquirir a chave do banheiro (mutex), da qual há apenas uma cópia. O lojista, que possui a chave, é análogo ao sistema operacional, que possui o mutex.

Você pode pensar em um mutex como nada mais do que um sinalizador binário com reconhecimento de multitarefa. O significado associado a um mutex particular deve, portanto, ser escolhido pelo designer de software e compreendido por cada uma das tarefas que o utilizam. Por exemplo, o buffer de dados compartilhado pela impressora e pela tarefa do computador no servidor de impressão teria um mutex associado a ele. Quando este sinalizador binário é definido, presume-se que uma das tarefas está usando o buffer de dados compartilhados. Todas as outras tarefas devem esperar até que o sinalizador seja limpo (e então até que eles mesmos o configurem novamente) antes de ler ou gravar qualquer dado dentro daquele buffer. Uma tarefa deve aguardar e adquirir um mutex antes de liberar o mutex.

Dizemos que os mutexes são multitarefa porque os processos de configuração e limpeza do sinalizador binário são atômicos. Uma tarefa pode alterar com segurança o estado do mutex sem correr o risco de ocorrer uma troca de contexto no meio da modificação. Se ocorrer uma troca de contexto, o sinalizador binário pode ser deixado em um estado imprevisível e pode ocorrer um conflito entre as tarefas. A atomicidade das operações mutex set e clear é imposta pelo sistema operacional, que desativa interrupções antes de ler ou modificar o estado do sinalizador binário.

Mutexes são usados para a proteção de recursos compartilhados entre tarefas em um sistema operacional. Recursos compartilhados são variáveis globais, buffers de memória ou registros de dispositivo que são acessados por várias tarefas. Um mutex pode ser usado para limitar o acesso a tal recurso a uma tarefa por vez.

No código do sistema operacional, as interrupções podem ser desabilitadas durante a seção crítica. Mas, geralmente, as tarefas não devem desativar as interrupções. Se eles tivessem permissão para fazer isso, outras tarefas - até mesmo tarefas de prioridade mais alta que não compartilhavam o mesmo recurso - não poderiam ser executadas durante esse intervalo. Mutexes fornecem um mecanismo para proteger seções críticas dentro de tarefas sem desativar interrupções.

Outro mecanismo de sincronização é chamado de *semáforo*. Um semáforo é usado para sincronização entre tarefas e é semelhante ao mutex. No entanto, o valor de um semáforo pode ser qualquer valor inteiro não negativo.

Voltemos à analogia do banheiro. Imagine agora que existem dois banheiros (recursos compartilhados) e duas chaves (representadas pelo semáforo). Nós queremos permitir duas pessoas (tarefas), mas não mais do que duas, para usar simultaneamente os banheiros. Se um banheiro estiver disponível (o valor do semáforo não é zero), a pessoa pode adquirir a chave e usar o banheiro. Se todas as chaves estão sendo usadas (o valor do semáforo é zero), a

Se um
chave
a

a próxima pessoa (tarefa) que chegar deve esperar até que uma chave esteja disponível. Cada vez que um semáforo é sinalizado, seu valor é incrementado. Quando um semáforo é adquirido (após a tarefa ter esperado por ele), seu valor é diminuído.

Os semáforos são geralmente usados como mecanismos de sincronização, com uma sinalização de tarefa e outra esperando. Em nosso exemplo do servidor de impressão, um semáforo pode ser usado para sinalizar para uma tarefa que os dados de entrada estão presentes. Conforme os dados chegam ao servidor de impressão, eles são armazenados em buffer. Depois que uma certa quantidade de dados é acumulada, um semáforo pode ser usado para sinalizar à tarefa do computador que é hora de processar esses dados. Este exemplo é semelhante a uma corrida de revezamento. Nesse caso, o bastão é o semáforo. Apenas o corredor (tarefa) com o bastão é capaz de correr. A tarefa do computador que espera pelos dados recebidos é como o corredor que espera pelo bastão. Uma vez que o bastão é passado, o próximo corredor (tarefa de computador) pode prosseguir.

Mutexes deve ser usado exclusivamente para controlar o acesso a recursos compartilhados. Enquanto os semáforos são normalmente usados como dispositivos de sinalização. Um semáforo pode ser usado para sinalizar uma tarefa de outra tarefa ou de um ISR - por exemplo, para sincronizar atividades.

Impasse e inversão de prioridade

Mutexes são ferramentas poderosas para sincronizar o acesso a recursos compartilhados. No entanto, eles têm seus próprios perigos. Dois dos problemas mais importantes a serem observados são deadlock e inversão de prioridade.

Impasse pode ocorrer sempre que houver uma dependência circular entre tarefas e recursos. O exemplo mais simples é o de duas tarefas: 1 e 2. Cada tarefa requer dois mutexes: A e B. Se a Tarefa 1 pegar o mutex A e esperar pelo mutex B enquanto a Tarefa 2 pegar o mutex B e esperar pelo mutex A, então cada tarefa é esperando que o outro libere o mutex. Aqui está um exemplo de código que demonstra esse problema de deadlock:

```
void task1 (void)
{
    os_wait_for_mutex (mutexA);
    os_wait_for_mutex (mutexB);

    /* Outro trabalho da tarefa1. */
}

void task2 (void)
{
    os_wait_for_mutex (mutexB);
    os_wait_for_mutex (mutexA);

    /* Outro trabalho de task2. */
}
```

Essas tarefas podem ser executadas sem problemas por um longo tempo, mas, eventualmente, uma tarefa pode ser interrompida entre as chamadas de espera e a outra tarefa será executada. Nesse caso, a Tarefa 1 precisa do mutex B para ser liberado pela Tarefa 2, enquanto a Tarefa 2 precisa do mutex A para ser liberado pela Tarefa 1. Nenhum desses eventos acontecerá.

Quando ocorre um deadlock, ele basicamente interrompe as duas tarefas, embora outras tarefas possam continuar em execução. A única maneira de acabar com o impasse é reinicializar o sistema inteiro, e mesmo isso não impedirá que aconteça novamente.

Inversão de prioridade ocorre sempre que uma tarefa de prioridade mais alta é bloqueada, aguardando o acesso a um recurso compartilhado que não está sendo usado no momento. Isso pode não parecer um grande negócio - afinal, o mutex está apenas fazendo seu trabalho de arbitrar o acesso ao recurso compartilhado - porque a tarefa de maior prioridade é escrita com o conhecimento de que às vezes a tarefa de menor prioridade será usando o recurso que compartilham. No entanto, considere o que acontece quando há uma terceira tarefa com um nível de prioridade em algum lugar entre as duas. Essa situação é ilustrada na Figura 10-8.

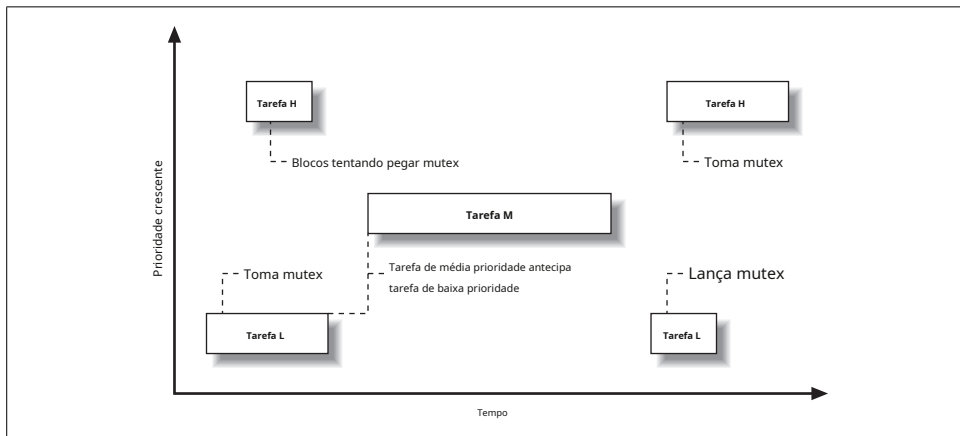


Figura 10-8. Um exemplo de inversão de prioridade

Na Figura 10-8, há três tarefas: Tarefa H (alta prioridade), Tarefa M (prioridade média) e Tarefa L (prioridade baixa). A tarefa L fica pronta primeiro e, logo em seguida, pega o mutex. Agora, quando a Tarefa H fica pronta, ela deve ser bloqueada até que a Tarefa L seja concluída com seu recurso compartilhado. O problema é que a Tarefa M, que nem mesmo requer acesso a esse recurso, consegue antecipar a Tarefa L e executar, embora isso atrase a Tarefa H de usar o processador. Depois que a Tarefa M é concluída, a Tarefa L é executada até liberar o semáforo. Finalmente, neste ponto, a Tarefa H tem sua chance de ser executada. Este exemplo mostra como as prioridades da tarefa podem ser violadas devido ao compartilhamento mutex entre a Tarefa H e a Tarefa L.

Várias soluções para este problema foram desenvolvidas. Uma das soluções mais utilizadas é chamada *herança de prioridade*. Essa técnica exige que uma tarefa de prioridade mais baixa herde a prioridade de qualquer tarefa de prioridade mais alta que esteja aguardando um recurso compartilhado por ela. Essa mudança de prioridade deve ocorrer assim que a tarefa de prioridade mais alta começa a esperar; deve terminar quando o recurso for liberado. Isso requer ajuda do sistema operacional. Se aplicarmos isso ao exemplo anterior, a prioridade da Tarefa L é aumentada para a Tarefa H assim que a Tarefa H começa a esperar pelo mutex.

Depois que a Tarefa L libera o mutex, sua prioridade é definida para o que era antes. A Tarefa L não pode ser interrompida pela Tarefa M até que ela libere o mutex, e a Tarefa H não pode ser atrasada desnecessariamente.

Outra solução é chamada *tetos prioritários*. Nesse caso, um valor de prioridade é associado a cada recurso; o planejador então transfere essa prioridade para qualquer tarefa que acesse o recurso. A prioridade atribuída ao recurso é a prioridade de seu usuário de maior prioridade, mais um. Assim que uma tarefa termina com o recurso, sua prioridade retorna ao normal. Uma desvantagem de usar tetos de prioridade é que o nível de prioridade para tarefas que usam o mutex deve ser conhecido com antecedência para que o valor de teto adequado possa ser definido. Isso significa que o sistema operacional não pode fazer o trabalho automaticamente para você. Outra desvantagem é que, se o valor do teto for definido muito alto, outras tarefas não relacionadas com níveis de prioridade abaixo do teto podem ser impedidas de executar. O teto de prioridade é utilizado mesmo quando não ocorre inversão de prioridade, como medida profilática.

A necessidade de mecanismos de sincronização precisa ser pensada durante o design do software. Uma vez desenvolvido no software, todos devem usar o mutex adequadamente para acessar o recurso compartilhado. Se alguém violar a regra, o software nem sempre funcionará conforme projetado. Em grandes projetos com vários programadores, pode ser difícil para diferentes programadores perceber que um recurso é compartilhado.

Passagem de mensagens

Embora os semáforos possam ser usados para sinalizar de uma tarefa para outra, pode haver momentos em que os dados precisam ser passados além do sinal. Para este propósito, os sistemas operacionais freqüentemente fornecem outro mecanismo chamado de *fila de mensagens* (ou *caixa de correio*).

O sistema operacional lida com o armazenamento em buffer e o gerenciamento de buffer para a passagem de mensagens, bem como a comunicação segura de dados entre as tarefas. A passagem de mensagens é, portanto, uma alternativa ao expediente simples de armazenar dados em uma variável global e oferece um método de troca de dados muito mais limpo e livre de erros. Os sistemas operacionais de tempo real normalmente usam ponteiros para acessar os dados da mensagem por razões de velocidade e conservação de memória.

Muitos aplicativos que usam filas de mensagens consistem em uma tarefa do produtor que envia os dados e uma tarefa do consumidor que os recebe. Pode haver várias tarefas de produtor e / ou consumidor. O conteúdo da mensagem é normalmente entendido entre o remetente e o destinatário com antecedência.

Se a fila de mensagens estiver cheia, o sistema operacional pode bloquear a tarefa de envio até que haja espaço disponível para a mensagem. Da mesma forma, se nenhuma mensagem estiver presente quando o receptor tentar ler a mensagem, o sistema operacional bloqueará a tarefa de recebimento. O tamanho da fila pode variar dependendo do tráfego de mensagens.

Outra Funcionalidade

Já cobrimos os mecanismos básicos comumente encontrados na maioria dos sistemas operacionais de tempo real. Vários outros recursos podem ser incluídos, dependendo do sistema operacional, para realizar várias outras operações úteis. Alguns dos outros mecanismos comumente encontrados em sistemas operacionais de tempo real são:

Bandeiras de eventos

Isso permite que uma tarefa aguarde a ocorrência de vários eventos antes de ser desbloqueada. Todos os eventos devem ocorrer (os eventos são colocados em AND juntos) ou qualquer um dos eventos pode ocorrer (os eventos são colocados em OR juntos).

Variáveis de condição

Eles são semelhantes a um semáforo de contagem em que uma tarefa sinaliza para outra tarefa despertar; entretanto, ao contrário de um semáforo, se nenhuma tarefa estiver aguardando a variável de condição quando ela for sinalizada, o sinal será perdido.

Spinlocks

Eles são semelhantes a um mutex e normalmente são usados em sistemas de multiprocessamento simétrico (SMP). Como um mutex, um spinlock é um sinalizador binário que uma tarefa tenta reivindicar. Se o sinalizador não for definido, a tarefa será capaz de obter o spinlock. Se o sinalizador estiver definido, a tarefa girará em um loop, verificando constantemente quando o sinalizador não está definido. Isso pode parecer um desperdício (e pode ser); entretanto, presume-se que o spinlock é mantido apenas por um período muito curto de tempo, e esta CPU deve esperar que o software rodando na outra CPU progrida primeiro de qualquer maneira.

Contadores e alarmes

Um contador controla o número de vezes que um evento específico ocorreu. Um alarme é usado em conjunto com um contador para despertar uma tarefa (para que ela execute uma ação) quando um determinado número de eventos ocorrer.

Manipulação de interrupção

Existem vários problemas que você precisa estar ciente ao lidar com interrupções em sistemas incorporados que usam um sistema operacional, incluindo:

Prioridade de interrupção

As interrupções têm a prioridade mais alta em um sistema - ainda mais alta do que a tarefa do sistema operacional mais alta. As interrupções não são programadas; o ISR é executado fora do programador do sistema operacional.

Desativando interrupções

Como o código do sistema operacional deve garantir a integridade de suas estruturas de dados quando elas são acessadas, o sistema operacional desabilita as interrupções durante as operações que alteram os dados internos do sistema operacional, como a lista pronta. Isso aumenta a latência de interrupção. A capacidade de resposta do sistema operacional tem o preço de uma latência de interrupção mais longa.

Quando uma tarefa desativa interrupções, ela impede que o planejador faça seu trabalho. As tarefas não devem desabilitar interrupções por conta própria.

Pilha de interrupção

Alguns sistemas operacionais têm um espaço de pilha separado para a execução de ISRs. Isso é importante porque, se as interrupções forem armazenadas na mesma pilha que as tarefas regulares, a pilha de cada tarefa deve acomodar o pior cenário de aninhamento de interrupções. Essas pilhas grandes aumentam os requisitos de RAM em todos n tarefas.

Tarefas de sinalização

Como os ISRs são executados fora do planejador, eles não têm permissão para fazer chamadas de sistema operacional que possam ser bloqueadas. Por exemplo, um ISR não pode esperar por um semáforo, embora possa sinalizá-lo.

Alguns sistemas operacionais usam um esquema de tratamento de interrupção dividido, onde o processamento de interrupção é dividido em duas partes. A primeira parte é um ISR que lida com o processamento mínimo da interrupção. A ideia é manter o ISR curto e rápido.

A segunda parte é tratada por um Representante de Vendas. O DSR lida com o processamento mais extenso do evento de interrupção. Ele é executado quando o agendamento de tarefas é permitido; no entanto, o DSR ainda tem uma prioridade mais alta do que qualquer tarefa no sistema. O DSR é capaz de sinalizar uma tarefa para realizar o trabalho disparado pelo evento de interrupção.

Por exemplo, no dispositivo do servidor de impressão, uma interrupção pode ser usada para controlar os dados de entrada dos computadores na rede Ethernet. O controlador Ethernet interromperia o processador quando um pacote fosse recebido. Usando o esquema de tratamento de interrupção de divisão, o ISR lidaria com o trabalho inicial mínimo: determinar o evento de interrupção, mascarar outras interrupções Ethernet e reconhecer a interrupção. O ISR então diria ao sistema operacional para executar o DSR, que então trataria do processamento do pacote de dados de baixo nível antes de passar os dados para uma tarefa para processamento posterior.

Características em tempo real

Os engenheiros costumam usar o termo *tempo real* para descrever problemas de computação para os quais uma resposta tardia é tão ruim quanto uma errada. Diz-se que esses problemas têm *prazos*, e os sistemas embarcados freqüentemente operam sob tais restrições. Por exemplo, se o software embutido que controla seus freios antitravamento perder um de seus prazos, você pode se envolver em um acidente. Portanto, é extremamente importante que os projetistas de sistemas embarcados de tempo real saibam tudo o que puderem sobre o comportamento e o desempenho de seu hardware e software. Nesta seção, discutiremos as características de desempenho dos sistemas operacionais de tempo real.

Os projetistas de sistemas de tempo real passam grande parte do tempo se preocupando com o desempenho do pior caso. Eles devem constantemente se fazer perguntas como: qual é o pior caso de tempo entre o momento em que um operador humano pressiona o pedal do freio e o momento em que um sinal de interrupção chega ao processador?

Qual é o pior caso de latência de interrupção? E qual é o pior caso de tempo para o software responder acionando o mecanismo de frenagem? A análise da média ou do caso esperado simplesmente não será suficiente em tais sistemas.

A maioria dos sistemas operacionais de tempo real disponíveis hoje são projetados para possível inclusão em sistemas de tempo real. Idealmente, seu desempenho no pior caso é bem compreendido e documentado. Para ganhar o título distintivo de “sistema operacional em tempo real”, um sistema operacional deve ser determinístico e ter latência de interrupção de pior caso garantida e tempos de troca de contexto. Dadas essas características e as prioridades relativas das tarefas e interrupções em seu sistema, é possível analisar o pior caso de desempenho do software.

Diz-se que um sistema operacional é *determinista* se o pior caso de tempo de execução de cada uma das chamadas do sistema for calculável. Os projetistas de sistemas operacionais que levam a sério o comportamento em tempo real geralmente publicam uma planilha de dados que fornece o número mínimo, médio e máximo de ciclos de clock exigidos por cada chamada do sistema. Esses números geralmente são diferentes para diferentes processadores. Portanto, é importante ao comparar esses números que hardware equivalente (ou melhor ainda, o mesmo) seja usado. Mas é razoável esperar que, se o algoritmo for determinístico em um processador, o seja em qualquer outro. Os tempos reais podem ser diferentes.

A latência de interrupção é a chave para determinar a capacidade de resposta de um RTOS. Um RTOS aumenta a latência de interrupção porque ele deve fazer algum processamento assim que ocorrer uma interrupção. A quantidade de tempo que esse processamento leva é uma característica importante do RTOS para um sistema de tempo real. Quando ocorre uma interrupção, o processador deve realizar várias etapas antes de executar o ISR. Primeiro, o processador deve terminar de executar a instrução atual. Isso provavelmente leva menos de um ciclo de clock, mas algumas instruções complexas requerem mais tempo do que isso. Em seguida, o tipo de interrupção deve ser reconhecido. Isso é feito pelo hardware do processador e não desacelera ou suspende a tarefa em execução. Em seguida, o RTOS deve processar a interrupção e determinar qual ISR é chamado. Finalmente, e somente se as interrupções estiverem habilitadas, o ISR associado à interrupção é iniciado.

Obviamente, se as interrupções forem desativadas no sistema operacional, a latência de interrupção do pior caso aumenta pelo tempo máximo em que são desativadas. Mas os sistemas operacionais têm certos lugares onde as interrupções devem ser desabilitadas. Essas são as seções internas críticas - relacionadas às estruturas do sistema operacional - descritas anteriormente; não há métodos alternativos para o sistema operacional protegê-los. Cada sistema operacional desabilitará as interrupções por um período de tempo diferente, portanto, é importante que você saiba quais são os requisitos do seu sistema. Um projeto em tempo real pode exigir um tempo de resposta de interrupção garantido tão curto quanto 10 μ s, enquanto outro requer apenas 100 ms.

A terceira característica em tempo real de um sistema operacional é a quantidade de tempo necessária para realizar uma troca de contexto. Isso é importante porque representa a sobrecarga em todo o sistema. Por exemplo, imagine que a execução média

O tempo de qualquer tarefa antes do bloqueio é de 100 ms, mas o tempo de mudança de contexto também é de 100 ms. Nesse caso, metade do tempo do processador é gasto na rotina de troca de contexto! Novamente, os tempos reais são geralmente específicos do processador porque dependem do número de registros que devem ser salvos. Certifique-se de obter esses números para qualquer sistema operacional que estiver pensando em usar. Dessa forma, não haverá surpresas de última hora.

Usar ou não usar um RTOS

A resposta é ... depende. Em muitos casos, não há uma resposta clara para essa pergunta. Muitos sistemas embarcados podem (e funcionam) exatamente como precisam, usando um loop infinito, como discutimos no Capítulo 3. Esses sistemas embarcados não precisam ser complicados pela adição de software adicional, como um RTOS. Não há prêmio por tornar um sistema embarcado mais complicado.

Cada projeto deve ser avaliado individualmente. Comece com a noção de que você não precisa de um RTOS. Em seguida, dê uma olhada nos requisitos gerais do sistema. Faça uma lista dos diferentes módulos de software de que o sistema precisará para atender a esses requisitos.

Vamos voltar ao exemplo do dispositivo de servidor de impressão. O diagrama de fluxo de dados no Capítulo 2 é um bom ponto de partida. Alguns dos módulos necessários para o servidor de impressão são um subsistema de interrupção para lidar com interrupções de temporizador e periféricas, um manipulador para a porta paralela para se comunicar e enviar dados para a impressora, uma pilha de rede para comunicação com computadores através do controlador Ethernet, um módulo de depuração que usa uma porta serial para saída (isso não é necessário, mas é útil) e, possivelmente, um monitor e uma interface de linha de comando de controle.

Agora você pode fazer algumas perguntas sobre esses módulos no sistema para descobrir as responsabilidades de cada um. Pode ajudar desenhar isso em seu caderno de projeto. Como esses módulos irão interagir uns com os outros? Os módulos são independentes e autônomos ou têm interdependências? Eles precisarão compartilhar memória ou outros recursos de hardware?

No exemplo do servidor de impressão, uma pilha de rede é necessária. Isso pode não ser algo que você gostaria de criar do zero. Várias pilhas de rede estão disponíveis e muitos RTOSs também as incluem.

Pode não haver respostas fáceis para algumas dessas perguntas. Eles não se baseiam apenas em questões técnicas. Na ausência de um vencedor inequívoco, provavelmente é melhor errar sobre o que torna o software mais fácil de ler e implementar. Fazer uma lista de prós e contras pode ajudar no processo de decisão.

Agora, se sua decisão for usar um RTOS, vá para a próxima seção para uma discussão de alguns critérios para determinar qual RTOS é o melhor para o projeto.

Processo de Seleção RTOS

A edição anterior deste livro mostrou como construir seu próprio RTOS. Apesar disso, é altamente recomendável usar um sistema operacional existente. Vamos dizer de novo: é altamente recomendável usar um sistema operacional pronto para uso em vez de escrever o seu próprio. Uma ampla variedade de sistemas operacionais está disponível para atender quase todos os projetos e carteiras. Usar um sistema operacional pronto para uso permite que sua equipe de software se concentre no desenvolvimento do aplicativo para o produto. Concedido, haverá uma curva de aprendizado para acelerar o uso do sistema operacional.

Nesta seção, discutiremos o processo de seleção do sistema operacional que melhor se adapta às necessidades do seu projeto. Selecionar um RTOS pode ser complicado. Existem muitos critérios a serem considerados ao tomar essa decisão. Claro, os critérios são normalmente ponderados de forma diferente de projeto para projeto e de empresa para empresa.

Vamos dar uma olhada em alguns dos critérios importantes usados para fazer uma seleção RTOS:

Suporte para processador

O processador é normalmente a primeira escolha no design de hardware de um projeto. A maioria dos RTOSs suporta os processadores populares (ou pelo menos famílias de processadores) usados em sistemas embarcados. Se o processador usado em seu projeto não for compatível, você precisa determinar se portar o RTOS para aquele processador é uma opção ou se é necessário escolher um RTOS diferente. Portar um RTOS nem sempre é trivial.

Características em tempo real

Já cobrimos as características de tempo real de um RTOS, que incluem latência de interrupção, tempo de mudança de contexto e tempo de execução de cada chamada do sistema. São critérios técnicos inerentes ao sistema e não podem ser alterados.

Restrições de orçamento

Os RTOSes abrangem o espectro de custos de código aberto e livre de royalties a dezenas de milhares de dólares por licença de desenvolvedor mais royalties para cada unidade enviada. Você precisa entender quais são seus custos em ambos os casos. O código aberto pode significar nenhum custo inicial, mas pode haver custos associados à obtenção de suporte quando necessário. Você também precisa entender os detalhes de licenciamento do RTOS que você escolher.

Uso de memória

Claramente, em um ambiente integrado, as restrições de memória são uma preocupação frequente. Alguns RTOSs podem ser dimensionados para caber no menor dos sistemas embarcados - por exemplo, removendo recursos para criar uma pegada menor. Outros requerem um conjunto mínimo de recursos comparáveis a um PC low-end. É importante ter em mente a necessidade potencial de alterar um RTOS no futuro, quando a memória não for tão abundante ou os custos precisarem ser reduzidos.

Drivers de dispositivo e componentes de software

Os drivers de dispositivo incluídos com um RTOS podem ajudar a manter o desenvolvimento dentro do cronograma. Isso reduz a quantidade de código que você precisa desenvolver para determinados periféricos. Muitos RTOSs suportam os dispositivos comuns encontrados em sistemas embarcados.

Se recursos adicionais forem necessários, como suporte de rede, bibliotecas gráficas, interfaces da web e sistemas de arquivos, um RTOS pode incluí-los e ter o código já integrado e testado. Alguns RTOSs podem exigir mais taxas para usar esses recursos adicionais. Se os recursos necessários não estiverem incluídos, você precisará identificar terceiros que os fornecem para que esses componentes possam ser integrados ao sistema.

Suporte técnico

Isso pode incluir vários incidentes ou um período de suporte por telefone. Alguns RTOS exigem que você pague uma taxa anual para manter um contrato de serviço. Para RTOSs de código aberto, um fórum aberto ou lista de discussão pode ser fornecida. Se for necessário suporte mais especializado, você terá que pesquisar para ver o que está disponível. Os RTOSs de código aberto populares têm empresas dedicadas a fornecer suporte.

Compatibilidade de ferramentas

Certifique-se de que o RTOS funcione com o montador, compilador, vinculador e depurador que você já obteve. Se o RTOS não oferece suporte a ferramentas com as quais você ou sua equipe estão familiarizados, a curva de aprendizado levará mais tempo.

Não importa qual RTOS você escolha, nosso conselho é obter o código-fonte, se possível. A razão para isso é que se você não conseguir suporte quando precisar (digamos, à 1h para um prazo final às 8h ou se o fornecedor do sistema operacional parar de oferecer suporte ao produto), você ficará feliz em ser capaz de encontrar e resolver o problema sozinho. Alguns RTOSs proprietários fornecem apenas código-objeto. Descubra o que é fornecido antes de tomar sua decisão final.

Com uma variedade tão grande de sistemas operacionais e recursos para escolher, pode ser difícil decidir qual é o melhor para o seu projeto. Experimente colocar o processador, o desempenho em tempo real e os requisitos orçamentários em primeiro lugar. Esses são critérios que você provavelmente não pode alterar, portanto, pode usá-los para restringir as opções possíveis a uma dúzia ou menos de produtos. Em seguida, você pode se concentrar nas informações técnicas mais detalhadas.

Neste ponto, muitos desenvolvedores tomam suas decisões com base na compatibilidade com compiladores, depuradores e outras ferramentas de desenvolvimento existentes. Mas é você quem decide quais recursos adicionais são mais importantes para o seu projeto. Não importa o que você decida, o kernel básico e os mecanismos de tarefa serão praticamente os mesmos que os descritos neste capítulo. As diferenças provavelmente serão medidas nos processadores suportados, requisitos mínimos e máximos de memória, disponibilidade de módulos de software adicionais (pilhas de protocolo de rede, drivers de dispositivo e sistemas de arquivos são exemplos comuns) e compatibilidade com ferramentas de desenvolvimento de terceiros.

Recursos adicionais

Outro bom conjunto de critérios para seleção de RTOS pode ser encontrado em março de 1999 *Programação de sistemas embarcados* artigo “Seleção de um sistema operacional em tempo real”, que pode ser encontrado online em <http://www.embedded.com>. A lista de fornecedores pode estar um pouco desatualizada, mas as informações ainda são muito úteis.

Se você quiser se aprofundar no funcionamento interno dos sistemas operacionais de tempo real, aqui estão dois recursos que sugerimos: *MicroC / OS-II: o kernel em tempo real*, por Jean J. Labrosse (CMP Books) e *Conceitos em tempo real para sistemas incorporados*, por Qing Li e Caroline Yao (CMP Books).

Exemplos eCos

Henry Hill: *Você é uma pistola, você é muito engraçado.*

Você é muito engraçado.

Tommy DeVito: *O que quer dizer com sou engraçado?*

Henry Hill: *É engraçado, você sabe. É uma boa história, é engraçada, você é um cara engraçado.*

Tommy DeVito: *O que você quer dizer com o jeito que eu falo? O que?*

Henry Hill: *É só, você sabe. Você é simplesmente engraçado, é ... engraçado, a maneira como você conta a história e tudo mais.*

Tommy DeVito: *Engraçado como? O que há de engraçado nisso?*

- o filme *Bons companheiros*

Neste capítulo, veremos alguns exemplos de código de sistema embarcado que usa os mecanismos do sistema operacional que abordamos no capítulo anterior, sob o princípio de que ver implementações reais contribui para um melhor entendimento dos mecanismos. Este capítulo usa o sistema operacional de tempo real eCos para os exemplos. Os conceitos e técnicas neste capítulo se aplicam a outros RTOSs também, mas diferentes sistemas operacionais usam APIs diferentes para realizar as técnicas. No próximo capítulo, examinaremos os mesmos exemplos usando o popular sistema operacional Linux.

Introdução

Decidimos usar dois sistemas operacionais para os exemplos de uso de sistema operacional - eCos e Linux. Por que escolhemos esses dois? Ambos são de código aberto, livres de royalties, ricos em recursos e cada vez mais populares. Aprender a programar usando-os provavelmente aumentará sua capacidade de ser produtivo com sistemas embarcados. Além disso, ambos os sistemas operacionais são compatíveis com as ferramentas gratuitas de desenvolvimento de software GNU. E ambos estão funcionando na placa da Arcom.

O eCos foi desenvolvido especificamente para uso em sistemas embarcados de tempo real, enquanto o Linux foi desenvolvido para uso em PCs e, posteriormente, transportado para vários processadores usados em sistemas embarcados. Algumas distribuições de Linux embarcado podem exigir um

quantidade significativa de recursos (principalmente memória e poder de processamento), que não são encontrados na maioria dos sistemas embarcados. O Linux não era originalmente em tempo real, mas as extensões agora estão disponíveis para adicionar recursos em tempo real.

Resumindo, eCos é mais adequado em geral para o trabalho de sistemas embarcados, mas o peso do conhecimento disseminado e das ferramentas de suporte em torno do Linux o tornam atraente também para desenvolvedores embarcados.

Tentamos manter os nomes das funções, e o que essas funções fazem, consistentes entre os diferentes exemplos neste capítulo e no seguinte. Desta forma, você pode se concentrar nos detalhes relacionados aos sistemas operacionais. eCos inclui uma API POSIX, que oferece suporte a um subconjunto de funções POSIX. * No entanto, nos exemplos de eCos a seguir, a API eCos nativa é usada.

As instruções para configurar o ambiente de construção eCos e construir os aplicativos eCos de exemplo são abordadas no Apêndice D. Informações adicionais sobre eCos podem ser encontradas online em <http://ecos.sourceware.org> bem como no livro *Desenvolvimento de software integrado com eCos*, por Anthony Massa (Prentice Hall PTR).



Para manter os exemplos neste capítulo mais curtos e fáceis de ler, não nos preocupamos em verificar os valores de retorno das chamadas de função do sistema operacional (embora muitas chamadas do sistema eCos não retornem um valor). Em geral, é uma boa ideia validar todos os códigos de retorno. Isso fornece feedback sobre problemas em potencial e permite que você, como desenvolvedor, tome decisões no software com base em chamadas com falha para o sistema operacional. Basicamente, isso torna seu código mais robusto e, com sorte, menos cheio de erros.

Mecânica da Tarefa

Neste primeiro exemplo de eCos, reutilizamos o programa de LED piscando que foi abordado anteriormente. A primeira coisa a aprender é como criar uma tarefa. Este exemplo cria uma tarefa para lidar com a alternância do LED em uma taxa constante. Primeiro, declaramos as variáveis específicas da tarefa, como a pilha e seu tamanho, a prioridade da tarefa e as variáveis específicas do sistema operacional.



eCos usa o termo *fio* ao invés de *tarefa* em sua API e tipos de variáveis. Esses termos significam a mesma coisa no contexto de sistemas embarcados.

O programa de exemplo fornece duas variáveis ao eCos para permitir que ele rastreie a tarefa. A variável `ledTaskObj` armazena informações sobre a tarefa, como seu estado atual; `ledTaskHdl` é um valor exclusivo atribuído à tarefa.

* A API eCos POSIX é atualmente compatível com a versão 1003.1-1996 do padrão e inclui elementos da versão 1004.1-2001.

A pilha para nossa tarefa é declarada estaticamente como a matriz `ledTaskStack`, que tem 4.096 bytes de tamanho. Uma prioridade arbitrária de 12 é atribuída para esta tarefa. O código também define o número de tiques do relógio por segundo, um valor específico para a configuração eCos da placa Arcom. Isso torna mais fácil alterar o intervalo de tique para ajustar o desempenho do sistema.

```
# define TICKS_PER_SECOND          (100)

# define LED_TASK_STACK_SIZE      (4096)
# define LED_TASK_PRIORITY        (12)

/* Declare as variáveis de tarefa. */
unsigned char ledTaskStack [LED_TASK_STACK_SIZE];
cyg_thread ledTaskObj;
cyg_handle_t ledTaskHdl;
```

A seguir, mostramos o código para realizar a alternância. Tentamos reutilizar o código do exemplo original de LED piscando. o `ledInit` e `ledToggle` As funções do driver de LED permanecem inalteradas em relação ao código descrito no Capítulo 3.

A tarefa `blinkLedTask` entra imediatamente em um loop infinito. O loop infinito é usado para manter a tarefa em execução contínua e o LED piscando. A primeira rotina chamada no loop infinito é `cyg_thread_delay`. Esta é uma função eCos que suspende uma tarefa até que um determinado número de tiques do relógio tenha decorrido. O parâmetro passado na rotina de atraso determina por quanto tempo suspender a tarefa e é baseado no relógio do sistema usado no eCos. Neste ponto, o `blinkLedTask` é bloqueado e colocado no estado de espera pelo agendador eCos.

Assim que o cronômetro expira, o agendador eCos coloca o `blinkLedTask` na fila de espera. Se nenhuma outra tarefa de prioridade mais alta estiver pronta para execução (que é a situação neste caso), o planejador executa o `blinkLedTask`; a tarefa continua em execução a partir do ponto em que foi bloqueada.

Próximo, `ledToggle` é chamado para alterar o estado do LED. Quando `ledToggle` completa e retorna, `cyg_thread_delay` é chamado para atrasar por mais 500 ms. o `blinkLedTask` é colocado de volta no estado de espera até que o tempo passe novamente.

```
# inclui <cyg / kernel / kapi.h>
# inclui "led.h"

/ *****
*
* Função:      blinkLedTask
*
* Descrição: Esta tarefa lida com alternar o LED verde em um
*             intervalo constante.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
```



```

void blinkLedTask (cyg_addrword_t data) {

    enquanto (1)
    {
        /* Atraso de 500 milissegundos. */
        cyg_thread_delay (TICKS_PER_SECOND / 2);

        ledToggle ();
    }
}

```

A seguir está o código para criar e iniciar a tarefa de LED. A primeira coisa a notar é que em vez da função a Principal, Os programas eCos usam uma função chamada `cyg_user_start`.

O primeiro trabalho de `cyg_user_start` é inicializar o LED chamando a função `ledInit`. A seguir, o `blinkLedTask` tarefa é criada. No eCos, as tarefas criadas durante a inicialização (quando o planejador não está em execução) são inicialmente suspensas. Para permitir que o planejador execute a tarefa, `cyg_thread_resume` é chamado. Além disso, o programador não corre até `cyg_user_start` saídas; em seguida, o agendador eCos assume o controle.

```

/ *****
*
* Função:      cyg_user_start
*
* Descrição: Rotina principal para o programa eCos Piscando LED. Esse
*             função cria a tarefa LED.
*
* Notas:      Esta rotina chama o planejador ao sair.
*
* Retorna:    Nenhum.
*
***** /
void cyg_user_start (void) {

    /* Configure o pino de controle do LED verde. */
    ledInit ();

    /* Cria a tarefa LED. */
    cyg_thread_create (LED_TASK_PRIORITY,
                      blinkLedTask,
                      (cyg_addrword_t) 0,
                      "Tarefa LED",
                      (vazio *) ledTaskStack,
                      LED_TASK_STACK_SIZE,
                      & ledTaskHdl,
                      & ledTaskObj);

    /* Notifica o planejador para iniciar a execução da tarefa. */
    cyg_thread_resume (ledTaskHdl);
}

```

O exemplo anterior demonstra como criar, retomar e atrasar uma tarefa no eCos. Outras operações de tarefa incluem a exclusão de tarefas (que às vezes pode ocorrer retornando da função de tarefa), cedendo a outras tarefas no sistema e outros mecanismos para suspender / retomar tarefas.

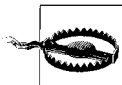
Sincronização Mutex de Tarefas

Agora atacamos a próxima grande tarefa no gerenciamento de tarefas, que é sincronizar tarefas. Como vimos no Capítulo 10, um mutex é um mecanismo comum para fazer com que duas tarefas independentes cooperem. Para nosso exemplo mutex eCos, duas tarefas compartilham uma variável comum - a primeira tarefa incrementando-a e a segunda diminuindo-a em intervalos definidos. O mutex é usado para proteger a variável compartilhada.

Antes de acessar um recurso compartilhado, uma tarefa pega o mutex; uma vez concluída, a tarefa libera o mutex para outras tarefas usarem. Cada sistema operacional define essas duas operações de sua própria maneira. Por exemplo, o eCos oferece funções de bloqueio (obtendo o mutex) e desbloqueio (liberando o mutex).

Para controlar as várias tarefas que aguardam o mesmo mutex, a estrutura do mutex contém uma fila de tarefas que aguardam esse mutex específico. Essa fila é normalmente classificada por prioridade, de modo que a tarefa de maior prioridade que aguarda o mutex é executada primeiro. Um possível resultado da liberação do mutex pode ser ativar uma tarefa de prioridade mais alta. Nesse caso, a tarefa de liberação seria imediatamente forçada (pelo escalonador) a ceder o controle do processador, em favor da tarefa de maior prioridade.

A função principal, `cyg_user_start`, que é mostrado a seguir, chama o `cyg_mutex_init` função para inicializar mutexes, como aquele que nomeamos `sharedVariableMutex`. Este mutex é usado para proteger a variável `gSharedVariable`. Depois de retornar da chamada de inicialização do mutex, o mutex estará disponível para a primeira tarefa que o fizer. Uma vez que o mutex é inicializado, as duas tarefas são criadas e então iniciadas da maneira mostrada anteriormente.



É importante observar que o mutex é inicializado antes da criação de qualquer tarefa que use esse mutex. Qualquer mecanismo de sincronização usado por uma tarefa deve ser inicializado antes de seu uso pela tarefa. Se uma tarefa tentar pegar um mutex não inicializado, pode ocorrer um comportamento indefinido ou uma falha do sistema.

Você pode notar que a função `debug_printf` é chamado no final de `cyg_user_start`. Esta é a versão leve da eCos de `printf`.

```
# inclui <cyg / kernel / kapi.h>
# inclui <cyg / infra / diag.h>

cyg_mutex_t sharedVariableMutex;
int32_t gSharedVariable = 0;
```

```

/ *****
*
* Função:      cyg_user_start
*
* Descrição: Rotina principal para o exemplo mutex eCos. Esta função
*             cria o mutex e, em seguida, o incremento e decremento
*             tarefas.
*
* Notas:      Esta rotina chama o planejador ao sair.
*
* Retorna:    Nenhum.
*
***** /
void cyg_user_start (void) {

    / * Crie o mutex para acessar a variável compartilhada. * /
    cyg_mutex_init (& sharedVariableMutex);

    / * Cria as tarefas de incremento e decremento. * /
    cyg_thread_create (INCREMENT_TASK_PRIORITY,
                       incrementTask,
                       (cyg_addrword_t) 0,
                       "Tarefa de incremento",
                       (vazio *) incrementTaskStack,
                       INCREMENT_TASK_STACK_SIZE,
                       & incrementTaskHdl,
                       & incrementTaskObj);

    cyg_thread_create (DECREMENT_TASK_PRIORITY,
                       decrementTask,
                       (cyg_addrword_t) 0,
                       "Tarefa de Decremento",
                       (void *) decrementTaskStack,
                       DECREMENT_TASK_STACK_SIZE,
                       & decrementTaskHdl,
                       & decrementTaskObj);

    / * Notifica o planejador para iniciar a execução das tarefas. * /
    cyg_thread_resume (incrementTaskHdl);
    cyg_thread_resume (decrementTaskHdl);

    diag_printf ("exemplo mutex eCos. \n");
}

```

o incrementTask função atrasa primeiro por três segundos. Após o atraso, a função tenta obter o mutex chamando cyg_mutex_lock, passando no mutex que deseja adquirir. Se o mutex estiver disponível, cyg_mutex_lock retorna e a tarefa pode prosseguir. Se o mutex não estiver disponível, a tarefa é bloqueada neste ponto (e é colocada no estado de espera pelo agendador) e espera que o mutex seja liberado.

Uma vez o incrementTask tarefa obtém o mutex, a variável compartilhada gSharedVariable é incrementado e seu valor é enviado. O mutex é então liberado chamando cyg_mutex_unlock, novamente passando o mutex para liberar como parâmetro. Ao contrário do

cyg_mutex_lock função, a função de desbloqueio nunca bloqueia, embora possa causar uma reprogramação.

```
/ *****
*
* Função:      incrementTask
*
* Descrição: Esta tarefa incrementa uma variável compartilhada.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
void incrementTask (cyg_addrword_t data) {

    enquanto (1)
    {
        /* Atraso de 3 segundos. */
        cyg_thread_delay (TICKS_PER_SECOND * 3);

        /* Aguarde até que o mutex se torne disponível. */
        cyg_mutex_lock (& sharedVariableMutex);

        gSharedVariable ++;

        diag_printf ("Tarefa de incremento: o valor da variável compartilhada é% d \n",
                     gSharedVariable);

        /* Libere o mutex. */ / cyg_mutex_unlock
        (& sharedVariableMutex);
    }
}
```

o decrementTask função é semelhante à tarefa de incremento anterior. Primeiro, a tarefa demora sete segundos. Em seguida, a tarefa espera para adquirir o sharedVariableMutex. Depois que a tarefa obtém o mutex, ela diminui o gSharedVariable valor e emite seu valor. Finalmente, a tarefa libera o mutex.

```
/ *****
*
* Função:      decrementTask
*
* Descrição: Esta tarefa diminui uma variável compartilhada.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
void decrementTask (cyg_addrword_t data) {

    enquanto (1)
    {
```

```

    /* Atraso de 7 segundos. */
    cyg_thread_delay (TICKS_PER_SECOND * 7);

    /* Aguarde até que o mutex se torne disponível. */
    cyg_mutex_lock (& sharedVariableMutex);

    gSharedVariable--;

    diag_printf ("Diminuir Tarefa: o valor da variável compartilhada é% d \n",
                gSharedVariable);

    /* Libere o mutex. */ cyg_mutex_unlock
    (& sharedVariableMutex);
}
}

```

Sincronização Semáforo de Tarefa

O exemplo do semáforo eCos é semelhante a um interruptor de luz de botão usando um LED para a luz. O exemplo a seguir tem duas tarefas: um produtor e um consumidor. A tarefa do produtor, `productTask`, monitora o botão identificado como SW0 no módulo complementar da placa Arcom. A Figura 11-1 mostra o botão usado neste exemplo.

Quando o botão SW0 é pressionado, a tarefa do produtor sinaliza a tarefa do consumidor usando um semáforo. O consumidor, `consumerTask`, aguarda o sinal do semáforo da tarefa do produtor. Ao receber o sinal, a tarefa do consumidor emite uma mensagem e alterna o LED verde.

A função principal, `cyg_user_start`, começa inicializando o LED chamando `ledInit`. Em seguida, o semáforo é inicializado com uma chamada para `cyg_semaphore_init`. O valor inicial do semáforo, `semButton`, é definido como zero para que a tarefa do consumidor que está aguardando não seja executada até que o semáforo seja sinalizado pela tarefa do produtor. Por último, as duas tarefas são criadas e retomadas, como no exemplo anterior, e então uma mensagem é saída significando o início do programa.

```

# inclui <cyg / kernel / kapi.h>
# inclui <cyg / infra / diag.h>
# inclui "led.h"

cyg_sem_t semButton;

/ *****
*
* Função:      cyg_user_start
*
* Descrição: Rotina principal do programa de semáforo eCos. Esse
*              função cria o semáforo e o produtor e
*              tarefas do consumidor.
*
* Notas:      Esta rotina chama o planejador ao sair.
*

```

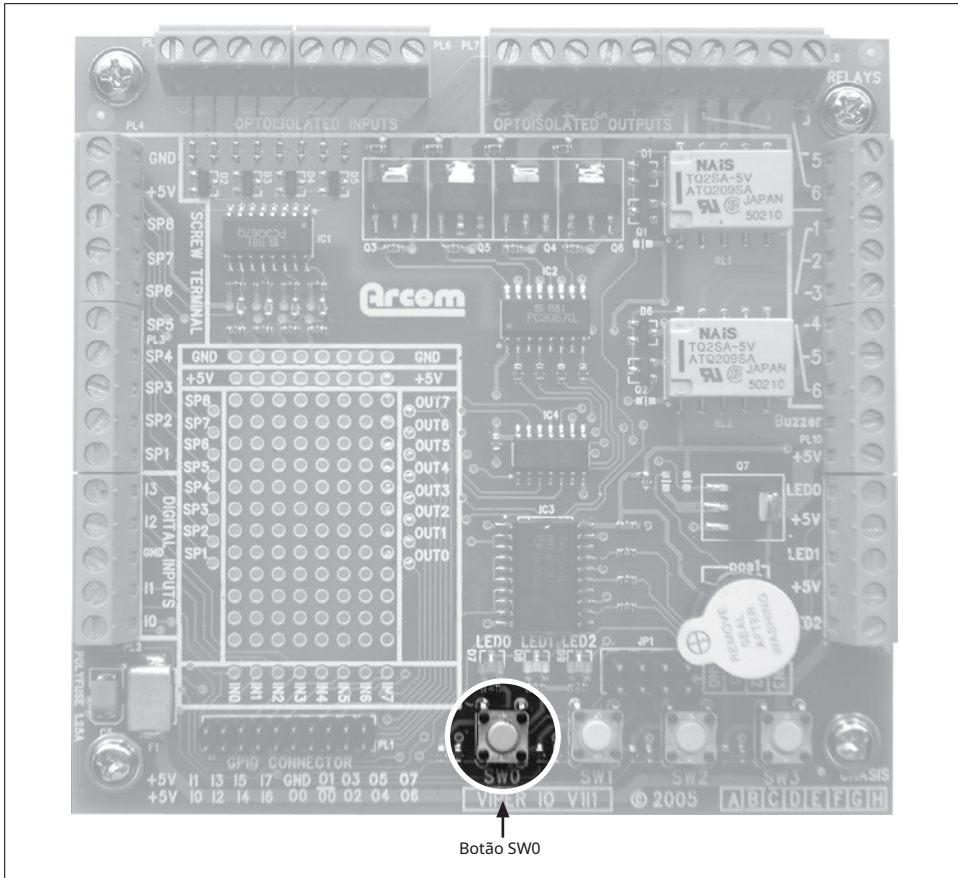


Figura 11-1. Botão SW0 do módulo complementar da placa Arcom

* Retorna: Nenhum.

*

```
void cyg_user_start (void) {
```

```
    /* Configure o pino de controle do LED verde. */
    ledInit ();
```

```
    /* Cria o semáforo com um valor inicial de zero. */
    cyg_semaphore_init (& semButton, 0);
```

```
    /* Crie as tarefas de produtor e consumidor. */
    cyg_thread_create (PRODUCER_TASK_PRIORITY,
                      productTask,
                      (cyg_addrword_t) 0,
                      "Tarefa do Produtor",
                      (vazio *) producerTaskStack,
                      PRODUCER_TASK_STACK_SIZE,
```

```

        & producerTaskHdl,
        & producerTaskObj);

    cyg_thread_create (CONSUMER_TASK_PRIORITY,
                      consumerTask,
                      (cyg_addrword_t) 0,
                      "Tarefa do consumidor",
                      (void *) consumerTaskStack,
                      CONSUMER_TASK_STACK_SIZE,
                      & consumerTaskHdl,
                      & consumerTaskObj);

    /* Notifica o planejador para iniciar a execução das tarefas. */
    cyg_thread_resume (producerTaskHdl);
    cyg_thread_resume (consumerTaskHdl);

    diag_printf ("exemplo de semáforo eCos - pressione o botão SW0. \n");
}

```

o productTask contém um loop infinito que primeiro retarda por 10 ms e, em seguida, verifica se o botão SW0 foi pressionado, chamando a função buttonDebounce (examinaremos essa função mais de perto em um momento). O intervalo de atraso foi selecionado para garantir que a tarefa responda quando o botão for pressionado. Para obter informações adicionais sobre como selecionar intervalos de amostragem, leia a barra lateral posteriormente neste capítulo, intitulada "Switch Debouncing".

Quando o botão SW0 é pressionado, o semáforo, semButton, é sinalizado chamando cyg_semaphore_post, que incrementa o valor do semáforo e ativa a tarefa do consumidor que está aguardando esse semáforo. A tarefa do produtor então retorna para monitorar com o botão SW0.

```

# inclui "button.h"

/ *****
*
* Função:      productTask
*
* Descrição: Esta tarefa monitora o botão SW0. Uma vez pressionado, o botão
*              é depurado e o semáforo é sinalizado, acordando o
*              espera tarefa do consumidor.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /

void producerTask (cyg_addrword_t data) {

    int buttonOn;

    enquanto (1)
    {
        /* Atraso de 10 milissegundos. */

```

```

        cyg_thread_delay (TICKS_PER_SECOND / 100);

        /* Verifique se o botão SW0 foi pressionado. */ / buttonOn
        = buttonDebounce ();

        /* Se o botão SW0 foi pressionado, sinaliza a tarefa do consumidor.
        * / if (buttonOn)
            cyg_semaphore_post (& semButton);
    }
}

```

Agora vamos dar uma olhada na função buttonDebounce. O código de debounce é de junho de 2004 *Programação de sistemas embarcados* artigo "My Favorite Software Debouncers," que pode ser encontrado online em <http://www.embedded.com>.

A função debounce é chamada na tarefa do produtor a cada 10 ms para determinar se o botão SW0 foi pressionado. Conforme mostrado na Figura 11-1, o botão SW0 está localizado no módulo adicional. O conselho da Arcom *Manual Técnico VIPER-Lite* e a *Manual Técnico VIPER-I / O* descreva como os botões do módulo adicional são conectados ao processador. Os esquemas do módulo add-on, que são encontrados no *Manual Técnico VIPER-I / O*, pode ser usado para rastrear a conexão do botão de volta ao processador.

O botão SW0 é lido a partir do sinal IN0, conforme mostrado na seção de interruptores no *Manual Técnico VIPER-I / O*. De acordo com *Manual Técnico VIPER-Lite*, a O valor do sinal IN0 é recuperado lendo o endereço 0x14500000. O bit menos significativo neste endereço contém o estado atual do botão SW0. A documentação da placa Arcom afirma que o nível de tensão padrão na chave do botão é alto e que, quando o botão é pressionado, o valor muda para baixo.

A função debounce primeiro chama buttonRead, que retorna o estado atual do botão SW0. O estado atual do botão SW0 é alterado para a variável buttonState. Quando a borda de ataque do fechamento do interruptor é depurada e detectada, VERDADE é devolvido.

Para obter informações adicionais sobre o debouncing de botões e interruptores, dê uma olhada na barra lateral "Switch Debouncing" posteriormente neste capítulo.

```

/ *****
*
* Função:      buttonDebounce
*
* Descrição: Esta função elimina botões.
*
* Notas:
*
* Retorna:     TRUE se a borda do botão for detectada, caso contrário,
*             FALSE será retornado.
*
***** /
int buttonDebounce (void)

```



```

{
    estático uint16_t buttonState = 0;
    uint8_t pinState;

    pinState = buttonRead ();

    /* Armazena o status de debounce atual. */
    buttonState = ((buttonState << 1) | pinState | 0xE000);

    if (buttonState == 0xF000)
        return TRUE;

    retorna falso;
}

```

o consumerTask contém um loop infinito simples: aguarde até que o semáforo seja sinalizado e imprima uma mensagem assim que o sinal for recebido. A tarefa do consumidor espera pelo sinal do semáforo chamando cyg_semaphore_wait, que bloqueia a tarefa se o valor do semáforo for igual a 0.

Assim que o sinal do semáforo é recebido, a tarefa do consumidor emite uma mensagem de que o botão foi pressionado e alterna o LED verde chamando ledToggle. Depois que o LED é alternado, a tarefa do consumidor volta a esperar por outro sinal de semáforo.

```

/ *****
*
* Função:      consumerTask
*
* Descrição: Esta tarefa aguarda o sinal do semáforo do
*             tarefa do produtor. Assim que o sinal for recebido, a tarefa
*             produz uma mensagem.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
void consumerTask (cyg_addrword_t data) {

    enquanto (1)
    {
        /* Aguarde o sinal. */
        cyg_semaphore_wait (& semButton);

        diag_printf ("O botão SW0 foi pressionado. \n");

        ledToggle ();
    }
}

```

Cobrimos outro exemplo de semáforos na seção “Tratamento de interrupções eCos” posteriormente neste capítulo.

Passagem de mensagens

Nesta seção, discutiremos uma técnica de programação que é útil para certas situações em que você divide tarefas - em particular, quando você pode especificar uma tarefa do produtor que gera dados e uma tarefa do consumidor que processa a saída do produtor. O uso da passagem de mensagens evita que as tarefas atropelem os dados umas das outras e também simplifica a codificação.

O exemplo de passagem de mensagem é semelhante ao exemplo de interruptor-semáforo de luz mostrado anteriormente neste capítulo, onde a tarefa do produtor espera que o botão SW0 seja pressionado e a tarefa do consumidor emite uma mensagem. A diferença desta vez é que uma mensagem que contém o número de vezes que o botão foi pressionado é passada do produtor para o consumidor. O consumidor, `consumerTask`, aguarda a mensagem da tarefa do produtor, `productTask`. Assim que a mensagem é recebida pela tarefa do consumidor, ela gera uma mensagem e alterna o LED verde.

As filas de mensagens em eCos são chamadas de caixas de correio. A seguinte função principal, `cyg_user_start`, começa inicializando o LED chamando `ledInit`. Em seguida, a caixa de correio é inicializada com uma chamada para `cyg_mbox_create`. A função de criação de caixa de correio é passada `mailboxHdl` (um identificador usado para chamadas subsequentes para realizar operações com essa caixa de correio específica) e `caixa de correio` (uma área de memória para a estrutura de caixa de correio do kernel). Por último, as duas tarefas são criadas e retomadas, como vimos no exemplo anterior, e então uma mensagem é emitida, significando o início do programa.

```
# inclui <cyg / kernel / kapi.h>
# inclui <cyg / infra / diag.h>
# inclui "led.h"

cyg_handle_t mailboxHdl; caixa
de correio cyg_mbox;

/ *****
*
* Função:      cyg_user_start
*
* Descrição: Rotina principal do programa de caixa de correio eCos. Esse
*              função cria a caixa de correio e o produtor e
*              tarefas do consumidor.
*
* Notas:      Esta rotina chama o planejador ao sair.
*
* Retorna:    Nenhum.
*
***** /
void cyg_user_start (void) {

    / * Configure o pino de controle do LED verde. * /
    ledInit ();

    / * Cria a caixa de correio para enviar mensagens entre tarefas. * /
```

```

cyg_mbox_create (& mailboxHdl, & mailbox);

/* Crie as tarefas de produtor e consumidor. */
cyg_thread_create (PRODUCER_TASK_PRIORITY,
    productTask,
    (cyg_addrword_t) 0,
    "Tarefa do Produtor",
    (void *) producerTaskStack,
    PRODUCER_TASK_STACK_SIZE,
    & producerTaskHdl,
    & producerTaskObj);

cyg_thread_create (CONSUMER_TASK_PRIORITY,
    consumerTask,
    (cyg_addrword_t) 0,
    "Tarefa do consumidor",
    (void *) consumerTaskStack,
    CONSUMER_TASK_STACK_SIZE,
    & consumerTaskHdl,
    & consumerTaskObj);

/* Notifica o planejador para iniciar a execução das tarefas. */
cyg_thread_resume (producerTaskHdl);
cyg_thread_resume (consumerTaskHdl);

diag_printf ("exemplo de caixa de correio eCos - pressione o botão SW0. \n");
}

```

o productTask mostrado a seguir começa com a inicialização da variável de contagem de pressionamento de botão, buttonPressCount, para zero e, em seguida, entra em um loop infinito. No loop, a tarefa atrasa 10 ms e, em seguida, verifica se o botão SW0 foi pressionado com uma chamada para a função buttonDebounce.

Quando o botão SW0 é pressionado, buttonPressCount é incrementado e o valor é enviado para a tarefa do consumidor em espera por meio da caixa de correio. No eCos, as mensagens são enviadas em caixas de correio usando a função cyg_mbox_put, que leva dois argumentos: o identificador da caixa de correio, neste caso mailboxHdl, e a mensagem a ser enviada. Se houver espaço no correio-caixa, a mensagem é colocada lá imediatamente; caso contrário, a tarefa bloqueia até que esteja disponível.

sala

```

# inclui "button.h"

/ *****
*
* Função:      productTask
*
* Descrição: Esta tarefa monitora o botão SW0. Uma vez pressionado, o botão
*              é depurado e uma mensagem é enviada para o
*              tarefa do consumidor.
*
* Notas:      Esta função é específica da placa Arcom.
*
* Retorna:    Nenhum.

```

```

*
***** /
void producerTask (cyg_addrword_t data) {

    uint32_t buttonPressCount = 0;
    int buttonOn;

    enquanto (1)
    {
        /* Atraso de 10 milissegundos. */
        cyg_thread_delay (TICKS_PER_SECOND / 100);

        /* Verifique se o botão SW0 foi pressionado. */
        buttonOn = buttonDebounce ();

        /* Se o botão SW0 foi pressionado, envia uma mensagem para a tarefa do consumidor.
        * / if (buttonOn)
        {
            buttonPressCount ++;
            cyg_mbox_put (mailboxHdl, (void *) buttonPressCount);
        }
    }
}

```

o consumerTask, mostrado no código a seguir, contém um loop infinito que espera por uma mensagem que chega e a imprime assim que é recebida. A tarefa do consumidor aguarda uma mensagem de entrada chamando cyg_mbox_get e passando o identificador da caixa de correio, mailboxHdl. A caixa de correio obtém blocos de chamada de função até que a tarefa do produtor envie uma mensagem. A mensagem, que é a contagem de pressionamento do botão, é armazenada na variável local rcvMsg. Depois que a mensagem é emitida e o LED verde é alternado, a tarefa do consumidor volta a esperar por outra mensagem.

```

/ *****
*
* Função:      consumerTask
*
* Descrição: Esta tarefa espera por uma mensagem da tarefa do produtor.
*              Assim que a mensagem é recebida pela caixa de correio,
*              emite uma mensagem e alterna o LED verde.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
void consumerTask (cyg_addrword_t data) {

    uint32_t rcvMsg;

    enquanto (1)
    {
        /* Aguarde uma nova mensagem. */
        rcvMsg = (uint32_t) cyg_mbox_get (& mailboxHdl);
    }
}

```

```

diag_printf ("Botão SW0 pressionado% d vezes. \n", rcvMsg);

    ledToggle ();
}
}

```

A maioria dos sistemas operacionais, incluindo eCos, tem funções API adicionais para várias operações de sincronização e passagem de mensagens. Por exemplo, eCos inclui as funções `cyg_mbox_tryput` e `cyg_mbox_tryget` que retornam `false` se não tiverem sucesso em colocar ou receber uma mensagem na caixa de correio, em vez de bloquear a tarefa. Essas funções podem ser usadas para fazer polling (ou seja, para verificar se uma caixa de correio está disponível, desligar e fazer outras tarefas se não estiver e, em seguida, retornar e tentar novamente).

Existem também funções para as quais você passa um valor de tempo limite que bloqueia por um determinado período de tempo enquanto a função tenta realizar a operação especificada. Informações adicionais sobre as APIs eCos RTOS podem ser encontradas no *eCos Reference* online em <http://ecos.sourceware.org/docs-latest>.

Switch Debouncing

Quando pressionado ou liberado, qualquer entrada mecânica, como uma chave ou botão, será aberta e fechada brevemente antes de se estabelecer. Os processadores são tão rápidos que podem detectar essa rápida sucessão de aberturas e fechamentos; portanto, é difícil saber se alguma leitura individual da posição de um switch é precisa. Este é um caso em que o processador vê as árvores, mas não a floresta. *Debouncing* é uma técnica usada para suavizar as amostras e garantir que o processador não fique confuso com o salto.

Você pode eliminar as entradas inserindo hardware ou software extra para filtrar esse ruído. As rotinas de debounce do software funcionam mais ou menos testando a entrada regularmente em um intervalo predeterminado e tomando decisões somente após uma sucessão de leituras que são iguais. Escrever código debounce é simples; selecionar o intervalo de amostragem e decidir quando a entrada foi estabelecida é mais difícil e específico para cada dispositivo de entrada e aplicação.

Para obter informações adicionais sobre os intervalos de amostragem, consulte o relatório de julho de 2002 *Programação de sistemas embarcados* artigo "How to Choose A Sensible Sampling Rate," que pode ser encontrado online em <http://www.embedded.com>.

Tratamento de interrupções eCos

Conforme explicado no Capítulo 8, é importante manter o processamento de interrupções em um mínimo para que outras interrupções (potencialmente de maior prioridade) no sistema possam ser atendidas e, assim, as tarefas de alta prioridade possam ser executadas. Assim, os programadores normalmente dividem o tratamento de interrupções em duas categorias: um ISR curto e um DSR mais tranquilo.

A questão de como fazer a divisão - o que colocar no ISR e o que colocar no DSR - depende do aplicativo e do processador. Basicamente, você deve adiar tudo o que puder. As funções do sistema operacional que podem ser bloqueadas não podem ser chamadas de um ISR ou DSR. E, ao contrário de muitos outros sistemas operacionais, o eCos em particular não permite que um ISR sinalize um semáforo por meio de uma chamada sem bloqueio. No eCos, a sinalização de semáforo deve ser feita por meio de uma chamada do DSR.

Para obter uma melhor compreensão do esquema de tratamento de interrupção de divisão, dê uma olhada no seguinte exemplo eCos, que mostra o uso de uma interrupção para controlar o tempo para o programa de LED piscando. Neste exemplo, um semáforo é usado para sinalizar uma tarefa da interrupção quando é hora de alternar o LED.

A sequência de inicialização entre o hardware e o software no `cyg_user_start` função é importante. Por exemplo, você não gostaria de ligar `timerInit` para iniciar a interrupção do cronômetro antes de criar e instalar um manipulador de interrupção para o cronômetro.

Primeiro, o LED é inicializado. Em seguida, o semáforo, `ledToggleSemaphore`, que sinaliza o `blinkLedTask` quando é hora de alternar o LED, é inicializado. Então o `blinkLedTask` é criado conforme mostrado anteriormente.

Em seguida, o ISR e o DSR são criados para a interrupção do cronômetro com uma chamada para `cyg_interrupt_create`, que preenche a estrutura de interrupção do kernel. O vetor de interrupção (27 para o temporizador 1) é definido pela macro `TIMER1_INT` e passado como o primeiro parâmetro para `cyg_interrupt_create`. Os próximos dois parâmetros são a prioridade da interrupção e os dados privados da interrupção, que são definidos como zero. A função ISR, `timerIsr`, e função DSR, `timerDsr`, são passados em seguida. Quando o `cyg_interrupt_create` retorna a função, ele define os dois argumentos finais: o identificador de interrupção, `timerInterruptHdl`, e interromper o objeto, `timerInterruptObj`. O identificador de interrupção é usado em operações subsequentes para esta interrupção. O objeto de interrupção fornece ao kernel uma área de memória contendo o manipulador de interrupção e seus dados associados (caso algum seja necessário).

Em seguida, a interrupção ISR e DSR são anexados à fonte de interrupção chamando `cyg_interrupt_attach` e passando o identificador da função de criação de interrupção. Como precaução, `cyg_interrupt_acknowledge` é chamado no caso de haver uma interrupção do temporizador pendente. Por último, a interrupção é desmascarada chamando `cyg_interrupt_unmask`.

A etapa final na sequência de inicialização é configurar os registradores do temporizador e habilitar a interrupção, o que é feito na função `timerInit`:

```
# inclui <cyg / kernel / kapi.h>
# inclui "timer.h"
# inclui "led.h"

/* Declare as variáveis ISR. */
cyg_handle_t timerInterruptHdl;
cyg_interrupt timerInterruptObj;
cyg_vector_t timerInterruptVector = TIMER1_INT;

cyg_sem_t ledToggleSemaphore;
```

```

/ *****
*
* Função:      cyg_user_start
*
* Descrição: Rotina principal para programa de LED piscando de interrupção eCos.
*            Esta função cria o semáforo de alternância de LED, o
*            Tarefa de LED e o manipulador de interrupção do cronômetro.
*
* Notas:      Esta rotina chama o planejador ao sair.
*
* Retorna:    Nenhum.
*
***** /
void cyg_user_start (void) {

    / * Configure o pino de controle do LED verde. * /
    ledInit ();

    / * Cria o semáforo para a sinalização de tarefa. Este semáforo
    * é inicializado com um valor de 0, então a tarefa de alternância deve esperar
    * para o primeiro período de tempo decorrido. * /
    cyg_semaphore_init (& ledToggleSemaphore, 0);

    / * Cria a tarefa LED. * /
    cyg_thread_create (LED_TASK_PRIORITY,
                      blinkLedTask,
                      (cyg_addrword_t) 0,
                      "Tarefa LED",
                      (vazio *) ledTaskStack,
                      LED_TASK_STACK_SIZE,
                      & ledTaskHdl,
                      & ledTaskObj);

    / * Notifica o planejador para iniciar a execução da tarefa. * /
    cyg_thread_resume (ledTaskHdl);

    / * Inicializa a interrupção do temporizador. * /
    cyg_interrupt_create (timerInterruptVector,
                          0,
                          0,
                          timerIsr,
                          timerDsr,
                          & timerInterruptHdl,
                          & timerInterruptObj);

    cyg_interrupt_attach (timerInterruptHdl);
    cyg_interrupt_acknowledge (timerInterruptVector);
    cyg_interrupt_unmask (timerInterruptVector);

    / * Inicializa os registros do temporizador.
    * / timerInit ();
}

```

O eCos fornece a funcionalidade de salvar e restaurar o contexto do processador quando ocorre uma interrupção, de forma que o ISR não precisa realizar essas operações. Assim, reduzimos a carga de trabalho do manipulador de interrupção do cronômetro, `timerIsr`, a três operações críticas que devem ser realizadas antes que as interrupções sejam habilitadas novamente.

A primeira operação mascara a interrupção do cronômetro, com uma chamada para `cyg_interrupt_mask` passando no vetor de interrupção `timerInterruptVector`, até que o DSR seja executado. Isso impede que o ISR seja chamado novamente até que a interrupção atual seja processada.

A segunda operação realizada no ISR é reconhecer a interrupção. A interrupção deve ser reconhecida no controlador de interrupção do processador e no periférico do temporizador. A interrupção é reconhecida no controlador de interrupção usando a função eCos `cyg_interrupt_acknowledge` e passando no vetor de interrupção `timerInterruptVector`. O o a interrupção é reconhecida no periférico do temporizador, escrevendo o `TIMER_1_MATCH ( 0x02)` bit para o registro de status do temporizador.

A terceira operação, realizada ao retornar, é informar ao sistema operacional que a interrupção do temporizador foi tratada (com a macro `CYG_ISR_HANDLED`) e que o DSR precisa ser executado (com a macro `CYG_ISR_CALL_DSR`). Notificar o eCos de que a interrupção foi tratada evita que ele chame qualquer outro ISR para lidar com a mesma interrupção.

A seguir `timerIsr` função mostra essas operações:

```
# inclui <cyg / hal / hal_intr.h>

/ *****
*
* Função:      timerIsr
*
* Descrição: Rotina de serviço de interrupção para a interrupção do cronômetro.
*
* Notas:
*
* Retorna:     Bitmask para informar ao sistema operacional que
*              a interrupção foi tratada e para agendar a rotina de
*              serviço adiada.
*
* ***** /
uint32_t timerIsr (vetor cyg_vector_t, dados cyg_addrword_t) {

    / * Bloqueia a ocorrência da interrupção do cronômetro até que o DSR seja executado.
    * / cyg_interrupt_mask (timerInterruptVector);

    / * Reconhecer a interrupção no controlador de interrupção e o
    * periférico temporizador. * /
    cyg_interrupt_acknowledge (timerInterruptVector);
    TIMER_STATUS_REG = TIMER_1_MATCH;
```



```

    /* Informa ao sistema operacional que a interrupção é tratada por este
    * ISR e que o DSR precisa ser executado. */
    retorno (CYG_ISR_HANDLED | CYG_ISR_CALL_DSR);
}

```

A função DSR, `timerDsr`, que é mostrado a seguir, está programado para ser executado pelo eCos assim que o ISR for concluído. O DSR sinaliza a tarefa de LED usando o semáforo `ledToggleSemaphore` com a chamada de função `cyg_semaphore_post`.

Em seguida, o novo intervalo do temporizador é programado no registro de correspondência do temporizador, que é definido para expirar 500 ms da contagem atual do temporizador. Finalmente, antes de sair, o DSR desmascara a interrupção do cronômetro no sistema operacional, chamando `cyg_interrupt_unmask` e passando no vetor de interrupção, que reativa o tratamento de interrupções de temporizador de entrada.

```

/ *****
*
* Função:      timerDsr
*
* Descrição: Rotina de serviço diferida para a interrupção do cronômetro.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
void timerDsr (vetor cyg_vector_t, contagem cyg_ucount32, dados cyg_addrword_t) {

    /* Sinaliza a tarefa para alternar o LED. */
    cyg_semaphore_post (& ledToggleSemaphore);

    /* Definir o novo intervalo do temporizador. */
    TIMER_1_MATCH_REG = (TIMER_COUNT_REG + TIMER_INTERVAL_500MS);

    /* Habilita o processamento de interrupções do temporizador de
    entrada. */
    cyg_interrupt_unmask (timerInterruptVector);
}

```

o `blinkLedTask` contém um loop infinito que espera pelo semáforo `ledToggleSemaphore` ser sinalizado chamando `cyg_semaphore_wait`. Quando o semáforo é finalizado pelo temporizador DSR, a tarefa chama `ledToggle` para alterar o estado do LED. assinar

```

/ *****
*
* Função:      blinkLedTask
*
* Descrição: Esta tarefa lida com alternar o LED quando é
*            sinalizado do manipulador de interrupção do temporizador.
*
*****

```

```

* Notas:
*
* Retorna:      Nenhum.
*
***** /
void blinkLedTask (cyg_addrword_t data) {

    enquanto (1)
    {
        /* Aguarde o sinal de que é hora de alternar o LED. */
        cyg_semaphore_wait (& ledToggleSemaphore);

        /* Altera o estado do LED verde. */
        ledToggle ();
    }
}

```

Isso conclui nossa breve introdução ao sistema operacional eCos e sua API. Felizmente, esses poucos exemplos esclareceram alguns dos pontos levantados em outras partes do livro. Essas são técnicas de programação valiosas usadas com frequência em sistemas embarcados.

Exemplos de Linux embarcado

TyWebb: *Não fique obcecado com seus desejos, Danny. O filósofo Zen Basho escreveu certa vez: “Uma flauta sem orifícios não é uma flauta ... e um donut com nenhum buraco é dinamarquês. “ Ele era um cara engraçado.*

- o filme *Caddyshack*

Neste capítulo, exploraremos alguns exemplos usando Linux embarcado. Os exemplos neste capítulo são semelhantes (ou em alguns casos iguais) aos exemplos eCos que abordamos no capítulo anterior. A ideia aqui é obter uma introdução ao Linux embarcado e compreender algumas funcionalidades básicas do sistema operacional.

Introdução

Os exemplos de Linux embarcado demonstram certas APIs básicas para várias operações. Existem APIs adicionais que oferecem outras funcionalidades. Você deve pesquisar as APIs adicionais por conta própria para determinar se existem outras maneiras melhores de executar as operações necessárias para seu sistema embarcado específico.

Um aspecto do Linux com o qual você precisa estar familiarizado é seu modelo de thread. A API do Linux está em conformidade com o principal padrão POSIX no espaço, POSIX 1003.1c, comumente chamado de *pthread* padrão. POSIX deixa muitos dos detalhes de implementação para o implementador do sistema operacional. Uma boa fonte de informação sobre pthreads é o livro *Programação de Pthreads*, por Bradford Nichols, Dick Buttlar e Jacqueline Farrell (O'Reilly).

A versão do Linux embutido usada na placa Arcom é uma árvore de kernel padrão (versão 2.6) com suporte adicional para ARM e XScale do Projeto ARM Linux no <http://www.arm.linux.org.uk>.

Uma infinidade de livros sobre Linux e Linux embarcado estão disponíveis. Alguns bons recursos incluem *Compreendendo o kernel do Linux*, por Daniel P. Bovet e Marco Cesati (O'Reilly), *Drivers de dispositivos Linux*, por Alessandro Rubini e Jonathan Corbet (O'Reilly), e *Construindo Sistemas Linux Embarcados*, por Karim Yaghmour (O'Reilly).

As instruções para configurar o ambiente de construção do Linux integrado e construir os aplicativos Linux de exemplo são detalhadas no Apêndice E. Informações adicionais sobre o uso do Linux integrado na placa Arcom podem ser encontradas no *Arcom Manual Técnico do Embedded Linux* e a *Manual Técnico VIPER-Lite*.



Para manter os exemplos neste capítulo mais curtos e fáceis de ler, não verificamos os valores de retorno das chamadas de função. Em geral, é uma boa ideia validar todos os códigos de retorno. Isso fornece feedback sobre problemas em potencial e permite que você, como desenvolvedor, tome decisões no software com base em chamadas com falha. Basicamente, isso torna seu código mais robusto e, com sorte, menos cheio de erros.

Acessando Hardware no Linux

Antes de prosseguir com os exemplos do Linux, é importante ter um conhecimento básico do acesso ao hardware no Linux. Linux, como a maioria dos sistemas operacionais de desktop, particiona seu gerenciamento de memória em *espaço do usuário* e *espaço do kernel*.

O espaço do usuário é onde os aplicativos são executados (incluindo os exemplos do Linux a seguir). Os aplicativos de espaço do usuário têm permissão para acessar o hardware apenas por meio de funções suportadas por kernel. O espaço do kernel é normalmente onde existem drivers de dispositivo. Isso permite que os drivers de dispositivo tenham acesso direto ao hardware.

Os exemplos do Linux usam uma função chamada mmap para acessar um determinado intervalo de endereços. O mmap A função pede ao kernel para fornecer acesso a um intervalo de endereço físico contido no hardware. Para obter detalhes sobre como usamos mmap, consulte o código-fonte do livro.

Mecânica da Tarefa

Para o primeiro exemplo do Linux, reutilizamos o programa LED piscando. Este exemplo mostra como criar uma tarefa para alternar o LED em uma taxa constante.

A tarefa blinkLedTask atrasa 500 ms e, em seguida, alterna o LED verde. A tarefa usa um loop infinito que chama a função `you_sleep` para suspender pelo período de tempo adequado. O `you_sleep` função recebe o número de microssegundos para suspender, que neste caso é 50.000 μ s.

Após a chamada da função de atraso, o `blinkLedTask` é bloqueado e colocado no estado de espera. A tarefa é colocada no estado pronto assim que o tempo decorre e é executada pelo planejador. Após o atraso, o `ledToggle` função alterna o LED verde.

```
# inclui <unistd.h>
# inclui "led.h"
```

```
/ *****
*
```

```

* Função:      blinkLedTask
*
* Descrição: Esta tarefa lida com alternar o LED verde em um
*             intervalo constante.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
void blinkLedTask (void * param) {

    enquanto (1)
    {
        /* Atraso de 500 milissegundos. */
        usleep (50000);

        ledToggle ();
    }
}

```

o a Principal o trabalho da função neste exemplo é criar a tarefa LED. A tarefa é criada chamando a função `pthread_create`. Para este exemplo, os atributos de tarefa padrão são usados e nenhum parâmetro é passado para a tarefa `blinkLedTask`.

A função `pthread_join` é usado para suspender o a Principal funcionar até o `blinkLedTask` a tarefa termina. Neste caso, o `blinkLedTask` tarefa é executada para sempre, então neia função sai.

```

# inclua <pthread.h>

/* Declare as variáveis de tarefa. */
pthread_t ledTaskObj;

/ *****
*
* Função:      a Principal
*
* Descrição: Rotina principal para o programa de LED piscando do Linux. Esse
*             função cria a tarefa LED.
*
* Notas:
*
* Retorna:      0
*
***** /
int main (vazio)
{
    /* Configure o pino de controle do LED verde. */
    ledInit ();

    /* Crie a tarefa de LED usando os atributos de tarefa padrão. Não
     * passe quaisquer parâmetros para a tarefa. */
    pthread_create (& ledTaskObj, NULL, (void *) blinkLedTask, NULL);
}

```

```

    /* Permite que a tarefa LED seja executada.
    * / pthread_join (ledTaskObj, NULL);

    return 0;
}

```

Operações de tarefas adicionais são suportadas no Linux. Essas operações incluem encerrar tarefas, modificar os atributos da tarefa, ceder a outras tarefas no sistema e suspender / retomar tarefas.

Sincronização Mutex de Tarefas

No exemplo mutex do Linux (assim como no exemplo eCos), duas tarefas compartilham uma variável comum chamada gSharedVariable. Uma tarefa aumenta a variável global em um intervalo definido e a outra tarefa diminui a variável em um intervalo definido. O mutex protege a variável compartilhada.

A função a Principal começa criando o mutex, sharedVariableMutex, chamando a função pthread_mutex_init. Como os atributos padrão são usados na criação de mutex, NULO é passado como o segundo parâmetro. No Linux, os mutexes têm atributos que você pode definir usando o segundo parâmetro, mas não os usaremos neste livro, então apenas passaremos NULO.

Por último, as duas tarefas incrementTask e decrementTask são criados. É importante criar o mutex antes de criar as tarefas que o utilizam, caso contrário, as tarefas podem travar o programa.

```

# inclua <pthread.h>

pthread_mutex_t sharedVariableMutex;
int32_t gSharedVariable = 0;

/ *****
*
* Função:          a Principal
*
* Descrição: Rotina principal para o exemplo mutex do Linux. Esse
*              função cria o mutex e, em seguida, o incremento e
*              tarefas de decremento.
*
* Notas:
*
* Retorna:        0
*
***** /

int main (vazio)
{
    /* Crie o mutex para acessar a variável compartilhada usando o
    * atributos padrão. */
    pthread_mutex_init (& sharedVariableMutex, NULL);

```

```

    /* Cria as tarefas de incremento e decremento usando a tarefa padrão
    * atributos. Não passe nenhum parâmetro para as tarefas. */
    pthread_create (& incrementTaskObj, NULL, (void *) incrementTask, NULL);
    pthread_create (& decrementTaskObj, NULL, (void *) decrementTask, NULL);

    /* Permitir que as tarefas sejam executadas. */
    pthread_join (incrementTaskObj, NULL);
    pthread_join (decrementTaskObj, NULL);

    return 0;
}

```

A tarefa `incrementTask`, mostrado a seguir, inclui um loop infinito que começa atrasando por três segundos chamando `dormir`, que suspende a tarefa por um determinado número de segundos.

Depois de decorrido o tempo de atraso, a tarefa de incremento continua de onde parou. o tarefa então chama `pthread_mutex_lock` e passa no `sharedVariableMutex` em ordem de pegue o mutex e acesse a variável compartilhada. Se o mutex estiver disponível, ele será bloqueado e a tarefa de incremento continuará a incrementar `gSharedVariable`. Se o mutex não estiver disponível, a tarefa de incremento é bloqueada até que possa adquirir o mutex.

Depois de incrementar a variável compartilhada e enviar uma mensagem, o mutex é liberado com uma chamada para `pthread_mutex_unlock`. A função de desbloqueio mutex nunca bloqueia.

```

# inclui <stdio.h>
# inclui <unistd.h>

/ *****
*
* Função:      incrementTask
*
* Descrição: Esta tarefa incrementa uma variável compartilhada.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
void incrementTask (void * param) {

    enquanto (1)
    {
        /* Atraso de 3 segundos. */
        dormir (3);

        /* Aguarde o mutex antes de acessar os registros GPIO. */
        pthread_mutex_lock (& sharedVariableMutex);

        gSharedVariable ++;

        printf ("Tarefa de incremento: o valor da variável compartilhada é% d \ n",
                gSharedVariable);
    }
}

```

```

        /* Libere o mutex para outra tarefa usar. */
        pthread_mutex_unlock (& sharedVariableMutex);
    }
}

```

A tarefa `decrementTask` é semelhante à tarefa de incremento. Em seu loop infinito, a tarefa primeiro é suspensa por sete segundos, depois espera para adquirir o `sharedVariableMutex`. Depois de obter o mutex, a tarefa diminui o valor de `gSharedVariable`, gera uma mensagem e, em seguida, libera o mutex, conforme mostrado aqui:

```

/ *****
*
* Função:      decrementTask
*
* Descrição: Esta tarefa diminui uma variável compartilhada.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
void decrementTask (void * param) {

    enquanto (1)
    {
        /* Atraso de 7 segundos. */
        dormir (7);

        /* Aguarde até que o mutex se torne disponível. */
        pthread_mutex_lock (& sharedVariableMutex);

        gSharedVariable--;

        printf ("Decrementar tarefa: o valor da variável compartilhada é %d \n",
                gSharedVariable);

        /* Libere o mutex. */ pthread_mutex_unlock
        (& sharedVariableMutex);
    }
}

```

A API `pthread` do Linux oferece suporte a funções mutex adicionais que fornecem outras funcionalidades. Por exemplo, a função `pthread_mutex_trylock` pode ser usado para tentar obter um mutex. Se o mutex estiver disponível, a tarefa adquire o mutex; se o mutex não estiver disponível, a tarefa pode prosseguir com outro trabalho sem esperar que o mutex seja liberado.

Sincronização Semáforo de Tarefa

O exemplo do semáforo do Linux é semelhante a um interruptor de luz de botão, usando um LED para a luz, como foi o caso no exemplo do semáforo `eCos`. Existem duas tarefas neste exemplo: um produtor e um consumidor. A tarefa do produtor, `productTask`,

monitora o botão identificado como SW0 no módulo complementar da placa Arcom. A Figura 11-1 no Capítulo 11 mostra o botão usado neste exemplo.

Quando o botão SW0 é pressionado, a tarefa do produtor sinaliza a tarefa do consumidor usando um semáforo. O consumidor, `consumerTask`, aguarda o sinal do semáforo da tarefa do produtor; uma vez recebida, a tarefa do consumidor emite uma mensagem e alterna o LED verde.

o a Principal função inicializa primeiro o LED chamando `ledInit`. Em seguida, o semáforo é inicializado com uma chamada para `sem_init`. O valor inicial do semáforo, `semButton`, é definido como zero pelo último parâmetro para que a tarefa do consumidor que está esperando não seja executada até que o semáforo seja sinalizado pela tarefa do produtor. O segundo parâmetro notifica o sistema operacional de que este semáforo pode ser usado apenas por este processo. Por último, as duas tarefas são criadas e uma mensagem é emitida indicando o programa de `do` início.

```
# inclui <stdio.h>
# inclua <pthread.h>
# inclua <semaphore.h>
# inclui "led.h"

sem_t semButton;

/ *****
*
* Função:          a Principal
*
* Descrição: Rotina principal para o exemplo do semáforo do Linux. Esse
*              função cria o semáforo e então o incremento
*              e tarefas de decremento.
*
* Notas:
*
* Retorna:        0
*
***** /

int main (vazio)
{
    /* Configure o pino de controle do LED verde. */
    ledInit ();

    /* Cria o semáforo apenas para este processo e com um inicial
     * valor zero. */
    sem_init (& semButton, 0, 0);

    /* Cria as tarefas de produtor e consumidor usando a tarefa padrão
     * atributos. Não passe nenhum parâmetro para as tarefas. */
    pthread_create (& producerTaskObj, NULL, (void *) producerTask, NULL);
    pthread_create (& consumerTaskObj, NULL, (void *) consumerTask, NULL);

    printf ("Exemplo de semáforo do Linux - pressione o botão SW0. \n");
}
```

```

        /* Permitir que as tarefas sejam executadas.
        * / pthread_join (producerTaskObj, NULL);
        pthread_join (consumerTaskObj, NULL);

        return 0;
    }

```

o productTask função mostrada a seguir contém um loop infinito que primeiro retarda por 10 ms e, em seguida, verifica se o botão SW0 foi pressionado, chamando a função buttonDebounce. Para obter informações adicionais sobre como selecionar intervalos de amostragem e debouncing do botão, leia a barra lateral “Switch Debouncing” no Capítulo 11.

Quando o botão SW0 é pressionado, o productTask função sinaliza o semButton semáforo chamando sem_post. Isso aumenta o valor do semáforo e ativa a tarefa do consumidor. A tarefa do produtor então retorna para monitorar o botão SW0.

```

# inclui <unistd.h>
# inclui "button.h"

/ *****
*
* Função:      productTask
*
* Descrição: Esta tarefa monitora o botão SW0. Uma vez pressionado, o botão
*              é depurado e o semáforo é sinalizado, acordando o
*              espera tarefa do consumidor.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
void producerTask (void * param) {

    int buttonOn;

    enquanto (1)
    {
        /* Atraso de 10 milissegundos. */
        usleep (10000);

        /* Verifique se o botão SW0 foi pressionado. */
        buttonOn = buttonDebounce ();

        /* Se o botão SW0 foi pressionado, sinaliza a tarefa do consumidor.
        * / if (buttonOn)
            sem_post (& semButton);
    }
}

```

A seguir consumerTask função contém um loop infinito que espera que o semáforo seja sinalizado pela chamada sem_wait. A função de espera bloqueia a tarefa se o valor do semáforo for 0.

Uma vez que o sinal do semáforo é recebido, a tarefa do consumidor emite uma mensagem e alterna o LED verde chamando `ledToggle`. Depois de alternar o LED, a tarefa do consumidor volta a esperar por outro sinal de semáforo.

```

/ *****
*
* Função:      consumerTask
*
* Descrição: Esta tarefa aguarda o sinal do semáforo do
*              tarefa do produtor. Assim que o sinal for recebido, a tarefa
*              emite uma mensagem e alterna o LED verde.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
void consumerTask (void * param) {

    enquanto (1)
    {
        /* Aguarde o sinal. */
        sem_wait (& semButton);

        printf ("O botão SW0 foi pressionado. \n");

        ledToggle ();
    }
}

```

Passagem de mensagens

O exemplo de passagem de mensagem do Linux é um interruptor de luz, como é o exemplo `eCos`, com uma tarefa de produtor e consumidor. A tarefa do produtor monitora o botão `SW0`. Assim que o botão é pressionado, o produtor envia a contagem de pressionamentos do botão para a tarefa do consumidor. A tarefa do consumidor espera por esta mensagem, gera a contagem de botões e, em seguida, alterna o LED verde. Este exemplo usa uma fila de mensagens POSIX para passar a mensagem do produtor para a tarefa do consumidor.



O código-fonte do exemplo de fila de mensagens não está incluído no site do livro porque não funciona na placa `Arcom` enviada. Para que o código da fila de mensagens seja executado na placa `Arcom`, o kernel do Linux precisa ser reconstruído com suporte para fila de mensagens incluído.

o a Principal rotina demonstra como criar a fila de mensagens. Primeiro, o LED é inicializado chamando `ledInit`. Em seguida, a fila de mensagens é criada chamando `mq_open`. O primeiro parâmetro especifica o nome da fila como fila de mensagens. O segundo

parâmetro, que é o OR dos sinalizadores de status do arquivo e modos de acesso, especifica o seguinte:

O_CREAT

Crie a fila de mensagens se ela não existir.

O_EXCL

Usado com O_CREAT para criar e abrir uma fila de mensagens se ainda não existir uma fila com o mesmo nome. Se houver uma fila com o mesmo nome, a fila de mensagens não será aberta.

O_RDWR

Abra para acesso de leitura e gravação.

Depois que a fila de mensagens é criada com sucesso, as tarefas são criadas conforme mostrado anteriormente. As técnicas que acabamos de discutir são mostradas a seguir a Principal função:

```
# inclua <pthread.h>
# include <mqueue.h>
# inclui "led.h"
```

```
int8_t messageQueuePath [] = "fila de mensagens";
```

```
/ *****
*
* Função:          a Principal
*
* Descrição: Rotina principal para o programa de fila de mensagens do Linux. Esse
*              função cria a fila de mensagens e o produtor
*              e tarefas do consumidor.
*
* Notas:
*
* Retorna:        0
*
***** /

int main (vazio)
{
    mqd_t messageQueueDescr;

    / * Configure o pino de controle do LED verde. * /
    ledInit ();

    / * Cria a fila de mensagens para enviar informações entre tarefas. * /
    messageQueueDescr = mq_open (messageQueuePath, (O_CREAT | O_EXCL | O_RDWR));

    / * Crie a tarefa do produtor usando os atributos de tarefa padrão. Não
      * passe quaisquer parâmetros para a tarefa. * /
    pthread_create (& producerTaskObj, NULL, (void *) producerTask, NULL);

    / * Crie a tarefa do consumidor usando os atributos de tarefa padrão. Não
      * passe quaisquer parâmetros para a tarefa. * /
    pthread_create (& consumerTaskObj, NULL, (void *) consumerTask, NULL);
```

```

    /* Permitir que as tarefas sejam executadas.
    * / pthread_join (producerTaskObj, NULL);
    pthread_join (consumerTaskObj, NULL);

    return 0;
}

```

Antes de entrar em seu loop infinito, o `productTask` começa inicializando a variável que controla o número de pressionamentos de botão, `buttonPressCount`, a zero. Em seguida, a tarefa do produtor abre a fila de mensagens, cujo nome é especificado por `messageQueuePath`, que foi criado na Principal função. A fila é aberta com permissão somente de gravação, especificada pelo segundo sinalizador de parâmetro `O_WRONLY`, porque o produtor envia apenas mensagens. Porque a bandeira `O_NONBLOCK` não é especificado no segundo parâmetro para `mq_open`, se uma mensagem não puder ser inserida na fila, a tarefa do produtor será bloqueada. A função `mq_open` retorna um descritor de fila de mensagens que é usado em acessos subsequentes à fila de mensagens.

No loop infinito, a tarefa do produtor primeiro atrasa 10 ms chamando `usleep`. O intervalo de atraso selecionado garante que a tarefa responda aos pressionamentos de botão. Em seguida, a função `buttonDebounce` é chamado para determinar se o botão SW0 foi pressionado.

Cada vez que o botão SW0 é pressionado, `buttonPressCount` é incrementado e o valor é enviado para a tarefa do consumidor em espera, usando a fila de mensagens. Para acomodar a função de envio da fila de mensagens, o `msgbuf_t` é usado para conter a mensagem. Essa união consiste em uma contagem de 32 bits e uma matriz de buffer de 4 bytes. A mensagem é enviada usando a função `mq_send`, com o descritor de fila de mensagens, `messageQueueDescr`, no primeiro parâmetro, a contagem de pressionamentos do botão no segundo parâmetro, o tamanho da mensagem no terceiro parâmetro e a prioridade da mensagem no último parâmetro.

Ao contrário da implementação da fila de mensagens `eCos`, as mensagens no Linux têm uma prioridade (que é especificada no último parâmetro passado para `mq_send`). Você pode usar este parâmetro para inserir mensagens de prioridade mais alta na frente da fila de mensagens, para serem lidas pela tarefa de recebimento.

Depois que a mensagem é enviada com sucesso, o loop volta a chamar a função de atraso. Aqui está o `productTask` função:

```

# inclui <unistd.h>
# inclui "button.h"

união typedef
{
    uint32_t count;
    uint8_t buf [4];
} msgbuf_t;

/ *****
*
* Função:      productTask
*

```

```

* Descrição: Esta tarefa monitora o botão SW0. Uma vez pressionado, o botão
*             é depurado e uma mensagem é enviada para o
*             tarefa do consumidor.
*
* Notas:
*
* Retorna:     Nenhum.
*
***** /
void producerTask (void * param) {

    uint32_t buttonPressCount = 0;
    mqd_t messageQueueDescr;
    botão uint8_t;
    msgbuf_t msg;

    /* Abra a fila de mensagens existente usando o sinalizador somente gravação porque
     * esta tarefa apenas envia mensagens. Defina a fila para bloquear se
     * um envio não pode ocorrer imediatamente. */
    messageQueueDescr = mq_open (messageQueuePath, O_WRONLY);

    enquanto (1)
    {
        /* Atraso de 10 milissegundos. */
        usleep (10000);

        /* Verifique se o botão SW0 foi pressionado. */ botão =
        botãoDebounce ();

        /* Se o botão SW0 foi pressionado, envia uma mensagem para a tarefa do consumidor.
         * / if (botão & BUTTON_SW0)
        {
            buttonPressCount++;
            msg.count = buttonPressCount;

            mq_send (messageQueueDescr, & msg.buf [0], sizeof (buttonPressCount), 0);
        }
    }
}

```

A tarefa consumerTask na seguinte função começa como a tarefa do produtor, abrindo a fila de mensagens com uma chamada para mq_open. Mas a fila é aberta com permissão somente leitura pelo segundo sinalizador de parâmetro, O_RDONLY, porque a tarefa do consumidor recebe apenas mensagens. Desde a bandeira O_NONBLOCK não é especificado no segundo parâmetro para mq_aberto, se nenhuma mensagem estiver disponível na fila quando o consumidor chamar a função de recebimento da fila de mensagens, ela será bloqueada até que uma mensagem esteja presente. A função mq_open retorna um descritor de fila de mensagens usado em acessos subsequentes à fila de mensagens.

A tarefa do consumidor então entra em um loop infinito onde espera por uma mensagem chamando mq_receive. Assim que uma mensagem estiver disponível, ela é copiada * (pelo Linux) para o rcvMsg

* Pode haver uma implementação da rotina da fila de mensagens que não usa uma cópia.

variável. o consumerTask em seguida, emite a mensagem e alterna o LED verde. A tarefa então volta a esperar pela próxima mensagem.

```
# inclui <stdio.h>

/ *****
*
* Função:      consumerTask
*
* Descrição: Esta tarefa espera por uma mensagem da tarefa do produtor.
*              Assim que a mensagem for recebida pela fila de mensagens,
*              a tarefa gera uma mensagem.
*
* Notas:
*
* Retorna:      Nenhum.
*
***** /
void consumerTask (void * param) {

    mqd_t messageQueueDescr;
    msgbuf_t rcvMsg;

    / * Abra a fila de mensagens existente usando o sinalizador somente leitura porque
    * esta tarefa só recebe mensagens. Defina a fila para bloquear se
    * uma mensagem não está disponível. * /
    messageQueueDescr = mq_open (messageQueuePath, O_RDONLY);

    enquanto (1)
    {
        / * Aguarde uma nova mensagem. * / mq_receive
        (messageQueueDescr, & rcvMsg.buf [0], 4, NULL);

        printf ("Botão SW0 pressionado %d vezes. \n", rcvMsg.count);

        ledToggle ();
    }
}
```

O Linux inclui várias outras funções de API que oferecem funcionalidade adicional para os mecanismos abordados anteriormente, bem como outros tipos de mecanismos de sincronização. Outros mecanismos incluem variáveis de condição e bloqueios de leitor-gravador. Variáveis de condição permitem que várias tarefas esperem até que um evento específico ocorra ou até que a variável atinja um valor específico. Os bloqueios de leitor-gravador permitem que várias tarefas leiam dados simultaneamente, enquanto qualquer tarefa de gravação de dados tem acesso exclusivo.

O tratamento de interrupções no Linux é mais complexo do que aquele encontrado nos RTOS. Por esse motivo, omitimos um exemplo de interrupção usando Linux. Para uma melhor compreensão do tratamento de interrupções do Linux, dê uma olhada em *Drivers de dispositivos Linux*, por Alessandro Rubini e Jonathan Corbet (O'Reilly).

Estendendo Funcionalidade

Kramer: *É apenas uma baixa para eles.*

Jerry: *Como é uma baixa?*

Kramer: *Eles apenas descartam.*

Jerry: *Gravar o quê?*

Kramer: *Jerry, todas essas grandes empresas dão baixa em tudo.*

Jerry: *Você nem sabe o que é uma baixa contábil.*

Kramer: *Você?*

Jerry: *Não. Eu não.*

Kramer: *Mas eles fazem e são eles que descartam.*

- a série de televisão “Seinfeld”, episódio “The Package”

Neste capítulo, apresentamos tecnologias adicionais de hardware e software que você pode encontrar em sistemas embarcados. Começamos com uma olhada em um par de barramentos de interconexão de chips chamados I²C e SPI. A seguir, apresentamos a lógica programável, incluindo FPGAs. E, finalmente, damos uma olhada na adição de uma rede TCP / IP.

Periféricos Comuns

Conforme você trabalha em mais e mais sistemas embarcados, encontrará diferentes periféricos que terá de usar. Nesta seção, daremos uma olhada em alguns dos periféricos embutidos comuns que você provavelmente encontrará. Às vezes, é necessário implementar esses protocolos inteiramente em software (consulte a barra lateral “Serial Bit Banging” posteriormente neste capítulo).

Os barramentos seriais podem ser *assíncrono* ou *síncrono*. Em uma conexão serial assíncrona, os dados são enviados sem usar um sinal de relógio de tempo comum. Para alinhar o receptor com o remetente, existe algum tipo de condição de início para indicar quando a transmissão começa e uma condição de parada para indicar o fim da transmissão. Uma conexão serial síncrona normalmente usa um sinal de relógio separado para sincronizar o receptor com o remetente. As conexões síncronas também podem usar uma condição de início e parada para sincronizar o receptor e o remetente, após a qual o remetente deve enviar caracteres um após o outro.

Uma interface serial que pode enviar e receber dados ao mesmo tempo é chamada *full-duplex*. Uma interface serial que deve alternar entre enviar e receber dados é chamada *meio duplex*.

Barramento de Circuito Integrado

Um dos barramentos seriais comuns usados em sistemas embarcados é o *Barramento de circuito integrado*, comumente referido como *I²C* (pronunciado “olho quadrado ver”) ônibus. Esse barramento é comum em sistemas embarcados porque não exige muitos recursos de hardware e é ideal para comunicações de baixa velocidade e curta distância. O *I²C* barramento C, criado pela Philips, é um sistema de comunicação de dois fios (dados e relógio). O *I²C* inclui o endereçamento de dispositivos individuais (até 127 no modo padrão, 1024 no modo estendido) para permitir vários dispositivos no mesmo barramento.

Várias taxas máximas de dados são suportadas por diferentes revisões do *I²C* Especificação C. Essas taxas de dados são de até 100 Kbps para o modo padrão, 400 Kbps para o modo rápido e 3,4 Mbps para o modo de alta velocidade. Dispositivos com taxas de dados diferentes podem ser combinados no mesmo barramento. O *I²C* A especificação C pode ser encontrada online em <http://www.semiconductors.philips.com>.

O processador PXA255 inclui um *I²C* Unidade de interface de barramento C. Informações adicionais sobre isso pode ser encontrado no *Manual do desenvolvedor do processador PXA255*.

Alguns dispositivos que normalmente contêm *I²C* As interfaces de barramento C são EEPROMs e chips de relógio em tempo real. A Figura 13-1 mostra um exemplo *I²C* Barramento C com vários dispositivos.

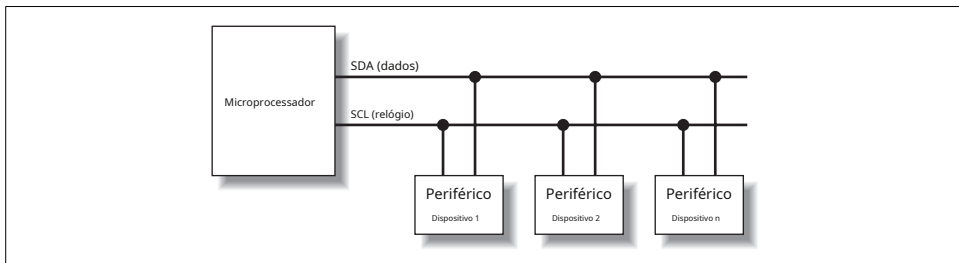


Figura 13-1. Exemplo *I²C* Estrutura do barramento C

Os dois *I²C* Os sinais de barramento C são dados seriais (SDA) e serial relógio (SCL). Os mestres no barramento o ônibus inicia todas as transferências; outros dispositivos no são chamados de escravos. No Figura 13-1, o microprocessador é o mestre e os outros dispositivos são os escravos. Tanto o mestre quanto os escravos podem receber e transmitir dados no barramento.

O mestre inicia as transações no barramento e controla o sinal do relógio. Por causa disso, um dispositivo escravo precisa de uma maneira de impedir o mestre durante uma transação. Quando um escravo retém o dispositivo mestre para realizar o controle de fluxo nos dados de entrada, ele é chamado *alongamento do relógio*. Durante este tempo, o escravo mantém a linha do relógio reduzida até que esteja pronto para continuar com a transação. É importante que todos os dispositivos principais

suporta esse recurso. A Figura 13-2 é o formato de um I₂ Transação de barramento C. Todos os dados no barramento são comunicados pelo bit mais significativo primeiro (MSB).

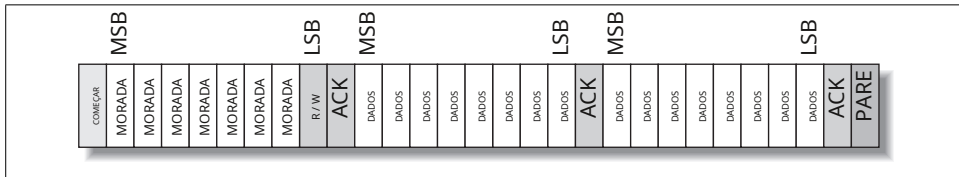


Figura 13-2. Formato de uma transação em um I₂ Ônibus C

Um eu 2 A transação de dados do barramento C começa com o mestre iniciando uma condição inicial. Uma condição de início ocorre quando o mestre causa uma transição de alto para baixo na linha de dados enquanto a linha do relógio é mantida alta.

Em seguida, o endereço exclusivo de 7 bits do dispositivo é enviado pelo dispositivo mestre. Cada dispositivo no barramento verifica este endereço com o seu próprio para determinar se o mestre está se comunicando com ele. Os dispositivos escravos C vêm com um endereço de dispositivo predefinido. Os bits mais baixos desse endereço às vezes são configuráveis no hardware.

Em seguida, o mestre emite o bit de leitura ou gravação. Se o bit for alto, a transação é uma leitura, onde os dados vão do dispositivo escravo para o mestre. Se o bit for baixo, a transação é uma gravação do mestre para o dispositivo escravo.

O dispositivo escravo então envia um bit de reconhecimento. Para o bit de reconhecimento, a linha de dados é mantida baixa enquanto o sinal do clock é alto. Os bits de reconhecimento sempre são enviados pelo dispositivo escravo.

Agora, dependendo do tipo de transação (leitura ou escrita), o transmissor (que pode ser escravo ou mestre) passa a enviar um byte de dados, a partir do MSB. No final do byte de dados, o receptor (escravo ou mestre) emite um bit de reconhecimento. Esse padrão continua até que todos os dados sejam transferidos.

A transação termina com o dispositivo mestre causando um *condição de parada*. Uma condição de parada ocorre quando o mestre causa uma transição de baixo para alto na linha de dados enquanto mantém a linha do relógio alta. Observe que eu 2 O protocolo C oferece suporte a vários mestres.

Interface Periférica Serial

Outro barramento serial comumente usado em sistemas embarcados é o Motorola *interface periférica serial* (SPI, pronunciado “espião”). Outra interface serial semelhante é *Microfio*, registrado pela National Semiconductor, que é um subconjunto restrito da SPI.

O SPI pode operar com taxas de dados de até 1 Mbps. Além disso, pode operar em modo full-duplex, tornando-o mais adequado do que o I²C para aplicativos em que os dados fluem constantemente. O I²C usa menos sinais do que o SPI, pode se comunicar a vários pés (um metro ou

Batida de bit serial

Alguns sistemas embarcados não possuem hardware dedicado a executar todas as funções de interface de uma interface serial. Nesse caso, os sinais de E / S de uso geral são conectados a dispositivos externos e cabe ao software implementar o protocolo de comunicação. *Batida de bits* é uma gíria para o processo de transferência de dados seriais sob controle de software.

O bit banging pode ser usado para qualquer interface serial, incluindo I²C, SPI e até UARTs. Ao implementar uma interface serial via bit banging, o software controla todos os sinais para operar a interface. Por exemplo, a seguinte função, `serialSendData`, demonstra o envio de um byte de dados, `dataByte`, em uma interface serial.

A função começa com o `bitMask` definido para o bit mais significativo. O E se instrução determina se o bit é alto ou baixo e define o sinal de dados GPIO de acordo. Para cada bit transmitido na interface serial, o sinal de clock GPIO é alternado. O enquanto o loop continua até que todos os oito bits do byte de dados tenham sido enviados.

```
void serialSendData (uint8_t dataByte) {

    uint8_t bitMask;

    /* Loop através de cada bit no byte de dados. */ for
    (bitMask = 0x80; bitMask != 0x00; bitMask >> = 1) {

        /* Ver se o próximo bit de dados é alto ou baixo
        * e defina a linha de dados GPIO de acordo. */
        if (dataByte & bitMask)
            gpioDataSignal = 1;
        outro
            gpioDataSignal = 0;

        /* Alterna a linha GPIO do relógio.
        * / gpioClkSignal = 1;

        /* Atraso pelo período de tempo adequado. */

        gpioClkSignal = 0;
    }
}
```

mais), e tem uma especificação bem definida. O SPI, por outro lado, tem um comprimento de comunicação limitado de alguns centímetros. O SPI não oferece suporte a vários mestres nem especifica um esquema de endereçamento de dispositivo; portanto, sinais de hardware adicionais são necessários para selecionar escravos específicos. Essa falta de endereçamento pode ser um benefício porque reduz a sobrecarga em interfaces SPI de mestre único e escravo único.

A Figura 13-3 mostra um exemplo de uma estrutura de barramento SPI. Nesta figura, há um mestre solteiro e dois dispositivos escravos conectados ao barramento SPI.

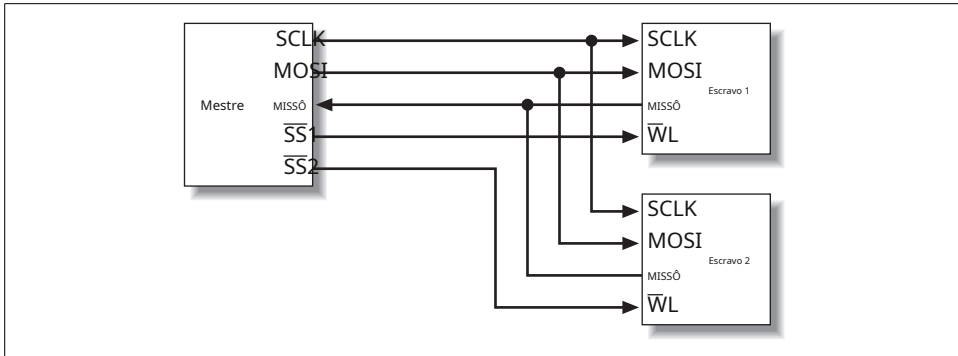


Figura 13-3. Exemplo de estrutura de barramento SPI

O barramento SPI inclui $3 + N$ sinais, onde N é o número de escravos no ônibus. Na Figura 13-3, há dois escravos, portanto, o barramento SPI requer cinco sinais. Esses sinais são relógio serial (SCLK), sinal de dados Master Out Slave In (MOSI), sinal de dados Master In Slave Out (MISO) e Slave Select (SS1 e SS2). O sinal de seleção de escravo é usado para selecionar com qual escravo o mestre deseja se comunicar. Neste exemplo, como existem dois dispositivos escravos, dois sinais de seleção escravos são necessários. Isso mostra como recursos de hardware adicionais são necessários na interface SPI para acomodar a falta de endereçamento no protocolo.

O SPI opera em modo full-duplex. Durante as comunicações, o dispositivo mestre inicia uma transação gerando um relógio e selecionando um dispositivo usando o sinal de seleção escravo. Os dados são então transferidos em ambas as direções nas linhas MOSI e MISO. Como os dados são transferidos em ambas as direções, cabe a cada dispositivo saber se os dados recebidos são significativos, ou seja, se a transação foi de leitura, gravação ou ambos.

Outra diferença entre SPI e I^2C é que o SPI não inclui nenhum tipo de mecanismo de confirmação. O transmissor não tem como saber que os dados foram recebidos no destino. Também não há mecanismo para controle de fluxo incluído no SPI. Se o controle de fluxo for necessário, ele deve ser implementado fora do SPI.

Lógica Programável

Os chips lógicos programáveis são amplamente usados em sistemas embarcados. Esses dispositivos permitem que os engenheiros de hardware executem várias tarefas (como lógica de seleção de chip) no hardware. Como um programador, você pode não projetar a lógica dentro do dispositivo programável, mas pode precisar escrever um driver para baixar o programa em um FPGA.

Esta seção fornecerá uma base melhor para se comunicar com os projetistas de hardware para determinar quais funções devem ser implementadas em lógica dedicada, lógica programável e / ou software. Descobrimos que existem razões válidas para escolher cada uma dessas três técnicas de implementação. Você deve prestar muita atenção aos requisitos do aplicativo específico para tomar a decisão correta.

Muitos tipos de lógica programável estão disponíveis. A gama atual de ofertas inclui tudo, desde pequenos dispositivos capazes de implementar apenas um punhado de equações lógicas a enormes dispositivos que podem conter um núcleo de processador inteiro (mais periféricos!). Além dessa incrível diferença de tamanho, as arquiteturas também variam muito. Apresentaremos os tipos mais comuns de lógica programável e destacaremos os recursos mais importantes de cada tipo.

Dispositivo Lógico Programável

Na extremidade inferior do espectro está o original *Dispositivo Lógico Programável* (PLD). Os PLDs foram os primeiros chips que puderam ser usados para implementar um projeto lógico digital flexível em hardware. No início, você poderia remover algumas das peças da série 7400 Transistor-Transistor-Logic (TTL) (ANDs, ORs e NOTs) de sua placa e substituí-las por um único PLD. Outros nomes que você pode encontrar para esta classe do dispositivo são *Matriz Lógica Programável* (PLA), *Lógica de matriz programável* (PAL), e *Lógica de matriz genérica* (GAROTA).

Os PLDs são frequentemente usados para decodificação de endereços, onde têm várias vantagens claras sobre as peças TTL da série 7400 que substituíram. Em primeiro lugar, um chip normalmente requer menos área de placa e fiação. Outra vantagem é que o design dentro do chip é flexível, portanto, uma mudança na lógica não requer nenhuma religação da placa. Em vez disso, a lógica de decodificação pode ser alterada simplesmente substituindo o único PLD instalado anteriormente por outra parte que foi programada com a nova lógica booleana.

Dentro de cada PLD está um conjunto de conectadas *macrocélulas*. Essas macrocélulas são normalmente compostas de alguma quantidade de lógica combinatória (portas AND e OR, por exemplo) e um flip-flop. Em outras palavras, uma pequena equação lógica booleana pode ser construída dentro de cada macrocélula. Essa equação combina o estado de um certo número de entradas binárias em uma saída binária e, se necessário, armazena essa saída no flip-flop até a próxima transição do clock. Obviamente, as particularidades das portas lógicas e flip-flops disponíveis são específicas para cada fabricante e família de produtos. Mas a ideia geral é sempre a mesma.

Projetos de hardware para esses PLDs simples são geralmente escritos em linguagens como ABEL ou PALASM (os equivalentes de hardware da linguagem assembly) ou desenhados com a ajuda de uma ferramenta de captura esquemática.

Dispositivo lógico programável complexo

À medida que a densidade do chip aumentava, era natural para os fabricantes de PLD evoluir seus produtos em peças maiores (logicamente, mas não necessariamente fisicamente) chamadas *Dispositivos lógicos programáveis complexos* (CPLDs). Para a maioria dos propósitos práticos, os CPLDs podem ser considerados como múltiplos PLDs (mais alguma interconexão programável) em um único chip. A maior capacidade de um CPLD permite que você implemente mais equações lógicas ou um projeto mais complicado. Na verdade, esses chips são grandes o suficiente para substituir dezenas de peças da série 7400.

A Figura 13-4 contém um diagrama de blocos de um CPLD hipotético. Cada um dos quatro blocos lógicos mostrados é equivalente a um PLD. No entanto, em um CPLD real, pode haver mais (ou menos) do que quatro blocos lógicos. A Figura 13-6 é uma versão simplificada. A matriz de comutação permite o roteamento de sinal e a comunicação entre os blocos lógicos. Observe também que esses blocos lógicos são compostos por *ing*, macrocélulas e cabos de interconexão assim como um PLD comum.

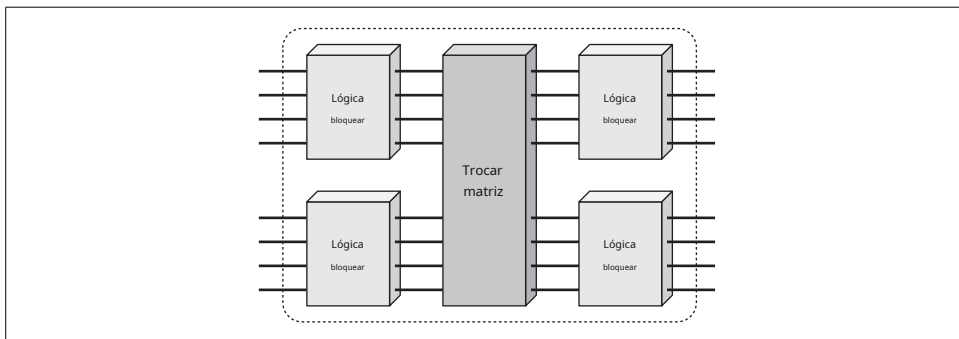


Figura 13-4. Estrutura interna do CPLD

Como os CPLDs podem conter projetos maiores do que os PLDs, seus usos potenciais são mais variados. Eles ainda são usados às vezes para aplicativos simples, como decodificação de endereço, mas mais frequentemente contêm lógica de controle de alto desempenho ou máquinas de estado finito. Na extremidade superior (em termos de número de portas), também há muita sobreposição em aplicações potenciais com FPGAs. Por causa de sua arquitetura interna menos flexível, o atraso por meio de um CPLD (medido em nanossegundos) é mais previsível e geralmente mais curto.

Matriz de portas programáveis em campo

UMA *Matriz de portas programáveis em campo* (FPGA) pode ser usado para implementar praticamente qualquer projeto de hardware. Um uso é prototipar um pedaço de hardware que eventualmente encontrará seu caminho em um ASIC. No entanto, não há nada que diga que o FPGA não pode permanecer no produto final, e freqüentemente permanece. Se isso acontecerá dependerá dos pesos relativos do custo de desenvolvimento e do custo de produção para um projeto específico, bem como da necessidade de atualizar o design do hardware após o produto ser enviado. (Custa significativamente mais desenvolver um ASIC, mas o custo por chip será menor se você produzi-los em quantidades suficientes. A compensação de custo envolve o número esperado de chips a serem produzidos e a probabilidade esperada de bugs de hardware e / ou alterações. Isso torna a análise de custos bastante complicada.)

O desenvolvimento histórico da tecnologia em um FPGA foi distinto da evolução do PLD / CPLD que acabamos de descrever. Isso fica aparente quando você olha para as estruturas internas. A Figura 13-5 ilustra uma arquitetura FPGA típica. Existem três partes principais em sua estrutura: blocos lógicos, interconexão e blocos de E / S. Os blocos de E / S formam um anel em torno da borda externa da peça. Cada um deles fornece selecionáveis individualmente

entrada, saída ou acesso bidirecional a um dos pinos GPIO no exterior do pacote FPGA. Dentro do anel de blocos de E / S encontra-se uma matriz retangular de blocos lógicos. E, finalmente, conectando blocos lógicos a blocos lógicos e blocos de E / S a blocos lógicos, a fiação de interconexão programável passa pelo array.

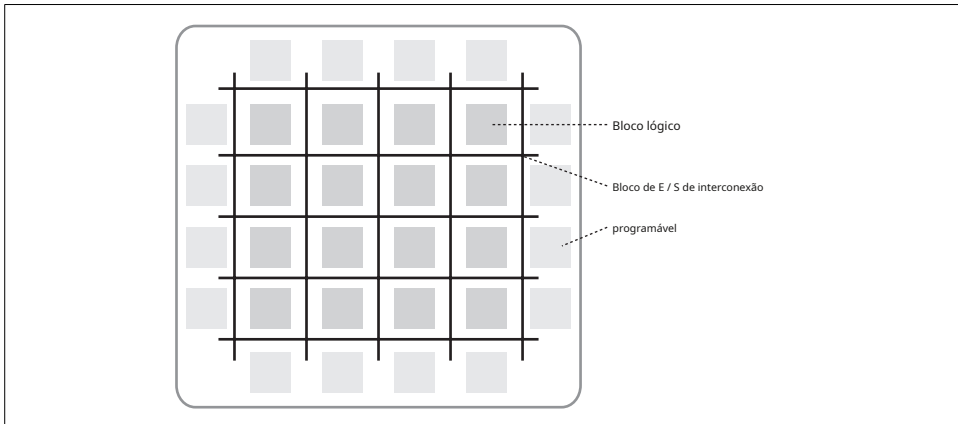


Figura 13-5. Estrutura interna FPGA

Os blocos lógicos em um FPGA podem ser tão pequenos e simples quanto as macrocélulas em um PLD (a chamada arquitetura de baixa granularidade) ou maiores e mais complexos (uma arquitetura de baixa granulação). No entanto, os blocos lógicos em um FPGA nunca são tão grandes quanto um PLD inteiro, como são os blocos lógicos de um CPLD. Lembre-se de que os blocos lógicos de um CPLD contêm várias macrocélulas. Mas os blocos lógicos em um FPGA geralmente nada mais são do que um par de portas lógicas ou uma tabela de consulta e um flip-flop.

Por causa de todos os flip-flops extras, a arquitetura de um FPGA é muito mais flexível do que a de um CPLD. Isso torna os FPGAs melhores em aplicativos com registros pesados e em pipeline. Eles também são frequentemente usados no lugar de uma solução de processador mais software, particularmente onde o processamento de fluxos de dados de entrada deve ser executado em um ritmo muito rápido. Além disso, os FPGAs geralmente são mais densos (mais portas em uma determinada área) do que seus primos CPLD, portanto, são a escolha certa para projetos lógicos maiores.

PulseWidthModulation

Modulação de largura de pulso (PWM) é uma técnica poderosa para controlar circuitos analógicos com saídas digitais de um processador. O PWM é empregado em uma ampla variedade de aplicações, desde medição e comunicações até controle de energia e conversão.

Circuitos analógicos

Um sinal analógico tem um valor que varia continuamente, com resolução efetivamente infinita em tempo e magnitude. Uma bateria de 9 V é um exemplo de dispositivo analógico, em que

sua tensão de saída não é precisamente 9 V, mas muda com o tempo e pode assumir qualquer valor real de 0,0 V a cerca de 9,5 V. Da mesma forma, a quantidade de corrente retirada de uma bateria não é limitada a um conjunto finito de valores possíveis. Os sinais analógicos são distinguíveis dos sinais digitais porque os últimos sempre tomam valores apenas de um conjunto finito de possibilidades predeterminadas, como o conjunto de dois valores (0 V, 5 V).

As tensões e correntes analógicas podem ser usadas para controlar coisas diretamente, como o volume do rádio de um carro. Em um rádio analógico simples, um botão é conectado a um resistor variável. Conforme você gira o botão, a resistência aumenta ou diminui. À medida que isso acontece, a corrente que flui através do resistor aumenta ou diminui. Isso pode alterar diretamente a quantidade de voltagem que impulsiona os alto-falantes, aumentando ou diminuindo o volume.

Por mais intuitivo e simples que o controle analógico possa parecer, nem sempre é economicamente atraente ou prático. Por um lado, os circuitos analógicos tendem a se desviar com o tempo e podem, portanto, ser muito difíceis de ajustar. Os circuitos analógicos de precisão, que resolvem esse problema, podem ser muito grandes, pesados (basta pensar em um equipamento estéreo doméstico antigo) e caros. Os circuitos analógicos também podem ficar muito quentes; a potência dissipada é proporcional à tensão através dos elementos ativos multiplicada pela corrente através deles. Os circuitos analógicos também podem ser sensíveis a ruídos. Por causa de sua resolução infinita, qualquer perturbação ou ruído em um sinal analógico altera necessariamente o valor atual.

Controle digital

O controle digital de circuitos analógicos pode reduzir drasticamente os custos do sistema e o consumo de energia. Além do mais, muitos microcontroladores e DSPs já incluem controladores PWM onchip, facilitando a implementação.

Resumindo, o PWM é uma forma de codificar digitalmente os níveis de sinal analógico. Por meio do uso de contadores de alta resolução, o *ciclo de trabalho* (a porcentagem de tempo que um sinal é afirmado) de uma onda quadrada é modulada para codificar um nível de sinal analógico específico. O sinal PWM ainda é digital porque, a qualquer momento, a alimentação CC completa está totalmente ligada ou totalmente desligada. A fonte de tensão ou corrente é fornecida à carga analógica por meio de uma série repetida de pulsos de ativação e desativação. o *na hora* é o tempo durante o qual a alimentação DC é aplicada à carga, e o *fora do horário* é o período durante o qual essa alimentação é desligada. Dado um período suficientemente pequeno do sinal PWM, qualquer valor analógico pode ser codificado com PWM.

Para ajudar a explicar a relação entre a codificação digital e os valores analógicos, mostramos três sinais PWM diferentes na Figura 13-6. A Figura 13-6 (a) mostra uma saída PWM em um ciclo de trabalho de 10 por cento. Ou seja, o sinal está ativado em 10% do período e desativado nos outros 90%. As Figuras 13-6 (b) e 13-6 (c) mostram saídas PWM em ciclos de trabalho de 50 por cento e 90 por cento, respectivamente. Essas três saídas PWM codificam três valores de sinal analógico diferentes, a 10 por cento, 50 por cento e 90 por cento da intensidade total. Se, por exemplo, a alimentação for de 9 V e o ciclo de trabalho for de 10 por cento, o resultado será um sinal analógico de 0,9 V.

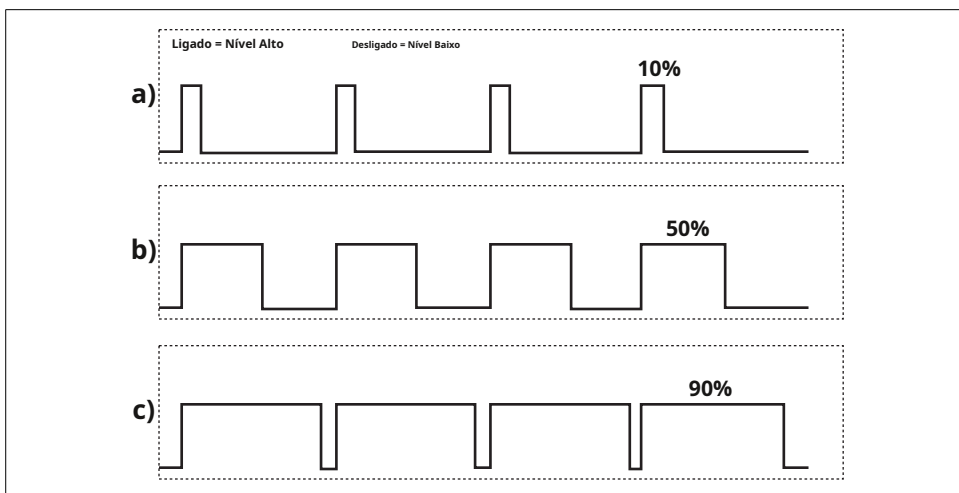


Figura 13-6. Sinais PWM com ciclos de trabalho variáveis

Na Figura 13-7, mostramos um circuito simples que pode ser acionado usando PWM. Nesta figura, uma bateria de 9 V alimenta uma lâmpada incandescente. Se a chave que conecta a bateria e a lâmpada estiver fechada por 50 ms, a lâmpada recebe os 9 V completos durante esse intervalo. Se abrirmos o interruptor pelos próximos 50 ms, a lâmpada recebe 0 V. Se repetirmos este ciclo 10 vezes por segundo, a lâmpada acenderá como se estivesse conectada a uma bateria de 4,5 V (50 por cento de 9 V) . Dizemos que o ciclo de trabalho é de 50 por cento e a frequência de modulação é de 10 Hz. (Observe que não estamos defendendo que você realmente ligue uma lâmpada dessa forma; apenas consideramos este um exemplo fácil de entender.)

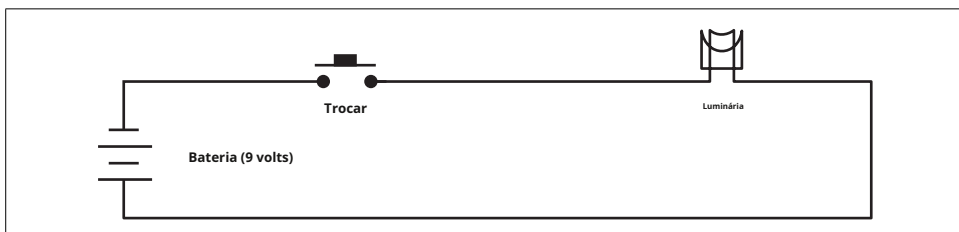


Figura 13-7. Um circuito PWM simples

A maioria das cargas requer uma frequência de modulação muito mais alta do que 10 Hz. Imagine que nossa lâmpada foi acesa por cinco segundos, depois apagada por cinco segundos e depois acesa novamente. O ciclo de trabalho ainda seria de 50 por cento, mas a lâmpada pareceria muito acesa nos primeiros cinco segundos e nem um pouco acesa nos próximos. Para que a lâmpada veja uma tensão de 4,5 V, o período do ciclo deve ser curto em relação ao tempo de resposta da carga a uma mudança no estado da chave. Para obter o efeito desejado de uma lâmpada dimmer (mas sempre acesa), é necessário aumentar a frequência de modulação. O mesmo é verdade em outras aplicações de PWM. As frequências de modulação comuns variam de 1 a 200 kHz.

Uma das vantagens do PWM é que o sinal permanece digital desde o processador até o sistema controlado; nenhuma conversão digital para analógico é necessária. Manter o sinal digital minimiza os efeitos do ruído. O ruído pode afetar um sinal digital apenas se o ruído for forte o suficiente para alterar um 1 lógico para um 0 lógico ou vice-versa.

Essa maior imunidade a ruído é outro benefício de escolher o PWM em vez do controle analógico e é o principal motivo pelo qual o PWM às vezes é usado para comunicações. Mudar de um sinal analógico para PWM pode aumentar drasticamente o comprimento de um canal de comunicação. Na extremidade de recepção, uma rede adequada de resistor-capacitor (RC) ou indutor-capacitor (LC) pode remover a onda quadrada de alta frequência modulante e retornar o sinal para a forma analógica.

O PWM encontra aplicação em uma variedade de sistemas. Como um exemplo concreto, considere um freio controlado por PWM. Para simplificar, um freio é um dispositivo que fixa com força em algo. Em muitos freios, a quantidade de pressão de aperto (ou potência de parada) é controlada com um sinal de entrada analógico. Quanto mais tensão ou corrente for aplicada ao freio, mais pressão ele exercerá.

A saída de um controlador PWM pode ser conectada a uma chave entre a alimentação e o freio. Para produzir mais potência de parada, o software precisa apenas aumentar o ciclo de trabalho da saída PWM. Se uma quantidade específica de pressão de frenagem é desejada, medições precisam ser feitas para determinar a relação matemática entre o ciclo de trabalho e a pressão. (E as fórmulas ou tabelas de pesquisa resultantes seriam ajustadas para temperatura operacional, desgaste de superfície e assim por diante.)

Para definir a pressão no freio para, digamos, 100 psi, o software faria uma pesquisa reversa para determinar o ciclo de trabalho que deveria produzir aquela quantidade de força. Em seguida, ele definiria o ciclo de trabalho PWM para o novo valor e o freio responderia de acordo. Se um sensor estiver disponível no sistema, o ciclo de trabalho pode ser ajustado, sob o controle de malha fechada, até que a pressão desejada seja alcançada com precisão.

Rede para todos os dispositivos, grandes e pequenos

Incorporar suporte de rede em um dispositivo embutido pode parecer uma tarefa assustadora à primeira vista. No entanto, até mesmo um sistema embarcado mais antigo pode ser atualizado com uma pilha de rede de software para estender seu conjunto de recursos e incorporar conveniências modernas, como enviar e-mail a um administrador quando ocorrem alarmes e um servidor da web para fornecer uma interface de usuário remota acessível a partir de qualquer navegador da web.

Certamente, há custos para incluir uma pilha de rede. A interface de rede (como Ethernet) pode rapidamente se tornar cara e complicada. Você pode precisar de hardware extra (com custos adicionais para chips e conectores, espaço na placa e consumo de energia) e software (com novos drivers). No entanto, esse não precisa ser o caso. Você pode executar um protocolo SLIP (Serial Line Interface Protocol) ou PPP (Point to Point Protocol) em uma porta UART para a interface de rede. A maioria dos processadores embarcados inclui pelo menos um UART, e SLIP e PPP são protocolos muito básicos para

implementar - não mais difícil do que o programa Monitorar e Controlar que vimos no Capítulo 9.

As demandas de uma pilha de rede podem fazer com que você se preocupe com a necessidade de muitos recursos do sistema. O poder de processamento e a memória necessários para acomodar o suporte de rede podem ser bastante reduzidos escolhendo a pilha de rede adequada. Várias pilhas de rede de software destinadas a sistemas embarcados, onde os ciclos do processador e a memória são limitados, estão atualmente disponíveis na comunidade de código aberto.

A próxima seção tem como objetivo fornecer uma visão geral de alguns dos benefícios de adicionar suporte de rede e inclui algumas opções de pilhas de rede com foco em recursos que são ideais para incorporação em sistemas embarcados.

Benefícios do suporte de rede

Adicionar suporte de rede, Protocolo de Controle de Transmissão / Protocolo de Internet (TCP / IP) e os outros protocolos de suporte agrupados neste conjunto permitem o acesso padronizado a um dispositivo. O TCP / IP permite que um dispositivo se comunique usando o protocolo nativo da maioria das infraestruturas de rede, o que permite que o dispositivo seja acessado de um PC, PDA ou telefone celular habilitado para web. O dispositivo habilitado para rede pode se comunicar por meio de uma rede local (LAN) ou pela Internet global.

Para obter informações adicionais sobre protocolos de rede, dê uma olhada na série de três volumes *TCP / IP ilustrado*, por Richard Stevens (Addison-Wesley). Outro recurso é *Guia TCP / IP: Uma referência abrangente e ilustrada de protocolos da Internet*, por Charles Kozierok (No Starch Press).

Cada dispositivo pode ser adaptado para usar o protocolo ideal para transmitir informações pela rede. O User Datagram Protocol (UDP) de muito baixo overhead pode ser usado para dados que não requerem confirmação, enquanto o TCP está disponível para dados que precisam de confirmação de seu recebimento do destino.

O suporte de rede pode melhorar o conjunto de recursos básicos que um dispositivo incorporado é capaz de suportar. Por exemplo, se ocorrer uma condição de alarme no sistema, o dispositivo pode gerar um e-mail e enviá-lo para notificar a operadora de rede sobre o erro. A operadora de rede pode, então, realizar rapidamente a manutenção necessária para corrigir o problema e fazer o sistema voltar ao normal - reduzindo o tempo de inatividade do sistema ao mínimo.

Outro benefício da rede é que *gestão baseada na web* pode ser facilmente incorporado no dispositivo. O gerenciamento baseado na web permite a configuração e o controle de um sistema usando protocolos TCP / IP e um navegador da web. Um técnico pode se conectar diretamente a um dispositivo para configuração e monitoramento usando um PDA padrão equipado com um navegador da web padrão. O servidor da web do dispositivo e as páginas HyperText Markup Language (HTML) são a nova interface do usuário para o dispositivo. Muitos dispositivos hoje, como roteadores de modem a cabo e firewalls, incluem uma interface da Web para configuração.

Figura 13-8 sh
servidor que ena

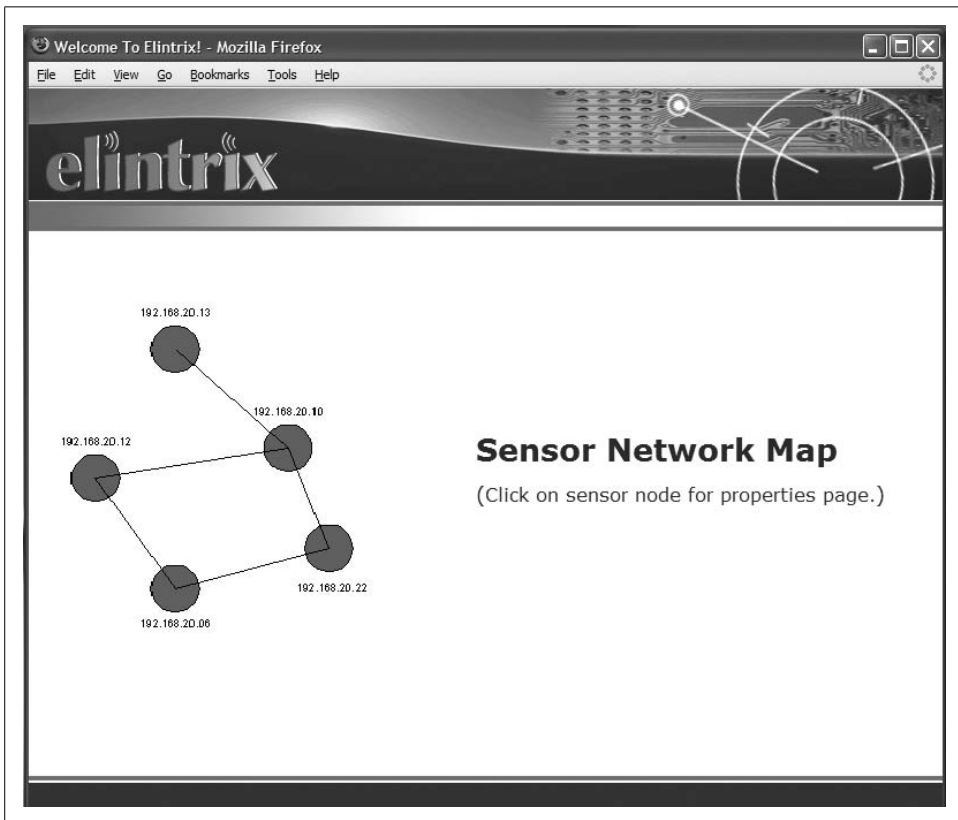


Figura 13-8. Exemplo de gerenciamento baseado na web com uma pilha de rede

Soluções de rede para sistemas embarcados

Existem várias soluções de pilha de rede comerciais e de código aberto disponíveis hoje. A maioria das pilhas oferece suítes de protocolo padrão e algumas incluem aplicativos de exemplo para ajudá-lo a estender o conjunto de recursos básicos do seu dispositivo. A Figura 13-9 mostra alguns dos componentes de protocolo de rede comuns incluídos na maioria das pilhas de rede. Listaremos ainda mais protocolos posteriormente neste capítulo, conforme descrevemos soluções de rede específicas.

Um dos segredos para decidir qual pilha se ajusta melhor ao seu dispositivo é determinar os requisitos de recursos de que o software precisa para operar. A quantidade de dados que o dispositivo transmite e recebe durante as sequências de comunicação também deve ditar a solução certa para o seu projeto. Por exemplo, se o seu dispositivo é um nó sensor que acorda a cada hora para transmitir alguns bytes de dados, uma rede compacta

Vantagens do gerenciamento baseado na web

O gerenciamento baseado na Web usa TCP para enviar pacotes pela rede, o que fornece um método confiável para a transferência de dados usando um esquema de confirmação e retransmissão.

O gerenciamento baseado na Web depende de um navegador padrão para a interface do lado do cliente. Isso oferece aos usuários uma interface padrão com a qual estão familiarizados e se sentem confortáveis para usar. Um dispositivo habilitado para web contém um servidor que simplesmente envia as páginas da web para o usuário quando o dispositivo é acessado. O navegador lida com a tarefa de renderizar as imagens e apresentar as imagens gráficas ao usuário.

O Protocolo Simples de Gerenciamento de Rede (SNMP) tem sido o padrão para monitorar e controlar dispositivos em rede. A integração de uma pilha de rede permite que o gerenciamento baseado na web seja utilizado. O SNMP pode ser implementado nos menores sistemas embarcados; no entanto, ele tem várias deficiências.

Uma deficiência do SNMP é que ele usa UDP para a transmissão de pacotes em uma rede. UDP é um protocolo sem conexão, não confiável e não possui mecanismo para a retransmissão de pacotes que são perdidos. Pode haver consequências desastrosas se informações cruciais sobre a integridade do sistema forem perdidas e o remetente não tiver como saber que não foram recebidas. O SNMP também muitas vezes requer o uso de redes complexas e caras software de gestão de trabalho do lado do cliente.

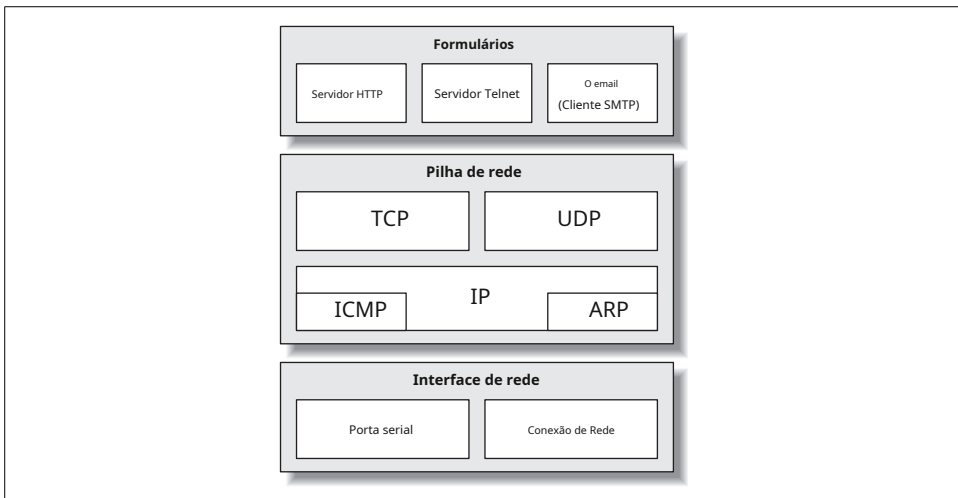


Figura 13-9. Componentes de protocolo de rede comuns

a implementação da pilha é ideal. Em contraste, um sistema de monitoramento de vídeo que está constantemente transmitindo grandes quantidades de dados pode precisar de uma implementação de pilha que ofereça melhor gerenciamento de buffer de pacote.

As implementações focadas em pequenos dispositivos incorporados permitem que a rede seja integrada até mesmo nos sistemas com recursos mais limitados. É importante que a implementação de pilha de rede “leve” que você escolher permita a comunicação com dispositivos TCP / IP padrão em escala real. Uma implementação especializada para um determinado dispositivo e rede pode causar problemas, limitando sua capacidade de estender os recursos de rede do dispositivo em outras redes genéricas.

Sempre é possível sair e rolar sua própria pilha de rede. No entanto, dada a ampla gama de soluções disponíveis na comunidade de código aberto, aproveitar a tecnologia existente geralmente é a melhor escolha e permite que você avance rapidamente em seu desenvolvimento.

Na lista de pilhas de rede de software a seguir, nos concentramos em soluções de código aberto que são ideais para dispositivos incorporados com recursos limitados. Esta lista não pretende ser abrangente, mas sim um ponto de partida para uma investigação mais aprofundada. Todas as pilhas de rede listadas incluem suporte ao protocolo TCP / IP. Uma breve descrição de cada pilha de rede está incluída; para obter informações mais detalhadas, consulte as páginas da web específicas listadas. (O código de rede BSD é uma espécie de padrão da indústria e, portanto, aparece como a base de muitos dos projetos listados.)

lwIP (<http://savannah.nongnu.org/projects/lwip>)

Essa pilha de “IP leve” é uma implementação TCP / IP simplificada, mas em grande escala. O lwIP foi projetado para ser executado em um sistema multithread com aplicativos em execução em threads simultâneos, mas também pode ser implementado em um sistema sem sistema operacional. Além do suporte ao protocolo TCP / IP padrão, o lwIP também inclui o protocolo ICMP (Internet Control Message Protocol), DHCP (Dynamic Host Configuration Protocol), ARP (Address Resolution Protocol) e UDP. Ele oferece suporte a várias interfaces de rede local. O lwIP tem uma configuração flexível que permite que seja facilmente usado em uma ampla variedade de dispositivos e dimensionado para se adequar a diferentes requisitos de recursos.

OpenTCP (<http://www.opentcp.org>)

Adaptado para microcontroladores de 8 e 16 bits, o OpenTCP incorpora ICMP, DHCP, protocolo Bootstrap (BOOTP), ARP e UDP. Este pacote também inclui vários aplicativos, como um servidor Trivial File Transfer Protocol (TFTP), um cliente Post Office Protocol versão 3 (POP3) para recuperar e-mail, suporte a protocolo SMTP para enviar e-mail e um protocolo de transferência de hipertexto (HTTP) servidor para gerenciamento de dispositivos baseado na web.

TinyTCP (<http://www.unusualresearch.com/tinytcp/tinytcp.htm>)

Essa pilha de rede é projetada para ser muito modular e incluir apenas o software exigido pelo sistema. Por exemplo, diferentes protocolos podem ser incluídos com base em sua configuração. O TinyTCP fornece uma biblioteca de soquetes compatível com BSD e inclui ARP, ICMP, UDP, DHCP, BOOTP e Internet Group Management Protocol (IGMP).

uC / IP (<http://ucip.sourceforge.net>)

uC / IP (pronuncia-se mew-kip) * é projetado para microcontroladores e baseado no software de rede BSD. O suporte de protocolo inclui ICMP e protocolo ponto a ponto (PPP).

uIP (<http://www.sics.se/~adam/uip>)

Esta pilha “micro IP” foi projetada para incorporar apenas o conjunto mínimo de componentes necessários para uma solução de pilha TCP / IP completa. Há suporte para apenas uma única interface de rede. Os exemplos de aplicativos incluídos no uIP são SMTP para envio de e-mail, um servidor e cliente Telnet, um servidor HTTP e cliente da Web e resolução de Sistema de Nomes de Domínio (DNS).

Cada pilha de rede foi transferida para vários processadores e microcontroladores. O suporte do driver de dispositivo para a interface de rede varia de pilha para pilha. É uma boa ideia revisar a licença da pilha de rede que você decidir usar, para certificar-se de que ela não impõe limitações ou requisitos indesejáveis ao seu produto.

Além disso, alguns sistemas operacionais incluem ou têm pilhas de rede portadas para eles. Os sistemas operacionais abordados anteriormente neste livro, eCos e Linux embarcado (consulte os Capítulos 11 e 12), oferecem módulos de suporte de rede. O eCos inclui as pilhas de rede OpenBSD, FreeBSD e lwIP, bem como o suporte da camada de aplicativo para muitos dos recursos estendidos discutidos anteriormente. O Embedded Linux, desenvolvido para um ambiente de PC desktop, oferece amplo suporte de rede.

Se uma pilha de rede estiver incluída ou já existir em seu dispositivo, vários servidores da web embutidos estarão disponíveis para incorporar o controle baseado na web. Uma dessas soluções de código aberto é o GoAhead WebServer (<http://www.goahead.com>).

Incorporar uma pilha de rede não é mais uma tarefa assustadora que requer uma enorme quantidade de recursos. As soluções listadas anteriormente podem ser rapidamente aproveitadas e integradas para trazer recursos de rede a qualquer sistema embarcado. Adaptar uma das soluções de pilha de rede às características específicas de um dispositivo garante que o sistema funcionará em seu nível ideal e utilizará melhor os recursos do sistema.

* O *você* na frente de uC / IP e uIP é uma forma tosca, mas comum de representar a letra grega *mu*.

Técnicas de Otimização

As coisas devem ser feitas o mais simples possível, mas não mais simples.

- Albert Einstein

Este capítulo oferece algumas dicas para otimizar o código para reduzir a utilização de recursos. Essas técnicas podem ser divididas em estratégias para reduzir o uso de memória, aumentar a eficiência do código e diminuir os requisitos de energia. A necessidade de versões de baixo custo de nossos produtos leva os designers de hardware a fornecerem memória e capacidade de processamento apenas suficientes para realizar o trabalho.

A maioria das otimizações realizadas no código envolve uma compensação entre a velocidade de execução e o tamanho do código. Seu programa pode ser mais rápido ou menor, mas não ambos. Na verdade, uma melhoria em uma dessas áreas pode ter um impacto negativo na outra. Cabe ao programador decidir qual dessas melhorias é mais importante. Dado esse único pedaço de informação, a fase de otimização do compilador pode fazer a escolha apropriada sempre que uma compensação entre velocidade e tamanho for encontrada.

A primeira etapa da otimização é determinar quais problemas você tem. Você pode ter problemas de tamanho, problemas de velocidade ou ambos. Se você tiver um tipo de problema, pode pedir para o compilador ajudá-lo com a otimização. Se você tiver problemas de tamanho e velocidade, recomendamos deixar o compilador fazer o possível para reduzir o tamanho do seu programa. Em seguida, você pode encontrar o código de tempo crítico ou gargalos (onde o programa está gastando a maior parte de seu tempo) e otimizar manualmente esse código para velocidade. (Em dispositivos alimentados por bateria, cada ciclo de processador desnecessário resulta em tempo de execução reduzido; portanto, a coisa a fazer é otimizar a velocidade em todo o aplicativo.)

A velocidade de execução geralmente é importante apenas dentro de algumas poucas partes do código que têm prazos curtos e aquelas executadas com mais frequência. Há muitas coisas que você pode fazer para melhorar a eficiência dessas seções manualmente. No entanto, o tamanho do código é difícil de influenciar manualmente, e o compilador está em uma posição muito melhor para fazer essa alteração em todos os módulos de software.

Aumento da eficiência do código

No momento em que seu programa estiver funcionando, você já deve saber, ou ter uma boa ideia, quais funções e módulos são os mais críticos para a eficiência geral do código. ISRs, tarefas de alta prioridade, cálculos com prazos em tempo real e funções que fazem uso intensivo de computação ou são chamadas com frequência são todos prováveis candidatos.

Uma ferramenta chamada *analisador*, incluído em alguns pacotes de desenvolvimento de software, pode ser usado para restringir seu foco às rotinas nas quais o programa passa a maior parte (ou muito) de seu tempo. Um criador de perfil coleta e relata estatísticas de execução de um programa. Essas estatísticas de execução incluem o número de chamadas e o tempo total gasto em cada rotina.

Depois de identificar as rotinas que exigem maior eficiência de código, você pode usar as técnicas a seguir para reduzir o tempo de execução. Observe que as técnicas descritas aqui são muito dependentes do compilador. Na maioria dos casos, não existem regras gerais que possam ser aplicadas em todas as situações. A melhor maneira de determinar se uma técnica fornecerá melhorias é examinar a saída do assembly do compilador.

Funções embutidas

Em C99, a palavra-chave na linha pode ser adicionado a qualquer declaração de função. Esta palavra-chave pede ao compilador para substituir todas as chamadas para a função indicada por cópias do código que está dentro. Isso elimina a sobrecarga de tempo de execução associada à chamada de função e é mais eficaz quando a função é usada com frequência, mas contém apenas algumas linhas de código.

As funções embutidas fornecem um exemplo perfeito de como a velocidade de execução e o tamanho do código às vezes são inversamente vinculados. A adição repetitiva do código embutido aumentará o tamanho do seu programa em proporção direta ao número de vezes que a função é chamada. E, obviamente, quanto maior a função, mais significativo será o aumento de tamanho. No entanto, você perderá a sobrecarga de configurar o quadro de pilha se os parâmetros forem passados para a função. O programa resultante é executado mais rápido, mas requer mais memória de código.

Consultas de mesa

UMA trocar declaração é uma técnica de programação comum que deve ser usada com cuidado. Cada teste e salto que compõe a implementação da linguagem de máquina usa um tempo valioso do processador simplesmente para decidir qual trabalho deve ser feito a seguir. Para acelerar as coisas, tente colocar os casos individuais em ordem por sua frequência relativa de ocorrência. Em outras palavras, coloque os casos mais prováveis primeiro e os menos prováveis por último. Isso reduzirá o tempo médio de execução, embora não melhore em nada no tempo do pior caso.

Se houver muito trabalho a ser feito em cada caso, pode ser mais eficiente substitua o todo trocar declaração com uma tabela de ponteiros para funções. Para exemplo, o seguinte bloco de código é um candidato para essa melhoria:

```
enum NodeType {NODE_A, NODE_B, NODE_C};

switch (getNodeType ())
```

```

{
    case NODE_A:
        .
        .
    case NODE_B:
        .
        .
    case NODE_C:
        .
        .
}

```

Para acelerar as coisas, substitua este trocar declaração com a seguinte alternativa. A primeira parte é a configuração: a criação de um array de ponteiros de função. A segunda parte é uma substituição de uma linha para o trocar instrução que executa com mais eficiência.

```

int processNodeA (vazio);
int processNodeB (vazio);
int processNodeC (vazio);

/* Estabelecimento de uma tabela de indicadores para funções. */
int (* nodeFunctions []) () = {processNodeA, processNodeB, processNodeC};

.
.

/* A instrução switch inteira é substituída pela próxima linha. */
status
= nodeFunctions [getNodeType ()] ();

```

Montagem codificada manualmente

Alguns módulos de software são melhor escritos em linguagem assembly. Isso dá ao programador a oportunidade de torná-los o mais eficientes possível. Embora a maioria dos compiladores C produza um código de máquina muito melhor do que o programador médio, um programador de montagem habilidoso e experiente pode fazer um trabalho melhor do que o compilador para uma determinada função.

Por exemplo, em um de nossos projetos anteriores, um algoritmo de filtragem digital foi implementado em C e direcionado para um TI TMS320C30 DSP. O compilador não conseguiu tirar proveito de uma instrução especial que executava exatamente as operações matemáticas necessárias. Substituindo manualmente um para loop do programa C com instruções de montagem embutidas que faziam a mesma coisa, o tempo geral de computação diminuiu em mais de um fator de 10.

Variáveis de registro

A palavra-chave registro pode ser usado ao declarar variáveis locais. Isso pede ao compilador para colocar a variável em um registrador de uso geral em vez de na pilha. Usada com cautela, essa técnica fornece dicas para o compilador sobre as variáveis acessadas com mais frequência e aumentará um pouco o desempenho da função. Quanto mais frequentemente a função é chamada, mais provável é que tal mudança melhore o desempenho do código. Mas alguns compiladores ignoram o registro palavra-chave.

Variáveis globais

Às vezes, é mais eficiente usar uma variável global do que passar um parâmetro para uma função. Isso elimina a necessidade de colocar o parâmetro na pilha antes da chamada da função e retirá-lo assim que a função for concluída. Na verdade, a implementação mais eficiente de qualquer sub-rotina não teria nenhum parâmetro. No entanto, a decisão de usar uma variável global também pode ter alguns efeitos negativos no programa. A comunidade de engenharia de software geralmente desencoraja o uso de variáveis globais em um esforço para promover as metas de modularidade e reentrada, que também são considerações importantes.

Polling

Os ISRs costumam ser usados para melhorar a capacidade de resposta de um programa. No entanto, existem alguns casos raros em que a sobrecarga associada às interrupções realmente causa ineficiência. São casos em que o tempo médio entre as interrupções é da mesma ordem de magnitude que a latência da interrupção. Nesses casos, pode ser melhor usar a pesquisa para se comunicar com o dispositivo de hardware. Mas isso também pode levar a um design de software menos modular.

Aritmética de ponto fixo

A menos que sua plataforma de destino tenha um processador de ponto flutuante, você pagará uma grande penalidade por manipular flutuador dados em seu programa. A biblioteca de ponto flutuante fornecida pelo compilador contém um conjunto de sub-rotinas de software que emulam as instruções de ponto flutuante. Muitas dessas funções demoram muito para serem executadas em relação às suas contrapartes inteiras e também podem não ser reentrantes.

Se você estiver usando ponto flutuante para apenas alguns cálculos, pode ser melhor implementar os próprios cálculos usando aritmética de ponto fixo. Por exemplo, dois bits fracionários representando um valor de 0,00, 0,25, 0,50 ou 0,75 são facilmente armazenados em qualquer número inteiro simplesmente multiplicando o valor real por 4 (por exemplo, $\ll 2$). A adição e a subtração podem ser realizadas por meio do conjunto de instruções inteiras, desde que ambos os valores tenham o mesmo ponto binário imaginário. A multiplicação e a divisão podem ser realizadas de forma semelhante, se o outro número for um inteiro inteiro.

É teoricamente possível realizar qualquer cálculo de ponto flutuante com aritmética de ponto fixo. (Afinal, é assim que a biblioteca de software de ponto flutuante faz, certo?) Sua maior vantagem é que você provavelmente não precisa implementar todo o padrão IEEE 754 apenas para realizar um ou dois cálculos. Se você precisa desse tipo de funcionalidade completa, use a biblioteca de ponto flutuante do compilador e procure outras maneiras de acelerar seu programa.

Tamanho variável

Normalmente, é melhor usar a largura do registro nativo do processador para variáveis sempre que possível (seja 8, 16 ou 32 bits). Isso permite que o compilador produza código que tira proveito dos registros rápidos embutidos nos opcodes da máquina do processador. Obviamente, você precisa garantir que o tamanho da variável acomode o intervalo de números que a variável representa. Por exemplo, se você precisa de uma contagem que vai de 0 a 512, não pode usar uma variável de 8 bits.

Um tamanho variável adaptado ao processador também pode acelerar o processamento, limitando o número de acessos à memória externa. Se um processador tiver um barramento de dados de 16 bits e precisar acessar uma variável de 32 bits na RAM externa, duas buscas de dados devem ser realizadas para que o processador obtenha a variável.

C99 define tipos inteiros `int_fastN_t` e `uint_fastN_t` (Onde N representa o comprimento inteiro) em *stdint.h*. Esses tipos devem ser usados quando você precisa de pelo menos “X bits” (por exemplo, X = 16) para armazenar seus dados, mas não se importa se o campo é maior que X de largura, para tornar o acesso o mais rápido possível. Esses tipos inteiros “rápidos”, portanto, não são adequados para uso com registradores periféricos, que sempre têm uma largura fixa que não pode ser maior ou menor que X.

Loop desenrolando

Em alguns casos, o código de loop repetitivo pode ser otimizado executando *loop desenrolando*. No desenrolamento do loop, a sobrecarga do loop no início e no final de um loop é eliminada. Aqui está um exemplo de para ciclo:

```
para (idx = 0; idx < 5; idx++) {  
  
    valor[idx] = entrandoDados[idx];  
}
```

Esta é a versão desenrolada sem a sobrecarga do loop:

```
valor[0] = dados recebidos[0];  
valor[1] = dados recebidos[1];  
valor[2] = dados recebidos[2];  
valor[3] = dados recebidos[3];  
valor[4] = dados recebidos[4];
```

Alguns compiladores oferecem o desenrolamento de loop como uma otimização; em outros casos, pode ser melhor para o desenvolvedor codificá-lo. É útil verificar a saída do assembly do compilador para ver se a eficiência foi realmente aprimorada.

A quantidade de rolagem que você - ou o compilador - escolhe fazer deve equilibrar o ganho de velocidade em relação ao aumento do tamanho do código. O desenrolamento do loop aumenta o tamanho do código - outra situação em que você deve trocar o tamanho do código por velocidade. Além disso, o desenrolamento do loop pode ser usado apenas quando o número de iterações através do loop é fixo. Um exemplo de uma implementação otimizada de desenrolamento de loop é o código-técnica conhecida como dispositivo de Duff (http://en.wikipedia.org/wiki/Duff's_device).

Diminuindo o tamanho do código

Conforme declarado anteriormente, quando se trata de reduzir o tamanho do código, sua melhor aposta é deixar o compilador fazer o trabalho por você. No entanto, se o programa resultante ainda for muito grande para a ROM disponível, existem várias técnicas de programação que você pode usar para reduzir ainda mais o tamanho do programa.

Depois de fazer com que as otimizações automáticas funcionem, dê uma olhada nestas dicas para reduzir ainda mais o tamanho do seu código manualmente:

Evite rotinas de biblioteca padrão

Uma das melhores coisas que você pode fazer para reduzir o tamanho do seu programa é evitar o uso de grandes rotinas de biblioteca padrão. Muitas das rotinas maiores são caras em termos de tamanho porque tentam lidar com todos os casos possíveis. Por exemplo, o Strupr função pode ser pequena, mas uma chamada para ela pode arrastar outras funções, como strlower, strcmp, strcpy, e outros em seu programa, sejam eles usados ou não.

Pode ser possível implementar um subconjunto da funcionalidade sozinho com muito menos código. Por exemplo, a biblioteca C padrão sprintf rotina é notoriamente grande. Muito desse volume está localizado nas rotinas de manipulação de ponto flutuante das quais depende. Mas se você não precisa formatar e exibir valores de ponto flutuante (% a, % e, % f, ou % g), você poderia escrever sua própria versão apenas de número inteiro de sprintf e economizar vários kilobytes de espaço de código. Na verdade, algumas implementações da biblioteca C padrão (Cygnus's *newlib* vem à mente) incluem apenas uma função, chamada siprintf.

Usar vamos para afirmações

Assim como acontece com as variáveis globais, as boas práticas de engenharia de software não permitem o uso dessa técnica. Mas em uma pitada, vamos para instruções podem ser usadas para remover estruturas de controle complicadas ou para compartilhar um bloco de código frequentemente repetido.

Por exemplo, muitos programadores usam o vamos para declaração para escapar de uma rotina em caso de erro. Desta forma, o programador pode agrupar quaisquer coisas que deve ser feito antes de sair da rotina, conforme mostrado aqui:

```
int functionWork(void)
{
    /* Faça algum trabalho aqui. */
    ...

    /* Se houver um erro ao fazer o trabalho, saia. */ goto
    CLEANUP;

    /* Faça mais trabalhos aqui. */
    ...

    /* Se houver um erro ao fazer o trabalho, saia. */ goto
    CLEANUP;

    ...

    /* Caso contrário, tudo deu certo. */
    retornar SUCESSO;
```

```

LIMPAR:
    / * Limpe o código aqui. * /

    retornar FALHA;
}

```

Além dessas técnicas para reduzir o tamanho do código, várias das descritas na seção anterior podem ser úteis, especificamente pesquisas de tabela, montagem codificada manualmente, variáveis de registro e variáveis globais. Dessas técnicas, o uso de assembly codificado manualmente geralmente produz a maior redução no tamanho do código.

Problemas com a otimização de compiladores

O compilador GNU C tem várias opções de linha de comando de otimização, todas variantes do `-O`. Especificando `-O3` liga todos disponíveis *gcc* otimizações, independentemente de seus efeitos na troca de velocidade versus tamanho. A opção de linha de comando `-Os` também otimiza o tamanho do código. Para uma explicação detalhada dos diferentes *gcc* níveis de otimização, consulte o *gcc* manual online em <http://gcc.gnu.org/onlinedocs>.

A Lei de Murphy determina que, na primeira vez que você habilitar o recurso de otimização do compilador, seu programa de trabalho anterior falhará repentinamente. Talvez a mais notória das otimizações automáticas seja a "eliminação de código morto". Essa otimização elimina o código que o compilador acredita ser redundante ou irrelevante. Por exemplo, adicionar zero a uma variável não requer nenhum cálculo de tempo de execução. Mas você ainda pode querer que o compilador gere aquelas instruções "irrelevantes" se elas executarem alguma função que o compilador não conhece.

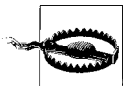
Por exemplo, dado o seguinte bloco de código, a maioria dos compiladores de otimização removeria a primeira instrução porque o valor de `*pControl` não é usado antes de ser substituído (na terceira linha):

```

* pControl = DISABLE;
* pData = 'a';
* pControl = ENABLE;

```

Mas e se `pControl` e `pData` são, na verdade, ponteiros para registros de dispositivos mapeados na memória? Nesse caso, o dispositivo periférico não receberia o DESATIVAR comando antes de o byte de dados ser gravado. Isso poderia causar estragos em todas as futuras interações entre o processador e este periférico. Para se proteger de tais problemas, você deve declarar todos os ponteiros para registros mapeados em memória e variáveis globais que são compartilhados entre tarefas (ou uma tarefa e um ISR) com a palavra-chave volátil. E se você perder apenas um deles, a Lei de Murphy voltará para assombrá-lo nos últimos dias de seu projeto - garantido.



Nunca cometa o erro de presumir que o programa otimizado se comportará da mesma maneira que o não otimizado. Você deve testar novamente seu software em cada novo nível de otimização para ter certeza de que seu comportamento não mudou.

Para piorar as coisas, depurar um programa otimizado é um desafio, para dizer o mínimo. Com a otimização do compilador habilitada, a correlação entre uma linha de código-fonte e o conjunto de instruções do processador que implementa essa linha é muito mais fraca. Essas instruções específicas podem ter sido movidas ou divididas, ou dois blocos de código semelhantes podem agora compartilhar uma implementação comum. Na verdade, algumas linhas do programa de linguagem de alto nível podem ter sido removidas completamente do programa (como foram no exemplo anterior)! Como resultado, você pode não conseguir definir um ponto de interrupção em uma linha específica do programa ou examinar o valor de uma variável de interesse.

Reduzindo o uso de memória

Em alguns casos, RAM, em vez de ROM, é o fator limitante para seu aplicativo. Nesses casos, você desejará reduzir sua dependência de dados globais, da pilha e do heap. Todas essas são otimizações feitas melhor pelo programador do que pelo compilador.

Como a ROM é geralmente mais barata do que a RAM (por byte), uma estratégia aceitável para reduzir a quantidade de dados globais pode ser mover dados constantes para a ROM. Isso pode ser feito automaticamente pelo compilador se você declarar todos os seus dados constantes com a palavra-chave `const`. A maioria dos compiladores C coloca todos os dados globais constantes que encontram em um segmento de dados especial que é reconhecível para o localizador como compatível com ROM. Essa técnica é mais valiosa se houver muitas strings ou dados orientados a tabela que não serão alterados no tempo de execução.

Se alguns dos dados forem fixados quando o programa estiver em execução, mas não necessariamente constantes, o segmento de dados constante pode ser colocado em um dispositivo de memória híbrido, como flash ou EEPROM. Esse dispositivo de memória pode então ser atualizado pela rede ou por um técnico designado para fazer a alteração. Um exemplo de tais dados é a taxa de imposto sobre vendas para cada local em que seu produto será implantado. Se uma taxa de imposto mudar, o dispositivo de memória pode ser atualizado, mas RAM adicional pode ser salva nesse meio tempo.

As reduções no tamanho da pilha também podem diminuir os requisitos de RAM do seu programa. Uma maneira de descobrir aproximadamente quanta pilha você precisa é preencher toda a área de memória reservada para a pilha com um padrão de dados especial, como `0xAAAA`. Então, depois que o software estiver em execução por um tempo - tanto em condições normais quanto estressantes - use um depurador para examinar a pilha modificada. A parte da área de memória da pilha que ainda contém seu padrão de dados especial nunca foi sobrescrita, portanto, você pode reduzir o tamanho da área da pilha nessa quantidade. *

Esteja especialmente atento ao espaço da pilha se estiver usando um sistema operacional em tempo real. Os sistemas operacionais preventivos criam uma pilha separada para cada tarefa. Essas pilhas são

* Claro, você provavelmente deseja deixar um pouco de espaço extra na pilha, caso seu teste não tenha durado o suficiente ou não reflita com precisão todos os cenários de tempo de execução possíveis. Nunca se esqueça de que um estouro de pilha é um evento potencialmente fatal para o seu software e deve ser evitado a todo custo.

usado para chamadas de função e ISRs que ocorrem no contexto de uma tarefa. Você pode determinar a quantidade de memória necessária para cada pilha de tarefas da maneira descrita anteriormente. Você também pode tentar reduzir o número de tarefas ou mudar para um sistema operacional que tenha uma “pilha de interrupções” distinta para a execução de todos os ISRs. O último método pode reduzir significativamente o requisito de tamanho da pilha de cada tarefa.

O tamanho do heap é limitado à quantidade de RAM restante depois que todos os dados globais e o espaço da pilha foram alocados. Se o heap for muito pequeno, seu programa não será capaz de alocar memória dinâmica quando for necessário, então sempre certifique-se de comparar o resultado de Malloc com NULO antes de remover a referência da memória que você tentou alocar. Se você tentou todas essas sugestões e seu programa ainda está exigindo muita memória, você pode não ter escolha a não ser eliminar o heap por completo. Isso não é totalmente ruim no caso de sistemas embarcados, que freqüentemente alocam toda a memória necessária ao sistema no momento da inicialização.

Observe que muitos programadores incorporados evitam o uso de malloc, e, portanto, a necessidade de uma pilha, por completo. Mas o principal benefício da alocação de memória dinâmica é que você não precisa gastar RAM para manter variáveis que são usadas apenas brevemente no programa. Essa é uma maneira de reduzir a utilização total da memória.

Técnicas de economia de energia

Uma das principais preocupações no projeto de sistemas embarcados alimentados por bateria é o consumo de energia. Nesta seção, daremos uma breve olhada nas áreas em que o software embarcado pode auxiliar na conservação da fonte de energia vital do sistema.

O consumo de energia é uma grande preocupação para dispositivos portáteis ou operados por bateria. Questões de energia, como quanto tempo o dispositivo precisa para funcionar e se as baterias podem ser recarregadas, precisam ser pensadas com antecedência. Em alguns sistemas, substituir a bateria de um dispositivo pode ser uma grande despesa. Isso significa que o sistema deve estar ciente da quantidade de energia que usa e tomar as medidas adequadas para conservar a vida útil da bateria.

Existem vários métodos para conservar energia em um sistema embarcado, incluindo controle de relógio, processadores sensíveis à energia, CIs de baixa tensão e desligamento do circuito. Algumas dessas técnicas devem ser abordadas pelo projetista de hardware em sua seleção dos diferentes CIs de sistema. Pode haver versões de baixo consumo de certos periféricos. Algumas técnicas de economia de energia estão sob controle de software.

Pode parecer ideal selecionar o processador mais rápido e poderoso disponível para um determinado sistema embarcado. No entanto, uma das tarefas do designer de hardware é usar apenas o poder de processamento suficiente para permitir que o dispositivo faça seu trabalho. Isso ajuda a reduzir a energia consumida pelo dispositivo. O processador selecionado desempenha um papel fundamental na determinação da quantidade de energia que um sistema embarcado irá consumir. Além disso, alguns processadores podem desligar automaticamente unidades de execução diferentes quando não estiverem em uso.

Modos de Processador

Uma técnica de software oferecida por muitos processadores embarcados para conservar energia são os diferentes modos de operação. Esses modos permitem que o software dimensione o consumo de energia do processador para atender às necessidades do aplicativo a cada momento. Por exemplo, o processador PXA255 da placa Arcom tem quatro modos de operação:

modo turbo

O núcleo de processamento é executado na frequência de pico. Minimizar os acessos à memória externa valeria a pena neste modo, porque o processador teria que esperar pela memória externa.

Modo corrida

O núcleo do processador funciona em sua frequência normal. Este é o modo de operação normal ou padrão.

Modo inativo

O núcleo do processador não tem clock, mas os outros componentes periféricos operam normalmente.

Modo dormir

Este é o estado de menor energia para o processador.

Compreender os detalhes desses modos e como entrar e sair deles é fundamental. Por exemplo, o PXA255 pode economizar energia entrando e saindo do modo ocioso várias vezes por segundo, porque o processador é reativado rapidamente no estado anterior. No entanto, no modo de hibernação, o estado do processador não é mantido e pode exigir uma reinicialização completa do sistema ao sair desse modo.

Operar o processador em diferentes modos pode economizar um pouco de energia. O consumo de energia do processador PXA255 (funcionando a 200 MHz) no modo de execução normal é normalmente de 178 mW. Enquanto no modo inativo, o PXA255 normalmente consome 63 mW.

Há várias questões a serem consideradas ao planejar o design do software de gerenciamento de energia. Primeiro, você deve garantir que cada tarefa seja capaz de obter ciclos suficientes para realizar seu trabalho atribuído. Se um sistema dobra a vida útil da bateria entrando em modos ociosos com frequência, mas é incapaz de realizar seu trabalho, o produto não atinge seus objetivos de design.

Você também precisa determinar quando o sistema não está fazendo nada e como despertar o processador quando ele precisa operar, e você precisa saber quais eventos despertarão o sistema. Por exemplo, em um sistema embarcado que envia alguns dados pela rede a cada poucos minutos, faz sentido desligar o dispositivo para conservar energia até a hora de enviar os dados. O dispositivo ainda deve ser capaz de acordar caso ocorra uma condição de erro. Portanto, você deve entender como um circuito periférico desperta o processador quando o processador precisa operar (incluindo quanto tempo leva para o circuito despertar e se alguma reinicialização precisa ser feita).

As técnicas de otimização apresentadas anteriormente neste capítulo também podem ser usadas para conservar energia. Ao reduzir a quantidade de tempo de execução das principais tarefas em um sistema, você permite que o sistema passe mais tempo em seu estado de baixa energia.

Mesmo que o processador esteja no modo ocioso, vários periféricos ainda operam e podem ser programados para ativar o processador. Normalmente, as interrupções podem ser usadas para despertar o processador para realizar alguma tarefa. É por isso que o gerenciamento de energia deve ser considerado ao projetar o software. Por exemplo, alguns comportamentos podem ser alcançados por votação. Mas quando os eventos são menos frequentes e o gerenciamento de energia é um problema, faz sentido usar interrupções em vez de sondagem, pois isso permite que o processador durma pelo máximo de tempo antes de acordar para lidar com um evento. Quando você opta por usar o polling, o processador deve executar constantemente a operação de polling, o que normalmente acontece em um intervalo definido. Isso desperdiça energia quando a operação de pesquisa é executada e nenhum evento ocorreu.

Você também pode tirar proveito dos periféricos que operam enquanto o processador está no modo ocioso. Por exemplo, se você estiver transferindo dados de um periférico externo para a RAM e precisar processar os dados assim que uma certa quantidade de dados for recebida, você pode usar o controlador DMA. Dessa forma, em vez de o processador manipular cada byte recebido, ele dorme enquanto os dados são transferidos. Você pode configurar o controlador DMA para interromper o processador assim que esses dados forem recebidos.

Frequência do Relógio

Outra técnica de economia de energia que pode ser controlada por software é variar a velocidade do clock do processador. Alguns processadores aceitam uma frequência de clock de entrada fixa, mas apresentam a capacidade de reduzir as velocidades de clock interno programando registradores de configuração de clock. O software pode reduzir a velocidade do clock para economizar energia durante a execução de tarefas não críticas e aumentar a velocidade do clock quando as demandas de processamento são altas.

A folha de dados do PXA255 mostra o consumo de energia enquanto o núcleo do processador opera em frequências diferentes. A Tabela 14-1 mostra uma comparação para três diferentes frequências de clock do núcleo do PXA255 e o consumo de energia associado em cada frequência.

Tabela 14-1. Comparação do consumo de energia do PXA255

Velocidade do clock do núcleo do processador (MHz)	Consumo de energia (mW)
400	411
300	283
200	178

Como designer de software, você precisa entender o que acontece durante o fogo sequência de mudança de frequência - o que fazer se ocorrer uma interrupção durante a mudança de frequência e o que precisa ser reconfigurado (como ciclos de atualização de DRAM) para a nova frequência.

Você precisará de um conhecimento abrangente de toda a operação do software no sistema se decidir alterar a frequência do processador em tempo real. Por exemplo, pode ser complicado saber quando diminuir a velocidade do clock quando um RTOS multitarefa é usado.

Em outros casos, determinados periféricos podem ser completamente desativados quando não estão em uso. Por exemplo, se um determinado módulo periférico não for usado no processador PXA255, o relógio para essa unidade pode ser desabilitado usando o Clock Enable Register (CKEN).

Acesso à memória externa

Existem várias coisas que podem ser feitas para reduzir os acessos à memória externa. Se um *esconderijo* estiver disponível, você pode ativá-lo para evitar que o processador busque dados ou instruções da memória externa. Um cache é uma memória no chip de altíssima velocidade que fornece as instruções e / ou dados usados mais recentemente ao processador com nenhum ou poucos estados de espera.

Da mesma forma, a memória do processador interno pode ser usada, se disponível. Em alguns casos, a memória interna pode ser usada para dados e código. Pode não ser viável incorporar todo o software do sistema na memória interna. Se não houver memória suficiente disponível para todo o software do sistema, você deve determinar quais dados e códigos devem ser incluídos. É melhor incluir a pilha e as variáveis ou funções usadas com frequência para limitar os acessos à memória externa. Em um sistema embarcado em que o consumo de energia é a principal prioridade, pode fazer sentido mudar para um processador com mais memória no chip para reduzir os acessos fora do chip.

As técnicas de otimização também podem ajudar aqui. Para otimização de energia, em vez de se concentrar apenas na velocidade ou no tamanho do código, você precisa se concentrar na análise do código para determinar como reduzir as transações de barramento externo.

Estas são apenas algumas considerações a serem lembradas ao trabalhar com um software de gerenciamento de energia. Os produtos futuros podem incluir tecnologia com fontes de energia auto-renováveis. Existem maneiras de coletar energia para circuitos que são autoalimentados usando fontes como vibração, luz e fontes térmicas. Essas técnicas são discutidas em junho de 2005 *Projeto de Sistemas Embarcados* artigo "Chips de Coleta de Energia: The Quest for Everlasting Life," que pode ser encontrado online em <http://www.embedded.com>.

Limitando o impacto do C ++

Um dos problemas que enfrentamos ao decidir escrever este livro foi incluir ou não C ++ na discussão e nos exemplos. Para quase todos os projetos em que trabalhamos ao longo de nossas respectivas carreiras, o software embarcado foi escrito em C e linguagem assembly. Além disso, tem havido muito debate na comunidade de software embarcado sobre a adequação do C ++. É amplamente aceito que os programas C ++ produzem executáveis maiores que são executados mais lentamente do que programas escritos inteiramente em C. No entanto, C ++ tem muitos benefícios para os programadores.

Acreditamos que muitos leitores enfrentarão a escolha de usar C ++ em sua programação embarcada. Esta seção cobre alguns dos recursos C ++ que são úteis para software de sistema embarcado e avisa sobre alguns dos recursos mais caros da linguagem.

C ++ incorporado

Você pode estar se perguntando por que os criadores da linguagem C ++ incluíram tantos recursos que são caros em termos de tempo de execução e tamanho do código. Você não está sozinho; pessoas ao redor do mundo se perguntam a mesma coisa - especialmente os usuários de C ++ para programação embarcada. Muitos desses recursos caros são adições recentes que não são estritamente necessárias nem fazem parte da especificação original do C ++. Esses recursos foram adicionados um a um como parte do processo de “padronização” em andamento.

Em 1996, um grupo de fornecedores de processadores japoneses se uniu para definir um subconjunto da linguagem C ++ e das bibliotecas que é mais adequado para o desenvolvimento de software embarcado. Eles chamam seu padrão da indústria de Embedded C ++ (EC ++). EC ++ gerou um grande interesse inicial e entusiasmo na comunidade embarcada.

Um subconjunto adequado do padrão de rascunho C ++, o EC ++ omite praticamente qualquer coisa que possa ser omitida sem limitar a expressividade da linguagem subjacente. Isso inclui não apenas recursos caros, como herança múltipla, classes de base virtuais, identificação de tipo de tempo de execução e tratamento de exceções, mas também algumas das adições mais recentes, como modelos, namespaces e novos estilos de conversão. O que resta é uma versão mais simples de C ++ que ainda é orientada a objetos e um superconjunto de C, mas tem significativamente menos sobrecarga de tempo de execução e bibliotecas de tempo de execução menores.

Vários compiladores C ++ comerciais oferecem suporte ao padrão EC ++ como uma opção. Vários outros permitem que você desabilite manualmente recursos de linguagem individuais, permitindo assim emular EC ++ (ou criar seu próprio sabor da linguagem C ++).

Obviamente, nem todos os recursos do C ++ são caros. Na verdade, os primeiros compiladores C ++ usavam uma tecnologia chamada C-front para transformar programas C ++ em C, que era então alimentado em um compilador C padrão. O fato de isso ser possível demonstra que muitas das diferenças sintáticas entre as linguagens têm pouco ou nenhum custo de tempo de execução. * Por exemplo, a definição de uma classe é completamente benigno. A lista de dados e funções de membros públicos e privados não é muito diferente de uma estrutura e uma lista de protótipos de função. No entanto, o compilador C ++ é capaz de usar o público e privado palavras-chave para determinar quais chamadas de método e acessos a dados são permitidos e proibidos. Como essa determinação é feita em tempo de compilação, não há penalidade paga em tempo de execução. Portanto, o uso de classes por si só não afeta o tamanho do código nem a eficiência de seus programas.

* Além disso, queremos deixar claro que não há penalidade para compilar um programa C original com um compilador C ++.

Os valores dos parâmetros padrão também não têm penalidade. O compilador simplesmente insere o código para passar o valor padrão sempre que a função é chamada sem um argumento nessa posição. Da mesma forma, a sobrecarga do nome da função envolve apenas uma modificação do código em tempo de compilação. Funções com os mesmos nomes, mas parâmetros diferentes, recebem nomes exclusivos atribuídos a cada uma durante o processo de compilação. O compilador altera o nome da função cada vez que ela aparece em seu programa, e o vinculador combina-os apropriadamente.

A sobrecarga do operador é outro recurso que pode ser usado em sistemas embarcados. Sempre que o compilador vê esse operador, ele simplesmente o substitui pela chamada de função apropriada. Portanto, na listagem de código C ++ a seguir, as duas últimas linhas são equivalentes e a penalidade de desempenho é facilmente compreendida:

Complexo a, b, c;

c = operador + (a, b); c
= a + b;

// A maneira tradicional: chamada de função // A
maneira C ++: sobrecarga de operador

Construtores e destruidores têm uma pequena penalidade. Esses métodos especiais têm garantia de serem chamados sempre que um objeto do tipo é criado ou sai do escopo, respectivamente. No entanto, essa pequena quantidade de sobrecarga é um preço razoável a pagar por menos bugs. Os construtores eliminam toda uma classe de erros de programação C relacionados a estruturas de dados não inicializadas. Esse recurso também se mostrou útil para ocultar as sequências de inicialização estranhas associadas a algumas classes.

As funções virtuais também têm uma relação custo / benefício razoável. Sem entrar em muitos detalhes sobre o que são funções virtuais, vamos apenas dizer que o polimorfismo seria impossível sem elas. E sem polimorfismo, C ++ não seria uma verdadeira linguagem orientada a objetos. O único custo significativo das funções virtuais é uma pesquisa de memória adicional antes que uma função virtual possa ser chamada. Chamadas de funções e métodos comuns não são afetadas.

Os recursos do C ++ que normalmente são muito caros para sistemas embarcados são modelos, exceções e identificação de tipo de tempo de execução. Todos os três impactam negativamente o tamanho do código, e as exceções e a identificação do tipo de tempo de execução também aumentam o tempo de execução. Antes de decidir se usará esses recursos, você pode querer fazer alguns experimentos para ver como eles afetarão o tamanho e a velocidade de seu próprio aplicativo.

O kit de desenvolvimento ArcomVIPER-Lite

Todos os exemplos neste livro foram escritos e testados em uma plataforma embarcada chamada VIPER-Lite. Esta placa é um controlador incorporado de alta velocidade projetado, fabricado e vendido pela Arcom. Os parágrafos a seguir contêm informações sobre o hardware, ferramentas de desenvolvimento de software e instruções para solicitar uma placa para você.

O hardware VIPER-Lite inclui o seguinte:

- Processador: PXA255 XScale (baseado na arquitetura ARM v.5TE) (200 MHz)
- RAM: 64 MB de SDRAM
- ROM: 16 MB de flash e 1 MB de ROM de inicialização
- Três portas seriais compatíveis com RS232 (com conectores DB9 externos)
- Porta Ethernet 10 / 100baseTx
- Porta do cliente USB v1.1
- Slot CompactFlash
- Quatro temporizadores / contadores programáveis
- Controlador DMA de dezesseis canais
- Cronômetro de vigilância
- Relógio de tempo real
- Oito entradas digitais com buffer
- Oito saídas digitais com buffer
- Programa de monitoramento de depuração RedBoot residente na ROM de inicialização
- Linux integrado (baseado na versão 2.6 do kernel) residente no flash *
- Porta JTAG para depuração do sistema

* O sistema operacional Windows CE pode ser especificado ao fazer o pedido da placa VIPER-Lite. Há um custo adicional para o VIPER-Lite com o sistema operacional Windows CE. Os exemplos no livro têm como alvo a versão do sistema operacional Linux embutido da placa.

A Arcom também construiu um módulo adicional ideal para aprender software integrado. A placa adicional foi projetada especificamente para os exemplos mostrados neste livro e está incluída com uma versão específica do livro do kit de desenvolvimento VIPER-Lite. O módulo complementar VIPER-Lite inclui o seguinte:

- Três LEDs
- Quatro botões e jumpers
- Quatro entradas opto-isoladas
- Quatro saídas opto-isoladas
- Duas saídas de relé
- Uma campainha
- Um pequeno protótipo

O dev VIPER-Lite

a placa e um kit de
posicionamento em azul



Figura A-1. Sistema de desenvolvimento Arcom VIPER-Lite

O CD-ROM que vem com o kit de desenvolvimento da Arcom inclui todos os manuais VIPER-Lite e documentos de referência, planilhas para todos os componentes da placa, código-fonte para RedBoot, pacotes Linux embutidos com código-fonte e imagens binárias para RedBoot e Linux embutido.

As ferramentas de desenvolvimento de software para a placa Arcom estão localizadas no site do livro. Nós mesmos construímos essas ferramentas para o processador ARM, seguindo as instruções mostradas no Apêndice C. As ferramentas de software incluem o compilador GNU C (*gcc*), assembler (*Como*), vinculador (*ld*), e depurador (*gdb*). Nós encorajamos você a investigar as outras ferramentas GNU incluídas no kit de desenvolvimento. Todos os programas neste livro foram criados usando as ferramentas contidas no site do livro.

Para os leitores deste livro, o kit de desenvolvimento VIPER-Lite está disponível com um desconto especial de \$ 295 (mais frete). Use um dos seguintes códigos de pedido ao entrar em contato com a Arcom, dependendo do sistema operacional que você deseja:

- Kit de desenvolvimento de Linux Embedded VIPER-Lite
- Kit de desenvolvimento VIPER-Lite Windows CE

Certifique-se de mencionar o livro para receber o módulo complementar. Aqui está Informações de contato da Arcom:

Arcom
7500 West 161st Street
Overland Park, KS 66085
Rede: <http://www.arcom.com>
América e Ásia: +1 913-549-1000 ou us-sales@arcom.com
EMEA: +44 (0) 1223-403410 ou sales@arcom.co.uk

Configurando Seu Ambiente de Desenvolvimento de Software

Este apêndice mostra o procedimento para configurar as ferramentas de desenvolvimento de software GNU e um exemplo de código-fonte. O procedimento de configuração das ferramentas de desenvolvimento de software GNU é dividido em duas seções: uma para Windows e outra para Linux. As ferramentas de software GNU que usamos para o código de exemplo incluem o *gcc* versão 3.4.4, *Como* versão 2.15, *ld* versão 2.15, *gdb* versão 6.3, e *binutils* versão 2.15.

O site deste livro contém vários arquivos compactados que se expandem para incluir os vários códigos-fonte e ferramentas usadas neste livro. Esses arquivos são:

windowshost.zip

Contém os arquivos de configuração do Cygwin e as ferramentas de desenvolvimento de software GNU baseadas no Windows

linuxhost.tar.gz

Contém as ferramentas de desenvolvimento de software GNU baseadas em Linux

ecos.tar.gz

Contém o repositório de código-fonte eCos e as ferramentas de desenvolvimento eCos (consulte o Apêndice D para obter informações adicionais sobre como configurar o ambiente de host eCos)

example.zip e examples.tar.gz

Contém o código de exemplo do livro



O código de exemplo do Linux no Capítulo 12 não foi criado e testado usando um computador host executando o Windows. É comum usar um sistema host Linux para desenvolver aplicativos Linux integrados.

Construir aplicativos para Linux usando um host Windows está além do escopo deste livro. Envolve o uso do conjunto de ferramentas de software livre Cygwin, um procedimento um pouco mais complicado do que o descrito neste capítulo.

Instalação de host do Windows

As ferramentas de desenvolvimento de software GNU foram executadas em um computador Pentium 4 com Windows XP (Service Pack 2). A primeira fase da configuração do Windows é instalar *Cygwin*, um ambiente Unix para Windows. Informações adicionais sobre o Cygwin podem ser encontrado online em <http://www.cygwin.com>.

A primeira etapa no procedimento de instalação do host do Windows é baixar o *windowshost.zip* arquivo e descompacte-o no diretório temporário.

Instalação Cygwin

Cygwin é usado para construir todos os exemplos neste livro no Windows. As instruções a seguir pressupõem que *C:* é a unidade de disco rígido onde os arquivos estão instalados. O ambiente Cygwin é instalado sob o *C: \ cygwin* diretório.

1. Execute o programa de instalação do Cygwin *setup.exe*. Os arquivos de instalação do Cygwin estão localizados no *cygwin* diretório no *windowshost.zip* Arquivo.
2. A primeira caixa de diálogo é intitulada Cygwin Net Release Setup Program. Isso fornece os detalhes sobre as informações da versão do programa de instalação. Clique em próximo para continuar.



Olhar para os comandos e diretórios aqui pode ficar um pouco confuso, porque os ambientes Windows e Unix diferem na maneira como separam os diretórios: o Windows usa a barra invertida (\) e o Unix usa a barra (/).

3. Agora selecione o diretório a partir do qual deseja instalar as ferramentas Cygwin. Nesse caso, selecionamos “Instalar do diretório local” e clique em Avançar.
4. Na próxima caixa de diálogo, selecione o local em seu disco rígido onde deseja que as ferramentas Cygwin sejam instaladas. Deixe o padrão como *C: \ cygwin*. (Se desejar escolher um destino alternativo, altere a localização da unidade e do diretório de acordo.) Na caixa de seleção Instalar para, selecione Todos os usuários e, para o Tipo de arquivo de texto padrão, selecione DOS. Em seguida, clique em Avançar.
5. Informe à configuração do Cygwin onde residem os arquivos locais que você deseja instalar. Navegue até o *cygwin* diretório temporário, onde você descompactou os arquivos de instalação do Cygwin e, em seguida, clique em Avançar. Isso fará com que o programa de configuração do Cygwin inventarie as ferramentas disponíveis e exiba a lista disponível.
6. Selecione as ferramentas a serem instaladas. Certifique-se de que todas as ferramentas que você deseja instalar estejam selecionadas para instalação. Para adicionar ferramentas, clique no botão Exibir para obter a lista completa de pacotes disponíveis para instalação. Em seguida, clique na seta circular na coluna Novo para selecionar o pacote apropriado para instalação. Isso deve alterar o texto ao lado da seta circular de “Pular” para a versão do pacote específico que será instalado. Depois que todos os pacotes forem selecionados para instalação, clique em Avançar para iniciar a instalação.

7. Após a conclusão da instalação, clique em Concluir. Isso adicionará os ícones do Cygwin à área de trabalho e ao menu iniciar. Clique em Ok na caixa de diálogo Instalação concluída.

Neste ponto, você deve verificar se tem um `C:\cygwin` diretório com os diretórios e arquivos Cygwin instalados.

Instalação de ferramentas de software GNU

A próxima fase da configuração das ferramentas de host do Windows é configurar as ferramentas de desenvolvimento de software GNU, conforme mostrado aqui. As ferramentas de software GNU são instaladas no `\cygwin\opt` diretório.

1. A versão Windows das ferramentas GNU baseadas em ARM está localizada no `\gnutools` diretório no `windowshost.zip` Arquivo. Descompacte o arquivo `gnutools.zip` ao `\cygwin\opt` diretório em seu disco rígido.
2. Defina o caminho para a localização das ferramentas GNU no Cygwin `bash` Concha. Para garantir que o caminho seja definido corretamente cada vez que o Cygwin `bash` shell é iniciado, edite o `bash` arquivo de perfil. O arquivo é nomeado `bash_profile` e está localizado sob `$CASA` diretório (que é específico para o seu ambiente). Adicione o seguinte à última linha deste arquivo:

```
PATH = / opt / gnutools / arm-elf / bin: $ PATH; exportar PATH
```

Você deve notar que as ferramentas de desenvolvimento GNU são instaladas em `C:\cygwin\opt\gnutools`. Se você olhar sob o `braço-elf\bin` diretório, você deve ver as ferramentas GNU (como `gcc`, `as`, `gdb`, e `ld`) arquivos executáveis, prefixados com o nome `elfo de braço`. Isso descreve o processador para o qual as ferramentas são construídas, `braço`, e o formato do arquivo do objeto, `elfo` que significa “formato executável e vinculável”.

Para testar se você instalou as ferramentas corretamente e configurou o caminho corretamente, abra um Cygwin `bash` shell e digite o comando:

```
# arm-elf-gcc -v
```

Você deve ver esta resposta:

```
Lendo especificações de /opt/gnutools/arm-elf/lib/gcc/arm-elf/3.4.4/specs
Configurado com: /src/gcc-3.4.4/configure --target = arm-elf --prefix = / opt / gnutool s /
arm-elf --enable-languages = c, c ++ --with-gnu-as - -with-gnu-ld --with-newlib --w ith-gxx-
include-dir = / opt / gnutools / arm-elf / arm-elf / include -v
Modelo de rosca: único
gcc versão 3.4.4
```

Instalação de host Linux

As ferramentas de desenvolvimento de software GNU foram testadas em um computador Celeron rodando Linux Fedora Core 5.



As ferramentas de software GNU para Linux que são configuradas neste procedimento permitem que você crie os exemplos para todos os capítulos, exceto o Capítulo 12. Os exemplos no Capítulo 12 devem ser executados no sistema operacional Linux integrado da placa Arcom. A instalação das ferramentas GNU para construir o código de exemplo do Capítulo 12 é abordada no Apêndice E.

Instalação de ferramentas de software GNU

As ferramentas de software GNU são instaladas sob o */opt* diretório. Para instalá-los e torná-los utilizáveis, siga estas etapas. (Você precisará garantir que tem permissão para se tornar superusuário (root) para executar a configuração do Linux com êxito).

1. Abra uma janela de terminal e mude para o */opt* diretório com o comando:

```
# cd /opt
```

2. A versão Linux das ferramentas GNU baseadas em ARM está localizada no arquivo *linuxhost.tar.gz*. Copie este arquivo para o */opt* diretório. Em seguida, descompacte o arquivo em seu disco rígido usando o comando:

```
# tar xvzf linuxhost.tar.gz
```

3. Finalmente, defina o caminho para a localização das ferramentas GNU em seu *bash* perfil de concha. Isso garante que o caminho seja definido corretamente cada vez que o *bash* shell é iniciado. Edite o *bash* arquivo de perfil chamado *\$HOME/.bash_profile* (onde *\$CASA* é específico para o seu ambiente). Adicione o seguinte à última linha deste arquivo:

```
PATH = /opt/gnutools/arm-elf/bin: $PATH; exportar PATH
```

Você deve notar que as ferramentas de desenvolvimento GNU são instaladas em */opt/gnutools*. Os arquivos executáveis, como *arm-elf-gcc*, estão contidos em */opt/gnutools/bin* diretório. O nome prefixado *elfo de braço* descreve o processador para o qual as ferramentas são construídas, *braço*, e o formato do arquivo do objeto, *duende*.

Para testar se você instalou as ferramentas corretamente e configurou o caminho corretamente, feche a janela do terminal existente. Abra uma nova janela de terminal (para garantir que o caminho está definido corretamente) e digite o comando:

```
# arm-elf-gcc -v
```

Você deve ver esta resposta:

```
Lendo especificações de /opt/gnutools/arm-elf/lib/gcc/arm-elf/3.4.4/specs
Configurado com: /src/gcc-3.4.4/configure --target = arm-elf --prefix = /opt/gnutools /
arm-elf --enable-languages = c, c++ --with-gnu-as - --with-gnu-ld --with-newlib --w
ith-gxx-include-dir = /opt/gnutools/arm-elf/arm-elf/include -v
Modelo de thread: único
gcc versão 3.4.4
```

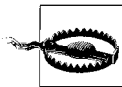
Exemplo de instalação de código

Os arquivos de exemplo do livro podem ser extraídos em qualquer diretório. Recomendamos instalar os arquivos de código-fonte de exemplo no `/opt/ProgEmbSys` diretório. O código de exemplo do livro está contido nos arquivos *example.zip* e *exemplos.tar.gz*.

O diretório de cada capítulo contém todos os exemplos de código-fonte necessários para construir os executáveis do capítulo. Makefiles são incluídos para simplificar o processo de construção. Os detalhes do procedimento de construção são cobertos no Capítulo 4. O procedimento para baixar e depurar os exemplos é coberto no Capítulo 5.

Construindo as ferramentas de software GNU

Este apêndice mostra como construir as ferramentas de software GNU voltadas para ARM (*arm-elf*) (*gcc*, *binutils*, *gdb*, *newlib*) usado neste livro. As instruções são para os sistemas operacionais Windows (executando Cygwin) e Linux.



A menos que haja alguma razão específica para construir um novo conjunto de ferramentas GNU, recomendamos que você use os binários GNU pré-construídos fornecidos no site do livro e incluídos no CD-ROM da placa Arcom. A instalação das ferramentas pré-construídas é abordada no Apêndice B e no Apêndice E, respectivamente.

Aproximadamente 1,2 GB de espaço em disco é necessário para o código-fonte das ferramentas GNU e os diretórios de saída da compilação. Estas instruções foram adaptadas da página da web “Construindo um conjunto de ferramentas para uso com eCos,” encontrada online em <http://ecos.sourceforge.org>.

Outro site popular para a construção de ferramentas de desenvolvimento cruzado é encontrado online em <http://kegel.com/crosstool>. Este site contém instruções e vários scripts para construir várias cadeias de ferramentas de desenvolvimento cruzado GNU para vários processadores e sistemas host. Você pode descobrir que as ferramentas neste site atendem melhor às suas necessidades.

As etapas a seguir devem ser seguidas em um *bash* prompt de shell no Cygwin ou um terminal no Linux. Construimos as ferramentas GNU no Windows XP (SP2) e Linux Fedora Core 5. Estas instruções localizam o código-fonte das ferramentas GNU sob o */src* diretório, a saída da compilação em */tmp/build*, e as ferramentas GNU resultantes em */opt/gnutools*. No Windows, todos esses subdiretórios estão contidos no principal *cygwin* diretório.

Extraindo os arquivos de origem

Depois de baixar o *gnutoolssrc.tar.gz* arquivo do site do livro, torne-se superusuário (*raiz*) e proceda da seguinte forma:

1. Crie um diretório para conter as fontes da ferramenta (evite nomes de diretório contendo espaços, pois eles podem confundir o sistema de construção). No Windows, este diretório está contido no *cygwin* diretório. No Linux, espera-se que esse diretório esteja localizado no diretório raiz (/).

```
# mkdir -p / src
```

2. Mude para o diretório recém-criado:

```
# cd / src
```

3. Copie o *gnutoolssrc.tar.gz* arquivo para o *src* diretório.

4. Extraia as fontes do arquivo compactado, conforme mostrado aqui. Isso deve criar os seguintes diretórios em *src*: *binutils-2.15*, *gcc-3.4.4*, *gdb-6.3*, e *newlib-1.13.0*.

```
# tar xvzf gnutoolssrc.tar.gz
```

Normalmente, você aplicaria quaisquer patches necessários neste ponto; entretanto, já aplicamos os patches necessários aos arquivos de origem. Existe um *correção* utilitário para auxiliar na aplicação de patches.

Construindo o conjunto de ferramentas

Agora você precisa compilar as ferramentas para seu hardware e sistema operacional:

- 1 Antes de tentar construir as ferramentas, certifique-se de que o diretório de ferramentas do compilador nativo GNU esteja no PATH e preceda o diretório atual.

```
# PATH = / bin: $ PATH; exportar PATH
```

- 2 Configure os utilitários binários GNU (*binutils*):

```
# mkdir -p / tmp / build / binutils
# cd / tmp / build / binutils
# /src/binutils-2.15/configure --target = arm-elf \
-- prefixo = / opt / gnutools / arm-elf -v 2> & 1 | tee configure.out
```

A saída resultante está contida no arquivo *configure.out*. Se houver algum problema na configuração das ferramentas, consulte este arquivo.

- 3 Construir e instalar o GNU *binutils* (esta etapa pode levar muito tempo):

```
# make -w all install 2> & 1 | tee make.out
```

A saída resultante está contida no arquivo *decifrar*. Se houver algum problema na construção das ferramentas, consulte este arquivo.

- 4 Certifique-se de que o *binutils* estão no topo do CAMINHO:

```
# PATH = / opt / gnutools / arm-elf / bin: $ PATH; exportar PATH
```

- 5 Configure *gcc*:

```
# mkdir -p / tmp / build / gcc
# cd / tmp / build / gcc
# /src/gcc-3.4.4/configure --target = arm-elf \
-- prefixo = / opt / gnutools / arm-elf --enable-languages = c, c ++ \
-- with-gnu-as --with-gnu-ld --with-newlib \
```



```
--with-gxx-include-dir = /opt / gnutools / arm-elf / arm-elf / include \  
-v 2> & 1 | tee configure.out
```

6. Construir e instalar *gcc* (esta etapa pode levar muito tempo):

```
# make -w all install 2> & 1 | tee make.out
```

7. Configure *gdb*:

```
# mkdir -p /tmp / build / gdb  
# cd /tmp / build / gdb  
# /src/gdb-6.3/configure --target = arm-elf \  
--prefix = /opt / gnutools / arm-elf --disable-nls \  
-v 2> & 1 | tee configure.out
```

8. Construir e instalar *gdb* (esta etapa pode levar muito tempo):

```
# make -w all install 2> & 1 | tee make.out
```

Após a construção e instalação bem-sucedidas das ferramentas GNU, a árvore de construção associada (localizada em */tmp / build*) pode ser excluído para economizar espaço, se necessário. O diretório de arquivos executáveis do conjunto de ferramentas (*/opt / gnutools / arm-elf / bin*) deve ser adicionado ao cabeçalho de seu PATH.

Configurando o Ambiente de Desenvolvimento eCos

Para construir um aplicativo eCos, você deve vincular o aplicativo à biblioteca eCos. Este apêndice descreve como construir uma nova biblioteca eCos para vincular com o código do aplicativo.

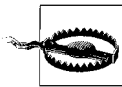
Uma biblioteca eCos pré-construída já está incluída no arquivo compactado do livro, mas você pode precisar saber o procedimento de construção da biblioteca eCos se mudar para uma nova placa de desenvolvimento ou precisar de funcionalidade adicional na biblioteca eCos.

Os exemplos de eCos, cobertos no Capítulo 11, são construídos da mesma maneira que os outros exemplos do livro. O procedimento de construção é abordado no Capítulo 4, e o procedimento de download e depuração no Capítulo 5. Os exemplos eCos são construídos usando as ferramentas GNU configuradas no Apêndice B, junto com a biblioteca eCos pré-construída.

Detalhes adicionais sobre a construção de aplicativos eCos podem ser encontrados no livro *Desenvolvimento de software integrado com eCos*, por Anthony Massa (Prentice Hall PTR).

O ambiente eCos Build

Insira os seguintes comandos em um Cygwin *bash* shell no Windows ou em uma janela de terminal no Linux. As instruções extraem o código-fonte do eCos para o `/opt/ecos` diretório. Outros diretórios podem ser usados, mas as instruções precisam ser ajustadas de acordo.



Usando Linux, você precisará de permissão para se tornar superusuário (*raiz*) para realizar a configuração com sucesso.

Instalação do código fonte eCos

O código-fonte do eCos está contido no arquivo *ecos.tar.gz*. Se você usar uma versão diferente do código-fonte eCos, altere as seguintes instruções de acordo:

1. Faça um diretório onde o código fonte do eCos seja extraído, com o seguinte comando:

```
# mkdir -p / opt / ecos
# cd / opt / ecos
```

2. Copie o arquivo fonte eCos *ecos.tar.gz* ao */ opt / ecos* diretório.

3. Extraia os arquivos de código-fonte eCos usando o seguinte:

```
# tar xvzf ecos.tar.gz
```

O diretório que contém o código-fonte do eCos agora deve estar disponível em */ opt / ecos / ecos-redboot-viper-v3i7*.

4. Configure as variáveis de ambiente. Edite o *\$ HOME / .bash_profile* arquivo (onde *\$ HOME* é específico para o seu ambiente) e adicione as seguintes linhas:

```
PATH = / opt / ecos / ecos-redboot-viper-v3i7 / tools: $ PATH; exportar PATH ECOS_REPOSITORY = /
opt / ecos / ecos-redboot-viper-v3i7 / packages; exportar ECOS_REPOSITORY
```

Fechar a corrente *bash* ambiente e abrir um novo. Isso permite que as alterações feitas no ambiente se tornem efetivas.

Construindo a Biblioteca eCos

Para este procedimento de construção, usamos a ferramenta de configuração de linha de comando eCos *ecosconfig*, que está incluído no */ opt / ecos / ecos-redboot-viper-v3i7 / tools* diretório. Portanto, esses comandos também são executados em um Cygwin *bash* prompt de shell em plataformas Windows ou em uma linha de comandos no Linux. Também existe uma ferramenta de configuração gráfica que pode ser usada para obter o mesmo resultado.

Os arquivos eCos resultantes estão localizados em */ opt / ProgEmbSys / capítulo11 / ecos* diretório. Se você instalou o código-fonte do livro, já deve haver uma biblioteca eCos presente neste local. Você precisa renomear ou mover o diretório da biblioteca eCos existente antes de continuar.

1. Faça um diretório onde os arquivos eCos serão construídos, usando os seguintes comandos:

```
# mkdir -p / opt / ProgEmbSys / capítulo11 / ecos
# cd / opt / ProgEmbSys / capítulo11 / ecos
```

2. Crie uma nova configuração para a placa Arcom usando o eCos predefinição *tem-placa* digitando o seguinte comando:

```
# ecosconfig novo padrão arcom-viper
```

3. Crie a árvore de construção eCos usando o comando:

```
# árvore ecosconfig
```

4. Finalmente, construa a biblioteca eCos digitando o comando:

```
# faça
```

Se você encontrar um erro, certifique-se de que o caminho esteja configurado corretamente, conforme mostrado anteriormente. Depois de construir com sucesso uma biblioteca eCos, você deverá ver a seguinte mensagem:

construção concluída

Você deve ter vários diretórios em */ opt / ProgEmbSys / capítulo11 / ecos*, incluindo o diretório *install / lib*. o *lib* O diretório contém os arquivos do sistema operacional eCos que são vinculados aos aplicativos eCos.

Os makefiles eCos contêm uma variável, `ECOS_INSTALL_DIR`, que é definido para a localização do diretório de instalação do eCos— */ opt / ProgEmbSys / capítulo11 / ecos / install* nesse caso. Se o local do diretório de instalação do eCos for alterado, essa variável também deverá ser alterada.

Configurando o Embedded Linux Ambiente de desenvolvimento

Neste apêndice, apresentamos o procedimento para configurar um ambiente de desenvolvimento de host Linux para construir os aplicativos de exemplo do Capítulo 12 para Linux embutido residente na placa Arcom. Essas ferramentas GNU são diferentes das ferramentas GNU cobertas no Apêndice B.

Dependendo da versão da placa Arcom que você solicitou, o Linux integrado vem pré-instalado na memória flash. Quando a placa Arcom é ligada, o RedBoot executa um script de inicialização para executar o kernel do Linux residente no flash.



O procedimento neste apêndice é baseado nas instruções da placa Arcom encontradas no *Manual Técnico Arcom Embedded Linux*, que está incluído no CD-ROM do kit de desenvolvimento. Consulte a seção intitulada "Instalando o Ambiente Host AEL," onde AEL significa Arcom Embedded Linux.

O procedimento de instalação foi verificado em um computador Celeron executando Linux Fedora Core 5. Se você encontrar problemas ao instalar o ambiente de compilação do host, sugerimos que contate a Arcom diretamente. Para suporte técnico na Europa, envie e-mail para euro-support@arcom.com. Para suporte técnico nos Estados Unidos, envie e-mail para ussupport@arcom.com.

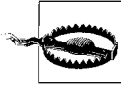
Configuração do ambiente de compilação do Linux

Para garantir que os aplicativos desenvolvidos por você sejam executados corretamente na placa da Arcom, você deve compilar e vincular seu código-fonte usando as bibliotecas presentes na versão do Linux da placa de destino.



Construir aplicativos para Linux usando um host Windows (e Cygwin) está além do escopo deste livro. A Internet está repleta de instruções de como fazer para configurar os ambientes de construção do Cygwin.

A distribuição do Linux em execução no sistema host deve ser compatível com o Linux Standard Base (LSB) versão 1.3 (consulte <http://www.linuxbase.org> Para maiores informações). Usamos o Fedora Core 5 (<http://fedora.redhat.com>) em nosso sistema de desenvolvimento de host.



Você precisará de permissão para se tornar superusuário (*raiz*) para executar o procedimento de instalação com sucesso.

Os comandos a seguir são executados a partir de uma janela de terminal com o CD-ROM de desenvolvimento da placa Arcom inserido na unidade.

- 1 Monte o CD-ROM onde o CD-ROM de desenvolvimento da placa Arcom está localizado usando o seguinte comando:

```
# montar / dev / cdrom / mnt
```

- 2 Execute o script de instalação:

```
# perl / mnt / install
```

Neste ponto, o programa de instalação emite uma mensagem semelhante à seguinte:

Tipo de distribuição de host: Red Hat Linux

Ambiente de host: / mnt / host

Pacotes: / mnt / packages

Armazenamento temporário: / tmp

Verificando o Linux Standard Base versão 1

```
lsb_release - / usr / bin / lsb_release
```

lsb_release relata a versão `: core-3.0-ia32: core-3.0-noarch: graphics-3.0-ia32: graphics-3.0-noarch '- versão LSB> = 2 detectada

Uso de valor não inicializado na correspondência de padrão (m //) em / mnt / install linha 700.

```
core-3.0-ia32
```

```
core-3.0-noarch
```

```
graphics-3.0-ia32
```

```
graphics-3.0-noarch
```

Vários componentes LSB foram encontrados, no entanto, nenhum deles parece fornecer a versão 1 conforme necessário.

Se possível, é recomendado que uma distribuição de host que suporte nativamente LSB versão 1 seja usada, pode haver um pacote adicional para suporte a LSB versão 1 disponível para a distribuição de host. No entanto, se a sua distribuição suportar apenas uma versão LSB diferente, pode ser possível instalar usando esses componentes LSB, no entanto, isso não é garantido.

Devo tentar instalar usando esta versão do LSB? [s / N] s

Verificando o linker dinâmico LSB /lib/

```
ld-lsb.so.1 - ok
```

Verificando os binários necessários em \$ PATH ...

rpm - / bin / rpm

Verificando os pacotes necessários ...

lsb versão 1.3 - 3.0-9.2 ok (não verificou a versão) rpm -

4.4.2-15.2 ok

rpm-build - 4.4.2-15.2 ok

wget - 1.10.2-3.2.1 ok

Verificando o espaço em disco disponível /

opt / arcom - 4095 megabytes - ok / tmp -

4095 megabytes - ok / var / tmp - 4095

megabytes - ok

A instalação do Arcom Embedded Linux Host Environment começará agora. Isso pode levar algum tempo para ser concluído.

Instalando o ambiente de host básico Arcom Embedded Linux

Verificar se um aplicativo LSB simples pode ser executado ok

Preparando o Arcom Embedded Linux Host Environment

Instalação do Arcom Embedded Linux Host Environment

3. Assim que a instalação for concluída com êxito, a seguinte mensagem será exibida:

Instalação completa.

Você deve adicionar `/opt/arcom/bin` ao seu PATH. Você pode fazer isso para a sessão de login atual com o seguinte comando:

```
export PATH = /opt/arcom/bin: $PATH
```

ou você pode modificar o caminho para todas as sessões de login para este usuário adicionando a mesma instrução a `~/.bash_profile` ou para todos os usuários adicionando-a a `/etc/profile`.

4. Para garantir que o caminho seja definido corretamente cada vez que você abrir um shell de terminal, edite o `~/.bash_profile` arquivo (onde `$CASA` é específico para o seu ambiente) adicionando a seguinte linha:

```
PATH = /opt/arcom/bin: $PATH; exportar PATH
```

O diretório `/opt/arcom` deve ser preenchido com vários arquivos. Os arquivos executáveis, como `arm-linux-gcc`, estão contidos em `/opt/arcom/bin` diretório.

Para testar se você instalou as ferramentas corretamente e configurou o caminho corretamente, feche a janela do terminal existente. Abra uma nova janela de terminal (para garantir que o caminho esteja definido corretamente) e digite o comando:

```
# arm-linux-gcc -v
```

Você deve ver uma resposta semelhante à seguinte:

```
Lendo especificações de /opt/arcom/lib/gcc/arm-linux/3.4.4/specs
Configurado com: ../gcc-3.4.4/configure --with-gxx-include-dir = / opt / arcom / arm-linux /
include / c ++ / 3.4.4 --target = arm-linux --host = i386-pc-linux-gnu --enable-crosstoolchain --
enable-languages = c, c ++ --with-gnu-as --with-gnu-ld --prefix = / opt / arcom -mandir = / opt /
arcom / share / man --with-slibdir = / opt / arcom / arm-linux / lib --enable-symvers = gnu --
enable-c99 --enable-long-long --program-prefix = arm-linux- --programsuffix = -3,4 --with-
headers = / opt / arcom / arm-linux / include --without-newlib --enable-threads --enable-shared --
enable-_cxa_atexit --with-arch = armv4t --with-float = modelo de rosca dura: posix
```

gcc versão 3.4.4

Exemplos de Linux embarcado

Os exemplos do Linux estão contidos no arquivo compactado do livro sob o *ProgEmbSys* / capítulo 12 diretório. Se você seguiu as instruções de instalação no Apêndice B, os arquivos devem estar localizados no diretório apropriado. Caso contrário, esses arquivos podem ser extraídos em qualquer diretório, mas recomendamos instalar o código-fonte de exemplo arquivos de código no / opt / *ProgEmbSys* / capítulo 12 diretório.



Não cobrimos a construção do kernel Linux que é executado na placa Arcom. Para obter informações adicionais sobre a construção do kernel Linux, consulte o *Manual técnico do Arcom Embedded Linux*.

Construindo os exemplos do Linux

Para construir qualquer um dos exemplos do Capítulo 12 Linux, proceda da seguinte forma.

1. Abra uma janela de terminal e mude para o diretório dos exemplos que você gostaria de construir. Por exemplo, para construir o *piscar* exemplo, digite o comando:

```
# cd / opt / ProgEmbSys / capítulo12 / blink
```

2. Em seguida, crie o código de exemplo usando o *makefile* com o seguinte comando:

```
# faça
```

Isso deve produzir dois arquivos executáveis chamados *piscar* e *blinkdbg*.

Baixando e executando os exemplos do Linux

Para baixar os exemplos e executá-los na placa Arcom, primeiro inicialize o Linux embarcado. Certifique-se de conectar a porta COM1 da placa Arcom à porta serial do seu PC e de estar executando um programa de console, como o minicom.



Certifique-se de ter a placa Ethernet da placa Arcom conectada à placa principal. Em seguida, conecte um cabo Ethernet entre a placa Arcom e seu computador (diretamente ou por meio de um hub). As instruções para conectar a placa Ethernet são mostradas no *Manual Técnico Arcom VIPER* e a *Manual Técnico VIPER-I / O*.

As instruções a seguir também presumem que um servidor DHCP (Dynamic Host Configuration Protocol) esteja presente em sua rede. Isso permite que a placa Arcom obtenha um endereço de protocolo de Internet (IP) dinâmico. Se você não tiver um servidor DHCP em sua rede, consulte o *Manual Técnico Arcom Embedded Linux* seção sobre como configurar estaticamente uma interface de rede.

Ligue a placa Arcom e permita que o RedBoot execute o script de inicialização do Linux. Você deve ver uma saída semelhante à seguinte quando o Linux começar seu processo de inicialização:

```
RedBoot> clock -l 27 -m 2 -n 10
mem: 99,532 MHz execução: 199,065 MHz turbo: 199,065 MHz. cccr 0x141 (L = 27 M = 1 N
= 1.0) RedBoot> mount -t jffs2 -f filesystem
RedBoot> load -r -b% {FREEMEMLO}% {kernel}
Usando protocolo padrão (arquivo)
Arquivo bruto carregado 0x00400000-0x004d4c3f, entrada assumida em 0x00400000
RedBoot> exec -c% {cmdline}
Usando endereço de base 0x00400000 e comprimento
0x000d4c40 Linux descompactando .....
```

Depois que o Linux for inicializado com sucesso, você deverá ver o prompt de login da placa Arcom:

login do Viper:

Você pode baixar e executar os exemplos do Linux seguindo estas etapas:

1. No prompt de login do conselho, digite raiz para o nome de login e arcom para a senha. A placa Arcom deve, então, gerar um prompt de comando:

```
root@ viper root #
```

2. Em seguida, baixe o programa do computador host para a placa Arcom. Por exemplo, para baixar o *piscar* Por exemplo, abra uma janela de terminal no sistema host Linux e digite os seguintes comandos:

```
# cd / opt / ProgEmbSys / capítulo12 / blink
# scp blink 192.168.0.4:/tmp/blink
```

Pode ser necessário substituir o endereço IP (192.168.0.4) pelo endereço IP apropriado apropriado para sua placa Arcom.

Em seguida, você será solicitado a inserir a senha do conselho, conforme mostrado aqui:

```
senha de root@192.168.0.4 :
```

Digite a senha arcom. O download ocorrerá e o terminal deve mostrar uma saída semelhante à seguinte:

```
piscar 100% 3620 3,5 KB / s 00:00
```

3. Para executar o programa baixado, digite o seguinte no prompt da placa Arcom:

```
root @ viper root # / tmp / blink
```

Se o programa foi baixado corretamente, o LED verde deve estar alternando.

Os exemplos do Linux assumem o controle da placa Arcom e devem ser executados para sempre. Para encerrar um exemplo específico, pressione Ctrl-C no console; a placa Arcom deve abortar o programa e retornar ao prompt VIPER-Lite.

Exemplos de depuração de Linux embarcado

Esta seção é uma introdução à depuração remota; instruções adicionais para depurar aplicativos Linux embarcados estão contidas no *Manual técnico do Arcom Embedded Linux*. O procedimento de depuração do Linux embarcado ocorre por meio da conexão Ethernet, em vez de por meio de uma conexão serial.

Depois de baixar um programa, como mostrado anteriormente, você pode iniciar um *gdb* sessão de depuração. O exemplo a seguir mostra como depurar o *piscar* exemplo.

- 1 Inicie a sessão de depuração, lançando o *gdb* processo do servidor na placa Arcom usando o comando:

```
root @ viper root # gdbserver: 9000 / tmp / blink "blink"
```

A placa Arcom irá então esperar que você conecte seu host ao destino, indicando que ele está esperando, exibindo a seguinte mensagem:

```
Processo / tmp / blink criado; pid = 706  
Ouvindo na porta 9000
```

- 2 Comece um *gdb* sessão no host digitando o seguinte em uma janela de terminal:

```
# arm-linux-gdb blinkdbg
```

Você deve então ver o familiar *gdb* prompt (conforme abordado no Capítulo 5), semelhante ao seguinte:

```
GNU gdb 6.1.1  
Copyright 2004 Free Software Foundation, Inc.  
GDB é um software livre, coberto pela GNU General Public License, e você está  
convidado a alterá-lo e / ou distribuir cópias sob certas condições. Digite "show  
copying" para ver as condições.  
Não há absolutamente nenhuma garantia para o GDB. Digite "mostrar garantia" para obter detalhes.  
Este GDB foi configurado como "--host = i386-redhat-linux-gnu --target = arm-linux" ... (gdb)
```

3. Conecte o host ao destino digitando o seguinte comando no host

gdb mensagem:

(gdb) **destino remoto 192.168.0.4:9000**

Você precisa alterar o endereço IP (192.168.0.4) para o endereço IP apropriado para sua placa Arcom.

Após a conexão bem-sucedida, a placa Arcom emite a seguinte mensagem (o seguinte endereço IP pode ser diferente para o seu host):

Depuração remota do host 192.168.0.3

Agora você está pronto para iniciar a depuração! Para obter informações adicionais sobre depuração com *gdb*, consulte o Capítulo 5.

Símbolos

prompt de shell # (sinal de número),
sintaxe de variável \$ () 62 em makefiles,
operador 67 & (AND), 124
 bits de limpeza, 125
 bits de teste, 124
 usando com sobreposição de estrutura,
130 / (barra), sinais baixos ativos, operador 26
<< (deslocamento para a esquerda), 124, 126
>> operador (deslocamento à direita), 124, 126
\\ (barra invertida), continuação da linha em
 makefiles, 68
Operador ^ (XOR), 124
 bits alternantes, 125
 usando com sobreposição de estrutura, 130
_ (sublinhado), comandos do vinculador começando
 com, 61
| (OR) operador, 124
 bits de configuração, 125
 usando com sobreposição de estrutura,
130 ~ (til)
 sinais baixos ativos,
 operador NOT bit a bit 26,
 operador NOT 124
 bits de limpeza, 125
 usando com sobreposição de estrutura, 130

Números

Família de processadores 80x86 (Intel), 35

UMA

reconhecendo uma interrupção, 147, 150
 eCos ISR, 216
 exemplo, 152
 interrupção do temporizador, 160
sinais altos ativos, 26
sinais baixos ativos, 26, 30
Ada, 14
 ônibus de endereço, 29
 sinais, 30
 teste, 108-110
rede de barramento de endereço,
26 decodificador de endereço, 29
Protocolo de resolução de endereço (ARP), 246
espaços de endereço, 28
 Processador PXA255 no chip
 periféricos, 36
endereços, atribuição com o vinculador GNU
 ferramenta, 59
alarmes, 191
pinos de função alternativa, 47
circuitos analógicos, 239
 controlando digitalmente, 240
sinais analógicos, conversão de sinais digitais
 a, 242
APIs (interfaces de programação de aplicativos)
 sistema operacional eCos, 199
 informações adicionais sobre, 213
 sistemas operacionais, diferenças em, 186
 driver de dispositivo serial (exemplo), 137

circuitos integrados específicos de aplicação
(ASICs), 38

Placa Arcom

informações de contato, 265
monitor de depuração (RedBoot), 60, 72
depuração usando gdb, 78
Linux embutido, 219
mapa de interrupção (parcial), mapa de
memória 149 (exemplo), dispositivo de
servidor de impressão 31, processador
PXA255 XScale 20, referência 36,
263-265
Placa de desenvolvimento VIPER-Lite, 17
processo de construção, 61

Processadores ARM, 36

prioridades de interrupção e exceção, 144
tabela de vetor de interrupção, 148
redefinir endereço e redefinir código, 75
versão Windows do GNU baseado em ARM
ferramentas, 268

ARP (Protocolo de Resolução de Endereço), 246

ASICs (integrado específico de aplicativo
circuitos), 38

montadores, 56

linguagem assembly, 14

interrupções globais, habilitando / desabilitando, 148 acesso
ao registro de espaço de E / S, 48
melhorando a eficiência do software, 250
testes de memória, 113
código de redefinição, 39
código de inicialização, 58
mudanças de contexto de tarefas,

181 eventos assíncronos, 133, 143

execução atômica, 154

variáveis automáticas, 41 sistemas

embarcados automotivos, 2

B

shell bash, 62

BDM (modo de depuração em segundo plano), 85

BGA (matriz de grade de bolas), 27

sinais bidirecionais, 27

big-endian, 99

pacote de utilitários binários (binutils), 64, 272

utilitário de tamanho, 66

bit banging, 235

manipulação de bits

bitfields, 127

bitmasks, 126

bitfields vs., 128

macro, 127

bits de limpeza, 125

bits de configuração, 125

bits de deslocamento, 126

bits de teste, 124

bits alternantes, 125

operadores bit a bit, 124

usando com bitmasks, 126

usando com sobreposição de estrutura para periférico
registros, 130

Programa de LED piscando (exemplo), 44-51

processo de construção, 61-66

compilar, 62

formatando arquivo de saída,

64 vinculação e localização, 63

makefile, usando, 67

função delay_ms, 50

baixando, 70-76

função ledInit, 46-49

função ledToggle, 50

script de linker, 60

usando temporizador de hardware para LED

pisca, 155-161

bloqueio, 181

Pranchas

questões básicas sobre, 20

(veja também a placa Arcom)

bootloader, 72

BOOTP (protocolo de bootstrap), 246 pontos

de interrupção

hardware e software, suporte por
emuladores, 85

removendo com gdb, 84 configuração

com depurador gdb, 80 processo de

compilação, 54

automatizando com makefiles, 66-69

Programa de LED piscando, 61-66

compilar, 62

arquivo de saída de formatação,

64 vinculação e localização, 63

compilação, 56

vinculando arquivos de objetos, 57-59

localização, 59

queimador, 75

redes de ônibus, 26

conectores fora da página, 27

ônibus

eu₂ C (circuito integrado), 233

diagramas de tempo, 30

botões, debouncing, 208, 213, 226

comunicação orientada a bytes, 78

C

Linguagem C, 13, 14

- operadores bit a bit, 124

- compilando fonte de programa de LED piscando
 - arquivos com gcc, 62 tipos de dados

- inteiros de tamanho fixo, ISO

 - padrão, 16**

 - arquivos de cabeçalho

 - mapa de memória para placa Arcom, 32

 - registros do processador PXA255, 37 arquivos

 - de cabeçalho para placas, 31

- pilha usada por programas, 59**

- biblioteca padrão para uso em**

 - sistemas, 58**

- código de inicialização para programas, 59

- sobreposições de estrutura no periférico

 - registros, 129**

- funções de wrapper para registro de espaço de E / S
 - acesso, 48**

C ++, 14, 15

- penalidade de desempenho, limitação,

- roteadores e firewalls de modem a cabo 259-261,

- cache 243, permitindo evitar memória externa

 - acessos, 259**

- capacitores, valores para, unidade de

- processamento central 24 (consulte CPU)

- Compilador frontal C, 260

- somas de verificação, 114

- sinais de seleção de chip, 29

 - Processador PXA255, 40

- CKEN (Clock Enable Register), 259**

- classes, C ++, 260**

- CLK (sinal do relógio), 30

- alongamento do relógio, 233

- relógios

 - reduzindo a frequência para conservar

 - poder, 258**

 - usado para temporizadores,

- código 156

 - eficiente, 11**

 - exemplo de código para este livro, 270

 - otimizando

 - diminuindo o tamanho do código,

 - 252-254 aumentando a eficiência, 249-252

 - portabilidade ou reutilização, 12

 - robusto, 12**

- diretrizes / padrões de codificação, 15

- COFF (Common Object File Format), 56 shell de comando, 62**

- interface de linha de comando (CLI), 71

 - comunicando-se com RedBoot CLI, 72

 - gdb (depurador GNU), 78, 84**

 - Programa de monitoramento e controle

 - (exemplo), 165, 167-172**

 - comandos**

 - vinculador, começando com sublinhado (_), 61

 - RedBoot, 73**

 - FIS, 76**

- Formato de arquivo de objeto comum (COFF), 56

- compiladores

 - C ++, 260**

 - Compatível com C99, 17

 - tamanho inteiro, escolha, 16

 - formatos de arquivo de objeto, 56

 - otimização

 - depuração e, 83**

 - problemas com, 254**

 - propósito de, 56**

 - compilando, 56

 - Programa de LED piscando, 62 dispositivos

- lógicos programáveis complexos

 - (CPLDs), 237**

- tipo de componente (símbolos IC), 24

- Concurrent Versions System (CVS), 91

- variáveis de condição, 191, 231

- conectores

 - net, 27**

 - fora da página, 27

- construtores (C ++),

- contexto 261

 - processador, 150

 - salvar e restaurar, 160, 216**

 - tarefa, 182**

- chaves de contexto, 181, 193

- barramento de controle, 29

- registros de controle, 122

 - manipulação de bits, 124-128

 - mapeado em memória, 123**

 - sobreposições de estrutura, 129

- multitarefa cooperativa, 174**

- contadores de custos (ver custo de desenvolvimento; custo de produção), 191

 - temporizadores vs., 156

- CPLDs (Complex Programmable Logic**

 - Dispositivos), 237**

- CPU (unidade de processamento central), conector

- off-page 35 CPU_RESET, 27 CRCs (verificações de redundância cíclica), 116-118

seções críticas, 154

desativando interrupções para, 193

protegendo com mutexes sem desativar

interrupções, 187

compilador cruzado, 56

saída de arquivo de objeto, 56

depuração de plataforma cruzada, 77

CVS (Concurrent Versions System), 91 verificações

de redundância cíclica (CRCs), 116-118 Cygwin,

configuração, 267

D

barramento de dados, 29

sinais, 30

teste, 106

rede de barramento de dados, 26

fluxo de dados através do sistema, 20

databooks, processador, 34 DataDisplay

Debugger, 78 datasheets, componentes de

hardware, 22 eliminação de código morto, 254

prazos, 192

impasse, 188

botões e interruptores de debouncing, 208, 213,
226

monitores de depuração, 60, 71-74, 77

desvantagens de usar, 78

RedBoot, 72-74

Emuladores de ROM vs.,

85 depuração, 77-92

otimização do compilador e, 83

cavando no hardware, 92

emuladores, usando, 84

hardware, 87-91

programas otimizados, 255

depuradores remotos, usando, 77-84

simuladores, usando, 86

software em execução a partir da memória flash ou

ROM, 76

controle de versão, 91

ferramentas de depuração, 12

valores de parâmetro padrão (C ++), 261

rotina de serviço adiado (consulte DSR)

função delay_ms, 50

medindo o tempo gasto em, 89

memória de desreferenciação,

256 requisitos de design, 7

exemplos de design incorporado, 8-11

relógio digital, 9

Mars rover, 10

jogador de videogame, 9

faixa de valores para sistemas embarcados, 8

destruidores (C ++), 261

sistemas operacionais determinísticos, 193

desenvolvedores, software embarcado, custo de

desenvolvimento 11-13, 7

relógios digitais, 9

jogadores de videogame, 9

ferramentas de desenvolvimento, 12

GNU, 55

configuração, 266-270

drivers de dispositivo, 5, 39, 130-141

filosofia de design, 130-133

benefícios de um bom design, 131

especificações de hardware ocultas,

132 design, 140

sistema embarcado vs. PC

contrapartes, 131

Programa de monitoramento e controle

(exemplo), 165

driver de dispositivo serial (exemplo), 133-140

API, 137

funcionalidade de extensão,

rotina de inicialização 139,

interface de registro 136, 135

variáveis de estado, 135

teste, 138

programador de dispositivo, 75, 96

registros de dispositivo

mapeado na memória, ponteiros para,

254 variáveis comuns na memória local

vs., 123

DHCP (Configuração Dinâmica de Host
Protocolo), 246

processadores de sinal digital (DSPs), 36

programa bootloader, 72

sinais digitais, 240

relógios digitais, 9

desativando interrupções, 191, 193

driver de vídeo, 44

divisor, 116

DMA (acesso direto à memória), 98
 canais, 36
 usando controlador DMA para conservar
 poder, 258

DNS (Sistema de Nomes de Domínio), 247

download de software incorporado, 70-76
 monitor de depuração, usando, 71-74
 depuradores remotos, usando, 77-84
 em ROM, 75

DRAM (RAM dinâmica), 94
 controladores DRAM, 94, 95

DSPs (processadores de sinal digital), 36
 programa bootloader, 72

DSR (rotina de serviço diferido), 150, 192
 manuseio de interrupção eCos, barramento de

linha dupla 213-218, 30

Dispositivo de Duff, 252

ciclos de trabalho, 240
 circuito PWM simples, protocolo de
 configuração dinâmica de host 241
 (DHCP), 246

programadores dinâmicos, 183

E

programador mais cedo prazo-primeiro, 175

sistema operacional eCos, 198-218

APIs
 informações adicionais sobre, 213
 POSIX, 199

tratamento de interrupções, passagem de
 mensagens 213-218, módulos de suporte de
 rede 210-213, 247 recursos para informações
 adicionais, 199 configuração de
 desenvolvimento

meio ambiente, 274-276

mecânica de tarefas, 199-202

tarefas
 sincronização mutex, sincronização de
 semáforo 202-205, interrupções sensíveis a
 borda 205-209, 146

EEPROM, 96, 255
 testes de memória ativados,

102 eficiência de código, 11
 aumentando, 249-252

fiação elétrica causando memória
 problemas, 103

ELF (formato executável e vinculável), 56
 convertendo para arquivo Intel Hex Format, 65

aplicações embutidas
 Monitorar e controlar (exemplo), 164-172
 interface de linha de comando, 167-172
 portas seriais, 166

padrão C ++ incorporado, 260

programação incorporada, 43-52
 "Olá Mundo!" (exemplo), 43
 Programa de LED piscando (exemplo), 44-51
 função delay_ms, 50
 função ledInit, 46-49
 função ledToggle, 50
 loops infinitos, 51

sistemas embarcados
 componentes comuns, 4
 definido, 1
 requisitos de design, 6
 história e desenvolvimentos futuros, 2
 camadas de software, 132

Projeto de Sistemas Embarcados
 "Design for Debugability", 27
 "Introdução ao In-Circuit
 Emuladores ", 85
 "My Favorite Software Debouncers", 208
 "Selecting a Real-Time Operating
 Sistema ", 197

Programação de sistemas embarcados (ver
 Projeto de sistemas embarcados)

emuladores
 JTAG, 85
 teste de memória, em execução a partir de,
 114 ROM, 85

endianness, 98-102
 em dispositivos, 99
 em rede, 100
 solução de macros do pré-processador, 101

ponto de entrada do programa, 74

EPROM (apagável e programável
 ROM), 96

documentos de errata para dispositivos, 23

sinalizadores de evento, 191

exceções, 143
 prioridades no processador ARM, 144

Executable and Linkable Format (ELF), 56
 convertendo para arquivo Intel Hex Format, 65

imagem binária executável
 conversão de programa relocável em, 59
 conversão de fonte de software embutido
 código para, 54

imagem binária executável (*continua*)
 baixando
 em ROM, 75
 depuradores remotos, usando, 77-84
 baixando para o sistema incorporado, 70-76
 formatação para o programa de LED piscando,
 64 gerando com o utilitário make, 68
 tamanho, localização, 66
memória externa, reduzindo o acesso a, 259
periféricos externos, 38, 122

F

Sistema de arquivos FAT (File Allocation Table), 121
FFUART (Full Function UART), 134
Field Programmable Gate Array (FPGA),
programação 238
FIFO (first-in-first-out)
 algoritmo, 174
sistema de arquivos, criando com memória flash, 121
firmware, 11
FIS (Flash Image System), 76
inteiros de largura fixa,
aritmética de 16 pontos fixos, 251
programação de prioridade fixa, limitações de,
183
memória flash, 76, 97, 118-121
 criando sistema de arquivos com, 121
 driver de dispositivo, 131
 drivers de flash, 120
 questões a serem consideradas nas atualizações de campo,
 118
 armazenando dados fixos, 255
teste, 102
 trabalhando com, 119
pino flutuante, 41
aritmética de ponto flutuante, 251
fluxo de dados através do sistema, 20
Forth, 14
FPGA (Field Programmable Gate Array), 238
Full Function UART (FFUART), 134
modo full-duplex
 interface serial, 233
 SPI, 236
funções
 verificando a execução com gdb, 83
 símbolos de depuração para, 82
 inline, 249
 tabela de símbolos em arquivo de objeto, 57
 referências não resolvidas em arquivos de objeto, 58

G

GAL (Generic Array Logic), 237
jogadores, vídeo, 9
Ganssle, Jack, 15
gcc (compilador GNU C), 17, 56
 __attribute__ palavra-chave, 151
 comando do compilador para LED piscando
 programa, 62
 invocar o vinculador indiretamente, 68
 arquivos de objetos de vinculação, 65
 otimização, 83
 opções de otimização, 254
 configuração, 272
 site, 56, 62
gdb (depurador GNU), 78-84
 pontos de interrupção
 removendo, 84
 alterando o valor do símbolo com impressão
 comando, 81
 verificando os valores dos símbolos,
 81
 verificando quais funções têm executado, 83
 comandos, informações sobre, 84
 problemas de conexão, 79
 valores de registro do processador, visualização e
 impressão, 83
 configuração, 273
 código-fonte de etapa única, pesquisa de
 valor de símbolo 82 usando memória
 mapa, 82
computadores de uso geral, 1
 interfaces para sistemas
embarcados, 2
 pinos de uso geral, 47
 Pinos de E / S, processador PXA255, 48
gerador polinomial, 116
Generic Array Logic (GAL), 237
bit de ativação / desativação de
interrupção global, 147
variáveis globais, 251
 declarando com a palavra-chave volátil, 254
ferramentas GNU, 17, 55
pacote binutils, informações do site,
compilador 66 C (ver gcc)
depurador (ver gdb)
pacote libgloss, 59
ligante (ld), 57
 atribuição de endereço, 59
 comando vinculador e localizador, 63

- scripts de linker, 61
- make utility**, 66-69
- objcopy**, 65
- utilitário de tamanho (pacote binutils), 66
- desenvolvimento de software, configuração
 - para cima, 266-270
 - exemplo de instalação de código, 270
 - Linux, 268
 - Instalação de host do Windows, ferramentas de software 267-268, construção, faixa 271-273, 64
- GoAhead WebServer**, 247
- declarações goto, 253
- GPIO Pin Level Register (GPLR)**, 50
- GPIO Pin Output Clear Register (GPCR)**, 50
- GPIO Pin Output Set Register (GPSR)**, 50
- pinos GPIO, processador PXA255, 46
- Registros GPIO, processador PXA255**, 47
- terra, símbolos para, 23

H

- modo half-duplex (interface serial), 233
- conjunto codificado à mão, 250
- sistema em tempo real rígido, 3
- hardware, 4, 19-42, 92
 - pontos de interrupção, 85
 - fluxo de dados através do sistema, 20
 - planilhas de dados para componentes, 22
 - ferramentas de depuração, 87-91
 - requisitos de design**, 6
 - problemas de fiação elétrica, 103
 - exemplos de projetos incorporados, 8-11
 - relógio digital, 9
 - Mars rover, 10
 - player de videogame, 9
 - plataforma incorporada, 15
 - desenvolvimento de software embarcado e, 11
 - componentes de sistema embarcado, 4
 - documentos de errata para dispositivos, 23
 - periféricos externos, 38
 - operação geral do sistema, 19-21
 - ocultação com drivers de dispositivo, 131
 - inicializando, 39-42
 - ambiente de processador e periféricos, 40
 - código de redefinição, 39
 - código de inicialização, 41
 - solução de problemas, 41

- Linux, acessando, 220
- mapa de memória, 29-32
- comunicação do processador com periféricos, 32
- processadores, 28
- esquemas**, 21
 - fundamentos**, 23-28
- estado de rastreamento de, 135
- verificação usando simulador, 87
- abstração de hardware, 130
- arquivos de cabeçalho para placas, 31
- cabeçalhos, arquivo de objeto, tamanho de pilha 56, limitação, 256
- "Olá Mundo!" programa (exemplo), 43
- computador host, 54
 - compiladores, 56
- host para macro longa de rede, 101
- host para macro curta de rede, 101
- HTML para dispositivos com base na web
 - gestão**, 243
- HTTP (protocolo de transferência de hipertexto), 246
- dispositivos de memória híbrida, 96, 255
 - resumo de**, 97
- HyperTerminal**, 72

eu

- Espaço de E / S, 28
 - acesso de registro, 48
- eu : C (barramento de circuito integrado), 233
- ICE** (emulador de circuito), 12, 84
- ICMP** (mensagem de controle da Internet Protocolo), 246
- ICMR** (máscara de controlador de interrupção Register), 147, 159
- ICs** (circuitos integrados), 2
 - específico do aplicativo (ASICs), 38
 - conservando a energia usada, 256
 - identificando pinos específicos, 90
 - representações esquemáticas, 24
 - modo inativo, 258
- tarefa ociosa, 182
- IGMP** (Internet Group Management Protocolo), 246
- IM** (máscara de interrupção), 147
- chips de memória inseridos incorretamente, 105
- emulador no circuito (ICE), 12, 84
- teste de incremento, 110
- indutor-capacitor (LC)**, 242

- indutores, valores para, 24
- loops infinitos, função de, 51
- inicializando
 - Controlador DRAM, 95
 - driver de dispositivo serial (exemplo), 136
 - temporizadores, 159
 - (ver também hardware,
- inicialização) funções inline, 249
- entrada e saída, 5
 - relógios digitais, 9
 - Pinos de I / O de processadores embutidos, 47
 - sinais de I / O, usando para depuração e medição de desempenho, pino de entrada de 88 no processador em desconhecido estado, 41
 - E / S mapeada em memória, 28
 - dispositivos de saída, 43
 - sinais, 27
- GUI do Insight, 78
- inteiros
 - endianness, 98
 - tamanho de, 16
- integração de circuitos
- integrados (ver ICs), 71
- Arquivo Intel Hex Format, 65
- processadores Intel
 - Família 80x86, 35
 - espaços de endereço, 28
 - PXA255 XScale, vendido para Marvell
 - Grupo de Tecnologia, 17
- Circuito Inter-integrado (I² C) barramento, 233
- periféricos internos, 122
- Organização Internacional para Padronização (ISO) padrão C (C99), 16
- Internet Control Message Protocol (ICMP), 246
- controlador de interrupção, 144
 - configurando para temporizador, 159
 - Registro de máscara de controlador de interrupção (ICMR), 147, 159
- manipuladores de interrupção (consulte
- ISRs) palavra-chave de interrupção, 150
- interromper aninhamento, 145
- tabela de vetor de interrupção de rotinas de serviço de interrupção (ver ISRs), 148
- interrupções, 33, 134, 142-163
 - drivers de dispositivo, níveis de prioridade para, 140
 - desativação, 193
 - sensível à borda, 146

- habilitando e desabilitando, 147
- manuseio
 - sistema operacional eCos, 213-218
 - sistemas incorporados usando sistema, 191
 - Sistema operacional Linux, 231
- implementando para UARTs, 140
- LED piscando aprimorado
 - programa, 155-161
- ISRs (rotinas de serviço de interrupção), latência 150-155, 193
- sensível ao nível e à borda, 146
- fonte de mapeamento para função ISR, visão geral 148-150, 142
- prioridades, determinar, 144
- informações do banco de dados do processador, 34
- eventos relacionados e, 143
- resumo dos pontos-chave, 161-163 (ver também ISRs)
- ISO (Organização Internacional para Padronização) padrão C (C99), 16
- ISRs (rotinas de serviço de interrupção), 33, 133, 150-155, 192
 - ações realizadas por, 150
 - determinando a fonte de interrupções, 151, 152
 - Manipulação de interrupção eCos, mapeamento 213-218
 - entre fontes de interrupção e Funções ISR, 148-150
 - pilhas de interrupção do sistema operacional, 256
 - dados compartilhados e condições de corrida, 153-155
 - temporizador, 160
 - usando votação em vez de, 251

J

- Depuradores JTAG, 85
- jumpers, designador de referência esquemática, 27
- junções, 26

K

- kernel, 174
- Linux, 219

eu

- último a entrar, primeiro a sair (LIFO), latência 41 (interrupções), 152, 191, 193
- frouxidão, 176
- LC (indutor-capacitor), 242

- Ferramenta ld (linker), 57
 - atribuição de endereço, 59
 - invocando direta ou indiretamente em **makefile**, 68
 - comando vinculador e localizador, 63
 - multilibs, especificando, 65
- função ledInit**, 46-49
- LEDs (diodos emissores de luz), 23
 - depurando com, 88
 - (veja também o programa de LED piscando) função **ledToggle**, 50
- interrupções sensíveis ao nível, 146
- pacote **libgloss** (GNU), 59
- bibliotecas
 - evitando rotinas padrão para diminuir tamanho do código, 253
 - multilibs, vinculando via **gcc**, biblioteca C padrão 65 para uso em embarcações sistemas, 58
- vida útil do sistema, 8 LIFO (último a entrar, primeiro a sair), 41
- diodos emissores de luz (consulte LEDs) IP leve (**lwIP**), 246
- script de linker, 60, 64
- Arquivos de script do vinculador GNU, site, 61
- arquivo **viperlite.ld**, 63
- ferramenta de vinculação (ver ferramenta **ld**)
- arquivos de objeto de vinculação, 57-59
 - Programa de LED piscando, compilador 63 **gcc**, usando, 65
 - utilitário **make**, usando, 68 código de inicialização, 58
- programa de **fiapos**, 90
- Sistemas Linux
 - escolher usar com incorporado sistemas, 198
 - shell de comando, 62
 - ambiente de desenvolvimento integrado, configuração, 277-283
 - exemplos incorporados, 219-231
 - acessando hardware Linux, 220
 - ferramentas de desenvolvimento de software GNU, 268 troca de mensagens, 227-231
 - minicom (programa de terminal), suporte de rede 72, 247
 - recursos para mais informações, 231
 - tarefas
 - mecânica de, 220-222
 - sincronização mutex, sincronização de semáforo 222, 224-227
 - little-endian, 99

- variáveis locais, 41
- localizando arquivos de objetos
 - Programa de LED piscando, 63
 - utilitário de **make**, usando, 68
 - localizadores, 59
- bloqueando o planejador, 179
- bloqueios (Linux), 231
- analísadores lógicos, 87
- dispositivos lógicos, 133
- desenrolamento de loop, 252
- lwIP** (IP leve), 246

M

- caixa de correio, 190
- programa principal, 41
 - Programa de LED piscando usando um cronômetro, 157
 - condições de corrida com **ISR** para compartilhamento recurso, 155
- make utility**, 66-69
 - alvos de construção, 68, 69
 - recursos para mais informações, 69
- makefiles**, 66-69
 - regra de construção, layout básico, programa de LED piscando de 67 edifícios, 67 instruções de execução de construção, 69 mantendo-se atualizado, 69
 - variáveis, 67
- função malloc**, 256
- arquivos de mapas, 64
 - ponto de entrada do programa, 74
- mapas, interrupção, 148-150
- Mars Pathfinder, 10
- Mars rover, 10
- interrupções mascaráveis, 144
 - habilitando e desabilitando, 147
- ROMs mascarados, 95
- mestre (eu: Ônibus C), 233
- Sinal Master In Slave Out (MISO), 236
- Master Out Slave In (MOSI) sinal, 236
- memória, 4, 7, 93-121
 - operações básicas realizadas pelo **RedBoot**
 - monitor de depuração, 60
 - dispositivo, 111
 - relógios digitais, 9
 - DMA (acesso direto à memória), 98
 - questões endian, 98-102
 - externo
 - chip-seleciona para o processador **PXA255**, 40
 - reduzindo o acesso para conservar poder, 259

memória (*continua*)

- memória flash, 76, 118-121 código de carregamento com bootloader, 72
- periféricos vs., 28
- localização física, estabelecendo para
 - seções relocáveis do programa com GNU vinculador, 60
- conexões de processador, 28
- reduzindo o uso, 255
- restrições de recursos para embutidos sistemas, 12
- estratégia de teste, desenvolvimento, 105-114
 - teste de barramento de endereço, 108-110
 - teste de barramento de dados, 106
 - teste do dispositivo de memória, 110-112 exemplo prático, teste 112-114, 102-105
 - problemas comuns, 103
 - problemas de fiação elétrica, 103
 - chips inseridos incorretamente, 105
 - chips de memória ausentes, 104
 - tipos de, 93
 - híbrido, 96
 - ROM, 95
- conteúdo de validação, 114-118
 - somas de verificação, usando, 114
 - verificações de redundância cíclica (CRCs), 116-118
- jogadores de videogame, 10
- mapas de memória, 29-32
 - Arquivos de cabeçalho em linguagem C para placas, 31 criando, 30
 - depurar valores de símbolo, olhando para cima, 82
- espaço de memória, 28
 - Controle periférico do processador PXA255 registros, 48
- E / S mapeada em memória, 28
- passagem de mensagem, 190
 - sistema operacional eCos, 210-213
 - sistema operacional Linux, fila de mensagens 227-231, 190
- pilha micro IP, 247
- microcontroladores, 35
- microprocessadores, 2, 35
- Microfio (interface serial), 234
- miliwatts por MIPS (mW / MIPS), 7
- minicom (programa de terminal), 72

- primeira programação de frouxidão mínima, 176
- MIPS (milhões de instruções por segundo), sinal
- 7 MISO (Master In Slave Out), 236 chips de memória ausentes, 104
- função mmap, 220
- frequência de modulação, circuitos PWM, aplicativo de monitoramento e controle 241
 - (exemplo), 164-172
 - interface de linha de comando, 167-172
- portas seriais, funcionando com sinal 166
- MOSI (Master Out Slave In), 236 Motorola
- SPI (periférico serial interface), 234
- multilibs, vinculando via gcc,
- 65 multitarefa, 173
 - agendamento de tarefas, 174
- mutexes (exclusões mútuas), 187
 - impasse e inversão de prioridade, 188-190
 - sistema operacional eCos, 202-205
 - Sincronização de tarefas do Linux, 222
 - inversão de prioridade, 189
- mW / MIPS (miliwatts por MIPS), 7

N

- prefixo n (nomes dos sinais), 26
- National Semiconductor, Microwire, 234
- pinos NC (sem conexão), 26
- interrupções de aninhamento, 145, 192
- rótulos de rede, 26
- redes, 26
 - rede de ônibus, 26
 - conectores fora da página, 27
- ordem de byte de rede, 100 rede para hospedar macro longa, 101 rede para hospedar macro curta, 101 rede
- endianness, 100
 - suporte em dispositivos incorporados, 242-247
 - benefícios da adição, 243
 - soluções para, 244-247
- pacote newlib (Cygnus), 58 sem conexão (pinos do processador), 26
- efeitos de ruído, minimização com PWM, 242
- interrupções não mascaráveis, 144, 147
- NRE (engenharia não recorrente), 7
- número de unidades, 7
- NVRAM (RAM não volátil), 97

O

- ferramenta objcopy, 65
- arquivos objeto, 54, 56
 - conteúdo, 56
 - copiando com objcopy, 65
 - vinculando, 57-59
 - Programa de LED piscando, compilador 63 gcc, usando, código de inicialização 65, 58
 - localizando, 59, 63
 - removendo seções com utilitário strip, 65 tamanhos de seção e tamanho total de arquivo, listagem, 66 código de inicialização, 57
- conectores fora da página, 27
- periféricos no chip, 36, 122
- dispositivos programáveis de uma só vez (OTP), 96
- pinos de processador aberto, 26
- software de código aberto, 53
 - Ferramentas GNU, 55
- fios abertos, 103
- OpenTCP, 246
- sistemas operacionais, 173-197
 - drivers de dispositivo vs., 131
 - eCos
 - exemplos, 198-218
 - configurando o desenvolvimento meio ambiente, 274-276
 - suporte ao compilador gcc, 56
 - história e propósito, 173
 - tratamento de interrupções, 191 kernel, 174
 - falta de API comum, 186
- Linux
 - ambiente de desenvolvimento integrado, configuração, 277-283
 - exemplos incorporados, 219-231
 - passagem de mensagens, 190
 - pilhas de rede, 247
 - em tempo real, 3, 6, 255
 - características de, 192
 - decidindo se usar, 194 outras funcionalidades, 191
 - selecionando, 195-197
 - programadores, 174-180
 - bloqueio e desbloqueio, 179
 - em tempo real, 175-178
 - pontos de agendamento, 178
 - plataforma de destino para embutido software, 54

- tarefas, 180-185
 - mecânica de, 184
 - estados de, 180
 - sincronização, 185-190 (ver também sistemas Windows)
- sobrecarga de operador (C ++), 261
- otimizações, 71, 248-261
 - eficiência de código, aumentando, tamanho de código 249-252, diminuindo, compilador 252-254, depuração e, 83
 - limitação do impacto de C ++, 259-261
 - economia de energia, 256-259
 - problemas com a otimização de compiladores, 254
 - reduzindo o uso de memória, 255
- osciloscópios, 87
 - conectando corretamente, 88
 - LED e sinal de E / S, depuração com, 88
- dispositivos OTP (programáveis uma vez), 96
- saídas (ver entrada e saída)

P

- PAL (Programmable Array Logic), 237
- valores de parâmetro, padrão (C ++), 261
- Pathfinder mission (NASA), 10 peeking (leitura), 72
- interrupções pendentes, 145
- desempenho
 - medição, usando sinal I / O e osciloscópio, 89
 - sistemas em tempo real, 192 (veja também otimizações)
- periféricos, 122-141
 - manipulação de bits (ver manipulação de bits)
 - comum, 232-242
 - lógica programável, modulação de largura de pulso 236-239 (PWM), 239-242
 - interface periférica serial (SPI), 234
 - registros de controle e status, 122
 - manipulação de bits, 124-128
 - mapeado em memória, 122
 - sobreposições de estrutura, 129
 - drivers de dispositivo, 130-141
 - filosofia de design, 130-133 design, 140
 - driver de dispositivo serial (exemplo), 133-140
 - desativando para conservar energia, 259
 - externo, 38

periféricos (*contínuo*)

- inicializando, 40
- interfaces, 11
- mapa de memória para placa Arcom (exemplo), 31
- mapeado na memória, 28
- no chip, 36
- operando enquanto o processador está ocioso
- modo, 258
- comunicação do processador com, 32
- conexões do processador para, 28
- Registros de controle periférico PXA255, 48
- simuladores e, 86
- Philips I₂ Especificação C, 233
- pinos, processador, 26
 - identificação de pinos específicos em um IC, pino de entrada 90 em estado desconhecido, processador 41 PXA255
 - pinos GPIO bidirecionais, 46
- PLA (Programmable Logic Array), plataforma 237, sistema embarcado, 15
- PLD (dispositivo lógico programável), 237
- protocolo ponto a ponto (PPP), 242
- ponteiros para registros mapeados na memória, 123, 254
- cutucando (escrevendo), 72
- votação, 33, 134
- UARTs e, 140
- usando em vez de ISRs, 251
- polimorfismo (C ++), 261
- POP (Post Office Protocol), Versão 3 (POP3), 246
- pinos da porta, 47
- portabilidade de código, 12
 - verificar o código-fonte em busca de problemas, usando fiapo, 90
- POSIX (Sistema Operacional Portátil Interface), 186
 - sistema operacional eCos, padrão 199
 - threads, API Linux, consumo de energia 219, 7
- poder, conservando, 256-259
 - frequência de relógio, 258
 - modos de processador, 257
 - reduzindo o acesso à memória externa, 259
- poder, símbolos para, 23
- endereços de potência de dois, 108
- PPP (protocolo ponto a ponto), 242
- # pragma, declarando um ISR, 151

preempção, 177

- programação preemptiva, 175
- macros de pré-processador para endianness
 - questões, 101
- servidor de impressão, projeto, 20
- tetos prioritários, 190
- herança de prioridade, 189
- inversão de prioridade, 189
- prioridade, 189
- interrupções, 144, 191
- agendamento baseado em prioridade
 - política de backup para tarefas com o mesmo prioridade, 178
 - prazo mais próximo primeiro, 175
 - fila de tarefas mutex, sistema eCos, algoritmo de prioridade 202, 175
- fila pronta, 181
- algoritmo de prioridade estática, 176
- suporte à preempção, 177
- prioridades de tarefa, configuração, 182-184
 - algoritmo monotônico de taxa (RMA), poder de processamento 183, 7
- contexto do processador, 150
 - salvando e restaurando, 160, 216
- processadores, 28
 - espaços de endereço, 28
- programa bootloade, 72
- comunicação com periféricos, 32
- sinais de controle, outros, 30
- software de depuração em execução em flash
 - memória ou ROM, 76
- relógios digitais, 9
- emuladores, usando para depuração, 84
- endianness, 99
- suporte gcc (compilador GNU C), 56
- suporte ao compilador gcc, 56
- tamanho inteiro e, 16
- aprendendo sobre, 34-38
 - livros de dados, 34
 - placa de desenvolvimento, 35
- em geral, 35
- Informações da Internet, 34
- PXA255 XScale, 36
- linguagem de máquina, 56
- Mars Pathfinder, 10
- mapa de memória, 29-32
- requisitos de memória e, 7
- modos de operação para economizar energia, 257
- valores de registro, visualização e impressão com gdb, 83

- redefinir regras, 75
- representações esquemáticas, 26
- simuladores, 86
- código de inicialização, 59
- jogadores de videogame,
 - 9 XScale ARM, 17
- custo de produção, 7
- relógios digitais, 9
- jogadores de videogame,
 - 9 profilers, 249
- perfilamento, 71
- lógica programável, 236-239
 - CPLDs (PLDs complexos), 237
 - Field Programmable Gate Array (FPGA), 238
 - Generic Array Logic (GAL), 237
 - Programmable Array Logic (PAL), 237
 - Programmable Logic Array (PLA), 237
 - Programmable Logic Device (PLD), 237
 - ROM programável (PROM), 96
 - linguagens de programação
 - C, 13
 - escolhendo para este livro, 14
 - incorporados (diferente de C), 14 para
 - projetos de hardware PLD, 237
 - (ver também linguagem assembly; linguagem C)
- caderno do projeto, 20
- PROM (ROM programável), 96
- prompts, shell e RedBoot, 62
- protocolos, rede, 243
 - componentes comuns de rede
 - pilhas, 244
 - suporte por pilhas de rede, 246
- pthreads, 219
- palavras-chave públicas e privadas (C ++), 260 PWM (modulação por largura de pulso), 239-242
 - vantagens de, 242
 - aplicações de, 242
 - codificando digitalmente sinal analógico
 - níveis, 240
 - Sinais PWM com tarefas variadas
 - ciclos, 240
 - circuito simples (exemplo),
- processador 241 PXA255
 - diminuição do consumo de energia,
 - controlador 257 DRAM, no chip,
 - registro 95 GPDRO, 47
 - Registros GPIO, 47
 - eu 2 Unidade de interface de barramento
 - C, 233 interrupções suportadas, 144

- modos de operação, 257
- endereço de redefinição, 40
- temporizadores, 156
- exemplos de diagrama de tempo,
 - 30 UART, 134
- Processador PXA255 XScale ARM, 17, 36

Q

- filas, tarefas prontas e em espera, 181

R

- condições de corrida, 153-155
- RAM (memória de acesso aleatório), 4, 93
 - Diagrama de blocos da placa Arcom
 - (exemplo), 31
 - diminuição do uso de, 255 velocidade
 - de execução de código, 72
 - inicialização de hardware e 41
 - dispositivos de memória híbrida, 96
 - testes, 102
- algoritmo monotônico de taxa (RMA),
 - 183 RC (resistor-capacitor), 242
- sinais de leitura e gravação, 29 bloqueios
- de leitor-gravador (Linux), 231 memória
- somente leitura (consulte ROM) estado
- pronto (tarefas), 180
 - tarefa ociosa, 182
- programador executivo em tempo real, 175
- sistema operacional em tempo real (consulte
- RTOS) sistemas em tempo real, 3
- rastreamento em tempo real, 85
- RedBoot, 60, 72-74
 - comandos, 73
 - baixando imagem binária executável, 73
 - monitor de depuração compatível com gdb, 78
 - mensagem de inicialização, 73
 - gerenciamento de ROM, 76
 - suporte de rede, 74
 - prompt (RedBoot>), 62
 - programas em execução, 74
- RedHat Embedded Debug and Bootstrap
 - programa (veja RedBoot)
- designadores de referência, 23
- conectores e jumpers, 27 ICs
- (circuitos integrados), 24
- pontos de teste, 27
- referências, não resolvidas, 57
- variáveis de registro, 250 largura de
- registro (processadores), 7

registra

ICMR (máscara de controlador de interrupção

Registre-se), 147

interface, driver de dispositivo de porta serial

(exemplo), 135

mapeado na memória, ponteiros para, 254

somente gravação, 129

confiabilidade, 8

Mars rover, 10

programa relocável, 54, 58

conversão para binário executável

imagem, 59

restante, 116

depuradores remotos, 77-84

uso por emuladores, 84 protocolo serial

remoto, 78 bits reservados, registros de

processador, 48 endereços de redefinição, 75

redefinir endereço ou vetor de redefinição, 39

código de redefinição, 39, 75

resistor-capacitor (RC), 242

resistores, valores para, 24

restrições de recursos, 12

agendamento de reserva de recursos,

176 recursos

considerações no driver de dispositivo

design, 141

compartilhada entre tarefas operacionais

sistema, 187

compartilhado, causando condições de corrida entre

ISR e programa principal, 155

código reutilizável, 12

Sistema de controle de revisão (RCS), 91

RMA (algoritmo monotônico de taxa), 183

código robusto, 12

ROM (memória somente leitura), 4, 93

Diagrama de blocos da placa Arcom

(exemplo), 31

baixando software embutido em, 75

gerenciamento de ROM com RedBoot, 76

emuladores, 85

velocidade de execução de código, 72

inicialização de hardware e 41 dispositivos de

memória híbrida, 96 movendo dados constantes

para, 255 monitores ROM (consulte monitores

de depuração;

RedBoot)

tipos de, 95

programação round robin, 175, 186

tarefas com a mesma prioridade, 178

RTOS (sistemas operacionais em tempo real), 6, 173,

255

APIs, 186

características de, 192

seções críticas e, 155

decidir se usar, 194

exemplos eCos, 198-218

bloquear e desbloquear o programador, 179

outras funcionalidades, 191

agendamento de tarefas, 175-178

baseado em prioridade, 177

selecionando, 195-197

recursos para informações, 197

estado de execução (tarefas), 180

S

Samek, Miro, 15

intervalos de amostragem para código de debounce,

213 planejadores, 174-180

algoritmos de programação comuns, 174

bloqueio e desbloqueio, 179

promovendo a tarefa para o estado de execução, 181

em tempo real, 175-178

baseado em prioridade, 177

pontos de agendamento, 178

pontos de agendamento, 178

esquemas, 21

fundamentos de, 23-28

Representações IC, 24

conectores fora da página, 27

esquemáticos parciais (exemplo), 25

designadores de referência, 23

símbolos para aterramento e potência, 23

pontos de teste, 27

blocos de título, 28

valores para componentes,

senal 24 SCLK (relógio serial), 233,

senal 236 SDA (dados seriais), 233

semáforos, 187

sistema operacional eCos, 205-209

sinalização para alternar LED, 214

sincronização de tarefas Linux, 224-227

barramentos seriais, assíncrono ou síncrono

conexões, 232

senal de relógio serial (SCLK), 233, 236

senal de dados seriais (SDA), 233

- driver de dispositivo serial (exemplo), 133-140
 - API, 137
 - funcionalidade de extensão,
 - rotina de inicialização 139,
 - interface de registro 136, 135
 - variáveis de estado, 135
 - teste, 138
- interface serial
 - bit banging, 235
- Protocolo de interface de linha serial (SLIP),
- interface periférica serial 242 (SPI), 234
 - estrutura de ônibus, 235
- portas seriais
 - driver de dispositivo (exemplo), diagrama de blocos de porta serial, 134
 - trabalhando com, 166
- dados compartilhados, condições de corrida e, 153-155
- projéteis, 62
- trabalho mais curto primeiro
- agendamento, 174 encurtamento na fiação
- elétrica, 103 tarefas de sinalização
- (interrupções), 192 sinais
 - ônibus de endereço, 30
 - analógico, 239
 - seleção de chips, 29
 - relógio (CLK), 30
 - ônibus de controle, 29
 - barramento de dados, 30
 - digital, 240
 - processadores de sinal digital (DSPs), 36
 - I / O, usando para depuração e desempenho
 - medição, 88
 - interrupção, nível e sensibilidade à borda, 146 níveis lógicos, ativação / desativação, 26 relacionados, representação esquemática como barramento
 - net, 26
 - SPI (interface periférica serial), 236
 - diagrama de tempo, 29
 - requisitos de tempo, 29
 - tipos, representação por off-page
 - conectores, 27
- Simple Mail Transfer Protocol (SMTP), 246
- Simple Network Management Protocol), 245
- simuladores, 86
 - verificação de hardware,
- 87 passo único, 82
- utilitário de tamanho, 66
- Sinais de seleção de escravos (SS1 e SS2), 236
- escravos I: Ônibus C, 233

- SMP (multiprocessamento simétrico)
 - sistemas, 191
- Controlador SMSC Ethernet, 31
- SMTP (Simple Mail Transfer Protocol), 246
- SNMP (Simple Network Management Protocol), 245
- sistema soft em tempo real, 3
- softwares
 - Ferramentas de desenvolvimento GNU, configuração para cima, 266-270
 - Ferramentas GNU, construção, 271-273
 - camadas para um sistema embarcado, 132
 - tarefas (ver tarefas)
- software, incorporado, 5
 - desenvolvedores, requisitos de, 11-13 ciclo de desenvolvimento, 70
- Código fonte
 - verificar se há problemas de portabilidade, usando fiapo, 90
 - software embutido, convertendo para imagem binária executável, 54
 - percorrendo as linhas, 82 controle de versão, 91
- SPI (interface periférica serial), 234
 - estrutura de ônibus, 235
- spinlocks, 191
- Programa Splint, 91
- SRAM (RAM estática), 94
- Sinais SS1 e SS2 (Slave Select), 236
- pilhas, 59
 - endianness, 100
 - interromper, 192
 - teste de memória e,
- 113 rede
 - protocolo de rede comum
 - componentes, 244
 - em sistemas operacionais, 247
 - incluídos no RedBoot, 74 lwIP (IP leve), 246 soluções de código aberto, 246 requisitos de recursos, 243 seleção para seu dispositivo, 244 TinyTCP, 246
 - configuração, 41
 - reduções de tamanho, 255
 - rotinas de biblioteca padrão, 253
 - código de inicialização, 41, 57, 58
 - para programas C, 59
 - inicialização de hardware e, 39 pedidos de arquivos de objetos e, 64

- startup.asm, arquivo crt0.s, 59
- algoritmo de programação de prioridade estática, 176
 - RMA (algoritmo monotônico de taxa), 183
- registros de status, 122
 - manipulação de bits, 124-128
 - mapeado em memória, 123
 - sobreposições de estrutura, 129
- arquivo de cabeçalho stdint.h,
 - utilitário de 16 faixas, 64
- estruturas
 - bitfields, 127
 - sobreposição para registros UART, 135
 - sobreposições em registros periféricos, 129
 - deslocamentos de endereço, 130
- Subversion, 91
- declaração switch, 249
 - substituição inline para, 250 switches,
- debouncing, 208, 213, 226 tabelas de
 - símbolos (em arquivos de objeto), 57
- símbolos
 - alterando o valor com impressão gdb
 - comando, 81
 - verificando valores com depurador remoto, 81
 - depuração, usando mapa de memória para pesquisa,
 - 82 para aterramento e energia, 23
 - remoção do arquivo de objeto, 65
 - não resolvido, 57
- multiprocessamento simétrico (SMP)
 - sistemas, 191
- sincronização (tarefas), 185-190
 - mutexes, 187
 - impasse e prioridade
 - inversão, 188-190
 - sistema operacional eCos, 202-205
 - sistema operacional Linux, 222
 - semáforos, 187
 - sistema operacional eCos, 205-209
- eventos síncronos, 143

T

- pesquisas de tabela, 249
- plataforma de destino, 54
 - Suporte ao compilador gcc, 56
- blocos de controle de tarefas, 182
- tarefas, 173, 180-185
 - contexto, 182
 - mudanças de contexto, 193
 - mecânica de, 184
 - sistema operacional eCos, 199-202
 - sistema operacional Linux, 220-222

- passando dados entre, 190, 210-213
 - Sistema operacional Linux,
- prioridades 227-231, 182-184
 - algoritmo monotônico de taxa (RMA), 183
- planejadores, 174-180
 - bloqueio e desbloqueio, 179
 - em tempo real, 175-178
 - pontos de agendamento, 178
- estados, 180
 - troca de contexto, 181
 - tarefa ociosa, 182
- sincronização, 185-190
 - eCos mutexes, 202-205
 - semáforos eCos, 205-209
 - mutexes Linux, 222
 - Semáforos do Linux, 224-227
 - mutexes e semáforos, 187-190
- Protocolos TCP / IP, 243
 - endianness, 100
 - pilha lwIP (IP leve), 246
 - OpenTCP, 246
 - pilha uC / IP, 247
 - pilha uIP, 247
- Telnet, 247
- programas de terminal, 72
- pontos de teste, 27
- TFTP (Protocolo de Transferência de Arquivos Trivial), 74,
 - 246
- tópicos
 - Modelo Linux, 219
 - (ver também tarefas)
- fração de tempo, 175
- temporizadores, 155-161
 - manuseio de interrupção eCos, 214-218
 - como eles funcionam, 156
 - calculando o intervalo de interrupção, 158
 - apagando interrupções pendentes, 158
 - permitindo interrupções do cronômetro, 159
 - inicializando o cronômetro, 159
 - Processador PXA255, 156
 - configurar temporizador para PXA255
 - processador, 159
 - compartilhamento entre tarefas,
 - 162 watchdog, 157
- diagramas de tempo, 29
 - Processador PXA255, 30
- requisitos de tempo, pilha de
 - rede 29 TinyTCP, bloco de título
- 246 (esquemas), 28
- rastreamento, tempo real, 85
- cama elástica, 148

Transistor-Transistor-Logic (TTL), 237
armadilhas, 34, 143
ônibus tristate, 30
Protocolo de Transferência de Arquivos Trivial (TFTP), 74,
246

solução de problemas

placas baseadas em processador, 39

temporizadores de watchdog, 157

Conector TX1 fora da página, 27 typedefs,

tipos inteiros de tamanho fixo, 16

você

UARTs, 133

Protocolos SLIP ou PPP para rede
interface, 242

Pilha de rede uC / IP, 247

UDP (protocolo de datagrama do usuário), 243,

246 pilha de rede uIP, 247

unidades, número de, 7

Receptor Assíncrono Universal

Transmissor (veja UARTs) desbloqueando
o agendador, 179 espaçonaves não tripuladas (Mars
rovers), 10 inteiros não assinados, ponteiros para
registradores, 124 User Datagram Protocol (UDP),
243, 246 tarefas de usuário, 182

utilização (tarefas), 176

capacidade de programação de tarefas de prioridade fixa, 183

V

variáveis

variáveis de condição (Linux), 231

símbolos de depuração para, 82

driver de dispositivo, estado de rastreamento de

dispositivos, 132

global, 251

makefile, 67

ponteiros para registros periféricos, 123

registro, 250

tamanho de, 251

variáveis de estado para hardware, 135 tabela

de símbolos em arquivos de objeto, 57

Designadores de referência VCC e VDD, tabela de

23 vetores, interrupções, 148

controle de versão, 91

jogadores de videogame,

9 placa VIPER-Lite, 17

kit de desenvolvimento, 263-265

(consulte também a placa Arcom)

funções virtuais (C ++), 261 palavra-

chave volátil, 123

declarando registro mapeado em memória

ponteiros e variáveis globais, 254

níveis de tensão, circuitos, 23

circuitos analógicos, 239

volume, produto, 7

C

estado de espera (tarefas), 180 teste

de caminhada 1, 106

temporizadores de vigilância, 157

navegadores da web, configuração e controle de
sistemas usando TCP / IP, 243

página da web para este livro, xviii

gerenciamento baseado na web, 243

vantagens de, 245

GoAhead WebServer, 247

Sistemas Windows

Shell bash Cygwin, 62

Ferramentas de desenvolvimento de software GNU

instalação de host, código-

fonte 267-268, 266

HyperTerminal, 72

desempenho de pior caso, 192

sinais de gravação, 29

X

protocolo xmodem, 73

Processador XScale ARM, 17

sobre os autores

Michael Barr é uma autoridade líder em design de software para dispositivos eletrônicos. Relacionado a isso, ele prestou testemunho de especialista no Tribunal Distrital dos Estados Unidos, apareceu no programa da PBS “American Business Review” e foi citado em artigos de jornal. Michael também é autor de mais de 40 artigos técnicos e co-autor do *Dicionário de sistemas incorporados*. Por três anos e meio ele atuou como editor-chefe do *Programação de sistemas embarcados* revista.

O software incorporado projetado ou escrito por Michael no início de sua carreira executa milhões de sistemas em todo o mundo, de eletrônicos de consumo a dispositivos médicos. No entanto, hoje Michael constrói negócios em vez de produtos individuais. Ele é CEO da Quantum Leaps, Inc. e fundador da Netrino, LLC. De maneiras diferentes, essas duas empresas ajudam os engenheiros a escrever um software embarcado melhor.

Anthony Massa tem mais de uma década de experiência no desenvolvimento de software embarcado. Ele trabalhou na arquitetura e no desenvolvimento de software para vários produtos em uso hoje, incluindo modems via satélite e a cabo, rádios sem fio, decodificadores e equipamentos de transmissão head-end.

Anthony escreveu vários artigos nas principais revistas de desenvolvimento de software com foco no desenvolvimento de software embarcado e é o autor do livro *Desenvolvimento de software embarcado com eCos*. Anthony é cofundador e engenheiro-chefe de software da Elintrix (<http://www.elintrix.com>), um fornecedor de produtos de rede sem fio e de processamento de sinal. Ele possui um diploma duplo de BS / BA em engenharia elétrica pela University of San Diego.

Colofão

Os insetos na capa de *Programando Sistemas Embarcados com C e GNU Devel-Ferramentas de opção*, Segunda edição, são carrapatos. Existem aproximadamente 850 espécies desses parasitas sanguíneos, pequenos a microscópicos, distribuídos em todo o mundo. Eles são particularmente abundantes em regiões tropicais e subtropicais. Existem duas famílias principais de carrapatos: carrapatos duros, cujas partes bucais são visíveis de cima, e carrapatos moles, cujas partes bucais ficam ocultas.

Tanto nos carrapatos duros quanto nos moles, a boca é composta por três partes principais: os palpos, as quelíceras e o hipostomo. É o hipostomo que é inserido na pele do hospedeiro enquanto o carrapato está se alimentando. Uma série de projeções voltadas para trás no hipostomo dificultam a remoção do carrapato da pele. A maioria dos carrapatos também secretam uma substância pegajosa que os cola no lugar. Esta substância se dissolve quando o carrapato termina de se alimentar. A superfície externa do corpo se expande de 200 a 600 por cento para acomodar o sangue ingerido.

Os carrapatos passam por três estágios de vida: larva, ninfa e adulto. Em cada estágio, eles se alimentam de um hospedeiro mamífero, réptil ou pássaro. Os carrapatos esperam por um hospedeiro empoleirando-se em folhas ou outras superfícies com as duas patas dianteiras estendidas. Quando um host esbarra em

eles se agarram e se prendem. Carrapatos duros fêmeas adultas põem um único lote de milhares de ovos e depois morrem. Carrapatos machos adultos também morrem após um único acasalamento.

No que diz respeito aos parasitas, os carrapatos podem ser muito desagradáveis. Eles transmitem mais doenças do que qualquer outro parasita sugador de sangue, incluindo a doença de Lyme, febre maculosa das Montanhas Rochosas e febre recorrente. Eles também podem causar perda excessiva de sangue. Alguns carrapatos secretam venenos nervosos que podem causar a morte. Um carrapato pode ser removido da pele segurando-o com uma pinça ou um dispositivo especial para remover carrapatos o mais próximo possível da pele e puxando com um movimento constante. Não aperte o carrapato. Jogue-o imediatamente no vaso sanitário - ou coloque-o em um recipiente lacrado e segure-o por um mês, para o caso de desenvolver sintomas de uma doença.

A imagem da capa é uma gravura do século 19 do Dover Pictorial Archive. A fonte da capa é Adobe ITC Garamond. A fonte do texto é Linotype Birka; a fonte do título é Adobe Myriad Condensed; e a fonte do código é TheSans Mono Condensed da LucasFont.